csharp/Platform.Data.Doublets/CriterionMatchers/TargetMatcher.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Interfaces;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.CriterionMatchers;

/// <summary>
///     <para>
///         Represents the target matcher.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="LinksOperatorBase{TLinkAddress}" />
/// <seealso cref="ICriterionMatcher{TLinkAddress}" />
public class TargetMatcher<TLinkAddress> : LinksOperatorBase<TLinkAddress>, ICriterionMatcher<TLinkAddress> where TLinkAddress
: IUnsignedNumber<TLinkAddress>
{
    private readonly TLinkAddress _targetToMatch;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="TargetMatcher{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    /// <param name="targetToMatch">
    ///     <para>A target to match.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public TargetMatcher(ILinks<TLinkAddress> links, TLinkAddress targetToMatch) : base(links: links)
    {
        _targetToMatch = targetToMatch;
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance is matched.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public bool IsMatched(TLinkAddress link)
    {
        return _links.GetTarget(link: link) == _targetToMatch;
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LinksCascadeUniquenessAndUsagesResolver.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators;

/// <summary>
///     <para>
///         Represents the links cascade uniqueness and usages resolver.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="LinksUniquenessResolver{TLinkAddress}" />
public class LinksCascadeUniquenessAndUsagesResolver<TLinkAddress> : LinksUniquenessResolver<TLinkAddress> where TLinkAddress
: IUnsignedNumber<TLinkAddress>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksCascadeUniquenessAndUsagesResolver{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public LinksCascadeUniquenessAndUsagesResolver(ILinks<TLinkAddress> links) : base(links: links) { }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Resolves the address change conflict using the specified old link address.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="oldLinkAddress">
    ///     <para>The old link address.</para>
    ///     <para></para>
    /// </param>
    /// <param name="newLinkAddress">
    ///     <para>The new link address.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override TLinkAddress ResolveAddressChangeConflict(TLinkAddress oldLinkAddress, TLinkAddress newLinkAddress, Wri
teHandler<TLinkAddress>? handler)
    {
        var constants = _links.Constants;
        WriteHandlerState<TLinkAddress> handlerState = new(@continue: constants.Continue, @break: constants.Break, handler: ha
ndler);
        // Use Facade (the last decorator) to ensure recursion working correctly
        handlerState.Apply(result: _facade.MergeUsages(oldLinkIndex: oldLinkAddress, newLinkIndex: newLinkAddress, handler: ha
ndlerState.Handler));
        handlerState.Apply(result: base.ResolveAddressChangeConflict(oldLinkAddress: oldLinkAddress, newLinkAddress: newLinkAd
dress, handler: handlerState.Handler));
        return handlerState.Result;
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LinksCascadeUsagesResolver.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators;

/// <summary>Deletes links together with their dependent usages.</summary>
/// <remarks>
///     <para>Must be used in conjunction with NonNullContentsLinkDeletionResolver.</para>
///     <para>Должен использоваться вместе с NonNullContentsLinkDeletionResolver.</para>
/// </remarks>
public class LinksCascadeUsagesResolver<TLinkAddress> : LinksDecoratorBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<
TLinkAddress>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksCascadeUsagesResolver{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public LinksCascadeUsagesResolver(ILinks<TLinkAddress> links) : base(links: links) { }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Deletes the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
    {
        var constants = _links.Constants;
        WriteHandlerState<TLinkAddress> handlerState = new(@continue: constants.Continue, @break: constants.Break, handler: ha
ndler);
        var linkIndex = _links.GetIndex(link: restriction);
        // Use Facade (the last decorator) to ensure recursion working correctly
        handlerState.Apply(result: _facade.DeleteAllUsages(linkIndex: linkIndex, handler: handlerState.Handler));
        handlerState.Apply(result: _links.Delete(restriction: restriction, handler: handlerState.Handler));
        return handlerState.Result;
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LinksDecoratorBase.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators;

/// <summary>
///     <para>
///         Represents the links decorator base.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="LinksOperatorBase{TLinkAddress}" />
/// <seealso cref="ILinks{TLinkAddress}" />
public abstract class LinksDecoratorBase<TLinkAddress> : LinksOperatorBase<TLinkAddress>, ILinks<TLinkAddress> where TLinkAddr
ess : IUnsignedNumber<TLinkAddress>
{
    /// <summary>
    ///     <para>
    ///         The constants.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly LinksConstants<TLinkAddress> _constants;

    /// <summary>
    ///     <para>
    ///         The facade.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected ILinks<TLinkAddress> _facade;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksDecoratorBase{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected LinksDecoratorBase(ILinks<TLinkAddress> links) : base(links: links)
    {
        _constants = links.Constants;
        _facade = this;
        Facade = this;
    }

    /// <summary>
    ///     <para>
    ///         Gets or sets the facade value.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public ILinks<TLinkAddress> Facade
    {
        [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
        get => _facade;
        [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
        set
        {
            _facade = value;
            if (_links is LinksDecoratorBase<TLinkAddress> decorator)
            {
                decorator.Facade = value;
            }
        }
    }

    /// <summary>
    ///     <para>
    ///         Gets the constants value.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public LinksConstants<TLinkAddress> Constants
    {
        [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
        get => _constants;
    }

    /// <summary>
    ///     <para>
    ///         Counts the restriction.
    ///     </para>
```

```
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public virtual TLinkAddress Count(IList<TLinkAddress>? restriction)
    {
        return _links.Count(restriction: restriction);
    }

    /// <summary>
    ///     <para>
    ///         Eaches the handler.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="handler">
    ///     <para>The handler.</para>
    ///     <para></para>
    /// </param>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public virtual TLinkAddress Each(IList<TLinkAddress>? restriction, ReadHandler<TLinkAddress>? handler)
    {
        return _links.Each(restriction: restriction, handler: handler);
    }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Creates the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="substitution">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public virtual TLinkAddress Create(IList<TLinkAddress>? substitution, WriteHandler<TLinkAddress>? handler)
    {
        return _links.Create(substitution: substitution, handler: handler);
    }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Updates the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <param name="substitution">
    ///     <para>The substitution.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public virtual TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLink
Address>? handler)
    {
        return _links.Update(restriction: restriction, substitution: substitution, handler: handler);
    }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Deletes the restriction.
    ///     </para>
```

```
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public virtual TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
    {
        return _links.Delete(restriction: restriction, handler: handler);
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LinksDisposableDecoratorBase.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Disposables;
using IDisposable = System.IDisposable;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member
#pragma warning disable CA1063 // Implement IDisposable Correctly

namespace Platform.Data.Doublets.Decorators;

/// <summary>
///     <para>
///         Represents the links disposable decorator base.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="LinksDecoratorBase{TLinkAddress}" />
/// <seealso cref="ILinks{TLinkAddress}" />
/// <seealso cref="System.IDisposable" />
public abstract class LinksDisposableDecoratorBase<TLinkAddress> : LinksDecoratorBase<TLinkAddress>, ILinks<TLinkAddress>, IDi
sposable where TLinkAddress : IUnsignedNumber<TLinkAddress>
{
    /// <summary>
    ///     <para>
    ///         The disposable.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly DisposableWithMultipleCallsAllowed Disposable;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksDisposableDecoratorBase{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected LinksDisposableDecoratorBase(ILinks<TLinkAddress> links) : base(links: links)
    {
        Disposable = new DisposableWithMultipleCallsAllowed(disposal: (manual, wasDisposed) => Dispose(manual, wasDisposed));
    }

    /// <summary>
    ///     <para>
    ///         Disposes this instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public void Dispose()
    {
        Disposable.Dispose();
    }

    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    ~LinksDisposableDecoratorBase()
    {
        Disposable.Destruct();
    }

    /// <summary>
    ///     <para>
    ///         Disposes the manual.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="manual">
    ///     <para>The manual.</para>
    ///     <para></para>
    /// </param>
    /// <param name="wasDisposed">
    ///     <para>The was disposed.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual void Dispose(bool manual, bool wasDisposed)
    {
        if (!wasDisposed)
        {
            _links.DisposeIfPossible();
        }
    }

    /// <summary>
    ///     <para>
    ///         Represents the disposable with multiple calls allowed.
    ///     </para>
```

```
    ///     <para></para>
    /// </summary>
    /// <seealso cref="Disposable" />
    protected class DisposableWithMultipleCallsAllowed : Disposable
    {
        /// <summary>
        ///     <para>
        ///         Initializes a new <see cref="DisposableWithMultipleCallsAllowed" /> instance.
        ///     </para>
        ///     <para></para>
        /// </summary>
        /// <param name="disposal">
        ///     <para>A disposal.</para>
        ///     <para></para>
        /// </param>
        [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
        public DisposableWithMultipleCallsAllowed(Disposal disposal) : base(disposal: disposal) { }

        /// <summary>
        ///     <para>
        ///         Gets the allow multiple dispose calls value.
        ///     </para>
        ///     <para></para>
        /// </summary>
        protected override bool AllowMultipleDisposeCalls
        {
            [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
            get => true;
        }
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LinksInnerReferenceExistenceValidator.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators;

// TODO: Make LinksExternalReferenceValidator. A layer that checks each link to exist or to be external (hybrid link's raw num
ber).
/// <summary>
///     <para>
///         Represents the links inner reference existence validator.
///     </para>
///     <para></para>
/// </summary>
/// <remarks>
/// Validates internal references in reads and both update argument lists, and checks
/// deleted addresses. Place outside resolver layers to reject missing references
/// before searches or mutations. Non-internal values are not validated as update
/// addresses; callers must still supply a concrete internal update address.
/// </remarks>
/// <seealso cref="LinksDecoratorBase{TLinkAddress}" />
public class LinksInnerReferenceExistenceValidator<TLinkAddress> : LinksDecoratorBase<TLinkAddress> where TLinkAddress : IUnsi
gnedNumber<TLinkAddress>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksInnerReferenceExistenceValidator{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public LinksInnerReferenceExistenceValidator(ILinks<TLinkAddress> links) : base(links: links) { }

    /// <summary>
    ///     <para>
    ///         Eaches the handler.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="handler">
    ///     <para>The handler.</para>
    ///     <para></para>
    /// </param>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Each(IList<TLinkAddress>? restriction, ReadHandler<TLinkAddress>? handler)
    {
        var links = _links;
        links.EnsureInnerReferenceExists(restriction: restriction, argumentName: nameof(restriction));
        return links.Each(restriction: restriction, handler: handler);
    }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Updates the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <param name="substitution">
    ///     <para>The substitution.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLin
kAddress>? handler)
    {
        // TODO: Possible values: null, ExistentLink or NonExistentHybrid(ExternalReference)
        var links = _links;
```

```
        links.EnsureInnerReferenceExists(restriction: restriction, argumentName: nameof(restriction));
        links.EnsureInnerReferenceExists(restriction: substitution, argumentName: nameof(substitution));
        return links.Update(restriction: restriction, substitution: substitution, handler: handler);
    }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Deletes the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
    {
        var links = _links;
        var link = links.GetIndex(link: restriction);
        links.EnsureLinkExists(link: link, argumentName: nameof(link));
        return links.Delete(restriction: restriction, handler: handler);
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LinksItselfConstantToSelfReferenceResolver.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators;

/// <summary>
///     <para>
///         Represents the links itself constant to self reference resolver.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="LinksDecoratorBase{TLinkAddress}" />
public class LinksItselfConstantToSelfReferenceResolver<TLinkAddress> : LinksDecoratorBase<TLinkAddress> where TLinkAddress :
IUnsignedNumber<TLinkAddress>
{

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksItselfConstantToSelfReferenceResolver{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public LinksItselfConstantToSelfReferenceResolver(ILinks<TLinkAddress> links) : base(links: links) { }

    /// <summary>
    ///     <para>
    ///         Eaches the handler.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="handler">
    ///     <para>The handler.</para>
    ///     <para></para>
    /// </param>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Each(IList<TLinkAddress>? restriction, ReadHandler<TLinkAddress>? handler)
    {
        var constants = _constants;
        var itselfConstant = constants.Itself;
        if (constants.Any != itselfConstant && restriction != null && restriction.Contains(item: itselfConstant))
        {
            // Itself constant is not supported for Each method right now, skipping execution
            return constants.Continue;
        }
        return _links.Each(restriction: restriction, handler: handler);
    }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Updates the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <param name="substitution">
    ///     <para>The substitution.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLin
kAddress>? handler)
    {
        return _links.Update(restriction: restriction, substitution: _links.ResolveConstantAsSelfReference(constant: _constant
s.Itself, restriction: restriction, substitution: substitution), handler: handler);
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LinksNonExistentDeletionResolver.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;

namespace Platform.Data.Doublets.Decorators;

/// <summary>Ignores deletion of a missing link without calling the inner store or handler.</summary>
/// <remarks>
/// Place this outside reset and cascade resolvers so it runs before they read the link.
/// Existing links are forwarded unchanged; endpoint reset and usages remain separate policies.
/// </remarks>
public class LinksNonExistentDeletionResolver<TLinkAddress> : LinksDecoratorBase<TLinkAddress>
    where TLinkAddress : IUnsignedNumber<TLinkAddress>
{
    /// <summary>Wraps a store with the ignore-missing-deletion policy.</summary>
    public LinksNonExistentDeletionResolver(ILinks<TLinkAddress> links) : base(links) { }

    /// <summary>Returns Continue for a missing link, or forwards an existing deletion.</summary>
    [MethodImpl(MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
    {
        var index = _links.GetIndex(restriction);
        return _links.Exists(index) ? _links.Delete(restriction, handler) : Constants.Continue;
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LinksNonExistentDependenciesCreator.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators;

/// <summary>Creates missing dependencies before updating links.</summary>
/// <remarks>
///     Not practical if newSource and newTarget are too big.
///     To be able to use practical version we should allow to create link at any specific location inside
///     ResizableDirectMemoryLinks.
///     This in turn will require to implement not a list of empty links, but a list of ranges to store it more
///     efficiently.
/// </remarks>
public class LinksNonExistentDependenciesCreator<TLinkAddress> : LinksDecoratorBase<TLinkAddress> where TLinkAddress : IUnsign
edNumber<TLinkAddress>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksNonExistentDependenciesCreator{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public LinksNonExistentDependenciesCreator(ILinks<TLinkAddress> links) : base(links: links) { }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Updates the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <param name="substitution">
    ///     <para>The substitution.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLin
kAddress>? handler)
    {
        var links = _links;
        links.EnsureCreated(links.GetSource(link: substitution), links.GetTarget(link: substitution));
        return links.Update(restriction: restriction, substitution: substitution, handler: handler);
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LinksNullConstantToSelfReferenceResolver.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators;

/// <summary>
///     <para>
///         Represents the links null constant to self reference resolver.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="LinksDecoratorBase{TLinkAddress}" />
public class LinksNullConstantToSelfReferenceResolver<TLinkAddress> : LinksDecoratorBase<TLinkAddress> where TLinkAddress : IU
nsignedNumber<TLinkAddress>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksNullConstantToSelfReferenceResolver{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public LinksNullConstantToSelfReferenceResolver(ILinks<TLinkAddress> links) : base(links: links) { }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Creates the substitution.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="substitution">
    ///     <para>The substitution.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Create(IList<TLinkAddress>? substitution, WriteHandler<TLinkAddress>? handler)
    {
        return _links.CreatePoint(handler: handler);
    }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Updates the substitution.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The substitution.</para>
    ///     <para></para>
    /// </param>
    /// <param name="substitution">
    ///     <para>The substitution.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLin
kAddress>? handler)
    {
        return _links.Update(restriction: restriction, substitution: _links.ResolveConstantAsSelfReference(constant: _constant
s.Null, restriction: restriction, substitution: substitution), handler: handler);
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LinksUniquenessResolver.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators;

/// <summary>
///     <para>
///         Represents the links uniqueness resolver.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="LinksDecoratorBase{TLinkAddress}" />
public class LinksUniquenessResolver<TLinkAddress> : LinksDecoratorBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLi
nkAddress>
{

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksUniquenessResolver{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public LinksUniquenessResolver(ILinks<TLinkAddress> links) : base(links: links) { }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Updates the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <param name="substitution">
    ///     <para>The substitution.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLin
kAddress>? handler)
    {
        var links = _links;
         var newLinkAddress = links.SearchOrDefault(source: links.GetSource(link: substitution), target: links.GetTarget(link:
substitution));
        if (newLinkAddress == default)
        {
            return links.Update(restriction: restriction, substitution: substitution, handler: handler);
        }
        return ResolveAddressChangeConflict(oldLinkAddress: links.GetIndex(link: restriction), newLinkAddress: newLinkAddress,
handler: handler);
    }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Resolves the address change conflict using the specified old link address.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="oldLinkAddress">
    ///     <para>The old link address.</para>
    ///     <para></para>
    /// </param>
    /// <param name="newLinkAddress">
    ///     <para>The new link address.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>Callback control (Continue or Break).</para>
    ///     <para></para>
    /// </returns>
    /// <remarks>
    /// Cleanup is followed by a survivor-to-survivor notification that identifies
    /// the resolved link, including unchanged pairs. Break suppresses later
    /// notifications while cleanup completes.
    /// </remarks>
```

```
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual TLinkAddress ResolveAddressChangeConflict(TLinkAddress oldLinkAddress, TLinkAddress newLinkAddress, Writ
eHandler<TLinkAddress>? handler)
    {
        var constants = _links.Constants;
        WriteHandlerState<TLinkAddress> handlerState = new(constants.Continue, constants.Break, handler);
        if (oldLinkAddress != newLinkAddress && _links.Exists(link: oldLinkAddress))
        {
            handlerState.Apply(_facade.Delete(linkToDelete: oldLinkAddress, handler: handlerState.Handler));
        }
        // Report the resolved result last: reset/deletion callbacks describe the
        // redundant link and must not become the result of address-returning APIs.
        // This also supplies a result when the requested pair is already at old.
        if (handlerState.Handler is { } resolvedHandler)
        {
            var survivor = _links.GetLink(newLinkAddress);
            handlerState.Apply(resolvedHandler(survivor, survivor));
        }
        return handlerState.Result;
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LinksUniquenessValidator.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators;

/// <summary>
///     <para>
///         Represents the links uniqueness validator.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="LinksDecoratorBase{TLinkAddress}" />
public class LinksUniquenessValidator<TLinkAddress> : LinksDecoratorBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<TL
inkAddress>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksUniquenessValidator{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public LinksUniquenessValidator(ILinks<TLinkAddress> links) : base(links: links) { }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Updates the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <param name="substitution">
    ///     <para>The substitution.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLin
kAddress>? handler)
    {
        var links = _links;
        links.EnsureDoesNotExists(source: links.GetSource(link: substitution), target: links.GetTarget(link: substitution));
        return links.Update(restriction: restriction, substitution: substitution, handler: handler);
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LinksUsagesValidator.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators;

/// <summary>
///     <para>
///         Represents the links usages validator.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="LinksDecoratorBase{TLinkAddress}" />
public class LinksUsagesValidator<TLinkAddress> : LinksDecoratorBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkA
ddress>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksUsagesValidator{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public LinksUsagesValidator(ILinks<TLinkAddress> links) : base(links: links) { }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Updates the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <param name="substitution">
    ///     <para>The substitution.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLin
kAddress>? handler)
    {
        var links = _links;
        links.EnsureNoUsages(link: links.GetIndex(link: restriction));
        return links.Update(restriction: restriction, substitution: substitution, handler: handler);
    }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Deletes the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
    {
        var links = _links;
        var link = links.GetIndex(link: restriction);
        links.EnsureNoUsages(link: link);
        return links.Delete(restriction: restriction, handler: handler);
    }
}
```

csharp/Platform.Data.Doublets/Decorators/LoggingDecorator.cs

```
using System.Collections.Generic;
using System.IO;
using System.Numerics;
using Platform.Delegates;

namespace Platform.Data.Doublets.Decorators;

/// <summary>Logs link mutations and forwards optional callbacks.</summary>
public class LoggingDecorator<TLinkAddress> : LinksDecoratorBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddre
ss>
{
    private readonly Stream _logStream;
    private readonly StreamWriter _logStreamWriter;

    /// <summary>Initializes the decorator with its underlying links.</summary>
    public LoggingDecorator(ILinks<TLinkAddress> links, Stream logStream) : base(links: links)
    {
        _logStream = logStream;
        _logStreamWriter = new StreamWriter(stream: _logStream);
        _logStreamWriter.AutoFlush = true;
    }

    /// <inheritdoc/>
    public override TLinkAddress Create(IList<TLinkAddress>? substitution, WriteHandler<TLinkAddress>? handler)
    {
         WriteHandlerState<TLinkAddress> handlerState = new(@continue: _constants.Continue, @break: _constants.Break, handler:
handler);
        return base.Create(substitution: substitution, handler: (before, after) =>
        {
            handlerState.HandleNullable(before: before, after: after);
             _logStreamWriter.WriteLine(value: $"Create. Before: {new Link<TLinkAddress>(values: before)}. After: {new Link<TLi
nkAddress>(values: after)}");
            return _constants.Continue;
        });
    }

    /// <inheritdoc/>
    public override TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLin
kAddress>? handler)
    {
         WriteHandlerState<TLinkAddress> handlerState = new(@continue: _constants.Continue, @break: _constants.Break, handler:
handler);
        return base.Update(restriction: restriction, substitution: substitution, handler: (before, after) =>
        {
            handlerState.HandleNullable(before: before, after: after);
             _logStreamWriter.WriteLine(value: $"Update. Before: {new Link<TLinkAddress>(values: before)}. After: {new Link<TLi
nkAddress>(values: after)}");
            return _constants.Continue;
        });
    }

    /// <inheritdoc/>
    public override TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
    {
         WriteHandlerState<TLinkAddress> handlerState = new(@continue: _constants.Continue, @break: _constants.Break, handler:
handler);
        return base.Delete(restriction: restriction, handler: (before, after) =>
        {
            handlerState.HandleNullable(before: before, after: after);
             _logStreamWriter.WriteLine(value: $"Delete. Before: {new Link<TLinkAddress>(values: before)}. After: {new Link<TLi
nkAddress>(values: after)}");
            return _constants.Continue;
        });
    }
}
```

csharp/Platform.Data.Doublets/Decorators/NoExceptionsDecorator.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;
using Platform.Delegates;
using Platform.Exceptions;

namespace Platform.Data.Doublets.Decorators;

/// <summary>Returns the error constant when an operation throws.</summary>
public class NoExceptionsDecorator<TLinkAddress> : LinksDecoratorBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLink
Address>
{
    /// <summary>Optionally selects which exceptions are converted to the error constant. Null accepts all exceptions.</summar
y>
    public Func<Exception, bool>? ExceptionFilter { get; set; }

    /// <summary>Optionally observes converted exceptions for diagnostics. Null disables diagnostic callbacks.</summary>
    public Action<Exception>? ExceptionHandler { get; set; }

    /// <summary>Initializes the decorator with its underlying links.</summary>
    public NoExceptionsDecorator(ILinks<TLinkAddress> storage) : base(links: storage) { }

    /// <inheritdoc/>
    public override TLinkAddress Count(IList<TLinkAddress>? restriction)
    {
        try
        {
            return base.Count(restriction: restriction);
        }
        catch (Exception exception) when (ExceptionFilter?.Invoke(exception) ?? true)
        {
            ExceptionHandler?.Invoke(exception);
            exception.Ignore();
            return Constants.Error;
        }
    }

    /// <inheritdoc/>
    public override TLinkAddress Each(IList<TLinkAddress>? restriction, ReadHandler<TLinkAddress>? handler)
    {
        try
        {
            return base.Each(restriction: restriction, handler: handler);
        }
        catch (Exception exception) when (ExceptionFilter?.Invoke(exception) ?? true)
        {
            ExceptionHandler?.Invoke(exception);
            exception.Ignore();
            return Constants.Error;
        }
    }

    /// <inheritdoc/>
    public override TLinkAddress Create(IList<TLinkAddress>? substitution, WriteHandler<TLinkAddress>? handler)
    {
        try
        {
            return base.Create(substitution: substitution, handler: handler);
        }
        catch (Exception exception) when (ExceptionFilter?.Invoke(exception) ?? true)
        {
            ExceptionHandler?.Invoke(exception);
            exception.Ignore();
            return Constants.Error;
        }
    }

    /// <inheritdoc/>
    public override TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLin
kAddress>? handler)
    {
        try
        {
            return base.Update(restriction: restriction, substitution: substitution, handler: handler);
        }
        catch (Exception exception) when (ExceptionFilter?.Invoke(exception) ?? true)
        {
            ExceptionHandler?.Invoke(exception);
            exception.Ignore();
            return Constants.Error;
        }
    }

    /// <inheritdoc/>
    public override TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
    {
        try
        {
            return base.Delete(restriction: restriction, handler: handler);
        }
        catch (Exception exception) when (ExceptionFilter?.Invoke(exception) ?? true)
```

```
        {
            ExceptionHandler?.Invoke(exception);
            exception.Ignore();
            return Constants.Error;
        }
    }
}
```

csharp/Platform.Data.Doublets/Decorators/NonNullContentsLinkDeletionResolver.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators;

/// <summary>
///     <para>
///         Represents the non null contents link deletion resolver.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="LinksDecoratorBase{TLinkAddress}" />
public class NonNullContentsLinkDeletionResolver<TLinkAddress> : LinksDecoratorBase<TLinkAddress> where TLinkAddress : IUnsign
edNumber<TLinkAddress>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="NonNullContentsLinkDeletionResolver{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public NonNullContentsLinkDeletionResolver(ILinks<TLinkAddress> links) : base(links: links) { }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Deletes the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <remarks>The caller must supply an existing link. Choose an outer existence policy for untrusted addresses.</remarks>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
    {
        var linkIndex = _links.GetIndex(link: restriction);
        var constants = _links.Constants;
        WriteHandlerState<TLinkAddress> handlerResult = new(@continue: constants.Continue, @break: constants.Break, handler: h
andler);
        handlerResult.Apply(result: _links.EnforceResetValues(linkIndex: linkIndex, handler: handlerResult.Handler));
        handlerResult.Apply(result: _links.Delete(restriction: restriction, handler: handlerResult.Handler));
        return handlerResult.Result;
    }
}
```

csharp/Platform.Data.Doublets/Decorators/UInt64Links.cs

```
using System.Collections.Generic;
using System.Net.Security;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Delegates;
using TLinkAddress = System.UInt64;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators
{
    /// <summary>
    /// <para>Represents a combined decorator that implements the basic logic for interacting with the links storage for links
with addresses represented as <see cref="System.UInt64" />.</para>
    /// <para>Представляет комбинированный декоратор, реализующий основную логику по взаимодействии с хранилищем связей, для с
вязей с адресами представленными в виде <see cref="System.UInt64"/>.</para>
    /// </summary>
    /// <remarks>
    /// Возможные оптимизации:
    /// Объединение в одном поле Source и Target с уменьшением до 32 бит.
    ///     + меньше объём БД
    ///     - меньше производительность
    ///     - больше ограничение на количество связей в БД)
    /// Ленивое хранение размеров поддеревьев (расчитываемое по мере использования БД)
    ///     + меньше объём БД
    ///     - больше сложность
    ///
    /// Текущее теоретическое ограничение на индекс связи, из-за использования 5 бит в размере поддеревьев для AVL баланса и ф
лагов нитей: 2 в степени(64 минус 5 равно 59 ) равно 576 460 752 303 423 488
    /// Желательно реализовать поддержку переключения между деревьями и битовыми индексами (битовыми строками) - вариант матри
цы (выстраеваемой лениво).
    ///
    /// Решить отключать ли проверки при компиляции под Release. Т.е. исключения будут выбрасываться только при #if DEBUG
    /// </remarks>
    public class CombinedDecorator<TLinkAddress> : LinksDisposableDecoratorBase<TLinkAddress> where TLinkAddress : IUnsignedNu
mber<TLinkAddress>
    {
        /// <summary>
        /// <para>
        /// Initializes a new <see cref="CombinedDecorator{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public CombinedDecorator(ILinks<TLinkAddress> links) : base(links) { }

        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Creates the substitution.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="substitution">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The TLinkAddress</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override TLinkAddress Create(IList<TLinkAddress>? substitution, WriteHandler<TLinkAddress>? handler) => _links.
CreatePoint(handler);

        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Updates the substitution.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="restriction">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        /// <param name="substitution">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The TLinkAddress</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<
TLinkAddress>? handler)
```

```
        {
            System.ArgumentNullException.ThrowIfNull(restriction);
            System.ArgumentNullException.ThrowIfNull(substitution);
            var constants = _constants;
            var indexPartConstant = constants.IndexPart;
            var sourcePartConstant = constants.SourcePart;
            var targetPartConstant = constants.TargetPart;
            var nullConstant = constants.Null;
            var itselfConstant = constants.Itself;
            var existedLink = nullConstant;
            var updatedLink = restriction[indexPartConstant];
            var newSource = substitution[sourcePartConstant];
            var newTarget = substitution[targetPartConstant];
            var links = _links;
            if (newSource != itselfConstant && newTarget != itselfConstant)
            {
                existedLink = links.SearchOrDefault(newSource, newTarget);
            }
            if (existedLink == nullConstant)
            {
                var before = links.GetRequiredLink(updatedLink);
                if (before[sourcePartConstant] != newSource || before[targetPartConstant] != newTarget)
                {
                    var source = newSource == itselfConstant ? updatedLink : newSource;
                    var target = newTarget == itselfConstant ? updatedLink : newTarget;
                    return links.Update(new Link<TLinkAddress>(updatedLink, source, target), handler);
                }
                return _links.Constants.Continue;
            }
            else
            {
                return _facade.MergeAndDelete(updatedLink, existedLink, handler);
            }
        }

        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Deletes the substitution.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="restriction">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
        {
            var linkIndex = _links.GetIndex(restriction);
            var constants = _links.Constants;
            WriteHandlerState<TLinkAddress> handlerState = new(constants.Continue, constants.Break, handler);
            handlerState.Apply(_links.EnforceResetValues(linkIndex, handlerState.Handler));
            handlerState.Apply(_facade.DeleteAllUsages(linkIndex, handlerState.Handler));
            handlerState.Apply(_links.Delete(restriction, handlerState.Handler));
            return handlerState.Result;
        }
    }
}
```

csharp/Platform.Data.Doublets/Decorators/UniLinks.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;
using Platform.Collections;
using Platform.Collections.Lists;
using Platform.Data.Universal;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Decorators;

/// <remarks>
///     What does empty pattern (for condition or substitution) mean? Nothing or Everything?
///     Now we go with nothing. And nothing is something one, but empty, and cannot be changed by itself. But can cause
///     creation (update from nothing) or deletion (update to nothing).
///     TODO: Decide to change to IDoubletLinks or not to change. (Better to create DefaultUniLinksBase, that contains
///     logic itself and can be implemented using both IDoubletLinks and ILinks.)
/// </remarks>
internal class UniLinks<TLinkAddress> : LinksDecoratorBase<TLinkAddress>, IUniLinks<TLinkAddress> where TLinkAddress : IUnsign
edNumber<TLinkAddress>
{

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="UniLinks{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    public UniLinks(ILinks<TLinkAddress> links) : base(links: links) { }

    //public static readonly TLinkAddress NullConstant = Use<LinksConstants<TLinkAddress>>.Single.Null;
    //public static readonly IReadOnlyList<TLinkAddress> NullLink = new ReadOnlyCollection<TLinkAddress>(new List<TLinkAddress
> { NullConstant, NullConstant, NullConstant });

    // TODO: Подумать о том, как реализовать древовидный Restriction и Substitution (Links-Expression)
    /// <summary>
    ///     <para>
    ///         Triggers the restriction.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction.</para>
    ///     <para></para>
    /// </param>
    /// <param name="matchedHandler">
    ///     <para>The matched handler.</para>
    ///     <para></para>
    /// </param>
    /// <param name="substitution">
    ///     <para>The substitution.</para>
    ///     <para></para>
    /// </param>
    /// <param name="substitutedHandler">
    ///     <para>The substituted handler.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    public TLinkAddress Trigger(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? matchedHandler, IList<TLinkAddre
ss>? substitution, WriteHandler<TLinkAddress>? substitutedHandler)
    {
        ////List<Transition> transitions = null;
        ////if (!restriction.IsNullOrEmpty())
        ////{
        ////    // Есть причина делать проход (чтение)
        ////    if (matchedHandler != null)
        ////    {
        ////        if (!(substitution is not { Count: > 0 }))
        ////        {
        ////            // restriction => { 0, 0, 0 } | { 0 } // Create
        ////            // substitution => { itself, 0, 0 } | { itself, itself, itself } // Create / Update
        ////            // substitution => { 0, 0, 0 } | { 0 } // Delete
        ////            transitions = new List<Transition>();
        ////            if (Equals(substitution[Constants.IndexPart], Constants.Null))
        ////            {
        ////                // If index is Null, that means we always ignore every other value (they are also Null by definiti
on)
        ////                var matchDecision = matchedHandler(, NullLink);
        ////                if (Equals(matchDecision, Constants.Break))
        ////                    return false;
        ////                if (!Equals(matchDecision, Constants.Skip))
        ////                    transitions.Add(new Transition(matchedLink, newValue));
        ////            }
```

```
        ////            else
        ////            {
        ////                Func<T, bool> handler;
        ////                handler = link =>
        ////                {
        ////                    var matchedLink = Memory.GetLinkValue(link);
        ////                    var newValue = Memory.GetLinkValue(link);
        ////                    newValue[Constants.IndexPart] = Constants.Itself;
        ////                     newValue[Constants.SourcePart] = Equals(substitution[Constants.SourcePart], Constants.Itself)
? matchedLink[Constants.IndexPart] : substitution[Constants.SourcePart];
        ////                     newValue[Constants.TargetPart] = Equals(substitution[Constants.TargetPart], Constants.Itself)
? matchedLink[Constants.IndexPart] : substitution[Constants.TargetPart];
        ////                    var matchDecision = matchedHandler(matchedLink, newValue);
        ////                    if (Equals(matchDecision, Constants.Break))
        ////                        return false;
        ////                    if (!Equals(matchDecision, Constants.Skip))
        ////                        transitions.Add(new Transition(matchedLink, newValue));
        ////                    return true;
        ////                };
        ////                if (!Memory.Each(handler, restriction))
        ////                    return Constants.Break;
        ////            }
        ////        }
        ////        else
        ////        {
        ////            Func<T, bool> handler = link =>
        ////            {
        ////                var matchedLink = Memory.GetLinkValue(link);
        ////                var matchDecision = matchedHandler(matchedLink, matchedLink);
        ////                return !Equals(matchDecision, Constants.Break);
        ////            };
        ////            if (!Memory.Each(handler, restriction))
        ////                return Constants.Break;
        ////        }
        ////    }
        ////    else
        ////    {
        ////        if (substitution != null)
        ////        {
        ////            transitions = new List<IList<T>>();
        ////            Func<T, bool> handler = link =>
        ////            {
        ////                var matchedLink = Memory.GetLinkValue(link);
        ////                transitions.Add(matchedLink);
        ////                return true;
        ////            };
        ////            if (!Memory.Each(handler, restriction))
        ////                return Constants.Break;
        ////        }
        ////        else
        ////        {
        ////            return Constants.Continue;
        ////        }
        ////    }
        ////}
        ////if (substitution != null)
        ////{
        ////    // Есть причина делать замену (запись)
        ////    if (substitutedHandler != null)
        ////    {
        ////    }
        ////    else
        ////    {
        ////    }
        ////}
        ////return Constants.Continue;

        //if (restriction.IsNullOrEmpty()) // Create
        //{
        //    substitution[Constants.IndexPart] = Memory.AllocateLink();
        //    Memory.SetLinkValue(substitution);
        //}
        //else if ((substitution is not { Count: > 0 })) // Delete
        //{
        //    Memory.FreeLink(restriction[Constants.IndexPart]);
        //}
        //else if (restriction.EqualTo(substitution)) // Read or ("repeat" the state) // Each
        //{
        //    // No need to collect links to list
        //    // Skip == Continue
        //    // No need to check substituedHandler
        //    if (!Memory.Each(link => !Equals(matchedHandler(Memory.GetLinkValue(link)), Constants.Break), restriction))
        //        return Constants.Break;
        //}
        //else // Update
        //{
        //    //List<IList<T>> matchedLinks = null;
        //    if (matchedHandler != null)
        //    {
        //        matchedLinks = new List<IList<T>>();
        //        Func<T, bool> handler = link =>
        //        {
```

```
        //            var matchedLink = Memory.GetLinkValue(link);
        //            var matchDecision = matchedHandler(matchedLink);
        //            if (Equals(matchDecision, Constants.Break))
        //                return false;
        //            if (!Equals(matchDecision, Constants.Skip))
        //                matchedLinks.Add(matchedLink);
        //            return true;
        //        };
        //        if (!Memory.Each(handler, restriction))
        //            return Constants.Break;
        //    }
        //    if (!matchedLinks.IsNullOrEmpty())
        //    {
        //        var totalMatchedLinks = matchedLinks.Count;
        //        for (var i = 0; i < totalMatchedLinks; i++)
        //        {
        //            var matchedLink = matchedLinks[i];
        //            if (substitutedHandler != null)
        //            {
        //                var newValue = new List<T>(); // TODO: Prepare value to update here
        //                // TODO: Decide is it actually needed to use Before and After substitution handling.
        //                var substitutedDecision = substitutedHandler(matchedLink, newValue);
        //                if (Equals(substitutedDecision, Constants.Break))
        //                    return Constants.Break;
        //                if (Equals(substitutedDecision, Constants.Continue))
        //                {
        //                    // Actual update here
        //                    Memory.SetLinkValue(newValue);
        //                }
        //                if (Equals(substitutedDecision, Constants.Skip))
        //                {
        //                    // Cancel the update. TODO: decide use separate Cancel constant or Skip is enough?
        //                }
        //            }
        //        }
        //    }
        //}
        return _constants.Continue;
    }

    /// <summary>
    ///     <para>
    ///         Triggers the pattern or condition.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="patternOrCondition">
    ///     <para>The pattern or condition.</para>
    ///     <para></para>
    /// </param>
    /// <param name="matchHandler">
    ///     <para>The match handler.</para>
    ///     <para></para>
    /// </param>
    /// <param name="substitution">
    ///     <para>The substitution.</para>
    ///     <para></para>
    /// </param>
    /// <param name="substitutionHandler">
    ///     <para>The substitution handler.</para>
    ///     <para></para>
    /// </param>
    /// <exception cref="NotImplementedException">
    ///     <para></para>
    ///     <para></para>
    /// </exception>
    /// <exception cref="NotSupportedException">
    ///     <para></para>
    ///     <para></para>
    /// </exception>
    /// <exception cref="NotSupportedException">
    ///     <para></para>
    ///     <para></para>
    /// </exception>
    /// <exception cref="NotSupportedException">
    ///     <para></para>
    ///     <para></para>
    /// </exception>
    /// <exception cref="NotSupportedException">
    ///     <para></para>
    ///     <para></para>
    /// </exception>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    public TLinkAddress Trigger(IList<TLinkAddress>? patternOrCondition, ReadHandler<TLinkAddress>? matchHandler, IList<TLinkA
ddress>? substitution, WriteHandler<TLinkAddress>? substitutionHandler)
    {
        var constants = _constants;
        if ((patternOrCondition is not { Count: > 0 }) && (substitution is not { Count: > 0 }))
        {
```

```
            return constants.Continue;
        }
        if (patternOrCondition != null && substitution != null && patternOrCondition.EqualTo(right: substitution)) // Should b
e Each here TODO: Check if it is a correct condition
        {
            // Or it only applies to trigger without matchHandler.
            throw new NotImplementedException();
        }
        if (patternOrCondition is not { Count: > 0 }) // Creation
        {
            var creation = substitution ?? throw new ArgumentNullException(nameof(substitution));
            var before = Array.Empty<TLinkAddress>();
            // Что должно означать False здесь? Остановиться (перестать идти) или пропустить (пройти мимо) или пустить (взять)
?
            if (matchHandler != null && matchHandler(link: before) == constants.Break)
            {
                return constants.Break;
            }
            var after = (IList<TLinkAddress>)creation.ToArray();
            if (after[index: 0] == default)
            {
                var newLink = _links.Create();
                after[index: 0] = newLink;
            }
            if (creation.Count == 1)
            {
                after = _links.GetRequiredLink(link: creation[index: 0]);
            }
            else if (creation.Count == 3)
            {
                //Links.Create(after);
            }
            else
            {
                throw new NotSupportedException();
            }
            return substitutionHandler != null ? substitutionHandler(before: before, after: after) : constants.Continue;
        }
        if (substitution is not { Count: > 0 }) // Deletion
        {
            ArgumentNullException.ThrowIfNull(patternOrCondition);
            if (patternOrCondition.Count == 1)
            {
                var linkToDelete = patternOrCondition[index: 0];
                var before = _links.GetRequiredLink(link: linkToDelete);
                if (matchHandler != null && matchHandler(link: before) == constants.Break)
                {
                    return constants.Break;
                }
                var after = Array.Empty<TLinkAddress>();
                _links.Update(link: linkToDelete, newSource: constants.Null, newTarget: constants.Null);
                _links.Delete(linkToDelete: linkToDelete);
                return substitutionHandler != null ? substitutionHandler(before: before, after: after) : constants.Continue;
            }
            throw new NotSupportedException();
        }
        // Replace / Update
        ArgumentNullException.ThrowIfNull(patternOrCondition);
        ArgumentNullException.ThrowIfNull(substitution);
        if (patternOrCondition.Count == 1) //-V3125
        {
            var linkToUpdate = patternOrCondition[index: 0];
            var before = _links.GetRequiredLink(link: linkToUpdate);
            if (matchHandler != null && matchHandler(link: before) == constants.Break)
            {
                return constants.Break;
            }
            var after = (IList<TLinkAddress>)substitution.ToArray(); //-V3125
            if (after[index: 0] == default)
            {
                after[index: 0] = linkToUpdate;
            }
            if (substitution.Count == 1)
            {
                if (substitution[index: 0] != linkToUpdate)
                {
                    after = _links.GetRequiredLink(link: substitution[index: 0]);
                    _links.Update(link: linkToUpdate, newSource: constants.Null, newTarget: constants.Null);
                    _links.Delete(linkToDelete: linkToUpdate);
                }
            }
            else if (substitution.Count == 3)
            {
                _links.Update(link: linkToUpdate, newSource: after[constants.SourcePart], newTarget: after[constants.TargetPar
t]);
            }
            else
            {
                throw new NotSupportedException();
            }
            return substitutionHandler != null ? substitutionHandler(before: before, after: after) : constants.Continue;
        }
```

```
        throw new NotSupportedException();
    }

    /// <summary>Executes the condition and substitution and returns the recorded changes.</summary>
    /// <remarks>
    ///     IList[IList[IList[T]]]
    ///     |     |     |      |||
    ///     |     |      ------ ||
    ///     |     |       link  ||
    ///     |      ------------- |
    ///     |         change     |
    ///     --------------------
    ///     changes
    /// </remarks>
    public IList<IList<IList<TLinkAddress>?>> Trigger(IList<TLinkAddress>? condition, IList<TLinkAddress>? substitution)
    {
        var changes = new List<IList<IList<TLinkAddress>?>>();
        var @continue = _constants.Continue;
         Trigger(patternOrCondition: condition, matchHandler: AlwaysContinue, substitution: substitution, substitutionHandler:
(before, after) =>
        {
            var change = new[] { before, after };
            changes.Add(item: change);
            return @continue;
        });
        return changes;
    }

    private TLinkAddress AlwaysContinue(IList<TLinkAddress>? linkToMatch)
    {
        return _constants.Continue;
    }

    private struct Transition
    {
        /// <summary>
        ///     <para>
        ///         The before.
        ///     </para>
        ///     <para></para>
        /// </summary>
        public readonly IList<TLinkAddress>? Before;
        /// <summary>
        ///     <para>
        ///         The after.
        ///     </para>
        ///     <para></para>
        /// </summary>
        public readonly IList<TLinkAddress>? After;

        /// <summary>
        ///     <para>
        ///         Initializes a new <see cref="Transition" /> instance.
        ///     </para>
        ///     <para></para>
        /// </summary>
        /// <param name="before">
        ///     <para>A before.</para>
        ///     <para></para>
        /// </param>
        /// <param name="after">
        ///     <para>A after.</para>
        ///     <para></para>
        /// </param>
        public Transition(IList<TLinkAddress>? before, IList<TLinkAddress>? after)
        {
            Before = before;
            After = after;
        }
    }
}
```

csharp/Platform.Data.Doublets/Doublet.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets
{

    /// <summary>
    /// <para>.</para>
    /// <para>.</para>
    /// </summary>
    /// <typeparam>
    /// <para>.</para>
    /// <para>.</para>
    /// </typeparam>
    public struct Doublet<T> : IEquatable<Doublet<T>> where T : IUnsignedNumber<T>
    {

        /// <summary>
        /// <para>.</para>
        /// <para>.</para>
        /// </summary>
        public readonly T Source;

        /// <summary>
        /// <para>.</para>
        /// <para>.</para>
        /// </summary>
        public readonly T Target;

        /// <summary>
        /// <para>.</para>
        /// <para>.</para>
        /// </summary>
        /// <param name="source">
        /// <para>.</para>
        /// <para>.</para>
        /// </param>
        /// <param name="target">
        /// <para>.</para>
        /// <para>.</para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Doublet(T source, T target)
        {
            Source = source;
            Target = target;
        }

        /// <summary>
        /// <para>.</para>
        /// <para>.</para>
        /// </summary>
        /// <returns>
        /// <para>.</para>
        /// <para>.</para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override string ToString() { return $"{Source}->{Target}"; }

        /// <summary>
        /// <para>.</para>
        /// <para>.</para>
        /// </summary>
        /// <typeparam>
        /// <para>.</para>
        /// <para>.</para>
        /// </typeparam>
        /// <param name="other">
        /// <para>.</para>
        /// <para>.</para>
        /// </param>
        /// <returns>
        /// <para>.</para>
        /// <para>.</para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public bool Equals(Doublet<T> other) { return Source == other.Source && Target == other.Target; }

        /// <summary>
        /// <para>.</para>
        /// <para>.</para>
        /// </summary>
        /// <typeparam>
        /// <para>.</para>
        /// <para>.</para>
        /// </typeparam>
        /// <param name="obj">
```

```
        /// <para>.</para>
        /// <para>.</para>
        /// </param>
        /// <returns>
        /// <para>.</para>
        /// <para>.</para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override bool Equals(object? obj) { return obj is Doublet<T> doublet && base.Equals(doublet); }

        /// <summary>
        /// <para>.</para>
        /// <para>.</para>
        /// </summary>
        /// <returns>
        /// <para>.</para>
        /// <para>.</para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override int GetHashCode() { return (Source, Target).GetHashCode(); }

        /// <summary>
        /// <para>.</para>
        /// <para>.</para>
        /// </summary>
        /// <param name="left">
        /// <para>.</para>
        /// <para>.</para>
        /// </param>
        /// <param name="right">
        /// <para>.</para>
        /// <para>.</para>
        /// </param>
        /// <returns>
        /// <para>.</para>
        /// <para>.</para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator ==(Doublet<T> left, Doublet<T> right) { return left.Equals(right); }

        /// <summary>
        /// <para>.</para>
        /// <para>.</para>
        /// </summary>
        /// <param name="left">
        /// <para>.</para>
        /// <para>.</para>
        /// </param>
        /// <param name="right">
        /// <para>.</para>
        /// <para>.</para>
        /// </param>
        /// <returns>
        /// <para>.</para>
        /// <para>.</para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator !=(Doublet<T> left, Doublet<T> right) { return !(left == right); }
    }
}
```

csharp/Platform.Data.Doublets/DoubletComparer.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets
{
    /// <summary>Compares doublets by their source and target.</summary>
    /// <remarks>
    /// TODO: Может стоит попробовать ref во всех методах (IRefEqualityComparer)
    /// 2x faster with comparer
    /// </remarks>
    public class DoubletComparer<T> : IEqualityComparer<Doublet<T>> where T : IUnsignedNumber<T>
    {
        /// <summary>
        /// <para>
        /// The .
        /// </para>
        /// <para></para>
        /// </summary>
        public static readonly DoubletComparer<T> Default = new DoubletComparer<T>();

        /// <summary>
        /// <para>
        /// Determines whether this instance equals.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="x">
        /// <para>The .</para>
        /// <para></para>
        /// </param>
        /// <param name="y">
        /// <para>The .</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public bool Equals(Doublet<T> x, Doublet<T> y) { return x.Equals(y); }

        /// <summary>
        /// <para>
        /// Gets the hash code using the specified obj.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="obj">
        /// <para>The obj.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The int</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public int GetHashCode(Doublet<T> obj) { return obj.GetHashCode(); }
    }
}
```

csharp/Platform.Data.Doublets/Ffi/Links.cs

```
/*
using System;
using System.Collections.Generic;
using System.Linq;
using System.Runtime.InteropServices;
using Platform.Converters;
using Platform.Delegates;
using Platform.Disposables;

namespace Platform.Data.Doublets.Ffi
{
    struct FfiLink_UInt8
    {
        public Byte Index;
        public Byte Source;
        public Byte Target;
    }

    struct FfiLink_UInt16
    {
        public UInt16 Index;
        public UInt16 Source;
        public UInt16 Target;
    }

    struct FfiLink_UInt32
    {
        public UInt32 Index;
        public UInt32 Source;
        public UInt32 Target;
    }

    struct FfiLink_UInt64
    {
        public UInt64 Index;
        public UInt64 Source;
        public UInt64 Target;
    }

    unsafe static class Methods
    {
        private const string DllName = "Platform.Doublets";

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate Byte EachCallback_UInt8(FfiLink_UInt8 link);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate UInt16 EachCallback_UInt16(FfiLink_UInt16 link);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate UInt32 EachCallback_UInt32(FfiLink_UInt32 link);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate UInt64 EachCallback_UInt64(FfiLink_UInt64 link);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate Byte CreateCallback_UInt8(FfiLink_UInt8 before, FfiLink_UInt8 after);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate UInt16 CreateCallback_UInt16(FfiLink_UInt16 before, FfiLink_UInt16 after);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate UInt32 CreateCallback_UInt32(FfiLink_UInt32 before, FfiLink_UInt32 after);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate UInt64 CreateCallback_UInt64(FfiLink_UInt64 before, FfiLink_UInt64 after);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate Byte UpdateCallback_UInt8(FfiLink_UInt8 before, FfiLink_UInt8 after);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate UInt16 UpdateCallback_UInt16(FfiLink_UInt16 before, FfiLink_UInt16 after);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate UInt32 UpdateCallback_UInt32(FfiLink_UInt32 before, FfiLink_UInt32 after);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate UInt64 UpdateCallback_UInt64(FfiLink_UInt64 before, FfiLink_UInt64 after);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate Byte DeleteCallback_UInt8(FfiLink_UInt8 before, FfiLink_UInt8 after);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate UInt16 DeleteCallback_UInt16(FfiLink_UInt16 before, FfiLink_UInt16 after);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate UInt32 DeleteCallback_UInt32(FfiLink_UInt32 before, FfiLink_UInt32 after);

        [UnmanagedFunctionPointer(CallingConvention.Cdecl)]
        public delegate UInt64 DeleteCallback_UInt64(FfiLink_UInt64 before, FfiLink_UInt64 after);
```

```
        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern void* ByteLinks_New(string path);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern void* UInt16Links_New(string path);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern void* UInt32Links_New(string path);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern void* UInt64Links_New(string path);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern void ByteLinks_Drop(void* self);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern void UInt16Links_Drop(void* self);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern void UInt32Links_Drop(void* self);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern void UInt64Links_Drop(void* self);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern byte ByteLinks_Create(void* self, byte* substitution, nuint substitutionLength, CreateCallback_UI
nt8 callback);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern ushort UInt16Links_Create(void* self, ushort* substitution, nuint substitutionLength, CreateCallb
ack_UInt16 callback);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern uint UInt32Links_Create(void* self, uint* substitution, nuint substitutionLength, CreateCallback_
UInt32 callback);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern ulong UInt64Links_Create(void* self, ulong* substitution, nuint substitutionLength, CreateCallbac
k_UInt64 callback);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern byte ByteLinks_Count(void* self, byte* restriction, nuint len);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern ushort UInt16Links_Count(void* self, ushort* restriction, nuint len);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern uint UInt32Links_Count(void* self, uint* restriction, nuint len);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern ulong UInt64Links_Count(void* self, ulong* restriction, nuint len);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern byte ByteLinks_Each(void* self, byte* restriction, nuint len, EachCallback_UInt8 callback);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern ushort UInt16Links_Each(void* self, ushort* restriction, nuint len, EachCallback_UInt16 callback)
;

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern uint UInt32Links_Each(void* self, uint* restriction, nuint len, EachCallback_UInt32 callback);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern ulong UInt64Links_Each(void* self, ulong* restriction, nuint len, EachCallback_UInt64 callback);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern byte ByteLinks_Update(void* self, byte* restriction, nuint restrictionLength,  byte* substitution
, nuint substitutionLength, UpdateCallback_UInt8 callback);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern ushort UInt16Links_Update(void* self, ushort* restriction, nuint restrictionLength,  ushort* subs
titution, nuint substitutionLength, UpdateCallback_UInt16 callback);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern uint UInt32Links_Update(void* self, uint* restriction, nuint restrictionLength,  uint* substituti
on, nuint substitutionLength, UpdateCallback_UInt32 callback);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern ulong UInt64Links_Update(void* self, ulong* restriction, nuint restrictionLength,  ulong* substit
ution, nuint substitutionLength, UpdateCallback_UInt64 callback);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern byte ByteLinks_Delete(void* self, byte* restriction, nuint restrictionLength, DeleteCallback_UInt
8 callback);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern ushort UInt16Links_Delete(void* self, ushort* restriction, nuint len, DeleteCallback_UInt16 callb
ack);

        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern uint UInt32Links_Delete(void* self, uint* restriction, nuint len, DeleteCallback_UInt32 callback)
;
```

```
        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
        public static extern ulong UInt64Links_Delete(void* self, ulong* restriction, nuint len, DeleteCallback_UInt64 callbac
k);
    }

    public class Links<TLinkAddress> : DisposableBase, ILinks<TLinkAddress>  where TLinkAddress : IUnsignedNumber<TLinkAddress
>
    {

        public LinksConstants<TLinkAddress> Constants { get; }

        private readonly unsafe void* _ptr;

        public Links(string path)
        {
            TLinkAddress t = default;
            unsafe
            {
                _ptr = t switch
                {
                    byte => Methods.ByteLinks_New(path),
                    ushort => Methods.UInt16Links_New(path),
                    uint => Methods.UInt32Links_New(path),
                    ulong => Methods.UInt64Links_New(path),
                    _ => throw new NotImplementedException()
                };

                // TODO: Update api
                Constants = new LinksConstants<TLinkAddress>(enableExternalReferencesSupport: true);
            }
        }

        public TLinkAddress Count(IList<TLinkAddress>? restriction)
        {
            var restrictionLength = restriction?.Count ?? 0;
            unsafe
            {
                TLinkAddress t = default;
                switch (t)
                {
                    case byte:
                    {
                        var restrictionArray = stackalloc byte[restrictionLength];
                        var byteRestrictionArray = (IList<byte>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = byteRestrictionArray[i];
                        }
                        return from_u8.Convert(Methods.ByteLinks_Count(_ptr, restrictionArray, (nuint)restrictionLength));
                    }
                    case ushort:
                    {
                        var restrictionArray = stackalloc ushort[restrictionLength];
                        var ushortRestrictionArray = (IList<ushort>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = ushortRestrictionArray[i];
                        }
                        return from_u16.Convert(Methods.UInt16Links_Count(_ptr, restrictionArray, (nuint)restrictionLength));
                    }
                    case uint:
                    {
                        var restrictionArray = stackalloc uint[restrictionLength];
                        var uintRestrictionArray = (IList<uint>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = uintRestrictionArray[i];
                        }
                        return from_u32.Convert(Methods.UInt32Links_Count(_ptr, restrictionArray, (nuint)restrictionLength));
                    }
                    case ulong:
                    {
                        {
                            var restrictionArray = stackalloc ulong[restrictionLength];
                            var ulongRestrictionArray = (IList<ulong>)restriction;
                            for (var i = 0; i < restrictionLength; i++)
                            {
                                restrictionArray[i] = ulongRestrictionArray[i];
                            }
                             return from_u64.Convert(Methods.UInt64Links_Count(_ptr, restrictionArray, (nuint)restrictionLength
));
                        }
                    }
                    default:
                    {
                        throw new NotImplementedException();
                    }
                }
            }
        }
```

```
        public TLinkAddress Each(IList<TLinkAddress>? restriction, ReadHandler<TLinkAddress>? handler)
        {
            var restrictionLength = restriction?.Count ?? 0;
            unsafe
            {
                TLinkAddress t = default;
                switch (t)
                {
                    case byte:
                    {
                        byte Callback(FfiLink_UInt8 link) => (byte)from_t.Convert(handler != null ? handler(new Link<TLinkAddr
ess>(from_u8.Convert(link.Index), from_u8.Convert(link.Source), from_u8.Convert(link.Target))) : Constants.Continue);
                        var restrictionArray = stackalloc byte[restrictionLength];
                        var byteRestrictionArray = (IList<byte>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = byteRestrictionArray[i];
                        }
                        return from_u8.Convert(Methods.ByteLinks_Each(_ptr, restrictionArray, (nuint)restrictionLength, Callba
ck));
                    }
                    case ushort:
                    {
                        ushort Callback(FfiLink_UInt16 link) => (ushort)from_t.Convert(handler != null ? handler(new Link<TLin
kAddress>(from_u16.Convert(link.Index), from_u16.Convert(link.Source), from_u16.Convert(link.Target))) : Constants.Continue);
                        var restrictionArray = stackalloc ushort[restrictionLength];
                        var ushortRestrictionArray = (IList<ushort>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = ushortRestrictionArray[i];
                        }
                        return from_u16.Convert(Methods.UInt16Links_Each(_ptr, restrictionArray, (nuint)restrictionLength, Cal
lback));
                    }
                    case uint:
                    {
                        uint Callback(FfiLink_UInt32 link) => (uint)from_t.Convert(handler != null ? handler(new Link<TLinkAdd
ress>(from_u32.Convert(link.Index), from_u32.Convert(link.Source), from_u32.Convert(link.Target))) : Constants.Continue);
                        var restrictionArray = stackalloc uint[restrictionLength];
                        var uintRestrictionArray = (IList<uint>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = uintRestrictionArray[i];
                        }
                        return from_u32.Convert(Methods.UInt32Links_Each(_ptr, restrictionArray, (nuint)restrictionLength, Cal
lback));
                    }
                    case ulong:
                    {
                        {
                            ulong Callback(FfiLink_UInt64 link) => from_t.Convert(handler != null ? handler(new Link<TLinkAddr
ess>(from_u64.Convert(link.Index), from_u64.Convert(link.Source), from_u64.Convert(link.Target))) : Constants.Continue);
                            var restrictionArray = stackalloc UInt64[restrictionLength];
                            var ulongRestrictionArray = (IList<ulong>)restriction;
                            for (var i = 0; i < restrictionLength; i++)
                            {
                                restrictionArray[i] = ulongRestrictionArray[i];
                            }
                            return from_u64.Convert(Methods.UInt64Links_Each(_ptr, restrictionArray, (nuint)restrictionLength,
Callback));
                        }
                    }
                    default:
                    {
                        throw new NotImplementedException();
                    }
                }
            }
        }

        public TLinkAddress Create(IList<TLinkAddress>? substitution, WriteHandler<TLinkAddress>? handler)
        {
            var substitutionLength = substitution?.Count ?? 0;
            unsafe
            {
                TLinkAddress t = default;
                switch (t)
                {
                    case byte:
                    {
                        byte Callback(FfiLink_UInt8 before, FfiLink_UInt8 after) => (byte)from_t.Convert(handler != null ? han
dler(new Link<TLinkAddress>(from_u8.Convert(before.Index), from_u8.Convert(before.Source), from_u8.Convert(before.Target)), ne
w Link<TLinkAddress>(from_u8.Convert(after.Index), from_u8.Convert(after.Source), from_u8.Convert(after.Target))) : Constants.
Continue);
                        var substitutionArray = stackalloc byte[substitutionLength];
                        var byteSubstitutionArray = (IList<byte>)substitution;
                        for (var i = 0; i < substitutionLength; i++)
                        {
                            substitutionArray[i] = byteSubstitutionArray[i];
                        }
                        return from_u8.Convert(Methods.ByteLinks_Create(_ptr, substitutionArray, (nuint)(substitution?.Count ?
? 0), Callback));
```

```
                    }
                    case ushort:
                    {
                        ushort Callback(FfiLink_UInt16 before, FfiLink_UInt16 after) => (ushort)from_t.Convert(handler != null
? handler(new Link<TLinkAddress>(from_u16.Convert(before.Index), from_u16.Convert(before.Source), from_u16.Convert(before.Targ
et)), new Link<TLinkAddress>(from_u16.Convert(after.Index), from_u16.Convert(after.Source), from_u16.Convert(after.Target))) :
Constants.Continue);
                        var substitutionArray = stackalloc ushort[substitutionLength];
                        var ushortSubstitutionArray = (IList<ushort>)substitution;
                        for (var i = 0; i < substitutionLength; i++)
                        {
                            substitutionArray[i] = ushortSubstitutionArray[i];
                        }
                        return from_u16.Convert(Methods.UInt16Links_Create(_ptr, substitutionArray, (nuint)(substitution?.Coun
t ?? 0), Callback));
                    }
                    case uint:
                    {
                        uint Callback(FfiLink_UInt32 before, FfiLink_UInt32 after) => (uint)from_t.Convert(handler != null ? h
andler(new Link<TLinkAddress>(from_u32.Convert(before.Index), from_u32.Convert(before.Source), from_u32.Convert(before.Target)
), new Link<TLinkAddress>(from_u32.Convert(after.Index), from_u32.Convert(after.Source), from_u32.Convert(after.Target))) : Co
nstants.Continue);
                        var substitutionArray = stackalloc uint[substitutionLength];
                        var uintSubstitutionArray = (IList<uint>)substitution;
                        for (var i = 0; i < substitutionLength; i++)
                        {
                            substitutionArray[i] = uintSubstitutionArray[i];
                        }
                        return from_u32.Convert(Methods.UInt32Links_Create(_ptr, substitutionArray, (nuint)(substitution?.Coun
t ?? 0), Callback));

                    }
                    case ulong:
                    {
                        ulong Callback(FfiLink_UInt64 before, FfiLink_UInt64 after) => (ulong)from_t.Convert(handler != null ?
handler(new Link<TLinkAddress>(from_u64.Convert(before.Index), from_u64.Convert(before.Source), from_u64.Convert(before.Target
)), new Link<TLinkAddress>(from_u64.Convert(after.Index), from_u64.Convert(after.Source), from_u64.Convert(after.Target))) : C
onstants.Continue);
                        var substitutionArray = stackalloc ulong[substitutionLength];
                        var ulongSubstitutionArray = (IList<ulong>)substitution;
                        for (var i = 0; i < substitutionLength; i++)
                        {
                            substitutionArray[i] = ulongSubstitutionArray[i];
                        }
                        return from_u64.Convert(Methods.UInt64Links_Create(_ptr, substitutionArray, (nuint)(substitution?.Coun
t ?? 0), Callback));

                    }
                    default:
                    {
                        throw new NotImplementedException();
                    }
                };
            }
        }

        public TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLinkAddr
ess>? handler)
        {
            var restrictionLength = restriction?.Count ?? 0;
            var substitutionLength = substitution?.Count ?? 0;
            unsafe
            {
                TLinkAddress t = default;
                switch (t)
                {
                    case byte:
                    {
                        var restrictionArray = stackalloc byte[restrictionLength];
                        var byteRestrictionArray = (IList<byte>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = byteRestrictionArray[i];
                        }
                        var substitutionArray = stackalloc byte[substitutionLength];
                        var byteSubstitutionArray = (IList<byte>)substitution;
                        for (var i = 0; i < substitutionLength; i++)
                        {
                            substitutionArray[i] = byteSubstitutionArray[i];
                        }
                        byte Callback(FfiLink_UInt8 before, FfiLink_UInt8 after) => (byte)from_t.Convert(handler != null ? han
dler(new Link<TLinkAddress>(from_u8.Convert(before.Index), from_u8.Convert(before.Source), from_u8.Convert(before.Target)), ne
w Link<TLinkAddress>(from_u8.Convert(after.Index), from_u8.Convert(after.Source), from_u8.Convert(after.Target))) : Constants.
Continue);
                        return from_u8.Convert(Methods.ByteLinks_Update(_ptr, restrictionArray, (nuint)restrictionLength, subs
titutionArray, (nuint)(substitution?.Count ?? 0), Callback));
                    }
                    case ushort:
                    {
                        var restrictionArray = stackalloc ushort[restrictionLength];
                        var ushortRestrictionArray = (IList<ushort>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
```

```
                        {
                            restrictionArray[i] = ushortRestrictionArray[i];
                        }
                        var substitutionArray = stackalloc ushort[substitutionLength];
                        var ushortSubstitutionArray = (IList<ushort>)substitution;
                        for (var i = 0; i < substitutionLength; i++)
                        {
                            substitutionArray[i] = ushortSubstitutionArray[i];
                        }
                        ushort Callback(FfiLink_UInt16 before, FfiLink_UInt16 after) => (ushort)from_t.Convert(handler != null
? handler(new Link<TLinkAddress>(from_u16.Convert(before.Index), from_u16.Convert(before.Source), from_u16.Convert(before.Targ
et)), new Link<TLinkAddress>(from_u16.Convert(after.Index), from_u16.Convert(after.Source), from_u16.Convert(after.Target))) :
Constants.Continue);
                        return from_u16.Convert(Methods.UInt16Links_Update(_ptr, restrictionArray, (nuint)restrictionLength, s
ubstitutionArray, (nuint)(substitution?.Count ?? 0), Callback));
                    }
                    case uint:
                    {
                        var restrictionArray = stackalloc uint[restrictionLength];
                        var uintRestrictionArray = (IList<uint>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = uintRestrictionArray[i];
                        }
                        var substitutionArray = stackalloc uint[substitutionLength];
                        var uintSubstitutionArray = (IList<uint>)substitution;
                        for (var i = 0; i < substitutionLength; i++)
                        {
                            substitutionArray[i] = uintSubstitutionArray[i];
                        }
                        uint Callback(FfiLink_UInt32 before, FfiLink_UInt32 after) => (uint)from_t.Convert(handler != null ? h
andler(new Link<TLinkAddress>(from_u32.Convert(before.Index), from_u32.Convert(before.Source), from_u32.Convert(before.Target)
), new Link<TLinkAddress>(from_u32.Convert(after.Index), from_u32.Convert(after.Source), from_u32.Convert(after.Target))) : Co
nstants.Continue);
                        return from_u32.Convert(Methods.UInt32Links_Update(_ptr, restrictionArray, (nuint)restrictionLength, s
ubstitutionArray, (nuint)(substitution?.Count ?? 0), Callback));
                    }
                    case ulong:
                    {
                        var restrictionArray = stackalloc ulong[restrictionLength];
                        var ulongRestrictionArray = (IList<ulong>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = ulongRestrictionArray[i];
                        }
                        var substitutionArray = stackalloc ulong[substitutionLength];
                        var ulongSubstitutionArray = (IList<ulong>)substitution;
                        for (var i = 0; i < substitutionLength; i++)
                        {
                            substitutionArray[i] = ulongSubstitutionArray[i];
                        }
                        ulong Callback(FfiLink_UInt64 before, FfiLink_UInt64 after) => (ulong)from_t.Convert(handler != null ?
handler(new Link<TLinkAddress>(from_u64.Convert(before.Index), from_u64.Convert(before.Source), from_u64.Convert(before.Target
)), new Link<TLinkAddress>(from_u64.Convert(after.Index), from_u64.Convert(after.Source), from_u64.Convert(after.Target))) : C
onstants.Continue);
                        return from_u64.Convert(Methods.UInt64Links_Update(_ptr, restrictionArray, (nuint)restrictionLength, s
ubstitutionArray, (nuint)(substitution?.Count ?? 0), Callback));
                    }
                    default:
                    {
                        throw new NotImplementedException();
                    }
                }
            }
        }

        public TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
        {
            var restrictionLength = restriction?.Count ?? 0;
            unsafe
            {
                TLinkAddress t = default;
                switch (t)
                {
                    case byte:
                    {
                        var restrictionArray = stackalloc byte[restrictionLength];
                        var byteRestrictionArray = (IList<byte>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = byteRestrictionArray[i];
                        }
                        byte Callback(FfiLink_UInt8 before, FfiLink_UInt8 after) => (byte)from_t.Convert(handler != null ? han
dler(new Link<TLinkAddress>(from_u8.Convert(before.Index), from_u8.Convert(before.Source), from_u8.Convert(before.Target)), ne
w Link<TLinkAddress>(from_u8.Convert(after.Index), from_u8.Convert(after.Source), from_u8.Convert(after.Target))) : Constants.
Continue);
                        return (TLinkAddress)(object)Methods.ByteLinks_Delete(_ptr, restrictionArray, (nuint)restrictionLength
, Callback);
                    }
                    case ushort:
                    {
                        var restrictionArray = stackalloc ushort[restrictionLength];
```

```
                        var ushortRestrictionArray = (IList<ushort>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = ushortRestrictionArray[i];
                        }
                        ushort Callback(FfiLink_UInt16 before, FfiLink_UInt16 after) => (ushort)from_t.Convert(handler != null
? handler(new Link<TLinkAddress>(from_u16.Convert(before.Index), from_u16.Convert(before.Source), from_u16.Convert(before.Targ
et)), new Link<TLinkAddress>(from_u16.Convert(after.Index), from_u16.Convert(after.Source), from_u16.Convert(after.Target))) :
Constants.Continue);
                        return (TLinkAddress)(object)Methods.UInt16Links_Delete(_ptr, restrictionArray, (nuint)restrictionLeng
th, Callback);
                    }
                    case uint:
                    {
                        var restrictionArray = stackalloc uint[restrictionLength];
                        var uintRestrictionArray = (IList<uint>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = uintRestrictionArray[i];
                        }
                        uint Callback(FfiLink_UInt32 before, FfiLink_UInt32 after) => (uint)from_t.Convert(handler != null ? h
andler(new Link<TLinkAddress>(from_u32.Convert(before.Index), from_u32.Convert(before.Source), from_u32.Convert(before.Target)
), new Link<TLinkAddress>(from_u32.Convert(after.Index), from_u32.Convert(after.Source), from_u32.Convert(after.Target))) : Co
nstants.Continue);
                        return (TLinkAddress)(object)Methods.UInt32Links_Delete(_ptr, restrictionArray, (nuint)restrictionLeng
th, Callback);
                    }
                    case ulong:
                    {
                        var restrictionArray = stackalloc ulong[restrictionLength];
                        var ulongRestrictionArray = (IList<ulong>)restriction;
                        for (var i = 0; i < restrictionLength; i++)
                        {
                            restrictionArray[i] = ulongRestrictionArray[i];
                        }
                        ulong Callback(FfiLink_UInt64 before, FfiLink_UInt64 after) => (ulong)from_t.Convert(handler != null ?
handler(new Link<TLinkAddress>(from_u64.Convert(before.Index), from_u64.Convert(before.Source), from_u64.Convert(before.Target
)), new Link<TLinkAddress>(from_u64.Convert(after.Index), from_u64.Convert(after.Source), from_u64.Convert(after.Target))) : C
onstants.Continue);
                        return (TLinkAddress)(object)Methods.UInt64Links_Delete(_ptr, restrictionArray, (nuint)restrictionLeng
th, Callback);
                    }
                    default:
                    {
                        throw new NotImplementedException();
                    }
                }
            }
        }

        protected override void Dispose(bool manual, bool wasDisposed)
        {
            unsafe
            {
                if (wasDisposed && _ptr != null)
                {
                    return;
                }
                TLinkAddress t = default;
                switch (t)
                    {
                        case byte:
                            Methods.ByteLinks_Drop(_ptr);
                            break;
                        case ushort:
                            Methods.UInt16Links_Drop(_ptr);
                            break;
                        case uint:
                            Methods.UInt32Links_Drop(_ptr);
                            break;
                        case ulong:
                            Methods.UInt64Links_Drop(_ptr);
                            break;
                        default:
                            throw new NotImplementedException();
                    }
            }
        }
    }
}
*/
```

csharp/Platform.Data.Doublets/Ffi/UInt32Links.cs

```
/*
using System;
using System.Collections.Generic;
using System.Linq;
using System.Runtime.InteropServices;
using Platform.Converters;
using Platform.Delegates;
using Platform.Disposables;

namespace Platform.Data.Doublets.Ffi
{
    using TLinkAddress = System.UInt32;

    public class UInt32Links : DisposableBase, ILinks<TLinkAddress>
    {
        public LinksConstants<TLinkAddress> Constants { get; }

        private readonly unsafe void* _ptr;

        public UInt32Links(string path)
        {
            unsafe
            {
                _ptr = Methods.UInt32Links_New(path);

                // TODO: Update api
                Constants = new LinksConstants<TLinkAddress>(enableExternalReferencesSupport: true);
            }
        }

        public TLinkAddress Count(IList<TLinkAddress>? restriction)
        {
            unsafe
            {
                var array = stackalloc uint[restriction.Count];
                for (var i = 0; i < restriction.Count; i++)
                {
                    array[i] = restriction[i];
                }
                return Methods.UInt32Links_Count(_ptr, array, (nuint)(restriction?.Count ?? 0));
            }
        }

        public TLinkAddress Each(IList<TLinkAddress>? restriction, ReadHandler<TLinkAddress>? handler)
        {
            unsafe
            {
                 Methods.EachCallback_UInt32 callback = (link) => handler != null ? handler(new Link<TLinkAddress>(link.Index,
link.Source, link.Target)) : Constants.Continue;
                var array = stackalloc uint[restriction.Count];
                for (var i = 0; i < restriction.Count; i++)
                {
                    array[i] = restriction[i];
                }
                return Methods.UInt32Links_Each(_ptr, array, (nuint)(restriction?.Count ?? 0), callback);
            }
        }

        public TLinkAddress Create(IList<TLinkAddress>? substitution, WriteHandler<TLinkAddress>? handler)
        {
            unsafe
            {
                Methods.CreateCallback_UInt32 callback = (before, after) => handler != null ? handler(new Link<TLinkAddress>(b
efore.Index, before.Source, before.Target), new Link<TLinkAddress>(after.Index, after.Source, after.Target)) : Constants.Conti
nue;
                fixed (uint* substitutionPtr = (uint[])substitution)
                {
                    return Methods.UInt32Links_Create(_ptr, substitutionPtr, (nuint)(substitution?.Count ?? 0), callback);
                }
            }
        }

        public TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLinkAddr
ess>? handler)
        {
            unsafe
            {
                var restrictionArray = stackalloc uint[restriction.Count];
                for (var i = 0; i < restriction.Count; i++)
                {
                    restrictionArray[i] = restriction[i];
                }
                var substitutionArray = stackalloc uint[substitution.Count];
                for (var i = 0; i < restriction.Count; i++)
                {
                    substitutionArray[i] = restriction[i];
                }
                Methods.UpdateCallback_UInt32 callback = (before, after) => handler != null ? handler(new Link<TLinkAddress>(b
efore.Index, before.Source, before.Target), new Link<TLinkAddress>(after.Index, after.Source, after.Target)) : Constants.Conti
nue;
                return Methods.UInt32Links_Update(_ptr, restrictionArray, (nuint)(restriction?.Count ?? 0), substitutionArray,
```

```
(nuint)(substitution?.Count ?? 0), callback);
            }
        }

        public TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
        {
            unsafe
            {
                var restrictionArray = stackalloc uint[restriction.Count];
                for (var i = 0; i < restriction.Count; i++)
                {
                    restrictionArray[i] = restriction[i];
                }
                Methods.DeleteCallback_UInt32 callback = (before, after) => handler != null ? handler(new Link<TLinkAddress>(b
efore.Index, before.Source, before.Target), new Link<TLinkAddress>(after.Index, after.Source, after.Target)) : Constants.Conti
nue;
                return Methods.UInt32Links_Delete(_ptr, restrictionArray, (nuint)(restriction?.Count ?? 0), callback);
            }
        }

        protected override void Dispose(bool manual, bool wasDisposed)
        {
            unsafe
            {
                if (wasDisposed || _ptr == null)
                {
                    return;
                }
                Methods.UInt32Links_Drop(_ptr);
            }
        }
    }
}
*/
```

csharp/Platform.Data.Doublets/ILinks.cs

```
#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

using System.Collections.Generic;
using System.Numerics;

namespace Platform.Data.Doublets
{
    /// <summary>
    /// <para>
    /// Defines the links.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="Platform.Data.ILinks{TLinkAddress, TConstants}"/>
    public interface ILinks<TLinkAddress> : ILinks<TLinkAddress, LinksConstants<TLinkAddress>> where TLinkAddress : IUnsignedN
umber<TLinkAddress>
    {
    }
}
```

csharp/Platform.Data.Doublets/ILinksExtensions.Validation.cs

```
using System;
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using System.Runtime.CompilerServices;
using System.Text;
using Platform.Ranges;
using Platform.Collections.Lists;
using Platform.Random;
using Platform.Setters;
using Platform.Converters;
using Platform.Numbers;
using Platform.Data.Exceptions;
using Platform.Data.Doublets.Decorators;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets
{
    /// <summary>
    /// <para>
    /// Represents the links extensions.
    /// </para>
    /// <para></para>
    /// </summary>
    public static partial class ILinksExtensions
    {
        #region Ensure
        // TODO: May be move to EnsureExtensions or make it both there and here

        /// <summary>
        /// <para>
        /// Ensures the link exists using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="restriction">
        /// <para>The restriction.</para>
        /// <para></para>
        /// </param>
        /// <exception cref="ArgumentLinkDoesNotExistsException{TLinkAddress}">
        /// <para>sequence[{i}]</para>
        /// <para></para>
        /// </exception>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void EnsureLinkExists<TLinkAddress>(this ILinks<TLinkAddress> links, IList<TLinkAddress>? restriction) w
here TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            restriction ??= Array.Empty<TLinkAddress>();
            for (var i = 0; i < restriction.Count; i++)
            {
                if (!links.Exists(restriction[i]))
                {
                    throw new ArgumentLinkDoesNotExistsException<TLinkAddress>(restriction[i], $"sequence[{i}]");
                }
            }
        }

        /// <summary>
        /// <para>
        /// Ensures the inner reference exists using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="reference">
        /// <para>The reference.</para>
        /// <para></para>
        /// </param>
        /// <param name="argumentName">
        /// <para>The argument name.</para>
        /// <para></para>
        /// </param>
        /// <exception cref="ArgumentLinkDoesNotExistsException{TLinkAddress}">
```

```
        /// <para></para>
        /// <para></para>
        /// </exception>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void EnsureInnerReferenceExists<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress reference, s
tring argumentName) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            if (links.Constants.IsInternalReference(reference) && !links.Exists(reference))
            {
                throw new ArgumentLinkDoesNotExistsException<TLinkAddress>(reference, argumentName);
            }
        }

        /// <summary>
        /// <para>
        /// Ensures the inner reference exists using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="restriction">
        /// <para>The restriction.</para>
        /// <para></para>
        /// </param>
        /// <param name="argumentName">
        /// <para>The argument name.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void EnsureInnerReferenceExists<TLinkAddress>(this ILinks<TLinkAddress> links, IList<TLinkAddress>? rest
riction, string argumentName) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            restriction ??= Array.Empty<TLinkAddress>();
            for (int i = 0; i < restriction.Count; i++)
            {
                links.EnsureInnerReferenceExists(restriction[i], argumentName);
            }
        }

        /// <summary>
        /// <para>
        /// Ensures the link is any or exists using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="restriction">
        /// <para>The restriction.</para>
        /// <para></para>
        /// </param>
        /// <exception cref="ArgumentLinkDoesNotExistsException{TLinkAddress}">
        /// <para>sequence[{i}]</para>
        /// <para></para>
        /// </exception>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void EnsureLinkIsAnyOrExists<TLinkAddress>(this ILinks<TLinkAddress> links, IList<TLinkAddress>? restric
tion) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            restriction ??= Array.Empty<TLinkAddress>();
            var any = links.Constants.Any;
            for (var i = 0; i < restriction.Count; i++)
            {
                if (restriction[i] != any && !links.Exists(restriction[i]))
                {
                    throw new ArgumentLinkDoesNotExistsException<TLinkAddress>(restriction[i], $"sequence[{i}]");
                }
            }
        }

        /// <summary>
        /// <para>
        /// Ensures the link is any or exists using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
```

```
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <param name="argumentName">
        /// <para>The argument name.</para>
        /// <para></para>
        /// </param>
        /// <exception cref="ArgumentLinkDoesNotExistsException{TLinkAddress}">
        /// <para></para>
        /// <para></para>
        /// </exception>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void EnsureLinkIsAnyOrExists<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link, string ar
gumentName) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            if (link != links.Constants.Any && !links.Exists(link))
            {
                throw new ArgumentLinkDoesNotExistsException<TLinkAddress>(link, argumentName);
            }
        }

        /// <summary>
        /// <para>
        /// Ensures the link is itself or exists using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <param name="argumentName">
        /// <para>The argument name.</para>
        /// <para></para>
        /// </param>
        /// <exception cref="ArgumentLinkDoesNotExistsException{TLinkAddress}">
        /// <para></para>
        /// <para></para>
        /// </exception>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void EnsureLinkIsItselfOrExists<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link, string
argumentName) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            if (link != links.Constants.Itself && !links.Exists(link))
            {
                throw new ArgumentLinkDoesNotExistsException<TLinkAddress>(link, argumentName);
            }
        }

        /// <summary>Throws when a link with the specified source and target exists.</summary>
        /// <param name="source">The source.</param>
        /// <param name="target">The target.</param>
        /// <param name="links">Хранилище связей.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void EnsureDoesNotExists<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress source, TLinkAddres
s target) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            if (links.Exists(source, target))
            {
                throw new LinkWithSameValueAlreadyExistsException();
            }
        }

        /// <summary>Throws when another link uses the specified link.</summary>
        /// <param name="link">The link.</param>
        /// <param name="links">Хранилище связей.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void EnsureNoUsages<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link) where TLinkAddress
: IUnsignedNumber<TLinkAddress>
        {
            if (links.HasUsages(link))
            {
                throw new ArgumentLinkHasDependenciesException<TLinkAddress>(link);
            }
        }

        /// <summary>Creates all requested internal link addresses, retaining existing links.</summary>
        /// <param name="addresses">The addresses.</param>
```

```
        /// <param name="links">Хранилище связей.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void EnsureCreated<TLinkAddress>(this ILinks<TLinkAddress> links, params TLinkAddress[] addresses) where
TLinkAddress : IUnsignedNumber<TLinkAddress> { links.EnsureCreated(links.Create, addresses); }

        /// <summary>Creates all requested internal link addresses, retaining existing links.</summary>
        /// <param name="addresses">The addresses.</param>
        /// <param name="links">Хранилище связей.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void EnsurePointsCreated<TLinkAddress>(this ILinks<TLinkAddress> links, params TLinkAddress[] addresses)
where TLinkAddress : IUnsignedNumber<TLinkAddress> { links.EnsureCreated(links.CreatePoint, addresses); }

        /// <summary>Creates all requested internal link addresses, retaining existing links.</summary>
        /// <param name="addresses">The addresses.</param>
        /// <param name="creator">The creator.</param>
        /// <param name="links">Хранилище связей.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void EnsureCreated<TLinkAddress>(this ILinks<TLinkAddress> links, Func<TLinkAddress> creator, params TLi
nkAddress[] addresses) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var remaining = new HashSet<TLinkAddress>(addresses.Where(address =>
                address != links.Constants.Null &&
                Comparer<TLinkAddress>.Default.Compare(address, links.Constants.InternalReferencesRange.Maximum) <= 0 &&
                !links.Exists(address)));
            var requested = new HashSet<TLinkAddress>(remaining);
            var created = new HashSet<TLinkAddress>();
            // Keep intermediate allocations until every requested address exists;
            // deleting one early could make the allocator return it repeatedly.
            while (remaining.Count > 0)
            {
                var address = creator();
                if (address == links.Constants.Null || !created.Add(address))
                {
                    throw new InvalidOperationException("The creator did not allocate a new link.");
                }
                remaining.Remove(address);
            }
            foreach (var address in created.Where(address => !requested.Contains(address)))
            {
                links.Delete(address, null);
            }
        }

        #endregion
    }
}
```

csharp/Platform.Data.Doublets/ILinksExtensions.cs

```
using System;
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using System.Runtime.CompilerServices;
using System.Text;
using Platform.Ranges;
using Platform.Collections.Lists;
using Platform.Random;
using Platform.Setters;
using Platform.Converters;
using Platform.Numbers;
using Platform.Data.Exceptions;
using Platform.Data.Doublets.Decorators;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets
{
    /// <summary>
    /// <para>
    /// Represents the links extensions.
    /// </para>
    /// <para></para>
    /// </summary>
    public static partial class ILinksExtensions
    {
        /// <summary>
        /// <para>
        /// Runs the random creations using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="amountOfCreations">
        /// <para>The amount of creations.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void RunRandomCreations<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress amountOfCreations) w
here TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
        {
            var random = RandomHelpers.Default;
            for (var i = TLinkAddress.Zero; i < amountOfCreations; i++)
            {
                var linksAddressRange = new Range<ulong>(ulong.CreateTruncating(TLinkAddress.Zero), ulong.CreateTruncating(lin
ks.Count()));
                var source = TLinkAddress.CreateTruncating(random.NextUInt64(linksAddressRange));
                var target = TLinkAddress.CreateTruncating(random.NextUInt64(linksAddressRange));
                links.GetOrCreate(source, target);
            }
        }

        /// <summary>
        /// <para>
        /// Runs the random searches using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="amountOfSearches">
        /// <para>The amount of searches.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void RunRandomSearches<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress amountOfSearches) whe
re TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
        {
            var random = RandomHelpers.Default;
            for (var i = TLinkAddress.Zero; i < amountOfSearches; i++)
            {
                var linksAddressRange = new Range<ulong>(ulong.CreateTruncating(TLinkAddress.Zero), ulong.CreateTruncating(lin
ks.Count()));
                var source = TLinkAddress.CreateTruncating(random.NextUInt64(linksAddressRange));
                var target = TLinkAddress.CreateTruncating(random.NextUInt64(linksAddressRange));
```

```
                links.SearchOrDefault(source, target);
            }
        }

        /// <summary>
        /// <para>
        /// Runs the random deletions using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="amountOfDeletions">
        /// <para>The amount of deletions.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void RunRandomDeletions<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress amountOfDeletions) w
here TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
        {
            var random = RandomHelpers.Default;
            var linksCount = links.Count();
            var min = amountOfDeletions > linksCount ? TLinkAddress.Zero : linksCount - amountOfDeletions;
            for (var i = TLinkAddress.Zero; i < amountOfDeletions; i++)
            {
                linksCount = links.Count();
                if (linksCount <= min)
                {
                    break;
                }
                var linksAddressRange = new Range<ulong>(ulong.CreateTruncating(min), ulong.CreateTruncating(linksCount));
                var link = (random.NextUInt64(linksAddressRange));
                links.Delete<TLinkAddress>(TLinkAddress.CreateTruncating(link));
            }
        }

        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Deletes the links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="linkToDelete">
        /// <para>The link to delete.</para>
        /// <para></para>
        /// </param>
        /// <remarks>
        /// Resets endpoints for an existing link, then forwards deletion. A missing address is
        /// still forwarded; choose EnsureLinkExists, an Exists guard, or an outer policy decorator.
        /// This helper does not resolve incoming usages. The overload without a handler from
        /// Platform.Data does not perform this reset.
        /// </remarks>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress Delete<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress linkToDelete, WriteHandl
er<TLinkAddress>? handler) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            if (links.Exists(linkToDelete))
            {
                links.EnforceResetValues(linkToDelete, handler);
            }
            return links.Delete(new LinkAddress<TLinkAddress>(linkToDelete), handler);
        }

        /// <summary>Deletes every link in the store.</summary>
        /// <remarks>
        /// <para>
        /// <c>Count()</c> is the number of links, not the highest address: deleted links leave
        /// freed addresses behind. The first loop therefore finds the highest existing address,
        /// and the second deletes existing links from that address down, so links usually go
        /// before the links they use. Each deletion resets the endpoints first, which keeps the
        /// source and target indexes consistent.
        /// </para>
        /// <para>
        /// TODO: Возможно есть очень простой способ это сделать.
        /// (Например просто удалить файл, или изменить его размер таким образом,
        /// чтобы удалился весь контент)
        /// Например через _header->AllocatedLinks в ResizableDirectMemoryLinks
```

```
        /// </para>
        /// </remarks>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void DeleteAll<TLinkAddress>(this ILinks<TLinkAddress> links) where TLinkAddress : IUnsignedNumber<TLink
Address>
        {
            // The scan stops at the last existing link, so it ends on the highest address.
            var last = TLinkAddress.Zero;
            var remaining = links.Count();
            while (remaining != TLinkAddress.Zero)
            {
                ++last;
                if (links.Exists(last))
                {
                    --remaining;
                }
            }
            for (var address = last; address != TLinkAddress.Zero; --address)
            {
                if (links.Exists(address))
                {
                    links.Delete(address, null);
                }
            }
        }

        /// <summary>
        /// <para>
        /// Firsts the links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <exception cref="InvalidOperationException">
        /// <para>В процессе поиска по хранилищу не было найдено связей.</para>
        /// <para></para>
        /// </exception>
        /// <exception cref="InvalidOperationException">
        /// <para>В хранилище нет связей.</para>
        /// <para></para>
        /// </exception>
        /// <returns>
        /// <para>The first link.</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress First<TLinkAddress>(this ILinks<TLinkAddress> links) where TLinkAddress : IUnsignedNumber<T
LinkAddress>
        {
            TLinkAddress firstLink = TLinkAddress.Zero;
            if (links.Count() == TLinkAddress.Zero)
            {
                throw new InvalidOperationException("В хранилище нет связей.");
            }
            links.Each(new Link<TLinkAddress>(links.Constants.Any, links.Constants.Any, links.Constants.Any), link =>
            {
                firstLink = (link ?? throw new InvalidOperationException("Read callback did not provide a link."))[links.Const
ants.IndexPart];
                return links.Constants.Break;
            });
            if (firstLink == TLinkAddress.Zero)
            {
                throw new InvalidOperationException("В процессе поиска по хранилищу не было найдено связей.");
            }
            return firstLink;
        }

        /// <summary>
        /// <para>
        /// Singles the or default using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="query">
        /// <para>The query.</para>
        /// <para></para>
        /// </param>
```

```
        /// <returns>
        /// <para>The result.</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static IList<TLinkAddress>? SingleOrDefault<TLinkAddress>(this ILinks<TLinkAddress> links, IList<TLinkAddress>?
query) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            IList<TLinkAddress>? result = null;
            var count = 0;
            var constants = links.Constants;
            var @continue = constants.Continue;
            var @break = constants.Break;
            links.Each(query, linkHandler);
            return result;

            TLinkAddress linkHandler(IList<TLinkAddress>? link)
            {
                if (count == 0)
                {
                    result = link;
                    count++;
                    return @continue;
                }
                else
                {
                    result = null;
                    return @break;
                }
            }
        }

        #region Paths

        /// <summary>Checks whether the specified path exists.</summary>
        /// <remarks>
        /// TODO: Как так? Как то что ниже может быть корректно?
        /// Скорее всего практически не применимо
        /// Предполагалось, что можно было конвертировать формируемый в проходе через SequenceWalker
        /// Stack в конкретный путь из Source, Target до связи, но это не всегда так.
        /// TODO: Возможно нужен метод, который именно выбрасывает исключения (EnsurePathExists)
        /// </remarks>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool CheckPathExistance<TLinkAddress>(this ILinks<TLinkAddress> links, params TLinkAddress[] path) where
TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var current = path[0];
            //EnsureLinkExists(current, "path");
            if (!links.Exists(current))
            {
                return false;
            }
            for (var i = 1; i < path.Length; i++)
            {
                var next = path[i];
                var values = links.GetRequiredLink(current);
                var source = links.GetSource(values);
                var target = links.GetTarget(values);
                if (source == target && source == next)
                {
                    //throw new InvalidOperationException(string.Format("Невозможно выбрать путь, так как и Source и Target со
впадают с элементом пути {0}.", next));
                    return false;
                }
                if (next != source && next != target)
                {
                    //throw new InvalidOperationException(string.Format("Невозможно продолжить путь через элемент пути {0}", n
ext));
                    return false;
                }
                current = next;
            }
            return true;
        }

        /// <summary>Gets the link reached by following the specified keys.</summary>
        /// <remarks>
        /// Может потребовать дополнительного стека для PathElement's при использовании SequenceWalker.
        /// </remarks>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress GetByKeys<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress root, params int[] pa
th) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            links.EnsureLinkExists(root, "root");
            var currentLink = root;
            for (var i = 0; i < path.Length; i++)
            {
                currentLink = links.GetRequiredLink(currentLink)[path[i]];
            }
            return currentLink;
        }
```

```
        /// <summary>
        /// <para>
        /// Gets the square matrix sequence element by index using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="root">
        /// <para>The root.</para>
        /// <para></para>
        /// </param>
        /// <param name="size">
        /// <para>The size.</para>
        /// <para></para>
        /// </param>
        /// <param name="index">
        /// <para>The index.</para>
        /// <para></para>
        /// </param>
        /// <exception cref="ArgumentOutOfRangeException">
        /// <para>Sequences with sizes other than powers of two are not supported.</para>
        /// <para></para>
        /// </exception>
        /// <returns>
        /// <para>The current link.</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress GetSquareMatrixSequenceElementByIndex<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkA
ddress root, TLinkAddress size, TLinkAddress index) where TLinkAddress : IUnsignedNumber<TLinkAddress>, IBitwiseOperators<TLin
kAddress, TLinkAddress, TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
        {
            var constants = links.Constants;
            var source = constants.SourcePart;
            var target = constants.TargetPart;
            if (!Platform.Numbers.Math.IsPowerOfTwo(size))
            {
                throw new ArgumentOutOfRangeException(nameof(size), "Sequences with sizes other than powers of two are not sup
ported.");
            }
            var path = new BitArray(BitConverter.GetBytes(ulong.CreateTruncating(index)));
            var length = Bit.GetLowestPosition(ulong.CreateTruncating(size));
            links.EnsureLinkExists(root, "root");
            var currentLink = root;
            for (var i = length - 1; i >= 0; i--)
            {
                currentLink = links.GetRequiredLink(currentLink)[path[i] ? target : source];
            }
            return currentLink;
        }

        #endregion

        /// <summary>
        /// Возвращает индекс указанной связи.
        /// </summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="link">Связь представленная списком, состоящим из её адреса и содержимого.</param>
        /// <returns>Индекс начальной связи для указанной связи.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress GetIndex<TLinkAddress>(this ILinks<TLinkAddress> links, IList<TLinkAddress>? link) where TL
inkAddress : IUnsignedNumber<TLinkAddress> { return (link ?? throw new ArgumentNullException(nameof(link)))[links.Constants.In
dexPart]; }

        /// <summary>
        /// Возвращает индекс начальной (Source) связи для указанной связи.
        /// </summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="link">Индекс связи.</param>
        /// <returns>Индекс начальной связи для указанной связи.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress GetSource<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link) where TLinkAddr
ess : IUnsignedNumber<TLinkAddress> { return links.GetRequiredLink(link)[links.Constants.SourcePart]; }

        /// <summary>
        /// Возвращает индекс начальной (Source) связи для указанной связи.
        /// </summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="link">Связь представленная списком, состоящим из её адреса и содержимого.</param>
        /// <returns>Индекс начальной связи для указанной связи.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress GetSource<TLinkAddress>(this ILinks<TLinkAddress> links, IList<TLinkAddress>? link) where T
LinkAddress : IUnsignedNumber<TLinkAddress> { return (link ?? throw new ArgumentNullException(nameof(link)))[links.Constants.S
ourcePart]; }
```

```
        /// <summary>
        /// Возвращает индекс конечной (Target) связи для указанной связи.
        /// </summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="link">Индекс связи.</param>
        /// <returns>Индекс конечной связи для указанной связи.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress GetTarget<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link) where TLinkAddr
ess : IUnsignedNumber<TLinkAddress> { return links.GetRequiredLink(link)[links.Constants.TargetPart]; }

        /// <summary>
        /// Возвращает индекс конечной (Target) связи для указанной связи.
        /// </summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="link">Связь представленная списком, состоящим из её адреса и содержимого.</param>
        /// <returns>Индекс конечной связи для указанной связи.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress GetTarget<TLinkAddress>(this ILinks<TLinkAddress> links, IList<TLinkAddress>? link) where T
LinkAddress : IUnsignedNumber<TLinkAddress> { return (link ?? throw new ArgumentNullException(nameof(link)))[links.Constants.T
argetPart]; }

        /// <summary>
        /// <para>
        /// Alls the links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="restriction">
        /// <para>The restriction.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>A list of i list t link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static IList<IList<TLinkAddress>?> All<TLinkAddress>(this ILinks<TLinkAddress> links, params TLinkAddress[] res
triction) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var allLinks = new List<IList<TLinkAddress>?>();
            var filler = new ListFiller<IList<TLinkAddress>?, TLinkAddress>(allLinks, links.Constants.Continue);
            links.Each(filler.AddAndReturnConstant, restriction);
            return allLinks;
        }

        /// <summary>
        /// <para>
        /// Alls the indices using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="restriction">
        /// <para>The restriction.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>A list of t link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static IList<TLinkAddress>? AllIndices<TLinkAddress>(this ILinks<TLinkAddress> links, params TLinkAddress[] res
triction) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var allIndices = new List<TLinkAddress>();
            var filler = new ListFiller<TLinkAddress, TLinkAddress>(allIndices, links.Constants.Continue);
            links.Each(link => filler.AddFirstAndReturnConstant(link ?? throw new InvalidOperationException("Read callback did
not provide a link.")), restriction);
            return allIndices;
        }

        /// <summary>
        /// Возвращает значение, определяющее существует ли связь с указанными началом и концом в хранилище связей.
        /// </summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="source">Начало связи.</param>
        /// <param name="target">Конец связи.</param>
```

```
        /// <returns>Значение, определяющее существует ли связь.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool Exists<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress source, TLinkAddress target) whe
re TLinkAddress : IUnsignedNumber<TLinkAddress> { return Comparer<TLinkAddress>.Default.Compare(links.Count(links.Constants.An
y, source, target), TLinkAddress.Zero) > 0; }



        /// <summary>Counts links that use the specified link.</summary>
        /// <param name="link">The link.</param>
        /// <param name="links">Хранилище связей.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress CountUsages<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link) where TLinkAd
dress : IUnsignedNumber<TLinkAddress>
        {
            var constants = links.Constants;
            var values = links.GetRequiredLink(link);
            TLinkAddress usagesAsSource = links.Count(new Link<TLinkAddress>(constants.Any, link, constants.Any));
            if (links.GetSource(values) == link)
            {
                --usagesAsSource;
            }
            TLinkAddress usagesAsTarget = links.Count(new Link<TLinkAddress>(constants.Any, constants.Any, link));
            if (links.GetTarget(values) == link)
            {
                --usagesAsTarget;
            }
            return usagesAsSource + usagesAsTarget;
        }

        /// <summary>Checks whether another link uses the specified link.</summary>
        /// <param name="link">The link.</param>
        /// <param name="links">Хранилище связей.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool HasUsages<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link) where TLinkAddress : IU
nsignedNumber<TLinkAddress> { return Comparer<TLinkAddress>.Default.Compare(links.CountUsages(link), TLinkAddress.Zero) > 0; }

        /// <summary>Checks whether the link has the specified source and target.</summary>
        /// <param name="link">The link.</param>
        /// <param name="source">The source.</param>
        /// <param name="target">The target.</param>
        /// <param name="links">Хранилище связей.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool Equals<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link, TLinkAddress source, TLink
Address target) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var values = links.GetRequiredLink(link);
            return links.GetSource(values) == source && links.GetTarget(values) == target;
        }

        /// <summary>
        /// Выполняет поиск связи с указанными Source (началом) и Target (концом).
        /// </summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="source">Индекс связи, которая является началом для искомой связи.</param>
        /// <param name="target">Индекс связи, которая является концом для искомой связи.</param>
        /// <returns>Индекс искомой связи с указанными Source (началом) и Target (концом).</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress SearchOrDefault<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress source, TLinkAd
dress target) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var contants = links.Constants;
            var setter = new Setter<TLinkAddress, TLinkAddress>(contants.Continue, contants.Break, TLinkAddress.Zero);
            links.Each(setter.SetFirstAndReturnFalse, contants.Any, source, target);
            return setter.Result;
        }

        public static TLinkAddress CreatePoint<TLinkAddress>(this ILinks<TLinkAddress> links) where TLinkAddress : IUnsignedNu
mber<TLinkAddress>
        {
            var constants = links.Constants;
            var setter = new Setter<TLinkAddress, TLinkAddress>(constants.Continue, constants.Break);
            links.CreatePoint(setter.SetFirstFromNonNullSecondListAndReturnTrue);
            return setter.Result;
        }

        /// <summary>Creates a self-referencing point and invokes the optional callback.</summary>
        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <param name="links">Хранилище связей.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress CreatePoint<TLinkAddress>(this ILinks<TLinkAddress> links, WriteHandler<TLinkAddress>? hand
ler) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var constants = links.Constants;
            WriteHandlerState<TLinkAddress> handlerState = new(constants.Continue, constants.Break, handler);
            TLinkAddress link = TLinkAddress.Zero;
            TLinkAddress HandlerWrapper(IList<TLinkAddress>? before, IList<TLinkAddress>? after)
            {
                link = links.GetIndex(after);
                return handlerState.HandleNullable(before, after); ;
            }
            handlerState.Apply(links.Create(null, HandlerWrapper));
```

```
            handlerState.Apply(links.Update(link, link, link, HandlerWrapper));
            return handlerState.Result;
        }

        public static TLinkAddress CreateAndUpdate<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress source, TLinkAd
dress target) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var constants = links.Constants;
            var setter = new Setter<TLinkAddress, TLinkAddress>(constants.Continue, constants.Break);
            links.CreateAndUpdate(source, target, setter.SetFirstFromNonNullSecondListAndReturnTrue);
            return setter.Result;
        }


        /// <summary>Creates a link with the specified source, target and callback.</summary>
        /// <remarks>
        /// Returns callback control (Continue or Break). Break suppresses later callbacks;
        /// creation and uniqueness cleanup still complete. Without a handler parameter,
        /// the overload returns the final surviving address instead.
        /// </remarks>
        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <param name="source">The source.</param>
        /// <param name="target">The target.</param>
        /// <param name="links">Хранилище связей.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress CreateAndUpdate<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress source, TLinkAd
dress target, WriteHandler<TLinkAddress>? handler) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var constants = links.Constants;
            TLinkAddress createdLink = TLinkAddress.Zero;
            WriteHandlerState<TLinkAddress> handlerState = new(constants.Continue, constants.Break, handler);
            handlerState.Apply(links.Create(null, (before, after) =>
            {
                createdLink = links.GetIndex(after);
                return handlerState.HandleNullable(before, after); ;
            }));
            handlerState.Apply(links.Update(createdLink, source, target,
                (before, after) => handlerState.HandleNullable(before, after)));
            return handlerState.Result;
        }

        /// <summary>
        /// Обновляет связь с указанными началом (Source) и концом (Target)
        /// на связь с указанными началом (NewSource) и концом (NewTarget).
        /// </summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="link">Индекс обновляемой связи.</param>
        /// <param name="newSource">Индекс связи, которая является началом связи, на которую выполняется обновление.</param>
        /// <param name="newTarget">Индекс связи, которая является концом связи, на которую выполняется обновление.</param>
        /// <returns>Индекс обновлённой связи.</returns>
        /// <remarks>
        /// Forwards to the selected stack without checking existence. Raw stores require
        /// a live internal address and valid endpoints. Choose an outer existence
        /// validator or check the address before updating when it may be missing.
        /// </remarks>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress Update<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link, TLinkAddress newSo
urce, TLinkAddress newTarget) where TLinkAddress : IUnsignedNumber<TLinkAddress> { return links.Update(new LinkAddress<TLinkAd
dress>(link), new Link<TLinkAddress>(link, newSource, newTarget)); }

        public static TLinkAddress Update<TLinkAddress>(this ILinks<TLinkAddress> links, params TLinkAddress[] restriction) wh
ere TLinkAddress : IUnsignedNumber<TLinkAddress> { return links.Update((IList<TLinkAddress>)restriction); }

         public static TLinkAddress Update<TLinkAddress>(this ILinks<TLinkAddress> links, WriteHandler<TLinkAddress>? handler,
params TLinkAddress[] restriction) where TLinkAddress : IUnsignedNumber<TLinkAddress> { return links.Update(restriction, handl
er); }

        public static TLinkAddress Update<TLinkAddress>(this ILinks<TLinkAddress> links, IList<TLinkAddress>? restriction) whe
re TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            restriction ??= Array.Empty<TLinkAddress>();
            var constants = links.Constants;
            var setter = new Setter<TLinkAddress, TLinkAddress>(constants.Continue, constants.Break);
            links.Update(restriction, setter.SetFirstFromNonNullSecondListAndReturnTrue);
            return setter.Result;
        }


        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// Обновляет связь с указанными началом (Source) и концом (Target)
        /// на связь с указанными началом (NewSource) и концом (NewTarget).
        /// </summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="restriction">Ограничения на содержимое связей. Каждое ограничение может иметь значения: Constants.Nul
l - 0-я связь, обозначающая ссылку на пустоту, Itself - требование установить ссылку на себя, 1..∞ конкретный адрес другой свя
зи.</param>
        /// <returns>Индекс обновлённой связи.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress Update<TLinkAddress>(this ILinks<TLinkAddress> links, IList<TLinkAddress>? restriction, Wri
teHandler<TLinkAddress>? handler) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
```

```
            restriction ??= Array.Empty<TLinkAddress>();
            return restriction.Count switch
            {
                2 => links.MergeAndDelete(restriction[0], restriction[1], handler),
                4 => links.UpdateOrCreateOrGet(restriction[0], restriction[1], restriction[2], restriction[3], handler),
                _ => links.Update(restriction[0], restriction[1], restriction[2], handler)
            };
        }

        public static TLinkAddress Update<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link, TLinkAddress newSo
urce, TLinkAddress newTarget, WriteHandler<TLinkAddress>? handler) where TLinkAddress : IUnsignedNumber<TLinkAddress> { return
links.Update(new LinkAddress<TLinkAddress>(link), new Link<TLinkAddress>(link, newSource, newTarget), handler); }

        /// <summary>
        /// <para>
        /// Resolves the constant as self reference using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="constant">
        /// <para>The constant.</para>
        /// <para></para>
        /// </param>
        /// <param name="restriction">
        /// <para>The restriction.</para>
        /// <para></para>
        /// </param>
        /// <param name="substitution">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>A list of t link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static IList<TLinkAddress>? ResolveConstantAsSelfReference<TLinkAddress>(this ILinks<TLinkAddress> links, TLink
Address constant, IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution) where TLinkAddress : IUnsignedNumber<TL
inkAddress>
        {
            var restrictionIndex = links.GetIndex(restriction);
            var substitutionIndex = links.GetIndex(substitution);
            if (substitutionIndex == TLinkAddress.Zero)
            {
                substitutionIndex = restrictionIndex;
            }
            var source = links.GetSource(substitution);
            var target = links.GetTarget(substitution);
            source = source == constant ? substitutionIndex : source;
            target = target == constant ? substitutionIndex : target;
            return new Link<TLinkAddress>(substitutionIndex, source, target);
        }

        /// <summary>
        /// Создаёт связь (если она не существовала), либо возвращает индекс существующей связи с указанными Source (началом)
и Target (концом).
        /// </summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="source">Индекс связи, которая является началом на создаваемой связи.</param>
        /// <param name="target">Индекс связи, которая является концом для создаваемой связи.</param>
        /// <returns>Индекс связи, с указанным Source (началом) и Target (концом)</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress GetOrCreate<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress source, TLinkAddres
s target) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var link = links.SearchOrDefault(source, target);
            if (EqualityComparer<TLinkAddress>.Default.Equals(link, TLinkAddress.Zero))
            {
                link = links.CreateAndUpdate(source, target);
            }
            return link;
        }

        public static TLinkAddress UpdateOrCreateOrGet<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress source, TLi
nkAddress target, TLinkAddress newSource, TLinkAddress newTarget) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var constants = links.Constants;
            var setter = new Setter<TLinkAddress, TLinkAddress>(constants.Continue, constants.Break);
            links.UpdateOrCreateOrGet(source, target, newSource, newTarget, setter.SetFirstFromNonNullSecondListAndReturnTrue)
;
            return setter.Result;
        }

        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
```

```
        /// <summary>
        /// Обновляет связь с указанными началом (Source) и концом (Target)
        /// на связь с указанными началом (NewSource) и концом (NewTarget).
        /// </summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="source">Индекс связи, которая является началом обновляемой связи.</param>
        /// <param name="target">Индекс связи, которая является концом обновляемой связи.</param>
        /// <param name="newSource">Индекс связи, которая является началом связи, на которую выполняется обновление.</param>
        /// <param name="newTarget">Индекс связи, которая является концом связи, на которую выполняется обновление.</param>
        /// <returns>Callback control (Continue or Break), including when the pair is unchanged.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress UpdateOrCreateOrGet<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress source, TLi
nkAddress target, TLinkAddress newSource, TLinkAddress newTarget, WriteHandler<TLinkAddress>? handler) where TLinkAddress : IU
nsignedNumber<TLinkAddress>
        {
            var link = links.SearchOrDefault(source, target);
            if (link == TLinkAddress.Zero)
            {
                return links.CreateAndUpdate(newSource, newTarget, handler);
            }
            if (newSource == source && newTarget == target)
            {
                var existing = links.GetLink(link);
                return handler is null ? links.Constants.Continue : handler(existing, existing);
            }
            return links.Update(link, newSource, newTarget, handler);
        }

        /// <summary>Удаляет связь с указанными началом (Source) и концом (Target).</summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="source">Индекс связи, которая является началом удаляемой связи.</param>
        /// <param name="target">Индекс связи, которая является концом удаляемой связи.</param>
        /// <remarks>
        /// Skips deletion when the pair is absent. A successful lookup does not supply endpoint
        /// reset or usage resolution; compose those policies or satisfy their preconditions.
        /// </remarks>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress DeleteIfExists<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress source, TLinkAdd
ress target) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var link = links.SearchOrDefault(source, target);
            if (!EqualityComparer<TLinkAddress>.Default.Equals(link, TLinkAddress.Zero))
            {
                links.Delete(link);
                return link;
            }
            return TLinkAddress.Zero;
        }

        /// <summary>Удаляет несколько связей.</summary>
        /// <param name="links">Хранилище связей.</param>
        /// <param name="deletedLinks">Список адресов связей к удалению.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
         public static void DeleteMany<TLinkAddress>(this ILinks<TLinkAddress> links, IList<TLinkAddress>? deletedLinks) where
TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            deletedLinks ??= Array.Empty<TLinkAddress>();
            for (int i = 0; i < deletedLinks.Count; i++)
            {
                links.Delete(deletedLinks[i]);
            }
        }

        public static void DeleteAllUsages<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress linkIndex) where TLinkA
ddress : IUnsignedNumber<TLinkAddress> { links.DeleteAllUsages(linkIndex, null); }

        /// <summary>Deletes links that use the specified link and invokes the optional callback.</summary>
         /// <remarks>Before execution of this method ensure that deleted link is detached (all values - source and target are
reset to null) or it might enter into infinite recursion.</remarks>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress DeleteAllUsages<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress linkIndex, Writ
eHandler<TLinkAddress>? handler) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var constants = links.Constants;
            var any = constants.Any;
            var usagesAsSourceQuery = new Link<TLinkAddress>(any, linkIndex, any);
            var usagesAsTargetQuery = new Link<TLinkAddress>(any, any, linkIndex);
            var usages = new List<IList<TLinkAddress>?>();
            var usagesFiller = new ListFiller<IList<TLinkAddress>?, TLinkAddress>(usages, constants.Continue);
            links.Each(usagesFiller.AddAndReturnConstant, usagesAsSourceQuery);
            links.Each(usagesFiller.AddAndReturnConstant, usagesAsTargetQuery);
            WriteHandlerState<TLinkAddress> handlerState = new(constants.Continue, constants.Break, handler);
            foreach (var usage in usages)
            {
                if (links.GetIndex(usage) == linkIndex || !links.Exists(links.GetIndex(usage)))
                {
                    continue;
                }
                handlerState.Apply(links.Delete(links.GetIndex(usage), handlerState.Handler));
            }
            return handlerState.Result;
        }
```

```
        /// <summary>
        /// <para>
        /// Deletes the by query using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="query">
        /// <para>The query.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void DeleteByQuery<TLinkAddress>(this ILinks<TLinkAddress> links, Link<TLinkAddress> query) where TLinkA
ddress : IUnsignedNumber<TLinkAddress>
        {
            var queryResult = new List<TLinkAddress>();
            var queryResultFiller = new ListFiller<TLinkAddress, TLinkAddress>(queryResult, links.Constants.Continue);
            links.Each(link => queryResultFiller.AddFirstAndReturnConstant(link ?? throw new InvalidOperationException("Read c
allback did not provide a link.")), query);
            foreach (var link in queryResult)
            {
                links.Delete(link);
            }
        }

        // TODO: Move to Platform.Data
        /// <summary>
        /// <para>
        /// Determines whether are values reset.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="linkIndex">
        /// <para>The link index.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        /// <remarks>Requires an existing link; this is not an existence validator.</remarks>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool AreValuesReset<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress linkIndex) where TLinkAd
dress : IUnsignedNumber<TLinkAddress>
        {
            var nullConstant = links.Constants.Null;
            var link = links.GetRequiredLink(linkIndex);
            for (int i = 1; i < link.Count; i++)
            {
                if (link[i] != nullConstant)
                {
                    return false;
                }
            }
            return true;
        }

        public static void ResetValues<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress linkIndex) where TLinkAddre
ss : IUnsignedNumber<TLinkAddress> { links.ResetValues(linkIndex, null); }

        // TODO: Create a universal version of this method in Platform.Data (with using of for loop)
        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Resets the values using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
```

```
        /// <param name="linkIndex">
        /// <para>The link index.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress ResetValues<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress linkIndex, WriteHan
dler<TLinkAddress>? handler) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var nullConstant = links.Constants.Null;
            var updateRequest = new Link<TLinkAddress>(linkIndex, nullConstant, nullConstant);
            return links.Update(updateRequest, handler);
        }

        public static void EnforceResetValues<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress linkIndex) where TLi
nkAddress : IUnsignedNumber<TLinkAddress> { links.EnforceResetValues(linkIndex, null); }


        // TODO: Create a universal version of this method in Platform.Data (with using of for loop)
        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Enforces the reset values using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="linkIndex">
        /// <para>The link index.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress EnforceResetValues<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress linkIndex, W
riteHandler<TLinkAddress>? handler) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            if (!links.AreValuesReset(linkIndex))
            {
                return links.ResetValues(linkIndex, handler);
            }
            return links.Constants.Continue;
        }

         public static void MergeUsages<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress oldLinkIndex, TLinkAddress
newLinkIndex) where TLinkAddress : IUnsignedNumber<TLinkAddress> { links.MergeUsages(oldLinkIndex, newLinkIndex, null); }

        /// <summary>
        /// Merging two usages graphs, all children of old link moved to be children of new link or deleted.
        /// </summary>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress MergeUsages<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress oldLinkIndex, TLink
Address newLinkIndex, WriteHandler<TLinkAddress>? handler) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            if (oldLinkIndex == newLinkIndex)
            {
                return newLinkIndex;
            }
            var constants = links.Constants;
            var usagesAsSource = links.All(new Link<TLinkAddress>(constants.Any, oldLinkIndex, constants.Any));
            WriteHandlerState<TLinkAddress> handlerState = new(constants.Continue, constants.Break, handler);
            for (var i = 0; i < usagesAsSource.Count; i++)
            {
                var usageAsSource = usagesAsSource[i];
                if (links.GetIndex(usageAsSource) == oldLinkIndex)
                {
                    continue;
                }
                var restriction = new LinkAddress<TLinkAddress>(links.GetIndex(usageAsSource));
                var substitution = new Link<TLinkAddress>(links.GetIndex(usageAsSource), newLinkIndex, links.GetTarget(usageAs
Source));
                handlerState.Apply(links.Update(restriction, substitution, handlerState.Handler));
            }
            var usagesAsTarget = links.All(new Link<TLinkAddress>(constants.Any, constants.Any, oldLinkIndex));
            for (var i = 0; i < usagesAsTarget.Count; i++)
            {
                var usageAsTarget = usagesAsTarget[i];
                if (links.GetIndex(usageAsTarget) == oldLinkIndex)
                {
                    continue;
                }
                var restriction = links.GetRequiredLink(links.GetIndex(usageAsTarget));
                var substitution = new Link<TLinkAddress>(links.GetIndex(usageAsTarget), links.GetSource(usageAsTarget), newLi
nkIndex);
                handlerState.Apply(links.Update(restriction, substitution, handlerState.Handler));
            }
            return handlerState.Result;
        }
```

```
        public static TLinkAddress MergeAndDelete<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress oldLinkIndex, TL
inkAddress newLinkIndex) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            if (oldLinkIndex != newLinkIndex)
            {
                links.MergeUsages(oldLinkIndex, newLinkIndex);
                links.Delete(oldLinkIndex);
            }
            return newLinkIndex;
        }

        /// <summary>
        /// Replace one link with another (replaced link is deleted, children are updated or deleted).
        /// </summary>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static TLinkAddress MergeAndDelete<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress oldLinkIndex, TL
inkAddress newLinkIndex, WriteHandler<TLinkAddress>? handler) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var constants = links.Constants;
            WriteHandlerState<TLinkAddress> handlerState = new(constants.Continue, constants.Break, handler);
            if (oldLinkIndex != newLinkIndex)
            {
                handlerState.Apply(links.MergeUsages(oldLinkIndex, newLinkIndex, handlerState.Handler));
                handlerState.Apply(links.Delete(oldLinkIndex, handlerState.Handler));
            }
            return handlerState.Result;
        }

        /// <summary>
        /// <para>
        /// Decorates the with automatic uniqueness and usages resolution using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The links.</para>
        /// <para></para>
        /// </returns>
        /// <remarks>
        /// Resolves uniqueness, contents and usages without selecting existence validation.
        /// Wrap the returned stack in LinksInnerReferenceExistenceValidator when missing
        /// internal references must be rejected before resolver searches or mutations.
        /// </remarks>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static ILinks<TLinkAddress> DecorateWithAutomaticUniquenessAndUsagesResolution<TLinkAddress>(this ILinks<TLinkA
ddress> links) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            links = new LinksCascadeUsagesResolver<TLinkAddress>(links);
            links = new NonNullContentsLinkDeletionResolver<TLinkAddress>(links);
            links = new LinksCascadeUniquenessAndUsagesResolver<TLinkAddress>(links);
            return links;
        }

        /// <summary>
        /// <para>
        /// Formats the links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The string</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string Format<TLinkAddress>(this ILinks<TLinkAddress> links, IList<TLinkAddress>? link) where TLinkAddre
ss : IUnsignedNumber<TLinkAddress>
        {
            return $"({links.GetIndex(link)}: {links.GetSource(link)} {links.GetTarget(link)})";
        }

        /// <summary>
```

```
        /// <para>
        /// Formats the links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The string</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string Format<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link) where TLinkAddress : IUn
signedNumber<TLinkAddress> { return links.Format(links.GetRequiredLink(link)); }

        /// <summary>
        /// <para>
        /// Determines whether any link is any.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="sequence">
        /// <para>The sequence.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool AnyLinkIsAny<TLinkAddress>(this ILinks<TLinkAddress> links, params TLinkAddress[] sequence) where T
LinkAddress : IUnsignedNumber<TLinkAddress>
        {
            if (sequence == null)
            {
                return false;
            }
            var constants = links.Constants;
            for (var i = 0; i < sequence.Length; i++)
            {
                if (sequence[i] == constants.Any)
                {
                    return true;
                }
            }
            return false;
        }

        /// <summary>
        /// <para>
        /// Formats the structure using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="linkIndex">
        /// <para>The link index.</para>
        /// <para></para>
        /// </param>
        /// <param name="isElement">
        /// <para>The is element.</para>
        /// <para></para>
        /// </param>
        /// <param name="renderIndex">
        /// <para>The render index.</para>
        /// <para></para>
        /// </param>
        /// <param name="renderDebug">
        /// <para>The render debug.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The string</para>
        /// <para></para>
        /// </returns>
```

```
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string FormatStructure<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress linkIndex, Func<Link<
TLinkAddress>, bool> isElement, bool renderIndex = false, bool renderDebug = false) where TLinkAddress : IUnsignedNumber<TLink
Address>
        {
            var sb = new StringBuilder();
            var visited = new HashSet<TLinkAddress>();
            links.AppendStructure<TLinkAddress>(sb, visited, linkIndex, isElement, (innerSb, link) => innerSb.Append(link.Inde
x), renderIndex, renderDebug);
            return sb.ToString();
        }

        /// <summary>
        /// <para>
        /// Formats the structure using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="linkIndex">
        /// <para>The link index.</para>
        /// <para></para>
        /// </param>
        /// <param name="isElement">
        /// <para>The is element.</para>
        /// <para></para>
        /// </param>
        /// <param name="appendElement">
        /// <para>The append element.</para>
        /// <para></para>
        /// </param>
        /// <param name="renderIndex">
        /// <para>The render index.</para>
        /// <para></para>
        /// </param>
        /// <param name="renderDebug">
        /// <para>The render debug.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The string</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string FormatStructure<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress linkIndex, Func<Link<
TLinkAddress>, bool> isElement, Action<StringBuilder, Link<TLinkAddress>> appendElement, bool renderIndex = false, bool render
Debug = false) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var sb = new StringBuilder();
            var visited = new HashSet<TLinkAddress>();
            links.AppendStructure<TLinkAddress>(sb, visited, linkIndex, isElement, appendElement, renderIndex, renderDebug);
            return sb.ToString();
        }

        /// <summary>
        /// <para>
        /// Appends the structure using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="sb">
        /// <para>The sb.</para>
        /// <para></para>
        /// </param>
        /// <param name="visited">
        /// <para>The visited.</para>
        /// <para></para>
        /// </param>
        /// <param name="linkIndex">
        /// <para>The link index.</para>
        /// <para></para>
        /// </param>
        /// <param name="isElement">
        /// <para>The is element.</para>
        /// <para></para>
        /// </param>
        /// <param name="appendElement">
        /// <para>The append element.</para>
        /// <para></para>
        /// </param>
        /// <param name="renderIndex">
        /// <para>The render index.</para>
        /// <para></para>
        /// </param>
        /// <param name="renderDebug">
```

```
        /// <para>The render debug.</para>
        /// <para></para>
        /// </param>
        /// <exception cref="ArgumentNullException">
        /// <para></para>
        /// <para></para>
        /// </exception>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void AppendStructure<TLinkAddress>(this ILinks<TLinkAddress> links, StringBuilder sb, HashSet<TLinkAddre
ss> visited, TLinkAddress linkIndex, Func<Link<TLinkAddress>, bool> isElement, Action<StringBuilder, Link<TLinkAddress>> appen
dElement, bool renderIndex = false, bool renderDebug = false) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            if (sb == null)
            {
                throw new ArgumentNullException(nameof(sb));
            }
            if (linkIndex == links.Constants.Null || linkIndex == links.Constants.Any || linkIndex == links.Constants.Itself)
            {
                return;
            }
            if (links.Exists(linkIndex))
            {
                if (visited.Add(linkIndex))
                {
                    sb.Append('(');
                    var link = new Link<TLinkAddress>(links.GetRequiredLink(linkIndex));
                    if (renderIndex)
                    {
                        sb.Append(link.Index);
                        sb.Append(':');
                    }
                    if (link.Source == link.Index)
                    {
                        sb.Append(link.Index);
                    }
                    else
                    {
                        var source = new Link<TLinkAddress>(links.GetRequiredLink(link.Source));
                        if (isElement(source))
                        {
                            appendElement(sb, source);
                        }
                        else
                        {
                            links.AppendStructure<TLinkAddress>(sb, visited, source.Index, isElement, appendElement, renderInd
ex);
                        }
                    }
                    sb.Append(' ');
                    if (link.Target == link.Index)
                    {
                        sb.Append(link.Index);
                    }
                    else
                    {
                        var target = new Link<TLinkAddress>(links.GetRequiredLink(link.Target));
                        if (isElement(target))
                        {
                            appendElement(sb, target);
                        }
                        else
                        {
                            links.AppendStructure<TLinkAddress>(sb, visited, target.Index, isElement, appendElement, renderInd
ex);
                        }
                    }
                    sb.Append(')');
                }
                else
                {
                    if (renderDebug)
                    {
                        sb.Append('*');
                    }
                    sb.Append(linkIndex);
                }
            }
            else
            {
                if (renderDebug)
                {
                    sb.Append('~');
                }
                sb.Append(linkIndex);
            }
        }

        #region Garbage Collection
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool IsGarbage<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link) where TLinkAddress : st
ruct, IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
        {
```

```
            return !links.IsPartialPoint(link) && links.Count(links.Constants.Any, link, links.Constants.Any) == TLinkAddress.
Zero && links.Count(links.Constants.Any, links.Constants.Any, link) == TLinkAddress.Zero;
        }

        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void ClearGarbage<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link) where TLinkAddress :
struct, IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
        {
            ClearGarbage(links, link, links.IsGarbage);
        }

        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void ClearGarbage<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress link, Func<TLinkAddress, b
ool> isGarbage) where TLinkAddress : struct, IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, b
ool>
        {
            if (isGarbage(link))
            {
                var contents = new Link<TLinkAddress>(links.GetRequiredLink(link));
                links.ResetValues(link);
                links.Delete(link);
                links.ClearGarbage(contents.Source, isGarbage);
                links.ClearGarbage(contents.Target, isGarbage);
            }
        }

        #endregion
    }
}
```

csharp/Platform.Data.Doublets/ISynchronizedLinks.cs

```
using System.Numerics;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets
{
    /// <summary>
    /// <para>
    /// Defines the synchronized links.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="Platform.Data.ISynchronizedLinks{TLinkAddress, TLinks, TConstants}"/>
    /// <seealso cref="ILinks{TLinkAddress}"/>
    public interface ISynchronizedLinks<TLinkAddress> : ISynchronizedLinks<TLinkAddress, ILinks<TLinkAddress>, LinksConstants<
TLinkAddress>>, ILinks<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>
    {
    }
}
```

csharp/Platform.Data.Doublets/Link.cs

```
using Platform.Collections.Lists;
using Platform.Exceptions;
using Platform.Ranges;
using Platform.Singletons;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets
{
    /// <summary>
    /// Структура описывающая уникальную связь.
    /// </summary>
     public struct Link<TLinkAddress> : IEquatable<Link<TLinkAddress>>, IReadOnlyList<TLinkAddress>, IList<TLinkAddress> where
TLinkAddress : IUnsignedNumber<TLinkAddress>
    {
        /// <summary>
        /// <para>
        /// The link.
        /// </para>
        /// <para></para>
        /// </summary>
        public static readonly Link<TLinkAddress> Null = new Link<TLinkAddress>();
        private static readonly LinksConstants<TLinkAddress> _constants = Default<LinksConstants<TLinkAddress>>.Instance;
        private const int Length = 3;

        /// <summary>
        /// <para>
        /// The index.
        /// </para>
        /// <para></para>
        /// </summary>
        public readonly TLinkAddress Index;
        /// <summary>
        /// <para>
        /// The source.
        /// </para>
        /// <para></para>
        /// </summary>
        public readonly TLinkAddress Source;
        /// <summary>
        /// <para>
        /// The target.
        /// </para>
        /// <para></para>
        /// </summary>
        public readonly TLinkAddress Target;

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="Link{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="values">
        /// <para>A values.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(params TLinkAddress[] values) { SetValues(values, out Index, out Source, out Target); }

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="Link{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="values">
        /// <para>A values.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(IList<TLinkAddress>? values) { SetValues(values, out Index, out Source, out Target); }

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="Link{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="other">
        /// <para>A other.</para>
        /// <para></para>
        /// </param>
        /// <exception cref="NotSupportedException">
        /// <para></para>
        /// <para></para>
```

```
        /// </exception>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(object other)
        {
            if (other is Link<TLinkAddress> otherLink)
            {
                SetValues(ref otherLink, out Index, out Source, out Target);
            }
            else if (other is IList<TLinkAddress> otherList)
            {
                SetValues(otherList, out Index, out Source, out Target);
            }
            else
            {
                throw new NotSupportedException();
            }
        }

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="Link{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="other">
        /// <para>A other.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(ref Link<TLinkAddress> other) { SetValues(ref other, out Index, out Source, out Target); }

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="Link{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="index">
        /// <para>A index.</para>
        /// <para></para>
        /// </param>
        /// <param name="source">
        /// <para>A source.</para>
        /// <para></para>
        /// </param>
        /// <param name="target">
        /// <para>A target.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(TLinkAddress index, TLinkAddress source, TLinkAddress target)
        {
            Index = index;
            Source = source;
            Target = target;
        }
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        private static void SetValues(ref Link<TLinkAddress> other, out TLinkAddress index, out TLinkAddress source, out TLink
Address target)
        {
            index = other.Index;
            source = other.Source;
            target = other.Target;
        }
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        private static void SetValues(IList<TLinkAddress>? values, out TLinkAddress index, out TLinkAddress source, out TLinkA
ddress target)
        {
            if (values == null)
            {
                index = TLinkAddress.Zero;
                source = TLinkAddress.Zero;
                target = TLinkAddress.Zero;
                return;
            }
            switch (values.Count)
            {
                case 3:
                    index = values[0];
                    source = values[1];
                    target = values[2];
                    break;
                case 2:
                    index = values[0];
                    source = values[1];
                    target = TLinkAddress.Zero;
                    break;
                case 1:
                    index = values[0];
                    source = TLinkAddress.Zero;
                    target = TLinkAddress.Zero;
                    break;
```

```
            default:
                index = TLinkAddress.Zero;
                source = TLinkAddress.Zero;
                target = TLinkAddress.Zero;
                break;
        }
    }

    /// <summary>
    /// <para>
    /// Gets the hash code.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <returns>
    /// <para>The int</para>
    /// <para></para>
    /// </returns>
    [MethodImpl(MethodImplOptions.AggressiveInlining)]
    public override int GetHashCode() { return (Index, Source, Target).GetHashCode(); }

    /// <summary>
    /// <para>
    /// Determines whether this instance is null.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <returns>
    /// <para>The bool</para>
    /// <para></para>
    /// </returns>
    [MethodImpl(MethodImplOptions.AggressiveInlining)]
    public bool IsNull() => Index == _constants.Null
                        && Source == _constants.Null
                        && Target == _constants.Null;

    /// <summary>
    /// <para>
    /// Determines whether this instance equals.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <param name="other">
    /// <para>The other.</para>
    /// <para></para>
    /// </param>
    /// <returns>
    /// <para>The bool</para>
    /// <para></para>
    /// </returns>
    [MethodImpl(MethodImplOptions.AggressiveInlining)]
     public override bool Equals(object? other) { return other is Link<TLinkAddress> && Equals((Link<TLinkAddress>)other);
}

    /// <summary>
    /// <para>
    /// Determines whether this instance equals.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <param name="other">
    /// <para>The other.</para>
    /// <para></para>
    /// </param>
    /// <returns>
    /// <para>The bool</para>
    /// <para></para>
    /// </returns>
    [MethodImpl(MethodImplOptions.AggressiveInlining)]
    public bool Equals(Link<TLinkAddress> other) => Index == other.Index
                                                 && Source == other.Source
                                                 && Target == other.Target;

    /// <summary>
    /// <para>
    /// Returns the string using the specified index.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <param name="index">
    /// <para>The index.</para>
    /// <para></para>
    /// </param>
    /// <param name="source">
    /// <para>The source.</para>
    /// <para></para>
    /// </param>
    /// <param name="target">
    /// <para>The target.</para>
    /// <para></para>
    /// </param>
    /// <returns>
```

```
        /// <para>The string</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string ToString(TLinkAddress index, TLinkAddress source, TLinkAddress target) { return $"({index}: {sour
ce}->{target})"; }

        /// <summary>
        /// <para>
        /// Returns the string using the specified source.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="source">
        /// <para>The source.</para>
        /// <para></para>
        /// </param>
        /// <param name="target">
        /// <para>The target.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The string</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string ToString(TLinkAddress source, TLinkAddress target) { return $"({source}->{target})"; }

        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator TLinkAddress[](Link<TLinkAddress> link) { return link.ToArray(); }

        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>(TLinkAddress[] linkArray) { return new Link<TLinkAddress>(linkArray
); }

        /// <summary>
        /// <para>
        /// Returns the string.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
        /// <para>The string</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override string ToString() { return Index == _constants.Null ? ToString(Source, Target) : ToString(Index, Sourc
e, Target); }

        #region IList

        /// <summary>
        /// <para>
        /// Gets the count value.
        /// </para>
        /// <para></para>
        /// </summary>
        public int Count
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get => Length;
        }

        /// <summary>
        /// <para>
        /// Gets the is read only value.
        /// </para>
        /// <para></para>
        /// </summary>
        public bool IsReadOnly
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get => true;
        }

        /// <summary>
        /// <para>
        /// The not supported exception.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress this[int index]
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get
            {
                Ensure.OnDebug.ArgumentInRange(index, new Range<int>(0, Length - 1), nameof(index));
                if (index == _constants.IndexPart)
                {
                    return Index;
                }
                if (index == _constants.SourcePart)
```

```
        {
            return Source;
        }
        if (index == _constants.TargetPart)
        {
            return Target;
        }
        throw new NotSupportedException(); // Impossible path due to Ensure.ArgumentInRange
    }
    [MethodImpl(MethodImplOptions.AggressiveInlining)]
    set => throw new NotSupportedException();
}

/// <summary>
/// <para>
/// Gets the enumerator.
/// </para>
/// <para></para>
/// </summary>
/// <returns>
/// <para>The enumerator</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();

/// <summary>
/// <para>
/// Gets the enumerator.
/// </para>
/// <para></para>
/// </summary>
/// <returns>
/// <para>An enumerator of t link</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public IEnumerator<TLinkAddress> GetEnumerator()
{
    yield return Index;
    yield return Source;
    yield return Target;
}

/// <summary>
/// <para>
/// Adds the item.
/// </para>
/// <para></para>
/// </summary>
/// <param name="item">
/// <para>The item.</para>
/// <para></para>
/// </param>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public void Add(TLinkAddress item) { throw new NotSupportedException(); }

/// <summary>
/// <para>
/// Clears this instance.
/// </para>
/// <para></para>
/// </summary>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public void Clear() { throw new NotSupportedException(); }

/// <summary>
/// <para>
/// Determines whether this instance contains.
/// </para>
/// <para></para>
/// </summary>
/// <param name="item">
/// <para>The item.</para>
/// <para></para>
/// </param>
/// <returns>
/// <para>The bool</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public bool Contains(TLinkAddress item) { return IndexOf(item) >= 0; }

/// <summary>
/// <para>
/// Copies the to using the specified array.
/// </para>
/// <para></para>
/// </summary>
/// <param name="array">
/// <para>The array.</para>
/// <para></para>
```

```
/// </param>
/// <param name="arrayIndex">
/// <para>The array index.</para>
/// <para></para>
/// </param>
/// <exception cref="InvalidOperationException">
/// <para></para>
/// <para></para>
/// </exception>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public void CopyTo(TLinkAddress[] array, int arrayIndex)
{
    Ensure.OnDebug.ArgumentNotNull(array, nameof(array));
    Ensure.OnDebug.ArgumentInRange(arrayIndex, new Range<int>(0, array.Length - 1), nameof(arrayIndex));
    if (arrayIndex + Length > array.Length)
    {
        throw new InvalidOperationException();
    }
    array[arrayIndex++] = Index;
    array[arrayIndex++] = Source;
    array[arrayIndex] = Target;
}

/// <summary>
/// <para>
/// Determines whether this instance remove.
/// </para>
/// <para></para>
/// </summary>
/// <param name="item">
/// <para>The item.</para>
/// <para></para>
/// </param>
/// <returns>
/// <para>The bool</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public bool Remove(TLinkAddress item) { return Throw.A.NotSupportedExceptionAndReturn<bool>(); }

/// <summary>
/// <para>
/// Indexes the of using the specified item.
/// </para>
/// <para></para>
/// </summary>
/// <param name="item">
/// <para>The item.</para>
/// <para></para>
/// </param>
/// <returns>
/// <para>The int</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public int IndexOf(TLinkAddress item)
{
    if (Index == item)
    {
        return _constants.IndexPart;
    }
    if (Source == item)
    {
        return _constants.SourcePart;
    }
    if (Target == item)
    {
        return _constants.TargetPart;
    }
    return -1;
}

/// <summary>
/// <para>
/// Inserts the index.
/// </para>
/// <para></para>
/// </summary>
/// <param name="index">
/// <para>The index.</para>
/// <para></para>
/// </param>
/// <param name="item">
/// <para>The item.</para>
/// <para></para>
/// </param>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public void Insert(int index, TLinkAddress item) { throw new NotSupportedException(); }

/// <summary>
/// <para>
/// Removes the at using the specified index.
```

```
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="index">
        /// <para>The index.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public void RemoveAt(int index) { throw new NotSupportedException(); }

        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator ==(Link<TLinkAddress> left, Link<TLinkAddress> right) { return left.Equals(right); }

        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator !=(Link<TLinkAddress> left, Link<TLinkAddress> right) { return !(left == right); }

        #endregion
    }
}
```

csharp/Platform.Data.Doublets/LinkExtensions.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets
{
    /// <summary>
    /// <para>
    /// Represents the link extensions.
    /// </para>
    /// <para></para>
    /// </summary>
    public static class LinkExtensions
    {
        /// <summary>
        /// <para>
        /// Determines whether is full point.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool IsFullPoint<TLinkAddress>(this Link<TLinkAddress> link) where TLinkAddress : IUnsignedNumber<TLinkA
ddress>
        {
            return Point<TLinkAddress>.IsFullPoint(link);
        }

        /// <summary>
        /// <para>
        /// Determines whether is partial point.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool IsPartialPoint<TLinkAddress>(this Link<TLinkAddress> link) where TLinkAddress : IUnsignedNumber<TLi
nkAddress>
        {
            return Point<TLinkAddress>.IsPartialPoint(link);
        }
    }
}
```

csharp/Platform.Data.Doublets/LinksOperatorBase.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets
{
    /// <summary>
    /// <para>
    /// Represents the links operator base.
    /// </para>
    /// <para></para>
    /// </summary>
    public abstract class LinksOperatorBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>
    {
        /// <summary>
        /// <para>
        /// The links.
        /// </para>
        /// <para></para>
        /// </summary>
        protected readonly ILinks<TLinkAddress> _links;

        /// <summary>
        /// <para>
        /// Gets the links value.
        /// </para>
        /// <para></para>
        /// </summary>
        public ILinks<TLinkAddress> Links
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get => _links;
        }

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="LinksOperatorBase{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected LinksOperatorBase(ILinks<TLinkAddress> links) => _links = links;
    }
}
```

csharp/Platform.Data.Doublets/Memory/ILinksListMethods.cs

```
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory
{
    /// <summary>
    /// <para>
    /// Defines the links list methods.
    /// </para>
    /// <para></para>
    /// </summary>
    public interface ILinksListMethods<TLinkAddress>
    {
        /// <summary>
        /// <para>
        /// Detaches the free link.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="freeLink">
        /// <para>The free link.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        void Detach(TLinkAddress freeLink);

        /// <summary>
        /// <para>
        /// Attaches the as first using the specified link.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        void AttachAsFirst(TLinkAddress link);
    }
}
```

csharp/Platform.Data.Doublets/Memory/ILinksTreeMethods.cs

```
using System;
using System.Collections.Generic;
using System.Runtime.CompilerServices;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory
{
    /// <summary>
    /// <para>
    /// Defines the links tree methods.
    /// </para>
    /// <para></para>
    /// </summary>
    public interface ILinksTreeMethods<TLinkAddress>
    {
        /// <summary>
        /// <para>
        /// Counts the usages using the specified root.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="root">
        /// <para>The root.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        TLinkAddress CountUsages(TLinkAddress root);

        /// <summary>
        /// <para>
        /// Searches the source.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="source">
        /// <para>The source.</para>
        /// <para></para>
        /// </param>
        /// <param name="target">
        /// <para>The target.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        TLinkAddress Search(TLinkAddress source, TLinkAddress target);

        /// <summary>
        /// <para>
        /// Eaches the usage using the specified root.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="root">
        /// <para>The root.</para>
        /// <para></para>
        /// </param>
        /// <param name="handler">
        /// <para>The handler.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        TLinkAddress EachUsage(TLinkAddress root, ReadHandler<TLinkAddress>? handler);

        /// <summary>
        /// <para>
        /// Detaches the root.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="root">
        /// <para>The root.</para>
        /// <para></para>
        /// </param>
        /// <param name="linkIndex">
        /// <para>The link index.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
```

```
        void Detach(ref TLinkAddress root, TLinkAddress linkIndex);

        /// <summary>
        /// <para>
        /// Attaches the root.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="root">
        /// <para>The root.</para>
        /// <para></para>
        /// </param>
        /// <param name="linkIndex">
        /// <para>The link index.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        void Attach(ref TLinkAddress root, TLinkAddress linkIndex);
    }
}
```

csharp/Platform.Data.Doublets/Memory/IndexTreeType.cs

```
#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory
{
    /// <summary>
    /// <para>
    /// The index tree type enum.
    /// </para>
    /// <para></para>
    /// </summary>
    public enum IndexTreeType
    {
        /// <summary>
        /// <para>
        /// The default index tree type.
        /// </para>
        /// <para></para>
        /// </summary>
        Default = 0,
        /// <summary>
        /// <para>
        /// The size balanced tree index tree type.
        /// </para>
        /// <para></para>
        /// </summary>
        SizeBalancedTree = 1,
        /// <summary>
        /// <para>
        /// The recursionless size balanced tree index tree type.
        /// </para>
        /// <para></para>
        /// </summary>
        RecursionlessSizeBalancedTree = 2,
        /// <summary>
        /// <para>
        /// The sized and threaded avl balanced tree index tree type.
        /// </para>
        /// <para></para>
        /// </summary>
        SizedAndThreadedAVLBalancedTree = 3
    }
}
```

csharp/Platform.Data.Doublets/Memory/LinksHeader.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory
{
    /// <summary>
    /// <para>
    /// The links header.
    /// </para>
    /// <para></para>
    /// </summary>
    public struct LinksHeader<TLinkAddress> : IEquatable<LinksHeader<TLinkAddress>> where TLinkAddress : IUnsignedNumber<TLink
Address>
    {

        /// <summary>
        /// <para>
        /// The size.
        /// </para>
        /// <para></para>
        /// </summary>
        public static readonly long SizeInBytes = Structure<LinksHeader<TLinkAddress>>.Size;

        /// <summary>
        /// <para>
        /// The allocated links.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress AllocatedLinks;
        /// <summary>
        /// <para>
        /// The reserved links.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress ReservedLinks;
        /// <summary>
        /// <para>
        /// The free links.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress FreeLinks;
        /// <summary>
        /// <para>
        /// The first free link.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress FirstFreeLink;
        /// <summary>
        /// <para>
        /// The root as source.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress RootAsSource;
        /// <summary>
        /// <para>
        /// The root as target.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress RootAsTarget;
        /// <summary>
        /// <para>
        /// The last free link.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress LastFreeLink;
        /// <summary>
        /// <para>
        /// The reserved.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress Reserved8;

        /// <summary>
        /// <para>
        /// Determines whether this instance equals.
        /// </para>
        /// <para></para>
        /// </summary>
```

```
        /// <param name="obj">
        /// <para>The obj.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override bool Equals(object? obj) { return obj is LinksHeader<TLinkAddress> linksHeader && Equals(linksHeader);
}

        /// <summary>
        /// <para>
        /// Determines whether this instance equals.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="other">
        /// <para>The other.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public bool Equals(LinksHeader<TLinkAddress> other)
            => AllocatedLinks == other.AllocatedLinks
            && ReservedLinks == other.ReservedLinks
            && FreeLinks == other.FreeLinks
            && FirstFreeLink == other.FirstFreeLink
            && RootAsSource == other.RootAsSource
            && RootAsTarget == other.RootAsTarget
            && LastFreeLink == other.LastFreeLink
            && Reserved8 == other.Reserved8;

        /// <summary>
        /// <para>
        /// Gets the hash code.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
        /// <para>The int</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override int GetHashCode() { return (AllocatedLinks, ReservedLinks, FreeLinks, FirstFreeLink, RootAsSource, Roo
tAsTarget, LastFreeLink, Reserved8).GetHashCode(); }

        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator ==(LinksHeader<TLinkAddress> left, LinksHeader<TLinkAddress> right) { return left.Equals(r
ight); }

        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator !=(LinksHeader<TLinkAddress> left, LinksHeader<TLinkAddress> right) { return !(left == rig
ht); }
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/ExternalLinksRecursionlessSizeBalancedTreeMethodsBase.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using System.Text;
using Platform.Collections.Methods.Trees;
using Platform.Converters;
using Platform.Delegates;
using static System.Runtime.CompilerServices.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the external links recursionless size balanced tree methods base.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="RecursionlessSizeBalancedTreeMethods{TLinkAddress}" />
/// <seealso cref="ILinksTreeMethods{TLinkAddress}" />
public abstract unsafe class ExternalLinksRecursionlessSizeBalancedTreeMethodsBase<TLinkAddress> : RecursionlessSizeBalancedTr
eeMethods<TLinkAddress>, ILinksTreeMethods<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperat
ors<TLinkAddress, TLinkAddress, bool>
{

    /// <summary>
    ///     <para>
    ///         The break.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Break;
    /// <summary>
    ///     <para>
    ///         The continue.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Continue;
    /// <summary>
    ///     <para>
    ///         The header.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* Header;
    /// <summary>
    ///     <para>
    ///         The links data parts.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* LinksDataParts;
    /// <summary>
    ///     <para>
    ///         The links index parts.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* LinksIndexParts;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="ExternalLinksRecursionlessSizeBalancedTreeMethodsBase{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected ExternalLinksRecursionlessSizeBalancedTreeMethodsBase(LinksConstants<TLinkAddress> constants, byte* linksDataPar
ts, byte* linksIndexParts, byte* header)
    {
        LinksDataParts = linksDataParts;
        LinksIndexParts = linksIndexParts;
        Header = header;
```

```
        Break = constants.Break;
        Continue = constants.Continue;
    }

    /// <summary>
    ///     <para>
    ///         The zero.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress this[TLinkAddress index]
    {
        [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
        get
        {
            var root = GetTreeRoot();
            if ((index >= GetSize(node: root)))
            {
                return TLinkAddress.Zero;
            }
            while (root != TLinkAddress.Zero)
            {
                var left = GetLeftOrDefault(node: root);
                var leftSize = GetSizeOrZero(node: left);
                if (index < leftSize)
                {
                    root = left;
                    continue;
                }
                if ((index == leftSize))
                {
                    return root;
                }
                root = GetRightOrDefault(node: root);
                index = (index) - (leftSize + TLinkAddress.One);
            }
            return TLinkAddress.Zero; // TODO: Impossible situation exception (only if tree structure broken)
        }
    }

    /// <summary>
    ///     Выполняет поиск и возвращает индекс связи с указанными Source (началом) и Target (концом).
    /// </summary>
    /// <param name="source">Индекс связи, которая является началом на искомой связи.</param>
    /// <param name="target">Индекс связи, которая является концом на искомой связи.</param>
    /// <returns>Индекс искомой связи.</returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public TLinkAddress Search(TLinkAddress source, TLinkAddress target)
    {
        var root = GetTreeRoot();
        while (root != TLinkAddress.Zero)
        {
            ref var rootLink = ref GetLinkDataPartReference(link: root);
            var rootSource = rootLink.Source;
            var rootTarget = rootLink.Target;
            if (FirstIsToTheLeftOfSecond(source: source, target: target, rootSource: rootSource, rootTarget: rootTarget)) // n
ode.Key < root.Key
            {
                root = GetLeftOrDefault(node: root);
            }
            else if (FirstIsToTheRightOfSecond(source: source, target: target, rootSource: rootSource, rootTarget: rootTarget)
) // node.Key > root.Key
            {
                root = GetRightOrDefault(node: root);
            }
            else // node.Key == root.Key
            {
                return root;
            }
        }
        return TLinkAddress.Zero;
    }

    // TODO: Return indices range instead of references count
    /// <summary>
    ///     <para>
    ///         Counts the usages using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public TLinkAddress CountUsages(TLinkAddress link)
    {
        var root = GetTreeRoot();
```

```
    var total = GetSize(node: root);
    var totalRightIgnore = TLinkAddress.Zero;
    while (root != TLinkAddress.Zero)
    {
        var @base = GetBasePartValue(link: root);
        if ((@base <= link))
        {
            root = GetRightOrDefault(node: root);
        }
        else
        {
            totalRightIgnore = (totalRightIgnore + GetRightSize(node: root) + TLinkAddress.One);
            root = GetLeftOrDefault(node: root);
        }
    }
    root = GetTreeRoot();
    var totalLeftIgnore = TLinkAddress.Zero;
    while (root != TLinkAddress.Zero)
    {
        var @base = GetBasePartValue(link: root);
        if ((@base >= link))
        {
            root = GetLeftOrDefault(node: root);
        }
        else
        {
            totalLeftIgnore = (totalLeftIgnore + GetLeftSize(node: root) + TLinkAddress.One);
            root = GetRightOrDefault(node: root);
        }
    }
    return (total - totalRightIgnore - totalLeftIgnore);
}
/// <summary>
///     <para>
///         Eaches the usage using the specified base.
///     </para>
///     <para></para>
/// </summary>
/// <param name="base">
///     <para>The base.</para>
///     <para></para>
/// </param>
/// <param name="handler">
///     <para>The handler.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
public TLinkAddress EachUsage(TLinkAddress @base, ReadHandler<TLinkAddress>? handler)
{
    return EachUsageCore(@base: @base, link: GetTreeRoot(), handler: handler);
}

/// <summary>
///     <para>
///         Gets the tree root.
///     </para>
///     <para></para>
/// </summary>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected abstract TLinkAddress GetTreeRoot();

/// <summary>
///     <para>
///         Gets the base part value using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected abstract TLinkAddress GetBasePartValue(TLinkAddress link);

/// <summary>
///     <para>
///         Determines whether this instance first is to the right of second.
///     </para>
///     <para></para>
/// </summary>
```

```
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="target">
    ///     <para>The target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootSource">
    ///     <para>The root source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootTarget">
    ///     <para>The root target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract bool FirstIsToTheRightOfSecond(TLinkAddress source, TLinkAddress target, TLinkAddress rootSource, TLink
Address rootTarget);

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="target">
    ///     <para>The target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootSource">
    ///     <para>The root source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootTarget">
    ///     <para>The root target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract bool FirstIsToTheLeftOfSecond(TLinkAddress source, TLinkAddress target, TLinkAddress rootSource, TLinkA
ddress rootTarget);

    /// <summary>
    ///     <para>
    ///         Gets the header reference.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>A ref links header of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref LinksHeader<TLinkAddress> GetHeaderReference()
    {
        return ref AsRef<LinksHeader<TLinkAddress>>(source: Header);
    }

    /// <summary>
    ///     <para>
    ///         Gets the link data part reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link data part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLinkDataPart<TLinkAddress> GetLinkDataPartReference(TLinkAddress link)
    {
        return ref AsRef<RawLinkDataPart<TLinkAddress>>(source: LinksDataParts + RawLinkDataPart<TLinkAddress>.SizeInBytes * l
ong.CreateTruncating(link));
    }

    /// <summary>
```

```
    ///     <para>
    ///         Gets the link index part reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link index part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLinkIndexPart<TLinkAddress> GetLinkIndexPartReference(TLinkAddress link)
    {
         return ref AsRef<RawLinkIndexPart<TLinkAddress>>(source: LinksIndexParts + RawLinkIndexPart<TLinkAddress>.SizeInBytes
* long.CreateTruncating(link));
    }

    /// <summary>
    ///     <para>
    ///         Gets the link values using the specified link index.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="linkIndex">
    ///     <para>The link index.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A list of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual IList<TLinkAddress>? GetLinkValues(TLinkAddress linkIndex)
    {
        ref var link = ref GetLinkDataPartReference(link: linkIndex);
        return new Link<TLinkAddress>(index: linkIndex, source: link.Source, target: link.Target);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="first">
    ///     <para>The first.</para>
    ///     <para></para>
    /// </param>
    /// <param name="second">
    ///     <para>The second.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheLeftOfSecond(TLinkAddress first, TLinkAddress second)
    {
        ref var firstLink = ref GetLinkDataPartReference(link: first);
        ref var secondLink = ref GetLinkDataPartReference(link: second);
        return FirstIsToTheLeftOfSecond(source: firstLink.Source, target: firstLink.Target, rootSource: secondLink.Source, roo
tTarget: secondLink.Target);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the right of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="first">
    ///     <para>The first.</para>
    ///     <para></para>
    /// </param>
    /// <param name="second">
    ///     <para>The second.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheRightOfSecond(TLinkAddress first, TLinkAddress second)
    {
        ref var firstLink = ref GetLinkDataPartReference(link: first);
        ref var secondLink = ref GetLinkDataPartReference(link: second);
        return FirstIsToTheRightOfSecond(source: firstLink.Source, target: firstLink.Target, rootSource: secondLink.Source, ro
```

```
otTarget: secondLink.Target);
    }

    // TODO: 1. Move target, handler to separate object. 2. Use stack or walker 3. Use low-level MSIL stack.
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    private TLinkAddress EachUsageCore(TLinkAddress @base, TLinkAddress link, ReadHandler<TLinkAddress>? handler)
    {
        var @continue = Continue;
        if (link == TLinkAddress.Zero)
        {
            return @continue;
        }
        var linkBasePart = GetBasePartValue(link: link);
        var @break = Break;
        if ((linkBasePart > @base))
        {
            if ((EachUsageCore(@base: @base, link: GetLeftOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
        }
        else if ((linkBasePart < @base))
        {
            if ((EachUsageCore(@base: @base, link: GetRightOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
        }
        else //if (linkBasePart == @base)
        {
            if (((handler == null ? Continue : handler(link: GetLinkValues(linkIndex: link))) == @break))
            {
                return @break;
            }
            if ((EachUsageCore(@base: @base, link: GetLeftOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
            if ((EachUsageCore(@base: @base, link: GetRightOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
        }
        return @continue;
    }

    /// <summary>
    ///     <para>
    ///         Prints the node value using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <param name="sb">
    ///     <para>The sb.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void PrintNodeValue(TLinkAddress node, StringBuilder sb)
    {
        ref var link = ref GetLinkDataPartReference(link: node);
        sb.Append(value: ' ');
        sb.Append(value: link.Source);
        sb.Append(value: '-');
        sb.Append(value: '>');
        sb.Append(value: link.Target);
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/ExternalLinksSizeBalancedTreeMethodsBase.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using System.Text;
using Platform.Collections.Methods.Trees;
using Platform.Converters;
using Platform.Delegates;
using static System.Runtime.CompilerServices.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the external links size balanced tree methods base.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="SizeBalancedTreeMethods{TLinkAddress}" />
/// <seealso cref="ILinksTreeMethods{TLinkAddress}" />
public abstract unsafe class ExternalLinksSizeBalancedTreeMethodsBase<TLinkAddress> : SizeBalancedTreeMethods<TLinkAddress>, I
LinksTreeMethods<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAdd
ress, bool>
{

    /// <summary>
    ///     <para>
    ///         The break.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Break;
    /// <summary>
    ///     <para>
    ///         The continue.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Continue;
    /// <summary>
    ///     <para>
    ///         The header.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* Header;
    /// <summary>
    ///     <para>
    ///         The links data parts.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* LinksDataParts;
    /// <summary>
    ///     <para>
    ///         The links index parts.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* LinksIndexParts;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="ExternalLinksSizeBalancedTreeMethodsBase{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected ExternalLinksSizeBalancedTreeMethodsBase(LinksConstants<TLinkAddress> constants, byte* linksDataParts, byte* lin
ksIndexParts, byte* header)
    {
        LinksDataParts = linksDataParts;
        LinksIndexParts = linksIndexParts;
        Header = header;
```

```
        Break = constants.Break;
        Continue = constants.Continue;
    }

    /// <summary>
    ///     <para>
    ///         The zero.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress this[TLinkAddress index]
    {
        [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
        get
        {
            var root = GetTreeRoot();
            if ((index >= GetSize(node: root)))
            {
                return TLinkAddress.Zero;
            }
            while (root != TLinkAddress.Zero)
            {
                var left = GetLeftOrDefault(node: root);
                var leftSize = GetSizeOrZero(node: left);
                if (index < leftSize)
                {
                    root = left;
                    continue;
                }
                if ((index == leftSize))
                {
                    return root;
                }
                root = GetRightOrDefault(node: root);
                index = (index) - (leftSize + TLinkAddress.One);
            }
            return TLinkAddress.Zero; // TODO: Impossible situation exception (only if tree structure broken)
        }
    }

    /// <summary>
    ///     Выполняет поиск и возвращает индекс связи с указанными Source (началом) и Target (концом).
    /// </summary>
    /// <param name="source">Индекс связи, которая является началом на искомой связи.</param>
    /// <param name="target">Индекс связи, которая является концом на искомой связи.</param>
    /// <returns>Индекс искомой связи.</returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public TLinkAddress Search(TLinkAddress source, TLinkAddress target)
    {
        var root = GetTreeRoot();
        while (root != TLinkAddress.Zero)
        {
            ref var rootLink = ref GetLinkDataPartReference(link: root);
            var rootSource = rootLink.Source;
            var rootTarget = rootLink.Target;
            if (FirstIsToTheLeftOfSecond(source: source, target: target, rootSource: rootSource, rootTarget: rootTarget)) // n
ode.Key < root.Key
            {
                root = GetLeftOrDefault(node: root);
            }
            else if (FirstIsToTheRightOfSecond(source: source, target: target, rootSource: rootSource, rootTarget: rootTarget)
) // node.Key > root.Key
            {
                root = GetRightOrDefault(node: root);
            }
            else // node.Key == root.Key
            {
                return root;
            }
        }
        return TLinkAddress.Zero;
    }

    // TODO: Return indices range instead of references count
    /// <summary>
    ///     <para>
    ///         Counts the usages using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public TLinkAddress CountUsages(TLinkAddress link)
    {
        var root = GetTreeRoot();
```

```
    var total = GetSize(node: root);
    var totalRightIgnore = TLinkAddress.Zero;
    while (root != TLinkAddress.Zero)
    {
        var @base = GetBasePartValue(link: root);
        if ((@base <= link))
        {
            root = GetRightOrDefault(node: root);
        }
        else
        {
            totalRightIgnore = (totalRightIgnore + GetRightSize(node: root) + TLinkAddress.One);
            root = GetLeftOrDefault(node: root);
        }
    }
    root = GetTreeRoot();
    var totalLeftIgnore = TLinkAddress.Zero;
    while (root != TLinkAddress.Zero)
    {
        var @base = GetBasePartValue(link: root);
        if ((@base >= link))
        {
            root = GetLeftOrDefault(node: root);
        }
        else
        {
            totalLeftIgnore = (totalLeftIgnore + GetLeftSize(node: root) + TLinkAddress.One);
            root = GetRightOrDefault(node: root);
        }
    }
    return total - totalRightIgnore - totalLeftIgnore;
}
/// <summary>
///     <para>
///         Eaches the usage using the specified base.
///     </para>
///     <para></para>
/// </summary>
/// <param name="base">
///     <para>The base.</para>
///     <para></para>
/// </param>
/// <param name="handler">
///     <para>The handler.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
public TLinkAddress EachUsage(TLinkAddress @base, ReadHandler<TLinkAddress>? handler)
{
    return EachUsageCore(@base: @base, link: GetTreeRoot(), handler: handler);
}

/// <summary>
///     <para>
///         Gets the tree root.
///     </para>
///     <para></para>
/// </summary>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected abstract TLinkAddress GetTreeRoot();

/// <summary>
///     <para>
///         Gets the base part value using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected abstract TLinkAddress GetBasePartValue(TLinkAddress link);

/// <summary>
///     <para>
///         Determines whether this instance first is to the right of second.
///     </para>
///     <para></para>
/// </summary>
```

```
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="target">
    ///     <para>The target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootSource">
    ///     <para>The root source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootTarget">
    ///     <para>The root target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract bool FirstIsToTheRightOfSecond(TLinkAddress source, TLinkAddress target, TLinkAddress rootSource, TLink
Address rootTarget);

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="target">
    ///     <para>The target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootSource">
    ///     <para>The root source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootTarget">
    ///     <para>The root target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract bool FirstIsToTheLeftOfSecond(TLinkAddress source, TLinkAddress target, TLinkAddress rootSource, TLinkA
ddress rootTarget);

    /// <summary>
    ///     <para>
    ///         Gets the header reference.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>A ref links header of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref LinksHeader<TLinkAddress> GetHeaderReference()
    {
        return ref AsRef<LinksHeader<TLinkAddress>>(source: Header);
    }

    /// <summary>
    ///     <para>
    ///         Gets the link data part reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link data part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLinkDataPart<TLinkAddress> GetLinkDataPartReference(TLinkAddress link)
    {
        return ref AsRef<RawLinkDataPart<TLinkAddress>>(source: LinksDataParts + RawLinkDataPart<TLinkAddress>.SizeInBytes * l
ong.CreateTruncating(link));
    }

    /// <summary>
```

```
    ///     <para>
    ///         Gets the link index part reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link index part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLinkIndexPart<TLinkAddress> GetLinkIndexPartReference(TLinkAddress link)
    {
         return ref AsRef<RawLinkIndexPart<TLinkAddress>>(source: LinksIndexParts + RawLinkIndexPart<TLinkAddress>.SizeInBytes
* long.CreateTruncating(link));
    }

    /// <summary>
    ///     <para>
    ///         Gets the link values using the specified link index.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="linkIndex">
    ///     <para>The link index.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A list of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual IList<TLinkAddress>? GetLinkValues(TLinkAddress linkIndex)
    {
        ref var link = ref GetLinkDataPartReference(link: linkIndex);
        return new Link<TLinkAddress>(index: linkIndex, source: link.Source, target: link.Target);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="first">
    ///     <para>The first.</para>
    ///     <para></para>
    /// </param>
    /// <param name="second">
    ///     <para>The second.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheLeftOfSecond(TLinkAddress first, TLinkAddress second)
    {
        ref var firstLink = ref GetLinkDataPartReference(link: first);
        ref var secondLink = ref GetLinkDataPartReference(link: second);
        return FirstIsToTheLeftOfSecond(source: firstLink.Source, target: firstLink.Target, rootSource: secondLink.Source, roo
tTarget: secondLink.Target);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the right of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="first">
    ///     <para>The first.</para>
    ///     <para></para>
    /// </param>
    /// <param name="second">
    ///     <para>The second.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheRightOfSecond(TLinkAddress first, TLinkAddress second)
    {
        ref var firstLink = ref GetLinkDataPartReference(link: first);
        ref var secondLink = ref GetLinkDataPartReference(link: second);
        return FirstIsToTheRightOfSecond(source: firstLink.Source, target: firstLink.Target, rootSource: secondLink.Source, ro
```

```
otTarget: secondLink.Target);
    }

    // TODO: 1. Move target, handler to separate object. 2. Use stack or walker 3. Use low-level MSIL stack.
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    private TLinkAddress EachUsageCore(TLinkAddress @base, TLinkAddress link, ReadHandler<TLinkAddress>? handler)
    {
        var @continue = Continue;
        if (link == TLinkAddress.Zero)
        {
            return @continue;
        }
        var linkBasePart = GetBasePartValue(link: link);
        var @break = Break;
        if ((linkBasePart > @base))
        {
            if ((EachUsageCore(@base: @base, link: GetLeftOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
        }
        else if ((linkBasePart < @base))
        {
            if ((EachUsageCore(@base: @base, link: GetRightOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
        }
        else //if (linkBasePart == @base)
        {
            if (((handler == null ? Continue : handler(link: GetLinkValues(linkIndex: link))) == @break))
            {
                return @break;
            }
            if ((EachUsageCore(@base: @base, link: GetLeftOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
            if ((EachUsageCore(@base: @base, link: GetRightOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
        }
        return @continue;
    }

    /// <summary>
    ///     <para>
    ///         Prints the node value using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <param name="sb">
    ///     <para>The sb.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void PrintNodeValue(TLinkAddress node, StringBuilder sb)
    {
        ref var link = ref GetLinkDataPartReference(link: node);
        sb.Append(value: ' ');
        sb.Append(value: link.Source);
        sb.Append(value: '-');
        sb.Append(value: '>');
        sb.Append(value: link.Target);
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/ExternalLinksSourcesRecursionlessSizeBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the external links sources recursionless size balanced tree methods.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="ExternalLinksRecursionlessSizeBalancedTreeMethodsBase{TLinkAddress}" />
public unsafe class ExternalLinksSourcesRecursionlessSizeBalancedTreeMethods<TLinkAddress> : ExternalLinksRecursionlessSizeBal
ancedTreeMethodsBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLin
kAddress, bool>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="ExternalLinksSourcesRecursionlessSizeBalancedTreeMethods{TLinkAddress}" /> instan
ce.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public ExternalLinksSourcesRecursionlessSizeBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* linksDataPar
ts, byte* linksIndexParts, byte* header) : base(constants: constants, linksDataParts: linksDataParts, linksIndexParts: linksIn
dexParts, header: header) { }

    /// <summary>
    ///     <para>
    ///         Gets the left reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetLeftReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).LeftAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Gets the right reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetRightReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).RightAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Gets the left using the specified node.
    ///     </para>
    ///     <para></para>
```

```
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetLeft(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).LeftAsSource;
}

/// <summary>
///     <para>
///         Gets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetRight(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).RightAsSource;
}

/// <summary>
///     <para>
///         Sets the left using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="left">
///     <para>The left.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetLeft(TLinkAddress node, TLinkAddress left)
{
    GetLinkIndexPartReference(link: node).LeftAsSource = left;
}

/// <summary>
///     <para>
///         Sets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="right">
///     <para>The right.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetRight(TLinkAddress node, TLinkAddress right)
{
    GetLinkIndexPartReference(link: node).RightAsSource = right;
}

/// <summary>
///     <para>
///         Gets the size using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetSize(TLinkAddress node)
{
```

```
        return GetLinkIndexPartReference(link: node).SizeAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Sets the size using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <param name="size">
    ///     <para>The size.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void SetSize(TLinkAddress node, TLinkAddress size)
    {
        GetLinkIndexPartReference(link: node).SizeAsSource = size;
    }

    /// <summary>
    ///     <para>
    ///         Gets the tree root.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override TLinkAddress GetTreeRoot()
    {
        return GetHeaderReference().RootAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Gets the base part value using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override TLinkAddress GetBasePartValue(TLinkAddress link)
    {
        return GetLinkDataPartReference(link: link).Source;
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="firstSource">
    ///     <para>The first source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="firstTarget">
    ///     <para>The first target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondSource">
    ///     <para>The second source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondTarget">
    ///     <para>The second target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheLeftOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress secondSo
urce, TLinkAddress secondTarget)
    {
        return firstSource < secondSource || ((firstSource == secondSource) && firstTarget < secondTarget);
    }
```

```
    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the right of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="firstSource">
    ///     <para>The first source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="firstTarget">
    ///     <para>The first target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondSource">
    ///     <para>The second source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondTarget">
    ///     <para>The second target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheRightOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress secondS
ource, TLinkAddress secondTarget)
    {
        return firstSource > secondSource || ((firstSource == secondSource) && firstTarget > secondTarget);
    }

    /// <summary>
    ///     <para>
    ///         Clears the node using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void ClearNode(TLinkAddress node)
    {
        ref var link = ref GetLinkIndexPartReference(link: node);
        link.LeftAsSource = TLinkAddress.Zero;
        link.RightAsSource = TLinkAddress.Zero;
        link.SizeAsSource = TLinkAddress.Zero;
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/ExternalLinksSourcesSizeBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the external links sources size balanced tree methods.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="ExternalLinksSizeBalancedTreeMethodsBase{TLinkAddress}" />
public unsafe class ExternalLinksSourcesSizeBalancedTreeMethods<TLinkAddress> : ExternalLinksSizeBalancedTreeMethodsBase<TLink
Address> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="ExternalLinksSourcesSizeBalancedTreeMethods{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public ExternalLinksSourcesSizeBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* linksDataParts, byte* lin
ksIndexParts, byte* header) : base(constants: constants, linksDataParts: linksDataParts, linksIndexParts: linksIndexParts, hea
der: header) { }

    /// <summary>
    ///     <para>
    ///         Gets the left reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetLeftReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).LeftAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Gets the right reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetRightReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).RightAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Gets the left using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
```

```
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetLeft(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).LeftAsSource;
}

/// <summary>
///     <para>
///         Gets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetRight(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).RightAsSource;
}

/// <summary>
///     <para>
///         Sets the left using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="left">
///     <para>The left.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetLeft(TLinkAddress node, TLinkAddress left)
{
    GetLinkIndexPartReference(link: node).LeftAsSource = left;
}

/// <summary>
///     <para>
///         Sets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="right">
///     <para>The right.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetRight(TLinkAddress node, TLinkAddress right)
{
    GetLinkIndexPartReference(link: node).RightAsSource = right;
}

/// <summary>
///     <para>
///         Gets the size using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetSize(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).SizeAsSource;
}
```

```
    /// <summary>
    ///     <para>
    ///         Sets the size using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <param name="size">
    ///     <para>The size.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void SetSize(TLinkAddress node, TLinkAddress size)
    {
        GetLinkIndexPartReference(link: node).SizeAsSource = size;
    }

    /// <summary>
    ///     <para>
    ///         Gets the tree root.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override TLinkAddress GetTreeRoot()
    {
        return GetHeaderReference().RootAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Gets the base part value using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override TLinkAddress GetBasePartValue(TLinkAddress link)
    {
        return GetLinkDataPartReference(link: link).Source;
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="firstSource">
    ///     <para>The first source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="firstTarget">
    ///     <para>The first target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondSource">
    ///     <para>The second source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondTarget">
    ///     <para>The second target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheLeftOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress secondSo
urce, TLinkAddress secondTarget)
    {
        return firstSource < secondSource || ((firstSource == secondSource) && firstTarget < secondTarget);
    }

    /// <summary>
    ///     <para>
```

```
    ///         Determines whether this instance first is to the right of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="firstSource">
    ///     <para>The first source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="firstTarget">
    ///     <para>The first target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondSource">
    ///     <para>The second source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondTarget">
    ///     <para>The second target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheRightOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress secondS
ource, TLinkAddress secondTarget)
    {
        return firstSource > secondSource || ((firstSource == secondSource) && firstTarget > secondTarget);
    }

    /// <summary>
    ///     <para>
    ///         Clears the node using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void ClearNode(TLinkAddress node)
    {
        ref var link = ref GetLinkIndexPartReference(link: node);
        link.LeftAsSource = TLinkAddress.Zero;
        link.RightAsSource = TLinkAddress.Zero;
        link.SizeAsSource = TLinkAddress.Zero;
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/ExternalLinksTargetsRecursionlessSizeBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the external links targets recursionless size balanced tree methods.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="ExternalLinksRecursionlessSizeBalancedTreeMethodsBase{TLinkAddress}" />
public unsafe class ExternalLinksTargetsRecursionlessSizeBalancedTreeMethods<TLinkAddress> : ExternalLinksRecursionlessSizeBal
ancedTreeMethodsBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLin
kAddress, bool>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="ExternalLinksTargetsRecursionlessSizeBalancedTreeMethods{TLinkAddress}" /> instan
ce.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public ExternalLinksTargetsRecursionlessSizeBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* linksDataPar
ts, byte* linksIndexParts, byte* header) : base(constants: constants, linksDataParts: linksDataParts, linksIndexParts: linksIn
dexParts, header: header) { }

    /// <summary>
    ///     <para>
    ///         Gets the left reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetLeftReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).LeftAsTarget;
    }

    /// <summary>
    ///     <para>
    ///         Gets the right reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetRightReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).RightAsTarget;
    }

    /// <summary>
    ///     <para>
    ///         Gets the left using the specified node.
    ///     </para>
    ///     <para></para>
```

```
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetLeft(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).LeftAsTarget;
}

/// <summary>
///     <para>
///         Gets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetRight(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).RightAsTarget;
}

/// <summary>
///     <para>
///         Sets the left using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="left">
///     <para>The left.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetLeft(TLinkAddress node, TLinkAddress left)
{
    GetLinkIndexPartReference(link: node).LeftAsTarget = left;
}

/// <summary>
///     <para>
///         Sets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="right">
///     <para>The right.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetRight(TLinkAddress node, TLinkAddress right)
{
    GetLinkIndexPartReference(link: node).RightAsTarget = right;
}

/// <summary>
///     <para>
///         Gets the size using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetSize(TLinkAddress node)
{
```

```
        return GetLinkIndexPartReference(link: node).SizeAsTarget;
    }

    /// <summary>
    ///     <para>
    ///         Sets the size using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <param name="size">
    ///     <para>The size.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void SetSize(TLinkAddress node, TLinkAddress size)
    {
        GetLinkIndexPartReference(link: node).SizeAsTarget = size;
    }

    /// <summary>
    ///     <para>
    ///         Gets the tree root.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override TLinkAddress GetTreeRoot()
    {
        return GetHeaderReference().RootAsTarget;
    }

    /// <summary>
    ///     <para>
    ///         Gets the base part value using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override TLinkAddress GetBasePartValue(TLinkAddress link)
    {
        return GetLinkDataPartReference(link: link).Target;
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="firstSource">
    ///     <para>The first source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="firstTarget">
    ///     <para>The first target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondSource">
    ///     <para>The second source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondTarget">
    ///     <para>The second target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheLeftOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress secondSo
urce, TLinkAddress secondTarget)
    {
        return firstTarget < secondTarget || ((firstTarget == secondTarget) && firstSource < secondSource);
    }
```

```
    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the right of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="firstSource">
    ///     <para>The first source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="firstTarget">
    ///     <para>The first target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondSource">
    ///     <para>The second source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondTarget">
    ///     <para>The second target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheRightOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress secondS
ource, TLinkAddress secondTarget)
    {
        return firstTarget > secondTarget || ((firstTarget == secondTarget) && firstSource > secondSource);
    }

    /// <summary>
    ///     <para>
    ///         Clears the node using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void ClearNode(TLinkAddress node)
    {
        ref var link = ref GetLinkIndexPartReference(link: node);
        link.LeftAsTarget = TLinkAddress.Zero;
        link.RightAsTarget = TLinkAddress.Zero;
        link.SizeAsTarget = TLinkAddress.Zero;
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/ExternalLinksTargetsSizeBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the external links targets size balanced tree methods.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="ExternalLinksSizeBalancedTreeMethodsBase{TLinkAddress}" />
public unsafe class ExternalLinksTargetsSizeBalancedTreeMethods<TLinkAddress> : ExternalLinksSizeBalancedTreeMethodsBase<TLink
Address> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="ExternalLinksTargetsSizeBalancedTreeMethods{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public ExternalLinksTargetsSizeBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* linksDataParts, byte* lin
ksIndexParts, byte* header) : base(constants: constants, linksDataParts: linksDataParts, linksIndexParts: linksIndexParts, hea
der: header) { }

    /// <summary>
    ///     <para>
    ///         Gets the left reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetLeftReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).LeftAsTarget;
    }

    /// <summary>
    ///     <para>
    ///         Gets the right reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetRightReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).RightAsTarget;
    }

    /// <summary>
    ///     <para>
    ///         Gets the left using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
```

```
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetLeft(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).LeftAsTarget;
}

/// <summary>
///     <para>
///         Gets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetRight(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).RightAsTarget;
}

/// <summary>
///     <para>
///         Sets the left using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="left">
///     <para>The left.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetLeft(TLinkAddress node, TLinkAddress left)
{
    GetLinkIndexPartReference(link: node).LeftAsTarget = left;
}

/// <summary>
///     <para>
///         Sets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="right">
///     <para>The right.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetRight(TLinkAddress node, TLinkAddress right)
{
    GetLinkIndexPartReference(link: node).RightAsTarget = right;
}

/// <summary>
///     <para>
///         Gets the size using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetSize(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).SizeAsTarget;
}
```

```
    /// <summary>
    ///     <para>
    ///         Sets the size using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <param name="size">
    ///     <para>The size.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void SetSize(TLinkAddress node, TLinkAddress size)
    {
        GetLinkIndexPartReference(link: node).SizeAsTarget = size;
    }

    /// <summary>
    ///     <para>
    ///         Gets the tree root.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override TLinkAddress GetTreeRoot()
    {
        return GetHeaderReference().RootAsTarget;
    }

    /// <summary>
    ///     <para>
    ///         Gets the base part value using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override TLinkAddress GetBasePartValue(TLinkAddress link)
    {
        return GetLinkDataPartReference(link: link).Target;
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="firstSource">
    ///     <para>The first source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="firstTarget">
    ///     <para>The first target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondSource">
    ///     <para>The second source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondTarget">
    ///     <para>The second target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheLeftOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress secondSo
urce, TLinkAddress secondTarget)
    {
        return firstTarget < secondTarget || ((firstTarget == secondTarget) && firstSource < secondSource);
    }

    /// <summary>
    ///     <para>
```

```
    ///         Determines whether this instance first is to the right of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="firstSource">
    ///     <para>The first source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="firstTarget">
    ///     <para>The first target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondSource">
    ///     <para>The second source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="secondTarget">
    ///     <para>The second target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheRightOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress secondS
ource, TLinkAddress secondTarget)
    {
        return firstTarget > secondTarget || ((firstTarget == secondTarget) && firstSource > secondSource);
    }

    /// <summary>
    ///     <para>
    ///         Clears the node using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void ClearNode(TLinkAddress node)
    {
        ref var link = ref GetLinkIndexPartReference(link: node);
        link.LeftAsTarget = TLinkAddress.Zero;
        link.RightAsTarget = TLinkAddress.Zero;
        link.SizeAsTarget = TLinkAddress.Zero;
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/InternalLinksRecursionlessSizeBalancedTreeMethodsBase.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using System.Text;
using Platform.Collections.Methods.Trees;
using Platform.Converters;
using Platform.Delegates;
using static System.Runtime.CompilerServices.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the internal links recursionless size balanced tree methods base.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="RecursionlessSizeBalancedTreeMethods{TLinkAddress}" />
/// <seealso cref="ILinksTreeMethods{TLinkAddress}" />
public abstract unsafe class InternalLinksRecursionlessSizeBalancedTreeMethodsBase<TLinkAddress> : RecursionlessSizeBalancedTr
eeMethods<TLinkAddress>, ILinksTreeMethods<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperat
ors<TLinkAddress, TLinkAddress, bool>
{

    /// <summary>
    ///     <para>
    ///         The break.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Break;
    /// <summary>
    ///     <para>
    ///         The continue.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Continue;
    /// <summary>
    ///     <para>
    ///         The header.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* Header;
    /// <summary>
    ///     <para>
    ///         The links data parts.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* LinksDataParts;
    /// <summary>
    ///     <para>
    ///         The links index parts.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* LinksIndexParts;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="InternalLinksRecursionlessSizeBalancedTreeMethodsBase{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected InternalLinksRecursionlessSizeBalancedTreeMethodsBase(LinksConstants<TLinkAddress> constants, byte* linksDataPar
ts, byte* linksIndexParts, byte* header)
    {
        LinksDataParts = linksDataParts;
        LinksIndexParts = linksIndexParts;
        Header = header;
```

```
    Break = constants.Break;
    Continue = constants.Continue;
}

/// <summary>
///     <para>
///         The zero.
///     </para>
///     <para></para>
/// </summary>
public TLinkAddress this[TLinkAddress link, TLinkAddress index]
{
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    get
    {
        var root = GetTreeRoot(link: link);
        if ((index >= GetSize(node: root)))
        {
            return TLinkAddress.Zero;
        }
        while (root != TLinkAddress.Zero)
        {
            var left = GetLeftOrDefault(node: root);
            var leftSize = GetSizeOrZero(node: left);
            if (index < leftSize)
            {
                root = left;
                continue;
            }
            if ((index == leftSize))
            {
                return root;
            }
            root = GetRightOrDefault(node: root);
            index = (index) - (leftSize + TLinkAddress.One);
        }
        return TLinkAddress.Zero; // TODO: Impossible situation exception (only if tree structure broken)
    }
}

/// <summary>
///     Выполняет поиск и возвращает индекс связи с указанными Source (началом) и Target (концом).
/// </summary>
/// <param name="source">Индекс связи, которая является началом на искомой связи.</param>
/// <param name="target">Индекс связи, которая является концом на искомой связи.</param>
/// <returns>Индекс искомой связи.</returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
public abstract TLinkAddress Search(TLinkAddress source, TLinkAddress target);

// TODO: Return indices range instead of references count
/// <summary>
///     <para>
///         Counts the usages using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
public TLinkAddress CountUsages(TLinkAddress link)
{
    return GetSizeOrZero(node: GetTreeRoot(link: link));
}
/// <summary>
///     <para>
///         Eaches the usage using the specified base.
///     </para>
///     <para></para>
/// </summary>
/// <param name="base">
///     <para>The base.</para>
///     <para></para>
/// </param>
/// <param name="handler">
///     <para>The handler.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
public TLinkAddress EachUsage(TLinkAddress @base, ReadHandler<TLinkAddress>? handler)
{
    return EachUsageCore(@base: @base, link: GetTreeRoot(link: @base), handler: handler);
}
```

```
    /// <summary>
    ///     <para>
    ///         Gets the tree root using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract TLinkAddress GetTreeRoot(TLinkAddress link);

    /// <summary>
    ///     <para>
    ///         Gets the base part value using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract TLinkAddress GetBasePartValue(TLinkAddress link);

    /// <summary>
    ///     <para>
    ///         Gets the key part value using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract TLinkAddress GetKeyPartValue(TLinkAddress link);

    /// <summary>
    ///     <para>
    ///         Gets the link data part reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link data part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLinkDataPart<TLinkAddress> GetLinkDataPartReference(TLinkAddress link)
    {
        return ref AsRef<RawLinkDataPart<TLinkAddress>>(source: LinksDataParts + RawLinkDataPart<TLinkAddress>.SizeInBytes * l
ong.CreateTruncating(link));
    }

    /// <summary>
    ///     <para>
    ///         Gets the link index part reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link index part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLinkIndexPart<TLinkAddress> GetLinkIndexPartReference(TLinkAddress link)
    {
         return ref AsRef<RawLinkIndexPart<TLinkAddress>>(source: LinksIndexParts + RawLinkIndexPart<TLinkAddress>.SizeInBytes
* long.CreateTruncating(link));
```

```
}

/// <summary>
///     <para>
///         Determines whether this instance first is to the left of second.
///     </para>
///     <para></para>
/// </summary>
/// <param name="first">
///     <para>The first.</para>
///     <para></para>
/// </param>
/// <param name="second">
///     <para>The second.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The bool</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override bool FirstIsToTheLeftOfSecond(TLinkAddress first, TLinkAddress second)
{
    return (GetKeyPartValue(link: first) < GetKeyPartValue(link: second));
}

/// <summary>
///     <para>
///         Determines whether this instance first is to the right of second.
///     </para>
///     <para></para>
/// </summary>
/// <param name="first">
///     <para>The first.</para>
///     <para></para>
/// </param>
/// <param name="second">
///     <para>The second.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The bool</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override bool FirstIsToTheRightOfSecond(TLinkAddress first, TLinkAddress second)
{
    return (GetKeyPartValue(link: first) > GetKeyPartValue(link: second));
}

/// <summary>
///     <para>
///         Gets the link values using the specified link index.
///     </para>
///     <para></para>
/// </summary>
/// <param name="linkIndex">
///     <para>The link index.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>A list of t link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected virtual IList<TLinkAddress>? GetLinkValues(TLinkAddress linkIndex)
{
    ref var link = ref GetLinkDataPartReference(link: linkIndex);
    return new Link<TLinkAddress>(index: linkIndex, source: link.Source, target: link.Target);
}

/// <summary>
///     <para>
///         Searches the core using the specified root.
///     </para>
///     <para></para>
/// </summary>
/// <param name="root">
///     <para>The root.</para>
///     <para></para>
/// </param>
/// <param name="key">
///     <para>The key.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The zero.</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected TLinkAddress SearchCore(TLinkAddress root, TLinkAddress key)
{
```

```
        while (root != TLinkAddress.Zero)
        {
            var rootKey = GetKeyPartValue(link: root);
            if (key < rootKey) // node.Key < root.Key
            {
                root = GetLeftOrDefault(node: root);
            }
            else if (key > rootKey) // node.Key > root.Key
            {
                root = GetRightOrDefault(node: root);
            }
            else // node.Key == root.Key
            {
                return root;
            }
        }
        return TLinkAddress.Zero;
    }

    // TODO: 1. Move target, handler to separate object. 2. Use stack or walker 3. Use low-level MSIL stack.
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    private TLinkAddress EachUsageCore(TLinkAddress @base, TLinkAddress link, ReadHandler<TLinkAddress>? handler)
    {
        var @continue = Continue;
        if (link == TLinkAddress.Zero)
        {
            return @continue;
        }
        var @break = Break;
        if ((EachUsageCore(@base: @base, link: GetLeftOrDefault(node: link), handler: handler) == @break))
        {
            return @break;
        }
        if (((handler == null ? Continue : handler(link: GetLinkValues(linkIndex: link))) == @break))
        {
            return @break;
        }
        if ((EachUsageCore(@base: @base, link: GetRightOrDefault(node: link), handler: handler) == @break))
        {
            return @break;
        }
        return @continue;
    }

    /// <summary>
    ///     <para>
    ///         Prints the node value using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <param name="sb">
    ///     <para>The sb.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void PrintNodeValue(TLinkAddress node, StringBuilder sb)
    {
        ref var link = ref GetLinkDataPartReference(link: node);
        sb.Append(value: ' ');
        sb.Append(value: link.Source);
        sb.Append(value: '-');
        sb.Append(value: '>');
        sb.Append(value: link.Target);
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/InternalLinksSizeBalancedTreeMethodsBase.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using System.Text;
using Platform.Collections.Methods.Trees;
using Platform.Converters;
using Platform.Delegates;
using static System.Runtime.CompilerServices.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the internal links size balanced tree methods base.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="SizeBalancedTreeMethods{TLinkAddress}" />
/// <seealso cref="ILinksTreeMethods{TLinkAddress}" />
public abstract unsafe class InternalLinksSizeBalancedTreeMethodsBase<TLinkAddress> : SizeBalancedTreeMethods<TLinkAddress>, I
LinksTreeMethods<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAdd
ress, bool>
{

    /// <summary>
    ///     <para>
    ///         The break.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Break;
    /// <summary>
    ///     <para>
    ///         The continue.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Continue;
    /// <summary>
    ///     <para>
    ///         The header.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* Header;
    /// <summary>
    ///     <para>
    ///         The links data parts.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* LinksDataParts;
    /// <summary>
    ///     <para>
    ///         The links index parts.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* LinksIndexParts;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="InternalLinksSizeBalancedTreeMethodsBase{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected InternalLinksSizeBalancedTreeMethodsBase(LinksConstants<TLinkAddress> constants, byte* linksDataParts, byte* lin
ksIndexParts, byte* header)
    {
        LinksDataParts = linksDataParts;
        LinksIndexParts = linksIndexParts;
        Header = header;
```

```
    Break = constants.Break;
    Continue = constants.Continue;
}

/// <summary>
///     <para>
///         The zero.
///     </para>
///     <para></para>
/// </summary>
public TLinkAddress this[TLinkAddress link, TLinkAddress index]
{
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    get
    {
        var root = GetTreeRoot(link: link);
        if ((index >= GetSize(node: root)))
        {
            return TLinkAddress.Zero;
        }
        while (root != TLinkAddress.Zero)
        {
            var left = GetLeftOrDefault(node: root);
            var leftSize = GetSizeOrZero(node: left);
            if (index < leftSize)
            {
                root = left;
                continue;
            }
            if ((index == leftSize))
            {
                return root;
            }
            root = GetRightOrDefault(node: root);
            index = (index) - (leftSize + TLinkAddress.One);
        }
        return TLinkAddress.Zero; // TODO: Impossible situation exception (only if tree structure broken)
    }
}

/// <summary>
///     Выполняет поиск и возвращает индекс связи с указанными Source (началом) и Target (концом).
/// </summary>
/// <param name="source">Индекс связи, которая является началом на искомой связи.</param>
/// <param name="target">Индекс связи, которая является концом на искомой связи.</param>
/// <returns>Индекс искомой связи.</returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
public abstract TLinkAddress Search(TLinkAddress source, TLinkAddress target);

// TODO: Return indices range instead of references count
/// <summary>
///     <para>
///         Counts the usages using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
public TLinkAddress CountUsages(TLinkAddress link)
{
    return GetSizeOrZero(node: GetTreeRoot(link: link));
}
/// <summary>
///     <para>
///         Eaches the usage using the specified base.
///     </para>
///     <para></para>
/// </summary>
/// <param name="base">
///     <para>The base.</para>
///     <para></para>
/// </param>
/// <param name="handler">
///     <para>The handler.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
public TLinkAddress EachUsage(TLinkAddress @base, ReadHandler<TLinkAddress>? handler)
{
    return EachUsageCore(@base: @base, link: GetTreeRoot(link: @base), handler: handler);
}
```

```
    /// <summary>
    ///     <para>
    ///         Gets the tree root using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract TLinkAddress GetTreeRoot(TLinkAddress link);

    /// <summary>
    ///     <para>
    ///         Gets the base part value using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract TLinkAddress GetBasePartValue(TLinkAddress link);

    /// <summary>
    ///     <para>
    ///         Gets the key part value using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract TLinkAddress GetKeyPartValue(TLinkAddress link);

    /// <summary>
    ///     <para>
    ///         Gets the link data part reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link data part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLinkDataPart<TLinkAddress> GetLinkDataPartReference(TLinkAddress link)
    {
        return ref AsRef<RawLinkDataPart<TLinkAddress>>(source: LinksDataParts + RawLinkDataPart<TLinkAddress>.SizeInBytes * l
ong.CreateTruncating(link));
    }

    /// <summary>
    ///     <para>
    ///         Gets the link index part reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link index part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLinkIndexPart<TLinkAddress> GetLinkIndexPartReference(TLinkAddress link)
    {
         return ref AsRef<RawLinkIndexPart<TLinkAddress>>(source: LinksIndexParts + RawLinkIndexPart<TLinkAddress>.SizeInBytes
* long.CreateTruncating(link));
```

```
}

/// <summary>
///     <para>
///         Determines whether this instance first is to the left of second.
///     </para>
///     <para></para>
/// </summary>
/// <param name="first">
///     <para>The first.</para>
///     <para></para>
/// </param>
/// <param name="second">
///     <para>The second.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The bool</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override bool FirstIsToTheLeftOfSecond(TLinkAddress first, TLinkAddress second)
{
    return (GetKeyPartValue(link: first) < GetKeyPartValue(link: second));
}

/// <summary>
///     <para>
///         Determines whether this instance first is to the right of second.
///     </para>
///     <para></para>
/// </summary>
/// <param name="first">
///     <para>The first.</para>
///     <para></para>
/// </param>
/// <param name="second">
///     <para>The second.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The bool</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override bool FirstIsToTheRightOfSecond(TLinkAddress first, TLinkAddress second)
{
    return (GetKeyPartValue(link: first) > GetKeyPartValue(link: second));
}

/// <summary>
///     <para>
///         Gets the link values using the specified link index.
///     </para>
///     <para></para>
/// </summary>
/// <param name="linkIndex">
///     <para>The link index.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>A list of t link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected virtual IList<TLinkAddress>? GetLinkValues(TLinkAddress linkIndex)
{
    ref var link = ref GetLinkDataPartReference(link: linkIndex);
    return new Link<TLinkAddress>(index: linkIndex, source: link.Source, target: link.Target);
}

/// <summary>
///     <para>
///         Searches the core using the specified root.
///     </para>
///     <para></para>
/// </summary>
/// <param name="root">
///     <para>The root.</para>
///     <para></para>
/// </param>
/// <param name="key">
///     <para>The key.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The zero.</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected TLinkAddress SearchCore(TLinkAddress root, TLinkAddress key)
{
```

```
        while (root != TLinkAddress.Zero)
        {
            var rootKey = GetKeyPartValue(link: root);
            if (key < rootKey) // node.Key < root.Key
            {
                root = GetLeftOrDefault(node: root);
            }
            else if (key > rootKey) // node.Key > root.Key
            {
                root = GetRightOrDefault(node: root);
            }
            else // node.Key == root.Key
            {
                return root;
            }
        }
        return TLinkAddress.Zero;
    }

    // TODO: 1. Move target, handler to separate object. 2. Use stack or walker 3. Use low-level MSIL stack.
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    private TLinkAddress EachUsageCore(TLinkAddress @base, TLinkAddress link, ReadHandler<TLinkAddress>? handler)
    {
        var @continue = Continue;
        if (link == TLinkAddress.Zero)
        {
            return @continue;
        }
        var @break = Break;
        if ((EachUsageCore(@base: @base, link: GetLeftOrDefault(node: link), handler: handler) == @break))
        {
            return @break;
        }
        if (((handler == null ? Continue : handler(link: GetLinkValues(linkIndex: link))) == @break))
        {
            return @break;
        }
        if ((EachUsageCore(@base: @base, link: GetRightOrDefault(node: link), handler: handler) == @break))
        {
            return @break;
        }
        return @continue;
    }

    /// <summary>
    ///     <para>
    ///         Prints the node value using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <param name="sb">
    ///     <para>The sb.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void PrintNodeValue(TLinkAddress node, StringBuilder sb)
    {
        ref var link = ref GetLinkDataPartReference(link: node);
        sb.Append(value: ' ');
        sb.Append(value: link.Source);
        sb.Append(value: '-');
        sb.Append(value: '>');
        sb.Append(value: link.Target);
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/InternalLinksSourcesLinkedListMethods.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Collections.Methods.Lists;
using Platform.Converters;
using Platform.Delegates;
using static System.Runtime.CompilerServices.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the internal links sources linked list methods.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="RelativeCircularDoublyLinkedListMethods{TLinkAddress}" />
public unsafe class InternalLinksSourcesLinkedListMethods<TLinkAddress> : RelativeCircularDoublyLinkedListMethods<TLinkAddress
> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
{
    private readonly byte* _linksDataParts;
    private readonly byte* _linksIndexParts;
    /// <summary>
    ///     <para>
    ///         The break.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Break;
    /// <summary>
    ///     <para>
    ///         The continue.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Continue;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="InternalLinksSourcesLinkedListMethods{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public InternalLinksSourcesLinkedListMethods(LinksConstants<TLinkAddress> constants, byte* linksDataParts, byte* linksInde
xParts)
    {
        _linksDataParts = linksDataParts;
        _linksIndexParts = linksIndexParts;
        Break = constants.Break;
        Continue = constants.Continue;
    }

    /// <summary>
    ///     <para>
    ///         Gets the link data part reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link data part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLinkDataPart<TLinkAddress> GetLinkDataPartReference(TLinkAddress link)
    {
         return ref AsRef<RawLinkDataPart<TLinkAddress>>(source: _linksDataParts + RawLinkDataPart<TLinkAddress>.SizeInBytes *
long.CreateTruncating(link));
    }

    /// <summary>
    ///     <para>
    ///         Gets the link index part reference using the specified link.
```

```
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link index part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLinkIndexPart<TLinkAddress> GetLinkIndexPartReference(TLinkAddress link)
    {
        return ref AsRef<RawLinkIndexPart<TLinkAddress>>(source: _linksIndexParts + RawLinkIndexPart<TLinkAddress>.SizeInBytes
* long.CreateTruncating(link));
    }

    /// <summary>
    ///     <para>
    ///         Gets the first using the specified head.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="head">
    ///     <para>The head.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override TLinkAddress GetFirst(TLinkAddress head)
    {
        return GetLinkIndexPartReference(link: head).RootAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Gets the last using the specified head.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="head">
    ///     <para>The head.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override TLinkAddress GetLast(TLinkAddress head)
    {
        var first = GetLinkIndexPartReference(link: head).RootAsSource;
        if (first == TLinkAddress.Zero)
        {
            return first;
        }
        return GetPrevious(element: first);
    }

    /// <summary>
    ///     <para>
    ///         Gets the previous using the specified element.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="element">
    ///     <para>The element.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override TLinkAddress GetPrevious(TLinkAddress element)
    {
        return GetLinkIndexPartReference(link: element).LeftAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Gets the next using the specified element.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="element">
    ///     <para>The element.</para>
    ///     <para></para>
```

```
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetNext(TLinkAddress element)
{
    return GetLinkIndexPartReference(link: element).RightAsSource;
}

/// <summary>
///     <para>
///         Gets the size using the specified head.
///     </para>
///     <para></para>
/// </summary>
/// <param name="head">
///     <para>The head.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetSize(TLinkAddress head)
{
    return GetLinkIndexPartReference(link: head).SizeAsSource;
}

/// <summary>
///     <para>
///         Sets the first using the specified head.
///     </para>
///     <para></para>
/// </summary>
/// <param name="head">
///     <para>The head.</para>
///     <para></para>
/// </param>
/// <param name="element">
///     <para>The element.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetFirst(TLinkAddress head, TLinkAddress element)
{
    GetLinkIndexPartReference(link: head).RootAsSource = element;
}

/// <summary>
///     <para>
///         Sets the last using the specified head.
///     </para>
///     <para></para>
/// </summary>
/// <param name="head">
///     <para>The head.</para>
///     <para></para>
/// </param>
/// <param name="element">
///     <para>The element.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetLast(TLinkAddress head, TLinkAddress element)
{
    //var first = GetLinkIndexPartReference(head).RootAsSource;
    //if (EqualToZero(first))
    //{
    //    SetFirst(head, element);
    //}
    //else
    //{
    //    SetPrevious(first, element);
    //}
}

/// <summary>
///     <para>
///         Sets the previous using the specified element.
///     </para>
///     <para></para>
/// </summary>
/// <param name="element">
///     <para>The element.</para>
///     <para></para>
/// </param>
/// <param name="previous">
///     <para>The previous.</para>
///     <para></para>
```

```
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetPrevious(TLinkAddress element, TLinkAddress previous)
{
    GetLinkIndexPartReference(link: element).LeftAsSource = previous;
}

/// <summary>
///     <para>
///         Sets the next using the specified element.
///     </para>
///     <para></para>
/// </summary>
/// <param name="element">
///     <para>The element.</para>
///     <para></para>
/// </param>
/// <param name="next">
///     <para>The next.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetNext(TLinkAddress element, TLinkAddress next)
{
    GetLinkIndexPartReference(link: element).RightAsSource = next;
}

/// <summary>
///     <para>
///         Sets the size using the specified head.
///     </para>
///     <para></para>
/// </summary>
/// <param name="head">
///     <para>The head.</para>
///     <para></para>
/// </param>
/// <param name="size">
///     <para>The size.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetSize(TLinkAddress head, TLinkAddress size)
{
    GetLinkIndexPartReference(link: head).SizeAsSource = size;
}

/// <summary>
///     <para>
///         Counts the usages using the specified head.
///     </para>
///     <para></para>
/// </summary>
/// <param name="head">
///     <para>The head.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
public TLinkAddress CountUsages(TLinkAddress head)
{
    return GetSize(head: head);
}

/// <summary>
///     <para>
///         Gets the link values using the specified link index.
///     </para>
///     <para></para>
/// </summary>
/// <param name="linkIndex">
///     <para>The link index.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>A list of t link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected virtual IList<TLinkAddress>? GetLinkValues(TLinkAddress linkIndex)
{
    ref var link = ref GetLinkDataPartReference(link: linkIndex);
    return new Link<TLinkAddress>(index: linkIndex, source: link.Source, target: link.Target);
}

/// <summary>
///     <para>
///         Eaches the usage using the specified source.
///     </para>
```

```
    ///     <para></para>
    /// </summary>
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="handler">
    ///     <para>The handler.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The continue.</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public TLinkAddress EachUsage(TLinkAddress source, ReadHandler<TLinkAddress>? handler)
    {
        var @continue = Continue;
        var @break = Break;
        var current = GetFirst(head: source);
        var first = current;
        while (current != TLinkAddress.Zero)
        {
            if (((handler == null ? Continue : handler(link: GetLinkValues(linkIndex: current))) == @break))
            {
                return @break;
            }
            current = GetNext(element: current);
            if ((current == first))
            {
                return @continue;
            }
        }
        return @continue;
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/InternalLinksSourcesRecursionlessSizeBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the internal links sources recursionless size balanced tree methods.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="InternalLinksRecursionlessSizeBalancedTreeMethodsBase{TLinkAddress}" />
public unsafe class InternalLinksSourcesRecursionlessSizeBalancedTreeMethods<TLinkAddress> : InternalLinksRecursionlessSizeBal
ancedTreeMethodsBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLin
kAddress, bool>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="InternalLinksSourcesRecursionlessSizeBalancedTreeMethods{TLinkAddress}" /> instan
ce.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public InternalLinksSourcesRecursionlessSizeBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* linksDataPar
ts, byte* linksIndexParts, byte* header) : base(constants: constants, linksDataParts: linksDataParts, linksIndexParts: linksIn
dexParts, header: header) { }

    /// <summary>
    ///     <para>
    ///         Gets the left reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetLeftReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).LeftAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Gets the right reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetRightReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).RightAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Gets the left using the specified node.
    ///     </para>
    ///     <para></para>
```

```
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetLeft(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).LeftAsSource;
}

/// <summary>
///     <para>
///         Gets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetRight(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).RightAsSource;
}

/// <summary>
///     <para>
///         Sets the left using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="left">
///     <para>The left.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetLeft(TLinkAddress node, TLinkAddress left)
{
    GetLinkIndexPartReference(link: node).LeftAsSource = left;
}

/// <summary>
///     <para>
///         Sets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="right">
///     <para>The right.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetRight(TLinkAddress node, TLinkAddress right)
{
    GetLinkIndexPartReference(link: node).RightAsSource = right;
}

/// <summary>
///     <para>
///         Gets the size using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetSize(TLinkAddress node)
{
```

```
    return GetLinkIndexPartReference(link: node).SizeAsSource;
}

/// <summary>
///     <para>
///         Sets the size using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="size">
///     <para>The size.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetSize(TLinkAddress node, TLinkAddress size)
{
    GetLinkIndexPartReference(link: node).SizeAsSource = size;
}

/// <summary>
///     <para>
///         Gets the tree root using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetTreeRoot(TLinkAddress link)
{
    return GetLinkIndexPartReference(link: link).RootAsSource;
}

/// <summary>
///     <para>
///         Gets the base part value using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetBasePartValue(TLinkAddress link)
{
    return GetLinkDataPartReference(link: link).Source;
}

/// <summary>
///     <para>
///         Gets the key part value using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetKeyPartValue(TLinkAddress link)
{
    return GetLinkDataPartReference(link: link).Target;
}

/// <summary>
///     <para>
///         Clears the node using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
```

```
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void ClearNode(TLinkAddress node)
    {
        ref var link = ref GetLinkIndexPartReference(link: node);
        link.LeftAsSource = TLinkAddress.Zero;
        link.RightAsSource = TLinkAddress.Zero;
        link.SizeAsSource = TLinkAddress.Zero;
    }

    /// <summary>
    ///     <para>
    ///         Searches the source.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="target">
    ///     <para>The target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    public override TLinkAddress Search(TLinkAddress source, TLinkAddress target)
    {
        return SearchCore(root: GetTreeRoot(link: source), key: target);
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/InternalLinksSourcesSizeBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the internal links sources size balanced tree methods.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="InternalLinksSizeBalancedTreeMethodsBase{TLinkAddress}" />
public unsafe class InternalLinksSourcesSizeBalancedTreeMethods<TLinkAddress> : InternalLinksSizeBalancedTreeMethodsBase<TLink
Address> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="InternalLinksSourcesSizeBalancedTreeMethods{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public InternalLinksSourcesSizeBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* linksDataParts, byte* lin
ksIndexParts, byte* header) : base(constants: constants, linksDataParts: linksDataParts, linksIndexParts: linksIndexParts, hea
der: header) { }

    /// <summary>
    ///     <para>
    ///         Gets the left reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetLeftReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).LeftAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Gets the right reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetRightReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).RightAsSource;
    }

    /// <summary>
    ///     <para>
    ///         Gets the left using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
```

```
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetLeft(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).LeftAsSource;
}

/// <summary>
///     <para>
///         Gets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetRight(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).RightAsSource;
}

/// <summary>
///     <para>
///         Sets the left using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="left">
///     <para>The left.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetLeft(TLinkAddress node, TLinkAddress left)
{
    GetLinkIndexPartReference(link: node).LeftAsSource = left;
}

/// <summary>
///     <para>
///         Sets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="right">
///     <para>The right.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetRight(TLinkAddress node, TLinkAddress right)
{
    GetLinkIndexPartReference(link: node).RightAsSource = right;
}

/// <summary>
///     <para>
///         Gets the size using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetSize(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).SizeAsSource;
}
```

```
/// <summary>
///     <para>
///         Sets the size using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="size">
///     <para>The size.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetSize(TLinkAddress node, TLinkAddress size)
{
    GetLinkIndexPartReference(link: node).SizeAsSource = size;
}

/// <summary>
///     <para>
///         Gets the tree root using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetTreeRoot(TLinkAddress link)
{
    return GetLinkIndexPartReference(link: link).RootAsSource;
}

/// <summary>
///     <para>
///         Gets the base part value using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetBasePartValue(TLinkAddress link)
{
    return GetLinkDataPartReference(link: link).Source;
}

/// <summary>
///     <para>
///         Gets the key part value using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetKeyPartValue(TLinkAddress link)
{
    return GetLinkDataPartReference(link: link).Target;
}

/// <summary>
///     <para>
///         Clears the node using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
```

```
    protected override void ClearNode(TLinkAddress node)
    {
        ref var link = ref GetLinkIndexPartReference(link: node);
        link.LeftAsSource = TLinkAddress.Zero;
        link.RightAsSource = TLinkAddress.Zero;
        link.SizeAsSource = TLinkAddress.Zero;
    }

    /// <summary>
    ///     <para>
    ///         Searches the source.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="target">
    ///     <para>The target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    public override TLinkAddress Search(TLinkAddress source, TLinkAddress target)
    {
        return SearchCore(root: GetTreeRoot(link: source), key: target);
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/InternalLinksTargetsRecursionlessSizeBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the internal links targets recursionless size balanced tree methods.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="InternalLinksRecursionlessSizeBalancedTreeMethodsBase{TLinkAddress}" />
public unsafe class InternalLinksTargetsRecursionlessSizeBalancedTreeMethods<TLinkAddress> : InternalLinksRecursionlessSizeBal
ancedTreeMethodsBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLin
kAddress, bool>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="InternalLinksTargetsRecursionlessSizeBalancedTreeMethods{TLinkAddress}" /> instan
ce.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public InternalLinksTargetsRecursionlessSizeBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* linksDataPar
ts, byte* linksIndexParts, byte* header) : base(constants: constants, linksDataParts: linksDataParts, linksIndexParts: linksIn
dexParts, header: header) { }

    /// <summary>
    ///     <para>
    ///         Gets the left reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetLeftReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).LeftAsTarget;
    }

    /// <summary>
    ///     <para>
    ///         Gets the right reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetRightReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).RightAsTarget;
    }

    /// <summary>
    ///     <para>
    ///         Gets the left using the specified node.
    ///     </para>
    ///     <para></para>
```

```
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetLeft(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).LeftAsTarget;
}

/// <summary>
///     <para>
///         Gets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetRight(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).RightAsTarget;
}

/// <summary>
///     <para>
///         Sets the left using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="left">
///     <para>The left.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetLeft(TLinkAddress node, TLinkAddress left)
{
    GetLinkIndexPartReference(link: node).LeftAsTarget = left;
}

/// <summary>
///     <para>
///         Sets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="right">
///     <para>The right.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetRight(TLinkAddress node, TLinkAddress right)
{
    GetLinkIndexPartReference(link: node).RightAsTarget = right;
}

/// <summary>
///     <para>
///         Gets the size using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetSize(TLinkAddress node)
{
```

```
    return GetLinkIndexPartReference(link: node).SizeAsTarget;
}

/// <summary>
///     <para>
///         Sets the size using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="size">
///     <para>The size.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetSize(TLinkAddress node, TLinkAddress size)
{
    GetLinkIndexPartReference(link: node).SizeAsTarget = size;
}

/// <summary>
///     <para>
///         Gets the tree root using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetTreeRoot(TLinkAddress link)
{
    return GetLinkIndexPartReference(link: link).RootAsTarget;
}

/// <summary>
///     <para>
///         Gets the base part value using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetBasePartValue(TLinkAddress link)
{
    return GetLinkDataPartReference(link: link).Target;
}

/// <summary>
///     <para>
///         Gets the key part value using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetKeyPartValue(TLinkAddress link)
{
    return GetLinkDataPartReference(link: link).Source;
}

/// <summary>
///     <para>
///         Clears the node using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
```

```
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void ClearNode(TLinkAddress node)
    {
        ref var link = ref GetLinkIndexPartReference(link: node);
        link.LeftAsTarget = TLinkAddress.Zero;
        link.RightAsTarget = TLinkAddress.Zero;
        link.SizeAsTarget = TLinkAddress.Zero;
    }

    /// <summary>
    ///     <para>
    ///         Searches the source.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="target">
    ///     <para>The target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    public override TLinkAddress Search(TLinkAddress source, TLinkAddress target)
    {
        return SearchCore(root: GetTreeRoot(link: target), key: source);
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/InternalLinksTargetsSizeBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the internal links targets size balanced tree methods.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="InternalLinksSizeBalancedTreeMethodsBase{TLinkAddress}" />
public unsafe class InternalLinksTargetsSizeBalancedTreeMethods<TLinkAddress> : InternalLinksSizeBalancedTreeMethodsBase<TLink
Address> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
{
    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="InternalLinksTargetsSizeBalancedTreeMethods{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksDataParts">
    ///     <para>A links data parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="linksIndexParts">
    ///     <para>A links index parts.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public InternalLinksTargetsSizeBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* linksDataParts, byte* lin
ksIndexParts, byte* header) : base(constants: constants, linksDataParts: linksDataParts, linksIndexParts: linksIndexParts, hea
der: header) { }

    /// <summary>
    ///     <para>
    ///         Gets the left reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetLeftReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).LeftAsTarget;
    }

    /// <summary>
    ///     <para>
    ///         Gets the right reference using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The ref link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref TLinkAddress GetRightReference(TLinkAddress node)
    {
        return ref GetLinkIndexPartReference(link: node).RightAsTarget;
    }

    /// <summary>
    ///     <para>
    ///         Gets the left using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
```

```
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetLeft(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).LeftAsTarget;
}

/// <summary>
///     <para>
///         Gets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetRight(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).RightAsTarget;
}

/// <summary>
///     <para>
///         Sets the left using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="left">
///     <para>The left.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetLeft(TLinkAddress node, TLinkAddress left)
{
    GetLinkIndexPartReference(link: node).LeftAsTarget = left;
}

/// <summary>
///     <para>
///         Sets the right using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="right">
///     <para>The right.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetRight(TLinkAddress node, TLinkAddress right)
{
    GetLinkIndexPartReference(link: node).RightAsTarget = right;
}

/// <summary>
///     <para>
///         Gets the size using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetSize(TLinkAddress node)
{
    return GetLinkIndexPartReference(link: node).SizeAsTarget;
}
```

```
/// <summary>
///     <para>
///         Sets the size using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
/// <param name="size">
///     <para>The size.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetSize(TLinkAddress node, TLinkAddress size)
{
    GetLinkIndexPartReference(link: node).SizeAsTarget = size;
}

/// <summary>
///     <para>
///         Gets the tree root using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetTreeRoot(TLinkAddress link)
{
    return GetLinkIndexPartReference(link: link).RootAsTarget;
}

/// <summary>
///     <para>
///         Gets the base part value using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetBasePartValue(TLinkAddress link)
{
    return GetLinkDataPartReference(link: link).Target;
}

/// <summary>
///     <para>
///         Gets the key part value using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetKeyPartValue(TLinkAddress link)
{
    return GetLinkDataPartReference(link: link).Source;
}

/// <summary>
///     <para>
///         Clears the node using the specified node.
///     </para>
///     <para></para>
/// </summary>
/// <param name="node">
///     <para>The node.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
```

```
    protected override void ClearNode(TLinkAddress node)
    {
        ref var link = ref GetLinkIndexPartReference(link: node);
        link.LeftAsTarget = TLinkAddress.Zero;
        link.RightAsTarget = TLinkAddress.Zero;
        link.SizeAsTarget = TLinkAddress.Zero;
    }

    /// <summary>
    ///     <para>
    ///         Searches the source.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="target">
    ///     <para>The target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    public override TLinkAddress Search(TLinkAddress source, TLinkAddress target)
    {
        return SearchCore(root: GetTreeRoot(link: target), key: source);
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/SplitMemoryLinks.cs

```
using System;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Memory;
using Platform.Singletons;
using static System.Runtime.CompilerServices.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the split memory links.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="SplitMemoryLinksBase{TLinkAddress}" />
public unsafe class SplitMemoryLinks<TLinkAddress> : SplitMemoryLinksBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<T
LinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
{
    private readonly Func<ILinksTreeMethods<TLinkAddress>> _createExternalSourceTreeMethods;
    private readonly Func<ILinksTreeMethods<TLinkAddress>> _createExternalTargetTreeMethods;
    private readonly Func<ILinksTreeMethods<TLinkAddress>> _createInternalSourceTreeMethods;
    private readonly Func<ILinksTreeMethods<TLinkAddress>> _createInternalTargetTreeMethods;
    private byte* _header;
    private byte* _linksDataParts;
    private byte* _linksIndexParts;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="SplitMemoryLinks{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="dataMemory">
    ///     <para>A data memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="indexMemory">
    ///     <para>A index memory.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public SplitMemoryLinks(string dataMemory, string indexMemory) : this(dataMemory: new FileMappedResizableDirectMemory(path
: dataMemory), indexMemory: new FileMappedResizableDirectMemory(path: indexMemory)) { }

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="SplitMemoryLinks{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="dataMemory">
    ///     <para>A data memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="indexMemory">
    ///     <para>A index memory.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public SplitMemoryLinks(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory) : this(dataMemory: dataMemo
ry, indexMemory: indexMemory, memoryReservationStep: DefaultLinksSizeStep) { }

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="SplitMemoryLinks{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="dataMemory">
    ///     <para>A data memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="indexMemory">
    ///     <para>A index memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="memoryReservationStep">
    ///     <para>A memory reservation step.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public SplitMemoryLinks(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory, long memoryReservationStep)
: this(dataMemory: dataMemory, indexMemory: indexMemory, memoryReservationStep: memoryReservationStep, constants: Default<Link
sConstants<TLinkAddress>>.Instance, indexTreeType: IndexTreeType.Default, useLinkedList: true) { }

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="SplitMemoryLinks{TLinkAddress}" /> instance.
```

```
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="dataMemory">
    ///     <para>A data memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="indexMemory">
    ///     <para>A index memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="memoryReservationStep">
    ///     <para>A memory reservation step.</para>
    ///     <para></para>
    /// </param>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public SplitMemoryLinks(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory, long memoryReservationStep,
LinksConstants<TLinkAddress> constants) : this(dataMemory: dataMemory, indexMemory: indexMemory, memoryReservationStep: memory
ReservationStep, constants: constants, indexTreeType: IndexTreeType.Default, useLinkedList: true) { }

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="SplitMemoryLinks{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="dataMemory">
    ///     <para>A data memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="indexMemory">
    ///     <para>A index memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="memoryReservationStep">
    ///     <para>A memory reservation step.</para>
    ///     <para></para>
    /// </param>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="indexTreeType">
    ///     <para>A index tree type.</para>
    ///     <para></para>
    /// </param>
    /// <param name="useLinkedList">
    ///     <para>A use linked list.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public SplitMemoryLinks(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory, long memoryReservationStep,
LinksConstants<TLinkAddress> constants, IndexTreeType indexTreeType, bool useLinkedList) : base(dataMemory: dataMemory, indexM
emory: indexMemory, memoryReservationStep: memoryReservationStep, constants: constants, useLinkedList: useLinkedList)
    {
        if (indexTreeType == IndexTreeType.SizeBalancedTree)
        {
            _createInternalSourceTreeMethods = () => new InternalLinksSourcesSizeBalancedTreeMethods<TLinkAddress>(constants:
_constants, linksDataParts: _linksDataParts, linksIndexParts: _linksIndexParts, header: _header);
            _createExternalSourceTreeMethods = () => new ExternalLinksSourcesSizeBalancedTreeMethods<TLinkAddress>(constants:
_constants, linksDataParts: _linksDataParts, linksIndexParts: _linksIndexParts, header: _header);
            _createInternalTargetTreeMethods = () => new InternalLinksTargetsSizeBalancedTreeMethods<TLinkAddress>(constants:
_constants, linksDataParts: _linksDataParts, linksIndexParts: _linksIndexParts, header: _header);
            _createExternalTargetTreeMethods = () => new ExternalLinksTargetsSizeBalancedTreeMethods<TLinkAddress>(constants:
_constants, linksDataParts: _linksDataParts, linksIndexParts: _linksIndexParts, header: _header);
        }
        else
        {
            _createInternalSourceTreeMethods = () => new InternalLinksSourcesRecursionlessSizeBalancedTreeMethods<TLinkAddress
>(constants: _constants, linksDataParts: _linksDataParts, linksIndexParts: _linksIndexParts, header: _header);
            _createExternalSourceTreeMethods = () => new ExternalLinksSourcesRecursionlessSizeBalancedTreeMethods<TLinkAddress
>(constants: _constants, linksDataParts: _linksDataParts, linksIndexParts: _linksIndexParts, header: _header);
            _createInternalTargetTreeMethods = () => new InternalLinksTargetsRecursionlessSizeBalancedTreeMethods<TLinkAddress
>(constants: _constants, linksDataParts: _linksDataParts, linksIndexParts: _linksIndexParts, header: _header);
            _createExternalTargetTreeMethods = () => new ExternalLinksTargetsRecursionlessSizeBalancedTreeMethods<TLinkAddress
>(constants: _constants, linksDataParts: _linksDataParts, linksIndexParts: _linksIndexParts, header: _header);
        }
        InitializeMemory(dataMemory, indexMemory, () => SetPointersCore(dataMemory, indexMemory));
    }

    /// <summary>
    ///     <para>
    ///         Sets the pointers using the specified data memory.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="dataMemory">
    ///     <para>The data memory.</para>
```

```
    ///     <para></para>
    /// </param>
    /// <param name="indexMemory">
    ///     <para>The index memory.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void SetPointers(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory) => SetPointersC
ore(dataMemory, indexMemory);

    private void SetPointersCore(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory)
    {
        _linksDataParts = (byte*)dataMemory.Pointer;
        _linksIndexParts = (byte*)indexMemory.Pointer;
        _header = _linksIndexParts;
        if (_useLinkedList)
        {
            InternalSourcesListMethods = new InternalLinksSourcesLinkedListMethods<TLinkAddress>(constants: _constants, linksD
ataParts: _linksDataParts, linksIndexParts: _linksIndexParts);
        }
        else
        {
            InternalSourcesTreeMethods = _createInternalSourceTreeMethods();
        }
        ExternalSourcesTreeMethods = _createExternalSourceTreeMethods();
        InternalTargetsTreeMethods = _createInternalTargetTreeMethods();
        ExternalTargetsTreeMethods = _createExternalTargetTreeMethods();
        UnusedLinksListMethods = new UnusedLinksListMethods<TLinkAddress>(links: _linksDataParts, header: _header);
    }

    /// <summary>
    ///     <para>
    ///         Resets the pointers.
    ///     </para>
    ///     <para></para>
    /// </summary>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void ResetPointers()
    {
        base.ResetPointers();
        _linksDataParts = null;
        _linksIndexParts = null;
        _header = null;
    }

    /// <summary>
    ///     <para>
    ///         Gets the header reference.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>A ref links header of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref LinksHeader<TLinkAddress> GetHeaderReference()
    {
        return ref AsRef<LinksHeader<TLinkAddress>>(source: _header);
    }

    /// <summary>
    ///     <para>
    ///         Gets the link data part reference using the specified link index.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="linkIndex">
    ///     <para>The link index.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link data part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref RawLinkDataPart<TLinkAddress> GetLinkDataPartReference(TLinkAddress linkIndex)
    {
        return ref AsRef<RawLinkDataPart<TLinkAddress>>(source: _linksDataParts + LinkDataPartSizeInBytes * long.CreateTruncat
ing(value: linkIndex));
    }

    /// <summary>
    ///     <para>
    ///         Gets the link index part reference using the specified link index.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="linkIndex">
    ///     <para>The link index.</para>
    ///     <para></para>
    /// </param>
```

```
    /// <returns>
    ///     <para>A ref raw link index part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override ref RawLinkIndexPart<TLinkAddress> GetLinkIndexPartReference(TLinkAddress linkIndex)
    {
        return ref AsRef<RawLinkIndexPart<TLinkAddress>>(source: _linksIndexParts + LinkIndexPartSizeInBytes * long.CreateTrun
cating(value: linkIndex));
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/SplitMemoryLinksBase.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Converters;
using Platform.Data.Exceptions;
using Platform.Delegates;
using Platform.Disposables;
using Platform.Memory;
using Platform.Singletons;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the split memory links base.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="DisposableBase" />
/// <seealso cref="ILinks{TLinkAddress}" />
public abstract class SplitMemoryLinksBase<TLinkAddress> : DisposableBase, ILinks<TLinkAddress> where TLinkAddress : IUnsigned
Number<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
{
    private static readonly Comparer<TLinkAddress> _comparer = Comparer<TLinkAddress>.Default;
    private static readonly TLinkAddress _zero = TLinkAddress.Zero;
    private static readonly TLinkAddress _one = TLinkAddress.One;

    /// <summary>Возвращает размер одной связи в байтах.</summary>
    /// <remarks>
    ///     Используется только во вне класса, не рекомедуется использовать внутри.
    ///     Так как во вне не обязательно будет доступен unsafe C#.
    /// </remarks>
    public static readonly long LinkDataPartSizeInBytes = RawLinkDataPart<TLinkAddress>.SizeInBytes;

    /// <summary>
    ///     <para>
    ///         The size in bytes.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public static readonly long LinkIndexPartSizeInBytes = RawLinkIndexPart<TLinkAddress>.SizeInBytes;

    /// <summary>
    ///     <para>
    ///         The size in bytes.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public static readonly long LinkHeaderSizeInBytes = LinksHeader<TLinkAddress>.SizeInBytes;

    /// <summary>
    ///     <para>
    ///         The TLinkAddress.Zero links size step.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public static readonly long DefaultLinksSizeStep = 1 * 1024 * 1024;

    /// <summary>
    ///     <para>
    ///         The data memory.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly IResizableDirectMemory _dataMemory;
    /// <summary>
    ///     <para>
    ///         The data memory reservation step in bytes.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly long _dataMemoryReservationStepInBytes;
    /// <summary>
    ///     <para>
    ///         The index memory.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly IResizableDirectMemory _indexMemory;
    /// <summary>
    ///     <para>
    ///         The index memory reservation step in bytes.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly long _indexMemoryReservationStepInBytes;
    /// <summary>
    ///     <para>
```

```
    ///         The use linked list.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly bool _useLinkedList;
    /// <summary>
    ///     <para>
    ///         The external sources tree methods.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected ILinksTreeMethods<TLinkAddress>? ExternalSourcesTreeMethods;
    private ILinksTreeMethods<TLinkAddress> RequiredExternalSourcesTreeMethods => ExternalSourcesTreeMethods ?? throw new Obje
ctDisposedException(GetType().Name);
    /// <summary>
    ///     <para>
    ///         The external targets tree methods.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected ILinksTreeMethods<TLinkAddress>? ExternalTargetsTreeMethods;
    private ILinksTreeMethods<TLinkAddress> RequiredExternalTargetsTreeMethods => ExternalTargetsTreeMethods ?? throw new Obje
ctDisposedException(GetType().Name);

    /// <summary>
    ///     <para>
    ///         The internal sources list methods.
    ///     </para>
    ///     <para></para>
    /// </summary>
    private InternalLinksSourcesLinkedListMethods<TLinkAddress>? _internalSourcesListMethods;
    protected InternalLinksSourcesLinkedListMethods<TLinkAddress> InternalSourcesListMethods
    {
        get => _internalSourcesListMethods ?? throw new System.ObjectDisposedException(GetType().Name);
        set => _internalSourcesListMethods = value;
    }
    /// <summary>
    ///     <para>
    ///         The internal sources tree methods.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected ILinksTreeMethods<TLinkAddress>? InternalSourcesTreeMethods;
    private ILinksTreeMethods<TLinkAddress> RequiredInternalSourcesTreeMethods => InternalSourcesTreeMethods ?? throw new Obje
ctDisposedException(GetType().Name);
    /// <summary>
    ///     <para>
    ///         The internal targets tree methods.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected ILinksTreeMethods<TLinkAddress>? InternalTargetsTreeMethods;
    private ILinksTreeMethods<TLinkAddress> RequiredInternalTargetsTreeMethods => InternalTargetsTreeMethods ?? throw new Obje
ctDisposedException(GetType().Name);
    // TODO: Возможно чтобы гарантированно проверять на то, является ли связь удалённой, нужно использовать не список а дерево
, так как так можно быстрее проверить на наличие связи внутри
    /// <summary>
    ///     <para>
    ///         The unused links list methods.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected ILinksListMethods<TLinkAddress>? UnusedLinksListMethods;
    private ILinksListMethods<TLinkAddress> RequiredUnusedLinksListMethods => UnusedLinksListMethods ?? throw new ObjectDispos
edException(GetType().Name);

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="SplitMemoryLinksBase{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="dataMemory">
    ///     <para>A data memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="indexMemory">
    ///     <para>A index memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="memoryReservationStep">
    ///     <para>A memory reservation step.</para>
    ///     <para></para>
    /// </param>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="useLinkedList">
    ///     <para>A use linked list.</para>
    ///     <para></para>
    /// </param>
```

```
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected SplitMemoryLinksBase(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory, long memoryReservati
onStep, LinksConstants<TLinkAddress> constants, bool useLinkedList)
    {
        _dataMemory = dataMemory;
        _indexMemory = indexMemory;
        _dataMemoryReservationStepInBytes = memoryReservationStep * LinkDataPartSizeInBytes;
        _indexMemoryReservationStepInBytes = memoryReservationStep * LinkIndexPartSizeInBytes;
        _useLinkedList = useLinkedList;
        _constants = constants;
    }

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="SplitMemoryLinksBase{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="dataMemory">
    ///     <para>A data memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="indexMemory">
    ///     <para>A index memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="memoryReservationStep">
    ///     <para>A memory reservation step.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected SplitMemoryLinksBase(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory, long memoryReservati
onStep) : this(dataMemory: dataMemory, indexMemory: indexMemory, memoryReservationStep: memoryReservationStep, constants: Defa
ult<LinksConstants<TLinkAddress>>.Instance, useLinkedList: true) { }

    /// <summary>
    ///     Возвращает общее число связей находящихся в хранилище.
    /// </summary>
    protected virtual TLinkAddress Total
    {
        [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
        get
        {
            ref var header = ref GetHeaderReference();
            return (header.AllocatedLinks) - (header.FreeLinks);
        }
    }

    /// <summary>
    ///     <para>
    ///         Gets the constants value.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly LinksConstants<TLinkAddress> _constants;

    /// <summary>Gets the configured constants for the store.</summary>
    public virtual LinksConstants<TLinkAddress> Constants => _constants;

    /// <summary>
    ///     <para>
    ///         Counts the substitution.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The query restriction. Null or an empty list means all links.</para>
    ///     <para></para>
    /// </param>
    /// <exception cref="NotSupportedException">
    ///     <para>Other restriction sizes and methods are not supported.</para>
    ///     <para></para>
    /// </exception>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public virtual TLinkAddress Count(IList<TLinkAddress>? restriction)
    {
        restriction ??= System.Array.Empty<TLinkAddress>();
        // Если нет ограничений, тогда возвращаем общее число связей находящихся в хранилище.
        if (restriction is null || restriction.Count == 0)
        {
            return Total;
        }
        var constants = Constants;
        var any = constants.Any;
        var index = this.GetIndex(link: restriction);
        if (restriction.Count == 1)
        {
            if ((index == any))
```

```
            {
                return Total;
            }
            return Exists(link: index) ? GetOne() : GetZero();
        }
        if (restriction.Count == 2)
        {
            var value = restriction[index: 1];
            if ((index == any))
            {
                if ((value == any))
                {
                    return Total; // Any - как отсутствие ограничения
                }
                var count = TLinkAddress.Zero;
                if (value == constants.Null)
                {
                    // Null endpoints are not indexed as internal usages.
                    Each(restriction, _ =>
                    {
                        count += TLinkAddress.One;
                        return constants.Continue;
                    });
                    return count;
                }
                count = Count(new Link<TLinkAddress>(index, value, any));
                // The target pass adds only records absent from the source pass.
                Each(new Link<TLinkAddress>(index, any, value), link =>
                {
                    if (this.GetSource(link) != value)
                    {
                        count += TLinkAddress.One;
                    }
                    return constants.Continue;
                });
                return count;
            }
            if (!Exists(link: index))
            {
                return GetZero();
            }
            if ((value == any))
            {
                return GetOne();
            }
            ref var storedLinkValue = ref GetLinkDataPartReference(linkIndex: index);
            if ((storedLinkValue.Source == value) || (storedLinkValue.Target == value))
            {
                return GetOne();
            }
            return GetZero();
        }
        if (restriction.Count == 3)
        {
            var externalReferencesRange = constants.ExternalReferencesRange;
            var source = this.GetSource(link: restriction);
            var target = this.GetTarget(link: restriction);
            if ((index == any))
            {
                if ((source == any) && (target == any))
                {
                    return Total;
                }
                if ((source == any))
                {
                    if (externalReferencesRange.HasValue && externalReferencesRange.Value.Contains(value: target))
                    {
                        return RequiredExternalTargetsTreeMethods.CountUsages(root: target);
                    }
                    return RequiredInternalTargetsTreeMethods.CountUsages(root: target);
                }
                if ((target == any))
                {
                    if (externalReferencesRange.HasValue && externalReferencesRange.Value.Contains(value: source))
                    {
                        return RequiredExternalSourcesTreeMethods.CountUsages(root: source);
                    }
                    if (_useLinkedList)
                    {
                        return InternalSourcesListMethods.CountUsages(head: source);
                    }
                    return RequiredInternalSourcesTreeMethods.CountUsages(root: source);
                }
                //if(source != Any && target != Any)
                // Эквивалент Exists(source, target) => Count(Any, source, target) > 0
                TLinkAddress link;
                if (externalReferencesRange.HasValue)
                {
                    if (externalReferencesRange.Value.Contains(value: source) && externalReferencesRange.Value.Contains(value:
target))
                    {
                        link = RequiredExternalSourcesTreeMethods.Search(source: source, target: target);
```

```
                    }
                    else if (externalReferencesRange.Value.Contains(value: source))
                    {
                        link = RequiredInternalTargetsTreeMethods.Search(source: source, target: target);
                    }
                    else if (externalReferencesRange.Value.Contains(value: target))
                    {
                        // The internal source has a list; the external target has a searchable tree.
                        link = _useLinkedList
                            ? RequiredExternalTargetsTreeMethods.Search(source: source, target: target)
                            : RequiredInternalSourcesTreeMethods.Search(source: source, target: target);
                    }
                    else
                    {
                        link = (_useLinkedList || (RequiredInternalSourcesTreeMethods.CountUsages(root: source) > RequiredInte
rnalTargetsTreeMethods.CountUsages(root: target)))
                        ? RequiredInternalTargetsTreeMethods.Search(source: source, target: target)
                        : RequiredInternalSourcesTreeMethods.Search(source: source, target: target);
                    }
                }
                else
                {
                    link = (_useLinkedList || (RequiredInternalSourcesTreeMethods.CountUsages(root: source) > RequiredInternal
TargetsTreeMethods.CountUsages(root: target)))
                        ? RequiredInternalTargetsTreeMethods.Search(source: source, target: target)
                        : RequiredInternalSourcesTreeMethods.Search(source: source, target: target);
                }
                return (link == constants.Null) ? GetZero() : GetOne();
            }
            if (!Exists(link: index))
            {
                return GetZero();
            }
            if ((source == any) && (target == any))
            {
                return GetOne();
            }
            ref var storedLinkValue = ref GetLinkDataPartReference(linkIndex: index);
            // Three-part restrictions match each endpoint in its own position.
            if ((source == any || storedLinkValue.Source == source) &&
                (target == any || storedLinkValue.Target == target))
            {
                return GetOne();
            }
            return GetZero();
        }
        throw new NotSupportedException(message: "Other restriction sizes and methods are not supported.");
    }

    /// <summary>
    ///     <para>
    ///         Eaches the handler.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="handler">
    ///     <para>The handler.</para>
    ///     <para></para>
    /// </param>
    /// <param name="restriction">
    ///     <para>The query restriction. Null or an empty list means all links.</para>
    ///     <para></para>
    /// </param>
    /// <exception cref="NotSupportedException">
    ///     <para>Other restriction sizes and methods are not supported.</para>
    ///     <para></para>
    /// </exception>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public virtual TLinkAddress Each(IList<TLinkAddress>? restriction, ReadHandler<TLinkAddress>? handler)
    {
        restriction ??= System.Array.Empty<TLinkAddress>();
        handler ??= _ => Constants.Continue;
        var constants = Constants;
        var @break = constants.Break;
        if (restriction is null || restriction.Count == 0)
        {
            for (var link = GetOne(); (link <= GetHeaderReference().AllocatedLinks); link = link + TLinkAddress.One)
            {
                if (Exists(link: link) && (handler(link: GetLinkStruct(linkIndex: link)) == @break))
                {
                    return @break;
                }
            }
            return constants.Continue;
        }
        var @continue = constants.Continue;
        var any = constants.Any;
        var index = this.GetIndex(link: restriction);
```

```
        if (restriction.Count == 1)
        {
            if ((index == any))
            {
                return Each(restriction: Array.Empty<TLinkAddress>(), handler: handler);
            }
            if (!Exists(link: index))
            {
                return @continue;
            }
            return handler(link: GetLinkStruct(linkIndex: index));
        }
        if (restriction.Count == 2)
        {
            var value = restriction[index: 1];
            if ((index == any))
            {
                if ((value == any))
                {
                    return Each(restriction: Array.Empty<TLinkAddress>(), handler: handler);
                }
                if (value == constants.Null)
                {
                    return Each(Array.Empty<TLinkAddress>(), link =>
                        this.GetSource(link) == value || this.GetTarget(link) == value ? handler(link) : @continue);
                }
                 if ((Each(restriction: new Link<TLinkAddress>(index: index, source: value, target: any), handler: handler) ==
@break))
                {
                    return @break;
                }
                return Each(restriction: new Link<TLinkAddress>(index: index, source: any, target: value), handler: link =>
                    this.GetSource(link) == value ? @continue : handler(link));
            }
            if (!Exists(link: index))
            {
                return @continue;
            }
            if ((value == any))
            {
                return handler(link: GetLinkStruct(linkIndex: index));
            }
            ref var storedLinkValue = ref GetLinkDataPartReference(linkIndex: index);
            if ((storedLinkValue.Source == value) || (storedLinkValue.Target == value))
            {
                return handler(link: GetLinkStruct(linkIndex: index));
            }
            return @continue;
        }
        if (restriction.Count == 3)
        {
            var externalReferencesRange = constants.ExternalReferencesRange;
            var source = this.GetSource(link: restriction);
            var target = this.GetTarget(link: restriction);
            if ((index == any))
            {
                if ((source == any) && (target == any))
                {
                    return Each(restriction: Array.Empty<TLinkAddress>(), handler: handler);
                }
                if ((source == any))
                {
                    if (externalReferencesRange.HasValue && externalReferencesRange.Value.Contains(value: target))
                    {
                        return RequiredExternalTargetsTreeMethods.EachUsage(root: target, handler: handler);
                    }
                    return RequiredInternalTargetsTreeMethods.EachUsage(root: target, handler: handler);
                }
                if ((target == any))
                {
                    if (externalReferencesRange.HasValue && externalReferencesRange.Value.Contains(value: source))
                    {
                        return RequiredExternalSourcesTreeMethods.EachUsage(root: source, handler: handler);
                    }
                    if (_useLinkedList)
                    {
                        return InternalSourcesListMethods.EachUsage(source: source, handler: handler);
                    }
                    return RequiredInternalSourcesTreeMethods.EachUsage(root: source, handler: handler);
                }
                //if(source != Any && target != Any)
                TLinkAddress link;
                if (externalReferencesRange.HasValue)
                {
                     if (externalReferencesRange.Value.Contains(value: source) && externalReferencesRange.Value.Contains(value:
target))
                    {
                        link = RequiredExternalSourcesTreeMethods.Search(source: source, target: target);
                    }
                    else if (externalReferencesRange.Value.Contains(value: source))
                    {
                        link = RequiredInternalTargetsTreeMethods.Search(source: source, target: target);
```

```
                    }
                    else if (externalReferencesRange.Value.Contains(value: target))
                    {
                        // The internal source has a list; the external target has a searchable tree.
                        link = _useLinkedList
                            ? RequiredExternalTargetsTreeMethods.Search(source: source, target: target)
                            : RequiredInternalSourcesTreeMethods.Search(source: source, target: target);
                    }
                    else
                    {
                        link = (_useLinkedList || (RequiredInternalSourcesTreeMethods.CountUsages(root: source) > RequiredInte
rnalTargetsTreeMethods.CountUsages(root: target)))
                            ? RequiredInternalTargetsTreeMethods.Search(source: source, target: target)
                            : RequiredInternalSourcesTreeMethods.Search(source: source, target: target);
                    }
                }
                else
                {
                    link = (_useLinkedList || (RequiredInternalSourcesTreeMethods.CountUsages(root: source) > RequiredInternal
TargetsTreeMethods.CountUsages(root: target)))
                        ? RequiredInternalTargetsTreeMethods.Search(source: source, target: target)
                        : RequiredInternalSourcesTreeMethods.Search(source: source, target: target);
                }
                return (link == constants.Null) ? @continue : handler(link: GetLinkStruct(linkIndex: link));
            }
            if (!Exists(link: index))
            {
                return @continue;
            }
            if ((source == any) && (target == any))
            {
                return handler(link: GetLinkStruct(linkIndex: index));
            }
            ref var storedLinkValue = ref GetLinkDataPartReference(linkIndex: index);
            // Three-part restrictions match each endpoint in its own position.
            if ((source == any || storedLinkValue.Source == source) &&
                (target == any || storedLinkValue.Target == target))
            {
                return handler(link: GetLinkStruct(linkIndex: index));
            }
            return @continue;
        }
        throw new NotSupportedException(message: "Other restriction sizes and methods are not supported.");
    }

    /// <summary>Updates the link that matches the restriction.</summary>
    /// <remarks>
    /// The caller must supply a live internal address and valid endpoint/index state.
    /// This raw operation does not validate existence. Select an outer
    /// LinksInnerReferenceExistenceValidator or a per-call EnsureLinkExists check
    /// before resolver work when missing internal addresses must be rejected.
    ///     TODO: Возможно можно перемещать значения, если указан индекс, но значение существует в другом месте (но не в
    ///     менеджере памяти, а в логике Links)
    /// </remarks>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public virtual TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLink
Address>? handler)
    {
        var constants = Constants;
        var @null = constants.Null;
        var externalReferencesRange = constants.ExternalReferencesRange;
        var linkIndex = this.GetIndex(link: restriction);
        var before = GetLinkStruct(linkIndex: linkIndex);
        ref var link = ref GetLinkDataPartReference(linkIndex: linkIndex);
        var source = link.Source;
        var target = link.Target;
        ref var header = ref GetHeaderReference();
        ref var rootAsSource = ref header.RootAsSource;
        ref var rootAsTarget = ref header.RootAsTarget;
        // Будет корректно работать только в том случае, если пространство выделенной связи предварительно заполнено нулями
        if ((source != @null))
        {
            if (externalReferencesRange.HasValue && externalReferencesRange.Value.Contains(value: source))
            {
                RequiredExternalSourcesTreeMethods.Detach(root: ref rootAsSource, linkIndex: linkIndex);
            }
            else
            {
                if (_useLinkedList)
                {
                    InternalSourcesListMethods.Detach(headElement: source, element: linkIndex);
                }
                else
                {
                    RequiredInternalSourcesTreeMethods.Detach(root: ref GetLinkIndexPartReference(linkIndex: source).RootAsSou
rce, linkIndex: linkIndex);
                }
            }
        }
        if ((target != @null))
        {
            if (externalReferencesRange.HasValue && externalReferencesRange.Value.Contains(value: target))
```

```
            {
                RequiredExternalTargetsTreeMethods.Detach(root: ref rootAsTarget, linkIndex: linkIndex);
            }
            else
            {
                RequiredInternalTargetsTreeMethods.Detach(root: ref GetLinkIndexPartReference(linkIndex: target).RootAsTarget,
linkIndex: linkIndex);
            }
        }
        source = link.Source = this.GetSource(link: substitution);
        target = link.Target = this.GetTarget(link: substitution);
        if ((source != @null))
        {
            if (externalReferencesRange.HasValue && externalReferencesRange.Value.Contains(value: source))
            {
                RequiredExternalSourcesTreeMethods.Attach(root: ref rootAsSource, linkIndex: linkIndex);
            }
            else
            {
                if (_useLinkedList)
                {
                    InternalSourcesListMethods.AttachAsLast(headElement: source, element: linkIndex);
                }
                else
                {
                    RequiredInternalSourcesTreeMethods.Attach(root: ref GetLinkIndexPartReference(linkIndex: source).RootAsSou
rce, linkIndex: linkIndex);
                }
            }
        }
        if ((target != @null))
        {
            if (externalReferencesRange.HasValue && externalReferencesRange.Value.Contains(value: target))
            {
                RequiredExternalTargetsTreeMethods.Attach(root: ref rootAsTarget, linkIndex: linkIndex);
            }
            else
            {
                RequiredInternalTargetsTreeMethods.Attach(root: ref GetLinkIndexPartReference(linkIndex: target).RootAsTarget,
linkIndex: linkIndex);
            }
        }
        return handler != null ? handler(before: before, after: new Link<TLinkAddress>(index: linkIndex, source: source, targe
t: target)) : Constants.Continue;
    }

    /// <summary>Creates a link and invokes the optional callback.</summary>
    /// <remarks>
    ///     TODO: Возможно нужно будет заполнение нулями, если внешнее API ими не заполняет пространство
    /// </remarks>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public virtual TLinkAddress Create(IList<TLinkAddress>? substitution, WriteHandler<TLinkAddress>? handler)
    {
        ref var header = ref GetHeaderReference();
        var freeLink = header.FirstFreeLink;
        if ((freeLink != Constants.Null))
        {
            RequiredUnusedLinksListMethods.Detach(freeLink: freeLink);
        }
        else
        {
            var maximumPossibleInnerReference = Constants.InternalReferencesRange.Maximum;
            if ((header.AllocatedLinks >= maximumPossibleInnerReference))
            {
                throw new LinksLimitReachedException<TLinkAddress>(limit: maximumPossibleInnerReference);
            }
            if (header.AllocatedLinks >= header.ReservedLinks)
            {
                _dataMemory.ReservedCapacity += _dataMemoryReservationStepInBytes;
                _indexMemory.ReservedCapacity += _indexMemoryReservationStepInBytes;
                SetPointers(dataMemory: _dataMemory, indexMemory: _indexMemory);
                header = ref GetHeaderReference();
                header.ReservedLinks = TLinkAddress.CreateTruncating(value: Math.Min(
                    (_dataMemory.ReservedCapacity - LinkDataPartSizeInBytes) / LinkDataPartSizeInBytes,
                    (_indexMemory.ReservedCapacity - LinkHeaderSizeInBytes) / LinkIndexPartSizeInBytes));
            }
            freeLink = header.AllocatedLinks = header.AllocatedLinks + TLinkAddress.One;
            _dataMemory.UsedCapacity += LinkDataPartSizeInBytes;
            _indexMemory.UsedCapacity += LinkIndexPartSizeInBytes;
        }
        return handler != null ? handler(before: null, after: GetLinkStruct(linkIndex: freeLink)) : Constants.Continue;
    }

    /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
    /// <summary>
    ///     <para>
    ///         Deletes a live link whose endpoints have already been reset. The caller owns these preconditions.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="restriction">
    ///     <para>The restriction containing the index of the link to delete.</para>
```

```
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public virtual TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
    {
        System.ArgumentNullException.ThrowIfNull(restriction);
        ref var header = ref GetHeaderReference();
        var link = restriction[index: Constants.IndexPart];
        var before = GetLinkStruct(linkIndex: link);
        if ((link < header.AllocatedLinks))
        {
            RequiredUnusedLinksListMethods.AttachAsFirst(link: link);
        }
        else if ((link == header.AllocatedLinks))
        {
            header.AllocatedLinks = header.AllocatedLinks - TLinkAddress.One;
            _dataMemory.UsedCapacity -= LinkDataPartSizeInBytes;
            _indexMemory.UsedCapacity -= LinkIndexPartSizeInBytes;
            // Убираем все связи, находящиеся в списке свободных в конце файла, до тех пор, пока не дойдём до первой существую
щей связи
            // Позволяет оптимизировать количество выделенных связей (AllocatedLinks)
            while ((header.AllocatedLinks > GetZero()) && IsUnusedLink(linkIndex: header.AllocatedLinks))
            {
                RequiredUnusedLinksListMethods.Detach(freeLink: header.AllocatedLinks);
                header.AllocatedLinks = header.AllocatedLinks - TLinkAddress.One;
                _dataMemory.UsedCapacity -= LinkDataPartSizeInBytes;
                _indexMemory.UsedCapacity -= LinkIndexPartSizeInBytes;
            }
        }
        return handler != null ? handler(before: before, after: null) : Constants.Continue;
    }

    /// <summary>
    ///     <para>
    ///         Inits the data memory.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="dataMemory">
    ///     <para>The data memory.</para>
    ///     <para></para>
    /// </param>
    /// <param name="indexMemory">
    ///     <para>The index memory.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual void Init(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory)
        => InitializeMemory(dataMemory, indexMemory, () => SetPointers(dataMemory, indexMemory));

    /// <summary>Initializes memory using a concrete pointer initializer during construction.</summary>
    protected unsafe void InitializeMemory(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory, Action setPo
inters)
    {
        // Read allocated links from header
        if (indexMemory.ReservedCapacity < LinkHeaderSizeInBytes)
        {
            indexMemory.ReservedCapacity = LinkHeaderSizeInBytes;
        }
        setPointers();
        ref var header = ref System.Runtime.CompilerServices.Unsafe.AsRef<LinksHeader<TLinkAddress>>((void*)indexMemory.Pointe
r);
        var allocatedLinks = long.CreateTruncating(value: header.AllocatedLinks);
        // Adjust reserved capacity
        var minimumDataReservedCapacity = (allocatedLinks + 1) * LinkDataPartSizeInBytes;
        if (minimumDataReservedCapacity < dataMemory.UsedCapacity)
        {
            minimumDataReservedCapacity = dataMemory.UsedCapacity;
        }
        if (minimumDataReservedCapacity < _dataMemoryReservationStepInBytes)
        {
            minimumDataReservedCapacity = _dataMemoryReservationStepInBytes;
        }
        var minimumIndexReservedCapacity = allocatedLinks * LinkIndexPartSizeInBytes + LinkHeaderSizeInBytes;
        if (minimumIndexReservedCapacity < indexMemory.UsedCapacity)
        {
            minimumIndexReservedCapacity = indexMemory.UsedCapacity;
        }
        if (minimumIndexReservedCapacity < _indexMemoryReservationStepInBytes)
        {
            minimumIndexReservedCapacity = _indexMemoryReservationStepInBytes;
        }
        // Check for alignment
        if (minimumDataReservedCapacity % _dataMemoryReservationStepInBytes > 0)
        {
            minimumDataReservedCapacity = minimumDataReservedCapacity / _dataMemoryReservationStepInBytes * _dataMemoryReserva
tionStepInBytes + _dataMemoryReservationStepInBytes;
        }
        if (minimumIndexReservedCapacity % _indexMemoryReservationStepInBytes > 0)
        {
            minimumIndexReservedCapacity = minimumIndexReservedCapacity / _indexMemoryReservationStepInBytes * _indexMemoryRes
ervationStepInBytes + _indexMemoryReservationStepInBytes;
```

```
        }
        if (dataMemory.ReservedCapacity < minimumDataReservedCapacity)
        {
            dataMemory.ReservedCapacity = minimumDataReservedCapacity;
        }
        if (indexMemory.ReservedCapacity < minimumIndexReservedCapacity)
        {
            indexMemory.ReservedCapacity = minimumIndexReservedCapacity;
        }
        setPointers();
        header = ref System.Runtime.CompilerServices.Unsafe.AsRef<LinksHeader<TLinkAddress>>((void*)indexMemory.Pointer);
        // Ensure correctness _memory.UsedCapacity over _header->AllocatedLinks
        // Гарантия корректности _memory.UsedCapacity относительно _header->AllocatedLinks
        dataMemory.UsedCapacity = ConvertToInt64(value: header.AllocatedLinks) * LinkDataPartSizeInBytes + LinkDataPartSizeInB
ytes; // First link is read only zero link.
        indexMemory.UsedCapacity = ConvertToInt64(value: header.AllocatedLinks) * LinkIndexPartSizeInBytes + LinkHeaderSizeInB
ytes;
        // Ensure correctness _memory.ReservedLinks over _header->ReservedCapacity
        // Гарантия корректности _header->ReservedLinks относительно _memory.ReservedCapacity
        header.ReservedLinks = TLinkAddress.CreateTruncating(value: Math.Min(
            (dataMemory.ReservedCapacity - LinkDataPartSizeInBytes) / LinkDataPartSizeInBytes,
            (indexMemory.ReservedCapacity - LinkHeaderSizeInBytes) / LinkIndexPartSizeInBytes));
    }

    /// <summary>
    ///     <para>
    ///         Gets the link struct using the specified link index.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="linkIndex">
    ///     <para>The link index.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A list of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public IList<TLinkAddress> GetLinkStruct(TLinkAddress linkIndex)
    {
        ref var link = ref GetLinkDataPartReference(linkIndex: linkIndex);
        return new Link<TLinkAddress>(index: linkIndex, source: link.Source, target: link.Target);
    }

    /// <remarks>
    ///     TODO: Возможно это должно быть событием, вызываемым из IMemory, в том случае, если адрес реально поменялся
    ///     Указатель this.links может быть в том же месте,
    ///     так как 0-я связь не используется и имеет такой же размер как Header,
    ///     поэтому header размещается в том же месте, что и 0-я связь
    /// </remarks>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract void SetPointers(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory);

    /// <summary>
    ///     <para>
    ///         Resets the pointers.
    ///     </para>
    ///     <para></para>
    /// </summary>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual void ResetPointers()
    {
        _internalSourcesListMethods = null;
        InternalSourcesTreeMethods = null;
        ExternalSourcesTreeMethods = null;
        InternalTargetsTreeMethods = null;
        ExternalTargetsTreeMethods = null;
        UnusedLinksListMethods = null;
    }

    /// <summary>
    ///     <para>
    ///         Gets the header reference.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>A ref links header of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract ref LinksHeader<TLinkAddress> GetHeaderReference();

    /// <summary>
    ///     <para>
    ///         Gets the link data part reference using the specified link index.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="linkIndex">
    ///     <para>The link index.</para>
```

```
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link data part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract ref RawLinkDataPart<TLinkAddress> GetLinkDataPartReference(TLinkAddress linkIndex);

    /// <summary>
    ///     <para>
    ///         Gets the link index part reference using the specified link index.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="linkIndex">
    ///     <para>The link index.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link index part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract ref RawLinkIndexPart<TLinkAddress> GetLinkIndexPartReference(TLinkAddress linkIndex);

    /// <summary>
    ///     <para>
    ///         Determines whether this instance exists.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual bool Exists(TLinkAddress link)
    {
        return (link >= Constants.InternalReferencesRange.Minimum) && (link <= GetHeaderReference().AllocatedLinks) && !IsUnus
edLink(linkIndex: link);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance is unused link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="linkIndex">
    ///     <para>The link index.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual bool IsUnusedLink(TLinkAddress linkIndex)
    {
        if ((GetHeaderReference().FirstFreeLink != linkIndex)) // May be this check is not needed
        {
            // TODO: Reduce access to memory in different location (should be enough to use just linkIndexPart)
            ref var linkDataPart = ref GetLinkDataPartReference(linkIndex: linkIndex);
            ref var linkIndexPart = ref GetLinkIndexPartReference(linkIndex: linkIndex);
            return (linkIndexPart.SizeAsTarget == TLinkAddress.Zero) && (linkDataPart.Source != TLinkAddress.Zero);
        }
        return true;
    }

    /// <summary>
    ///     <para>
    ///         Gets the one.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual TLinkAddress GetOne()
    {
        return _one;
    }

    /// <summary>
    ///     <para>
```

```
///         Gets the zero.
///     </para>
///     <para></para>
/// </summary>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected virtual TLinkAddress GetZero()
{
    return TLinkAddress.Zero;
}

/// <summary>
///     <para>
///         Determines whether this instance are equal.
///     </para>
///     <para></para>
/// </summary>
/// <param name="first">
///     <para>The first.</para>
///     <para></para>
/// </param>
/// <param name="second">
///     <para>The second.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The bool</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected virtual bool AreEqual(TLinkAddress first, TLinkAddress second)
{
    return first == second;
}

/// <summary>
///     <para>
///         Determines whether this instance less than.
///     </para>
///     <para></para>
/// </summary>
/// <param name="first">
///     <para>The first.</para>
///     <para></para>
/// </param>
/// <param name="second">
///     <para>The second.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The bool</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected virtual bool LessThan(TLinkAddress first, TLinkAddress second)
{
    return _comparer.Compare(x: first, y: second) < 0;
}

/// <summary>
///     <para>
///         Determines whether this instance less or equal than.
///     </para>
///     <para></para>
/// </summary>
/// <param name="first">
///     <para>The first.</para>
///     <para></para>
/// </param>
/// <param name="second">
///     <para>The second.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The bool</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected virtual bool LessOrEqualThan(TLinkAddress first, TLinkAddress second)
{
    return _comparer.Compare(x: first, y: second) <= 0;
}

/// <summary>
///     <para>
///         Determines whether this instance greater than.
///     </para>
///     <para></para>
/// </summary>
```

```
/// <param name="first">
///     <para>The first.</para>
///     <para></para>
/// </param>
/// <param name="second">
///     <para>The second.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The bool</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected virtual bool GreaterThan(TLinkAddress first, TLinkAddress second)
{
    return _comparer.Compare(x: first, y: second) > 0;
}

/// <summary>
///     <para>
///         Determines whether this instance greater or equal than.
///     </para>
///     <para></para>
/// </summary>
/// <param name="first">
///     <para>The first.</para>
///     <para></para>
/// </param>
/// <param name="second">
///     <para>The second.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The bool</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected virtual bool GreaterOrEqualThan(TLinkAddress first, TLinkAddress second)
{
    return _comparer.Compare(x: first, y: second) >= 0;
}

/// <summary>
///     <para>
///         Converts the to int 64 using the specified value.
///     </para>
///     <para></para>
/// </summary>
/// <param name="value">
///     <para>The value.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The long</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected virtual long ConvertToInt64(TLinkAddress value)
{
    return long.CreateTruncating(value); ;
}

/// <summary>
///     <para>
///         Converts the to address using the specified value.
///     </para>
///     <para></para>
/// </summary>
/// <param name="value">
///     <para>The value.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected virtual TLinkAddress ConvertToAddress(long value)
{
    return TLinkAddress.CreateTruncating(value);
}

/// <summary>
///     <para>
///         Adds the first.
///     </para>
///     <para></para>
/// </summary>
/// <param name="first">
///     <para>The first.</para>
///     <para></para>
/// </param>
```

```
    /// <param name="second">
    ///     <para>The second.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual TLinkAddress Add(TLinkAddress first, TLinkAddress second)
    {
        return first + second;
    }

    /// <summary>
    ///     <para>
    ///         Subtracts the first.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="first">
    ///     <para>The first.</para>
    ///     <para></para>
    /// </param>
    /// <param name="second">
    ///     <para>The second.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual TLinkAddress Subtract(TLinkAddress first, TLinkAddress second)
    {
        return first - second;
    }

    #region Disposable

    /// <summary>
    ///     <para>
    ///         Gets the allow multiple dispose calls value.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected override bool AllowMultipleDisposeCalls
    {
        [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
        get => true;
    }

    /// <summary>
    ///     <para>
    ///         Disposes the manual.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="manual">
    ///     <para>The manual.</para>
    ///     <para></para>
    /// </param>
    /// <param name="wasDisposed">
    ///     <para>The was disposed.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void Dispose(bool manual, bool wasDisposed)
    {
        if (!wasDisposed)
        {
            ResetPointers();
            _dataMemory.DisposeIfPossible();
            _indexMemory.DisposeIfPossible();
        }
    }

    #endregion
}
```

csharp/Platform.Data.Doublets/Memory/Split/Generic/UnusedLinksListMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Collections.Methods.Lists;
using Platform.Converters;
using static System.Runtime.CompilerServices.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split.Generic;

/// <summary>
///     <para>
///         Represents the unused links list methods.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="AbsoluteCircularDoublyLinkedListMethods{TLinkAddress}" />
/// <seealso cref="ILinksListMethods{TLinkAddress}" />
public unsafe class UnusedLinksListMethods<TLinkAddress> : AbsoluteCircularDoublyLinkedListMethods<TLinkAddress>, ILinksListMe
thods<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
{
    private readonly byte* _header;
    private readonly byte* _links;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="UnusedLinksListMethods{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public UnusedLinksListMethods(byte* links, byte* header)
    {
        _links = links;
        _header = header;
    }

    /// <summary>
    ///     <para>
    ///         Gets the header reference.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>A ref links header of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref LinksHeader<TLinkAddress> GetHeaderReference()
    {
        return ref AsRef<LinksHeader<TLinkAddress>>(source: _header);
    }

    /// <summary>
    ///     <para>
    ///         Gets the link data part reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link data part of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLinkDataPart<TLinkAddress> GetLinkDataPartReference(TLinkAddress link)
    {
        return ref AsRef<RawLinkDataPart<TLinkAddress>>(source: _links + RawLinkDataPart<TLinkAddress>.SizeInBytes * long.Crea
teTruncating(link));
    }

    /// <summary>
    ///     <para>
    ///         Gets the first.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
```

```
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetFirst()
{
    return GetHeaderReference().FirstFreeLink;
}

/// <summary>
///     <para>
///         Gets the last.
///     </para>
///     <para></para>
/// </summary>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetLast()
{
    return GetHeaderReference().LastFreeLink;
}

/// <summary>
///     <para>
///         Gets the previous using the specified element.
///     </para>
///     <para></para>
/// </summary>
/// <param name="element">
///     <para>The element.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetPrevious(TLinkAddress element)
{
    return GetLinkDataPartReference(link: element).Source;
}

/// <summary>
///     <para>
///         Gets the next using the specified element.
///     </para>
///     <para></para>
/// </summary>
/// <param name="element">
///     <para>The element.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetNext(TLinkAddress element)
{
    return GetLinkDataPartReference(link: element).Target;
}

/// <summary>
///     <para>
///         Gets the size.
///     </para>
///     <para></para>
/// </summary>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetSize()
{
    return GetHeaderReference().FreeLinks;
}

/// <summary>
///     <para>
///         Sets the first using the specified element.
///     </para>
///     <para></para>
/// </summary>
/// <param name="element">
///     <para>The element.</para>
///     <para></para>
/// </param>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected override void SetFirst(TLinkAddress element)
{
```

```
        GetHeaderReference().FirstFreeLink = element;
    }

    /// <summary>
    ///     <para>
    ///         Sets the last using the specified element.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="element">
    ///     <para>The element.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void SetLast(TLinkAddress element)
    {
        GetHeaderReference().LastFreeLink = element;
    }

    /// <summary>
    ///     <para>
    ///         Sets the previous using the specified element.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="element">
    ///     <para>The element.</para>
    ///     <para></para>
    /// </param>
    /// <param name="previous">
    ///     <para>The previous.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void SetPrevious(TLinkAddress element, TLinkAddress previous)
    {
        GetLinkDataPartReference(link: element).Source = previous;
    }

    /// <summary>
    ///     <para>
    ///         Sets the next using the specified element.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="element">
    ///     <para>The element.</para>
    ///     <para></para>
    /// </param>
    /// <param name="next">
    ///     <para>The next.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void SetNext(TLinkAddress element, TLinkAddress next)
    {
        GetLinkDataPartReference(link: element).Target = next;
    }

    /// <summary>
    ///     <para>
    ///         Sets the size using the specified size.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="size">
    ///     <para>The size.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void SetSize(TLinkAddress size)
    {
        GetHeaderReference().FreeLinks = size;
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/RawLinkDataPart.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split;

/// <summary>
///     <para>
///         The raw link data part.
///     </para>
///     <para></para>
/// </summary>
public struct RawLinkDataPart<TLinkAddress> : IEquatable<RawLinkDataPart<TLinkAddress>> where TLinkAddress : IUnsignedNumber<T
LinkAddress>
{

    /// <summary>
    ///     <para>
    ///         The size.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public static readonly long SizeInBytes = Structure<RawLinkDataPart<TLinkAddress>>.Size;

    /// <summary>
    ///     <para>
    ///         The source.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress Source;
    /// <summary>
    ///     <para>
    ///         The target.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress Target;

    /// <summary>
    ///     <para>
    ///         Determines whether this instance equals.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="obj">
    ///     <para>The obj.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override bool Equals(object? obj)
    {
        return obj is RawLinkDataPart<TLinkAddress> link && Equals(other: link);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance equals.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="other">
    ///     <para>The other.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public bool Equals(RawLinkDataPart<TLinkAddress> other)
    {
        return Source == other.Source && Target == other.Target;
    }

    /// <summary>
    ///     <para>
    ///         Gets the hash code.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>The int</para>
```

```
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override int GetHashCode()
    {
        return (Source, Target).GetHashCode();
    }

    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public static bool operator ==(RawLinkDataPart<TLinkAddress> left, RawLinkDataPart<TLinkAddress> right)
    {
        return left.Equals(other: right);
    }

    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public static bool operator !=(RawLinkDataPart<TLinkAddress> left, RawLinkDataPart<TLinkAddress> right)
    {
        return !(left == right);
    }
}
```

csharp/Platform.Data.Doublets/Memory/Split/RawLinkIndexPart.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.Split;

/// <summary>
///     <para>
///         The raw link index part.
///     </para>
///     <para></para>
/// </summary>
public struct RawLinkIndexPart<TLinkAddress> : IEquatable<RawLinkIndexPart<TLinkAddress>> where TLinkAddress : IUnsignedNumber
<TLinkAddress>
{

    /// <summary>
    ///     <para>
    ///         The size.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public static readonly long SizeInBytes = Structure<RawLinkIndexPart<TLinkAddress>>.Size;

    /// <summary>
    ///     <para>
    ///         The root as source.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress RootAsSource;
    /// <summary>
    ///     <para>
    ///         The left as source.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress LeftAsSource;
    /// <summary>
    ///     <para>
    ///         The right as source.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress RightAsSource;
    /// <summary>
    ///     <para>
    ///         The size as source.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress SizeAsSource;
    /// <summary>
    ///     <para>
    ///         The root as target.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress RootAsTarget;
    /// <summary>
    ///     <para>
    ///         The left as target.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress LeftAsTarget;
    /// <summary>
    ///     <para>
    ///         The right as target.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress RightAsTarget;
    /// <summary>
    ///     <para>
    ///         The size as target.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress SizeAsTarget;

    /// <summary>
    ///     <para>
    ///         Determines whether this instance equals.
    ///     </para>
    ///     <para></para>
    /// </summary>
```

```
    /// <param name="obj">
    ///     <para>The obj.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override bool Equals(object? obj)
    {
        return obj is RawLinkIndexPart<TLinkAddress> link && Equals(other: link);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance equals.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="other">
    ///     <para>The other.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public bool Equals(RawLinkIndexPart<TLinkAddress> other)
    {
        return RootAsSource == other.RootAsSource && LeftAsSource == other.LeftAsSource && RightAsSource == other.RightAsSourc
e && SizeAsSource == other.SizeAsSource && RootAsTarget == other.RootAsTarget && LeftAsTarget == other.LeftAsTarget && RightAs
Target == other.RightAsTarget && SizeAsTarget == other.SizeAsTarget;
    }

    /// <summary>
    ///     <para>
    ///         Gets the hash code.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>The int</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public override int GetHashCode()
    {
        return (RootAsSource, LeftAsSource, RightAsSource, SizeAsSource, RootAsTarget, LeftAsTarget, RightAsTarget, SizeAsTarg
et).GetHashCode();
    }

    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public static bool operator ==(RawLinkIndexPart<TLinkAddress> left, RawLinkIndexPart<TLinkAddress> right)
    {
        return left.Equals(other: right);
    }

    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public static bool operator !=(RawLinkIndexPart<TLinkAddress> left, RawLinkIndexPart<TLinkAddress> right)
    {
        return !(left == right);
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/Generic/LinksAvlBalancedTreeMethodsBase.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using System.Text;
using Platform.Collections.Methods.Trees;
using Platform.Converters;
using Platform.Delegates;
using Platform.Numbers;
using static System.Runtime.CompilerServices.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United.Generic;

/// <summary>
///     <para>
///         Represents the links avl balanced tree methods base.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="SizedAndThreadedAVLBalancedTreeMethods{TLinkAddress}" />
/// <seealso cref="ILinksTreeMethods{TLinkAddress}" />
public abstract unsafe class LinksAvlBalancedTreeMethodsBase<TLinkAddress> : SizedAndThreadedAVLBalancedTreeMethods<TLinkAddre
ss>, ILinksTreeMethods<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IShiftOperators<TLinkAddress, int, TL
inkAddress>, IBitwiseOperators<TLinkAddress, TLinkAddress, TLinkAddress>, IMinMaxValue<TLinkAddress>, IComparisonOperators<TLi
nkAddress, TLinkAddress, bool>
{

    /// <summary>
    ///     <para>
    ///         The break.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Break;
    /// <summary>
    ///     <para>
    ///         The continue.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Continue;
    /// <summary>
    ///     <para>
    ///         The header.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* Header;
    /// <summary>
    ///     <para>
    ///         The links.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* Links;

    private static readonly UncheckedConverter<TLinkAddress, bool> _addressToBoolConverter = UncheckedConverter<TLinkAddress,
bool>.Default;
    private static readonly UncheckedConverter<bool, TLinkAddress> _boolToAddressConverter = UncheckedConverter<bool, TLinkAdd
ress>.Default;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksAvlBalancedTreeMethodsBase{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected LinksAvlBalancedTreeMethodsBase(LinksConstants<TLinkAddress> constants, byte* links, byte* header)
    {
        Links = links;
        Header = header;
        Break = constants.Break;
        Continue = constants.Continue;
    }

    /// <summary>
    ///     <para>
```

```
    ///         The zero.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress this[TLinkAddress index]
    {
        [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
        get
        {
            var root = GetTreeRoot();
            if ((index >= GetSize(node: root)))
            {
                return TLinkAddress.Zero;
            }
            while (root != TLinkAddress.Zero)
            {
                var left = GetLeftOrDefault(node: root);
                var leftSize = GetSizeOrZero(node: left);
                if (index < leftSize)
                {
                    root = left;
                    continue;
                }
                if ((index == leftSize))
                {
                    return root;
                }
                root = GetRightOrDefault(node: root);
                index = (index) - (leftSize + TLinkAddress.One);
            }
            return TLinkAddress.Zero; // TODO: Impossible situation exception (only if tree structure broken)
        }
    }

    /// <summary>
    ///     Выполняет поиск и возвращает индекс связи с указанными Source (началом) и Target (концом).
    /// </summary>
    /// <param name="source">Индекс связи, которая является началом на искомой связи.</param>
    /// <param name="target">Индекс связи, которая является концом на искомой связи.</param>
    /// <returns>Индекс искомой связи.</returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public TLinkAddress Search(TLinkAddress source, TLinkAddress target)
    {
        var root = GetTreeRoot();
        while (root != TLinkAddress.Zero)
        {
            ref var rootLink = ref GetLinkReference(link: root);
            var rootSource = rootLink.Source;
            var rootTarget = rootLink.Target;
            if (FirstIsToTheLeftOfSecond(source: source, target: target, rootSource: rootSource, rootTarget: rootTarget)) // n
ode.Key < root.Key
            {
                root = GetLeftOrDefault(node: root);
            }
            else if (FirstIsToTheRightOfSecond(source: source, target: target, rootSource: rootSource, rootTarget: rootTarget)
) // node.Key > root.Key
            {
                root = GetRightOrDefault(node: root);
            }
            else // node.Key == root.Key
            {
                return root;
            }
        }
        return TLinkAddress.Zero;
    }

    // TODO: Return indices range instead of references count
    /// <summary>
    ///     <para>
    ///         Counts the usages using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public TLinkAddress CountUsages(TLinkAddress link)
    {
        var root = GetTreeRoot();
        var total = GetSize(node: root);
        var totalRightIgnore = TLinkAddress.Zero;
        while (root != TLinkAddress.Zero)
        {
            var @base = GetBasePartValue(link: root);
            if ((@base <= link))
```

```
        {
            root = GetRightOrDefault(node: root);
        }
        else
        {
            totalRightIgnore = (totalRightIgnore + GetRightSize(node: root) + TLinkAddress.One);
            root = GetLeftOrDefault(node: root);
        }
    }
    root = GetTreeRoot();
    var totalLeftIgnore = TLinkAddress.Zero;
    while (root != TLinkAddress.Zero)
    {
        var @base = GetBasePartValue(link: root);
        if ((@base >= link))
        {
            root = GetLeftOrDefault(node: root);
        }
        else
        {
            totalLeftIgnore = (totalLeftIgnore + GetLeftSize(node: root) + TLinkAddress.One);
            root = GetRightOrDefault(node: root);
        }
    }
    return total - totalRightIgnore - totalLeftIgnore;
}

/// <summary>
///     <para>
///         Eaches the usage using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <param name="handler">
///     <para>The handler.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The continue.</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
public TLinkAddress EachUsage(TLinkAddress link, ReadHandler<TLinkAddress>? handler)
{
    var root = GetTreeRoot();
    if (root == TLinkAddress.Zero)
    {
        return Continue;
    }
    TLinkAddress first = TLinkAddress.Zero, current = root;
    while (current != TLinkAddress.Zero)
    {
        var @base = GetBasePartValue(link: current);
        if ((@base >= link))
        {
            if ((@base == link))
            {
                first = current;
            }
            current = GetLeftOrDefault(node: current);
        }
        else
        {
            current = GetRightOrDefault(node: current);
        }
    }
    if (first != TLinkAddress.Zero)
    {
        current = first;
        while (true)
        {
            if (((handler == null ? Continue : handler(link: GetLinkValues(linkIndex: current))) == Break))
            {
                return Break;
            }
            current = GetNext(node: current);
            if (current == TLinkAddress.Zero || (GetBasePartValue(link: current) != link))
            {
                break;
            }
        }
    }
    return Continue;
}

/// <summary>
///     <para>
///         Gets the tree root.
```

```
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract TLinkAddress GetTreeRoot();

    /// <summary>
    ///     <para>
    ///         Gets the base part value using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract TLinkAddress GetBasePartValue(TLinkAddress link);

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the right of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="target">
    ///     <para>The target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootSource">
    ///     <para>The root source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootTarget">
    ///     <para>The root target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract bool FirstIsToTheRightOfSecond(TLinkAddress source, TLinkAddress target, TLinkAddress rootSource, TLink
Address rootTarget);

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="target">
    ///     <para>The target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootSource">
    ///     <para>The root source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootTarget">
    ///     <para>The root target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract bool FirstIsToTheLeftOfSecond(TLinkAddress source, TLinkAddress target, TLinkAddress rootSource, TLinkA
ddress rootTarget);

    /// <summary>
    ///     <para>
    ///         Gets the header reference.
    ///     </para>
    ///     <para></para>
```

```
    /// </summary>
    /// <returns>
    ///     <para>A ref links header of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref LinksHeader<TLinkAddress> GetHeaderReference()
    {
        return ref AsRef<LinksHeader<TLinkAddress>>(source: Header);
    }

    /// <summary>
    ///     <para>
    ///         Gets the link reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLink<TLinkAddress> GetLinkReference(TLinkAddress link)
    {
        return ref AsRef<RawLink<TLinkAddress>>(source: Links + RawLink<TLinkAddress>.SizeInBytes * long.CreateTruncating(link
));
    }

    /// <summary>
    ///     <para>
    ///         Gets the link values using the specified link index.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="linkIndex">
    ///     <para>The link index.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A list of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual IList<TLinkAddress>? GetLinkValues(TLinkAddress linkIndex)
    {
        ref var link = ref GetLinkReference(link: linkIndex);
        return new Link<TLinkAddress>(index: linkIndex, source: link.Source, target: link.Target);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="first">
    ///     <para>The first.</para>
    ///     <para></para>
    /// </param>
    /// <param name="second">
    ///     <para>The second.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheLeftOfSecond(TLinkAddress first, TLinkAddress second)
    {
        ref var firstLink = ref GetLinkReference(link: first);
        ref var secondLink = ref GetLinkReference(link: second);
        return FirstIsToTheLeftOfSecond(source: firstLink.Source, target: firstLink.Target, rootSource: secondLink.Source, roo
tTarget: secondLink.Target);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the right of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="first">
    ///     <para>The first.</para>
    ///     <para></para>
    /// </param>
    /// <param name="second">
    ///     <para>The second.</para>
```

```
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheRightOfSecond(TLinkAddress first, TLinkAddress second)
    {
        ref var firstLink = ref GetLinkReference(link: first);
        ref var secondLink = ref GetLinkReference(link: second);
        return FirstIsToTheRightOfSecond(source: firstLink.Source, target: firstLink.Target, rootSource: secondLink.Source, ro
otTarget: secondLink.Target);
    }

    /// <summary>
    ///     <para>
    ///         Gets the size value using the specified value.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="value">
    ///     <para>The value.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual TLinkAddress GetSizeValue(TLinkAddress value)
    {
        return Bit<TLinkAddress>.PartialRead(target: value, shift: 5, limit: -5);
    }

    /// <summary>
    ///     <para>
    ///         Sets the size value using the specified stored value.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="storedValue">
    ///     <para>The stored value.</para>
    ///     <para></para>
    /// </param>
    /// <param name="size">
    ///     <para>The size.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual void SetSizeValue(ref TLinkAddress storedValue, TLinkAddress size)
    {
        storedValue = Bit<TLinkAddress>.PartialWrite(target: storedValue, source: size, shift: 5, limit: -5);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance get left is child value.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="value">
    ///     <para>The value.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual bool GetLeftIsChildValue(TLinkAddress value)
    {
        return _addressToBoolConverter.Convert(source: Bit<TLinkAddress>.PartialRead(target: value, shift: 4, limit: 1));
        //return Bit<TLinkAddress>.PartialRead(value !=  4, 1, default);
    }

    /// <summary>
    ///     <para>
    ///         Sets the left is child value using the specified stored value.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="storedValue">
    ///     <para>The stored value.</para>
    ///     <para></para>
    /// </param>
    /// <param name="value">
    ///     <para>The value.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual void SetLeftIsChildValue(ref TLinkAddress storedValue, bool value)
```

```
    {
        unchecked
        {
            var previousValue = storedValue;
            var modified = Bit<TLinkAddress>.PartialWrite(target: previousValue, source: _boolToAddressConverter.Convert(sourc
e: value), shift: 4, limit: 1);
            storedValue = modified;
        }
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance get right is child value.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="value">
    ///     <para>The value.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual bool GetRightIsChildValue(TLinkAddress value)
    {
        return _addressToBoolConverter.Convert(source: Bit<TLinkAddress>.PartialRead(target: value, shift: 3, limit: 1));
        //return Bit<TLinkAddress>.PartialRead(value !=  3, 1, default);
    }

    /// <summary>
    ///     <para>
    ///         Sets the right is child value using the specified stored value.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="storedValue">
    ///     <para>The stored value.</para>
    ///     <para></para>
    /// </param>
    /// <param name="value">
    ///     <para>The value.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual void SetRightIsChildValue(ref TLinkAddress storedValue, bool value)
    {
        unchecked
        {
            var previousValue = storedValue;
            var modified = Bit<TLinkAddress>.PartialWrite(target: previousValue, source: _boolToAddressConverter.Convert(sourc
e: value), shift: 3, limit: 1);
            storedValue = modified;
        }
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance is child.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="parent">
    ///     <para>The parent.</para>
    ///     <para></para>
    /// </param>
    /// <param name="possibleChild">
    ///     <para>The possible child.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected bool IsChild(TLinkAddress parent, TLinkAddress possibleChild)
    {
        var parentSize = GetSize(node: parent);
        var childSize = GetSizeOrZero(node: possibleChild);
        return (childSize > TLinkAddress.Zero) && (childSize <= parentSize);
    }

    /// <summary>
    ///     <para>
    ///         Gets the balance value using the specified stored value.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="storedValue">
    ///     <para>The stored value.</para>
    ///     <para></para>
```

```
    /// </param>
    /// <returns>
    ///     <para>The sbyte</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual sbyte GetBalanceValue(TLinkAddress storedValue)
    {
        unchecked
        {
            var value = int.CreateTruncating(Bit<TLinkAddress>.PartialRead(target: storedValue, shift: 0, limit: 3));
            value |= 0xF8 * ((value & 4) >> 2); // if negative, then continue ones to the end of sbyte
            return (sbyte)value;
        }
    }

    /// <summary>
    ///     <para>
    ///         Sets the balance value using the specified stored value.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="storedValue">
    ///     <para>The stored value.</para>
    ///     <para></para>
    /// </param>
    /// <param name="value">
    ///     <para>The value.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual void SetBalanceValue(ref TLinkAddress storedValue, sbyte value)
    {
        unchecked
        {
            var packagedValue = TLinkAddress.CreateTruncating((((byte)value >> 5) & 4) | (value & 3));
            var modified = Bit<TLinkAddress>.PartialWrite(target: storedValue, source: packagedValue, shift: 0, limit: 3);
            storedValue = modified;
        }
    }

    /// <summary>
    ///     <para>
    ///         Prints the node value using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <param name="sb">
    ///     <para>The sb.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void PrintNodeValue(TLinkAddress node, StringBuilder sb)
    {
        ref var link = ref GetLinkReference(link: node);
        sb.Append(value: ' ');
        sb.Append(value: link.Source);
        sb.Append(value: '-');
        sb.Append(value: '>');
        sb.Append(value: link.Target);
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/Generic/LinksRecursionlessSizeBalancedTreeMethodsBase.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using System.Text;
using Platform.Collections.Methods.Trees;
using Platform.Converters;
using Platform.Delegates;
using static System.Runtime.CompilerServices.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United.Generic;

/// <summary>
///     <para>
///         Represents the links recursionless size balanced tree methods base.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="RecursionlessSizeBalancedTreeMethods{TLinkAddress}" />
/// <seealso cref="ILinksTreeMethods{TLinkAddress}" />
public abstract unsafe class LinksRecursionlessSizeBalancedTreeMethodsBase<TLinkAddress> : RecursionlessSizeBalancedTreeMethod
s<TLinkAddress>, ILinksTreeMethods<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLin
kAddress, TLinkAddress, bool>
{

    /// <summary>
    ///     <para>
    ///         The break.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Break;
    /// <summary>
    ///     <para>
    ///         The continue.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Continue;
    /// <summary>
    ///     <para>
    ///         The header.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* Header;
    /// <summary>
    ///     <para>
    ///         The links.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* Links;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksRecursionlessSizeBalancedTreeMethodsBase{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected LinksRecursionlessSizeBalancedTreeMethodsBase(LinksConstants<TLinkAddress> constants, byte* links, byte* header)
    {
        Links = links;
        Header = header;
        Break = constants.Break;
        Continue = constants.Continue;
    }

    /// <summary>
    ///     <para>
    ///         The zero.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress this[TLinkAddress index]
    {
        [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
```

```
        get
        {
            var root = GetTreeRoot();
            if ((index >= GetSize(node: root)))
            {
                return TLinkAddress.Zero;
            }
            while (root != TLinkAddress.Zero)
            {
                var left = GetLeftOrDefault(node: root);
                var leftSize = GetSizeOrZero(node: left);
                if (index < leftSize)
                {
                    root = left;
                    continue;
                }
                if ((index == leftSize))
                {
                    return root;
                }
                root = GetRightOrDefault(node: root);
                index = (index) - (leftSize + TLinkAddress.One);
            }
            return TLinkAddress.Zero; // TODO: Impossible situation exception (only if tree structure broken)
        }
    }

    /// <summary>
    ///     Выполняет поиск и возвращает индекс связи с указанными Source (началом) и Target (концом).
    /// </summary>
    /// <param name="source">Индекс связи, которая является началом на искомой связи.</param>
    /// <param name="target">Индекс связи, которая является концом на искомой связи.</param>
    /// <returns>Индекс искомой связи.</returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public TLinkAddress Search(TLinkAddress source, TLinkAddress target)
    {
        var root = GetTreeRoot();
        while (root != TLinkAddress.Zero)
        {
            ref var rootLink = ref GetLinkReference(link: root);
            var rootSource = rootLink.Source;
            var rootTarget = rootLink.Target;
            if (FirstIsToTheLeftOfSecond(source: source, target: target, rootSource: rootSource, rootTarget: rootTarget)) // n
ode.Key < root.Key
            {
                root = GetLeftOrDefault(node: root);
            }
            else if (FirstIsToTheRightOfSecond(source: source, target: target, rootSource: rootSource, rootTarget: rootTarget)
) // node.Key > root.Key
            {
                root = GetRightOrDefault(node: root);
            }
            else // node.Key == root.Key
            {
                return root;
            }
        }
        return TLinkAddress.Zero;
    }

    // TODO: Return indices range instead of references count
    /// <summary>
    ///     <para>
    ///         Counts the usages using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public TLinkAddress CountUsages(TLinkAddress link)
    {
        var root = GetTreeRoot();
        var total = GetSize(node: root);
        var totalRightIgnore = TLinkAddress.Zero;
        while (root != TLinkAddress.Zero)
        {
            var @base = GetBasePartValue(link: root);
            if ((@base <= link))
            {
                root = GetRightOrDefault(node: root);
            }
            else
            {
                totalRightIgnore = (totalRightIgnore + GetRightSize(node: root) + TLinkAddress.One);
                root = GetLeftOrDefault(node: root);
```

```
        }
    }
    root = GetTreeRoot();
    var totalLeftIgnore = TLinkAddress.Zero;
    while (root != TLinkAddress.Zero)
    {
        var @base = GetBasePartValue(link: root);
        if ((@base >= link))
        {
            root = GetLeftOrDefault(node: root);
        }
        else
        {
            totalLeftIgnore = (totalLeftIgnore + GetLeftSize(node: root) + TLinkAddress.One);
            root = GetRightOrDefault(node: root);
        }
    }
    return total - totalRightIgnore - totalLeftIgnore;
}
/// <summary>
///     <para>
///         Eaches the usage using the specified base.
///     </para>
///     <para></para>
/// </summary>
/// <param name="base">
///     <para>The base.</para>
///     <para></para>
/// </param>
/// <param name="handler">
///     <para>The handler.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
public TLinkAddress EachUsage(TLinkAddress @base, ReadHandler<TLinkAddress>? handler)
{
    return EachUsageCore(@base: @base, link: GetTreeRoot(), handler: handler);
}

/// <summary>
///     <para>
///         Gets the tree root.
///     </para>
///     <para></para>
/// </summary>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected abstract TLinkAddress GetTreeRoot();

/// <summary>
///     <para>
///         Gets the base part value using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected abstract TLinkAddress GetBasePartValue(TLinkAddress link);

/// <summary>
///     <para>
///         Determines whether this instance first is to the right of second.
///     </para>
///     <para></para>
/// </summary>
/// <param name="source">
///     <para>The source.</para>
///     <para></para>
/// </param>
/// <param name="target">
///     <para>The target.</para>
///     <para></para>
/// </param>
/// <param name="rootSource">
///     <para>The root source.</para>
///     <para></para>
/// </param>
/// <param name="rootTarget">
```

```
    ///     <para>The root target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract bool FirstIsToTheRightOfSecond(TLinkAddress source, TLinkAddress target, TLinkAddress rootSource, TLink
Address rootTarget);

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="target">
    ///     <para>The target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootSource">
    ///     <para>The root source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootTarget">
    ///     <para>The root target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract bool FirstIsToTheLeftOfSecond(TLinkAddress source, TLinkAddress target, TLinkAddress rootSource, TLinkA
ddress rootTarget);

    /// <summary>
    ///     <para>
    ///         Gets the header reference.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>A ref links header of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref LinksHeader<TLinkAddress> GetHeaderReference()
    {
        return ref AsRef<LinksHeader<TLinkAddress>>(source: Header);
    }

    /// <summary>
    ///     <para>
    ///         Gets the link reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLink<TLinkAddress> GetLinkReference(TLinkAddress link)
    {
        return ref AsRef<RawLink<TLinkAddress>>(source: Links + RawLink<TLinkAddress>.SizeInBytes * long.CreateTruncating(link
));
    }

    /// <summary>
    ///     <para>
    ///         Gets the link values using the specified link index.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="linkIndex">
    ///     <para>The link index.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A list of t link</para>
    ///     <para></para>
    /// </returns>
```

```
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual IList<TLinkAddress>? GetLinkValues(TLinkAddress linkIndex)
    {
        ref var link = ref GetLinkReference(link: linkIndex);
        return new Link<TLinkAddress>(index: linkIndex, source: link.Source, target: link.Target);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="first">
    ///     <para>The first.</para>
    ///     <para></para>
    /// </param>
    /// <param name="second">
    ///     <para>The second.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheLeftOfSecond(TLinkAddress first, TLinkAddress second)
    {
        ref var firstLink = ref GetLinkReference(link: first);
        ref var secondLink = ref GetLinkReference(link: second);
        return FirstIsToTheLeftOfSecond(source: firstLink.Source, target: firstLink.Target, rootSource: secondLink.Source, roo
tTarget: secondLink.Target);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the right of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="first">
    ///     <para>The first.</para>
    ///     <para></para>
    /// </param>
    /// <param name="second">
    ///     <para>The second.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheRightOfSecond(TLinkAddress first, TLinkAddress second)
    {
        ref var firstLink = ref GetLinkReference(link: first);
        ref var secondLink = ref GetLinkReference(link: second);
        return FirstIsToTheRightOfSecond(source: firstLink.Source, target: firstLink.Target, rootSource: secondLink.Source, ro
otTarget: secondLink.Target);
    }

    // TODO: 1. Move target, handler to separate object. 2. Use stack or walker 3. Use low-level MSIL stack.
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    private TLinkAddress EachUsageCore(TLinkAddress @base, TLinkAddress link, ReadHandler<TLinkAddress>? handler)
    {
        var @continue = Continue;
        if (link == TLinkAddress.Zero)
        {
            return @continue;
        }
        var linkBasePart = GetBasePartValue(link: link);
        var @break = Break;
        if ((linkBasePart > @base))
        {
            if ((EachUsageCore(@base: @base, link: GetLeftOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
        }
        else if ((linkBasePart < @base))
        {
            if ((EachUsageCore(@base: @base, link: GetRightOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
        }
        else //if (linkBasePart == @base)
        {
            if (((handler == null ? Continue : handler(link: GetLinkValues(linkIndex: link))) == @break))
            {
                return @break;
            }
```

```
            if ((EachUsageCore(@base: @base, link: GetLeftOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
            if ((EachUsageCore(@base: @base, link: GetRightOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
        }
        return @continue;
    }

    /// <summary>
    ///     <para>
    ///         Prints the node value using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <param name="sb">
    ///     <para>The sb.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void PrintNodeValue(TLinkAddress node, StringBuilder sb)
    {
        ref var link = ref GetLinkReference(link: node);
        sb.Append(value: ' ');
        sb.Append(value: link.Source);
        sb.Append(value: '-');
        sb.Append(value: '>');
        sb.Append(value: link.Target);
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/Generic/LinksSizeBalancedTreeMethodsBase.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using System.Text;
using Platform.Collections.Methods.Trees;
using Platform.Converters;
using Platform.Delegates;
using static System.Runtime.CompilerServices.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United.Generic;

/// <summary>
///     <para>
///         Represents the links size balanced tree methods base.
///     </para>
///     <para></para>
/// </summary>
/// <seealso cref="SizeBalancedTreeMethods{TLinkAddress}" />
/// <seealso cref="ILinksTreeMethods{TLinkAddress}" />
public abstract unsafe class LinksSizeBalancedTreeMethodsBase<TLinkAddress> : SizeBalancedTreeMethods<TLinkAddress>, ILinksTre
eMethods<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bo
ol>
{

    /// <summary>
    ///     <para>
    ///         The break.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Break;
    /// <summary>
    ///     <para>
    ///         The continue.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly TLinkAddress Continue;
    /// <summary>
    ///     <para>
    ///         The header.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* Header;
    /// <summary>
    ///     <para>
    ///         The links.
    ///     </para>
    ///     <para></para>
    /// </summary>
    protected readonly byte* Links;

    /// <summary>
    ///     <para>
    ///         Initializes a new <see cref="LinksSizeBalancedTreeMethodsBase{TLinkAddress}" /> instance.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="constants">
    ///     <para>A constants.</para>
    ///     <para></para>
    /// </param>
    /// <param name="links">
    ///     <para>A links.</para>
    ///     <para></para>
    /// </param>
    /// <param name="header">
    ///     <para>A header.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected LinksSizeBalancedTreeMethodsBase(LinksConstants<TLinkAddress> constants, byte* links, byte* header)
    {
        Links = links;
        Header = header;
        Break = constants.Break;
        Continue = constants.Continue;
    }

    /// <summary>
    ///     <para>
    ///         The zero.
    ///     </para>
    ///     <para></para>
    /// </summary>
    public TLinkAddress this[TLinkAddress index]
    {
        [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
```

```
        get
        {
            var root = GetTreeRoot();
            if ((index >= GetSize(node: root)))
            {
                return TLinkAddress.Zero;
            }
            while (root != TLinkAddress.Zero)
            {
                var left = GetLeftOrDefault(node: root);
                var leftSize = GetSizeOrZero(node: left);
                if (index < leftSize)
                {
                    root = left;
                    continue;
                }
                if ((index == leftSize))
                {
                    return root;
                }
                root = GetRightOrDefault(node: root);
                index = (index) - (leftSize + TLinkAddress.One);
            }
            return TLinkAddress.Zero; // TODO: Impossible situation exception (only if tree structure broken)
        }
    }

    /// <summary>
    ///     Выполняет поиск и возвращает индекс связи с указанными Source (началом) и Target (концом).
    /// </summary>
    /// <param name="source">Индекс связи, которая является началом на искомой связи.</param>
    /// <param name="target">Индекс связи, которая является концом на искомой связи.</param>
    /// <returns>Индекс искомой связи.</returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public TLinkAddress Search(TLinkAddress source, TLinkAddress target)
    {
        var root = GetTreeRoot();
        while (root != TLinkAddress.Zero)
        {
            ref var rootLink = ref GetLinkReference(link: root);
            var rootSource = rootLink.Source;
            var rootTarget = rootLink.Target;
            if (FirstIsToTheLeftOfSecond(source: source, target: target, rootSource: rootSource, rootTarget: rootTarget)) // n
ode.Key < root.Key
            {
                root = GetLeftOrDefault(node: root);
            }
            else if (FirstIsToTheRightOfSecond(source: source, target: target, rootSource: rootSource, rootTarget: rootTarget)
) // node.Key > root.Key
            {
                root = GetRightOrDefault(node: root);
            }
            else // node.Key == root.Key
            {
                return root;
            }
        }
        return TLinkAddress.Zero;
    }

    // TODO: Return indices range instead of references count
    /// <summary>
    ///     <para>
    ///         Counts the usages using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    public TLinkAddress CountUsages(TLinkAddress link)
    {
        var root = GetTreeRoot();
        var total = GetSize(node: root);
        var totalRightIgnore = TLinkAddress.Zero;
        while (root != TLinkAddress.Zero)
        {
            var @base = GetBasePartValue(link: root);
            if ((@base <= link))
            {
                root = GetRightOrDefault(node: root);
            }
            else
            {
                totalRightIgnore = (totalRightIgnore + GetRightSize(node: root) + TLinkAddress.One);
                root = GetLeftOrDefault(node: root);
```

```
        }
    }
    root = GetTreeRoot();
    var totalLeftIgnore = TLinkAddress.Zero;
    while (root != TLinkAddress.Zero)
    {
        var @base = GetBasePartValue(link: root);
        if ((@base >= link))
        {
            root = GetLeftOrDefault(node: root);
        }
        else
        {
            totalLeftIgnore = (totalLeftIgnore + GetLeftSize(node: root) + TLinkAddress.One);
            root = GetRightOrDefault(node: root);
        }
    }
    return total - totalRightIgnore - totalLeftIgnore;
}
/// <summary>
///     <para>
///         Eaches the usage using the specified base.
///     </para>
///     <para></para>
/// </summary>
/// <param name="base">
///     <para>The base.</para>
///     <para></para>
/// </param>
/// <param name="handler">
///     <para>The handler.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
public TLinkAddress EachUsage(TLinkAddress @base, ReadHandler<TLinkAddress>? handler)
{
    return EachUsageCore(@base: @base, link: GetTreeRoot(), handler: handler);
}

/// <summary>
///     <para>
///         Gets the tree root.
///     </para>
///     <para></para>
/// </summary>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected abstract TLinkAddress GetTreeRoot();

/// <summary>
///     <para>
///         Gets the base part value using the specified link.
///     </para>
///     <para></para>
/// </summary>
/// <param name="link">
///     <para>The link.</para>
///     <para></para>
/// </param>
/// <returns>
///     <para>The link</para>
///     <para></para>
/// </returns>
[MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
protected abstract TLinkAddress GetBasePartValue(TLinkAddress link);

/// <summary>
///     <para>
///         Determines whether this instance first is to the right of second.
///     </para>
///     <para></para>
/// </summary>
/// <param name="source">
///     <para>The source.</para>
///     <para></para>
/// </param>
/// <param name="target">
///     <para>The target.</para>
///     <para></para>
/// </param>
/// <param name="rootSource">
///     <para>The root source.</para>
///     <para></para>
/// </param>
/// <param name="rootTarget">
```

```
    ///     <para>The root target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract bool FirstIsToTheRightOfSecond(TLinkAddress source, TLinkAddress target, TLinkAddress rootSource, TLink
Address rootTarget);

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="source">
    ///     <para>The source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="target">
    ///     <para>The target.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootSource">
    ///     <para>The root source.</para>
    ///     <para></para>
    /// </param>
    /// <param name="rootTarget">
    ///     <para>The root target.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected abstract bool FirstIsToTheLeftOfSecond(TLinkAddress source, TLinkAddress target, TLinkAddress rootSource, TLinkA
ddress rootTarget);

    /// <summary>
    ///     <para>
    ///         Gets the header reference.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <returns>
    ///     <para>A ref links header of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref LinksHeader<TLinkAddress> GetHeaderReference()
    {
        return ref AsRef<LinksHeader<TLinkAddress>>(source: Header);
    }

    /// <summary>
    ///     <para>
    ///         Gets the link reference using the specified link.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="link">
    ///     <para>The link.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A ref raw link of t link</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual ref RawLink<TLinkAddress> GetLinkReference(TLinkAddress link)
    {
        return ref AsRef<RawLink<TLinkAddress>>(source: Links + RawLink<TLinkAddress>.SizeInBytes * long.CreateTruncating(link
));
    }

    /// <summary>
    ///     <para>
    ///         Gets the link values using the specified link index.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="linkIndex">
    ///     <para>The link index.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>A list of t link</para>
    ///     <para></para>
    /// </returns>
```

```
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected virtual IList<TLinkAddress>? GetLinkValues(TLinkAddress linkIndex)
    {
        ref var link = ref GetLinkReference(link: linkIndex);
        return new Link<TLinkAddress>(index: linkIndex, source: link.Source, target: link.Target);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the left of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="first">
    ///     <para>The first.</para>
    ///     <para></para>
    /// </param>
    /// <param name="second">
    ///     <para>The second.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheLeftOfSecond(TLinkAddress first, TLinkAddress second)
    {
        ref var firstLink = ref GetLinkReference(link: first);
        ref var secondLink = ref GetLinkReference(link: second);
        return FirstIsToTheLeftOfSecond(source: firstLink.Source, target: firstLink.Target, rootSource: secondLink.Source, roo
tTarget: secondLink.Target);
    }

    /// <summary>
    ///     <para>
    ///         Determines whether this instance first is to the right of second.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="first">
    ///     <para>The first.</para>
    ///     <para></para>
    /// </param>
    /// <param name="second">
    ///     <para>The second.</para>
    ///     <para></para>
    /// </param>
    /// <returns>
    ///     <para>The bool</para>
    ///     <para></para>
    /// </returns>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override bool FirstIsToTheRightOfSecond(TLinkAddress first, TLinkAddress second)
    {
        ref var firstLink = ref GetLinkReference(link: first);
        ref var secondLink = ref GetLinkReference(link: second);
        return FirstIsToTheRightOfSecond(source: firstLink.Source, target: firstLink.Target, rootSource: secondLink.Source, ro
otTarget: secondLink.Target);
    }

    // TODO: 1. Move target, handler to separate object. 2. Use stack or walker 3. Use low-level MSIL stack.
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    private TLinkAddress EachUsageCore(TLinkAddress @base, TLinkAddress link, ReadHandler<TLinkAddress>? handler)
    {
        var @continue = Continue;
        if (link == TLinkAddress.Zero)
        {
            return @continue;
        }
        var linkBasePart = GetBasePartValue(link: link);
        var @break = Break;
        if ((linkBasePart > @base))
        {
            if ((EachUsageCore(@base: @base, link: GetLeftOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
        }
        else if ((linkBasePart < @base))
        {
            if ((EachUsageCore(@base: @base, link: GetRightOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
        }
        else //if (linkBasePart == @base)
        {
            if (((handler == null ? Continue : handler(link: GetLinkValues(linkIndex: link))) == @break))
            {
                return @break;
            }
```

```
            if ((EachUsageCore(@base: @base, link: GetLeftOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
            if ((EachUsageCore(@base: @base, link: GetRightOrDefault(node: link), handler: handler) == @break))
            {
                return @break;
            }
        }
        return @continue;
    }

    /// <summary>
    ///     <para>
    ///         Prints the node value using the specified node.
    ///     </para>
    ///     <para></para>
    /// </summary>
    /// <param name="node">
    ///     <para>The node.</para>
    ///     <para></para>
    /// </param>
    /// <param name="sb">
    ///     <para>The sb.</para>
    ///     <para></para>
    /// </param>
    [MethodImpl(methodImplOptions: MethodImplOptions.AggressiveInlining)]
    protected override void PrintNodeValue(TLinkAddress node, StringBuilder sb)
    {
        ref var link = ref GetLinkReference(link: node);
        sb.Append(value: ' ');
        sb.Append(value: link.Source);
        sb.Append(value: '-');
        sb.Append(value: '>');
        sb.Append(value: link.Target);
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/Generic/LinksSourcesAvlBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United.Generic
{
    /// <summary>
    /// <para>
    /// Represents the links sources avl balanced tree methods.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="LinksAvlBalancedTreeMethodsBase{TLinkAddress}"/>
    public unsafe class LinksSourcesAvlBalancedTreeMethods<TLinkAddress> : LinksAvlBalancedTreeMethodsBase<TLinkAddress> where
TLinkAddress : IUnsignedNumber<TLinkAddress>, IShiftOperators<TLinkAddress, int, TLinkAddress>, IBitwiseOperators<TLinkAddress
, TLinkAddress, TLinkAddress>, IMinMaxValue<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
    {
        /// <summary>
        /// <para>
        /// Initializes a new <see cref="LinksSourcesAvlBalancedTreeMethods{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="constants">
        /// <para>A constants.</para>
        /// <para></para>
        /// </param>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        /// <param name="header">
        /// <para>A header.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public LinksSourcesAvlBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* links, byte* header) : base(co
nstants, links, header) { }

        /// <summary>
        /// <para>
        /// Gets the left reference using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The ref link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref TLinkAddress GetLeftReference(TLinkAddress node) => ref GetLinkReference(node).LeftAsSource;

        /// <summary>
        /// <para>
        /// Gets the right reference using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The ref link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref TLinkAddress GetRightReference(TLinkAddress node) => ref GetLinkReference(node).RightAsSource;

        /// <summary>
        /// <para>
        /// Gets the left using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetLeft(TLinkAddress node) => GetLinkReference(node).LeftAsSource;
```

```
        /// <summary>
        /// <para>
        /// Gets the right using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetRight(TLinkAddress node) => GetLinkReference(node).RightAsSource;

        /// <summary>
        /// <para>
        /// Sets the left using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="left">
        /// <para>The left.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetLeft(TLinkAddress node, TLinkAddress left) => GetLinkReference(node).LeftAsSource = left;

        /// <summary>
        /// <para>
        /// Sets the right using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="right">
        /// <para>The right.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetRight(TLinkAddress node, TLinkAddress right) => GetLinkReference(node).RightAsSource = righ
t;

        /// <summary>
        /// <para>
        /// Gets the size using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetSize(TLinkAddress node) => GetSizeValue(GetLinkReference(node).SizeAsSource);

        /// <summary>
        /// <para>
        /// Sets the size using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="size">
        /// <para>The size.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetSize(TLinkAddress node, TLinkAddress size) => SetSizeValue(ref GetLinkReference(node).SizeA
sSource, size);

        /// <summary>
        /// <para>
        /// Determines whether this instance get left is child.
        /// </para>
        /// <para></para>
```

```
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool GetLeftIsChild(TLinkAddress node) => GetLeftIsChildValue(GetLinkReference(node).SizeAsSource);

        /// <summary>
        /// <para>
        /// Sets the left is child using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="value">
        /// <para>The value.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetLeftIsChild(TLinkAddress node, bool value) => SetLeftIsChildValue(ref GetLinkReference(node
).SizeAsSource, value);

        /// <summary>
        /// <para>
        /// Determines whether this instance get right is child.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool GetRightIsChild(TLinkAddress node) => GetRightIsChildValue(GetLinkReference(node).SizeAsSource
);

        /// <summary>
        /// <para>
        /// Sets the right is child using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="value">
        /// <para>The value.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetRightIsChild(TLinkAddress node, bool value) => SetRightIsChildValue(ref GetLinkReference(no
de).SizeAsSource, value);

        /// <summary>
        /// <para>
        /// Gets the balance using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The sbyte</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override sbyte GetBalance(TLinkAddress node) => GetBalanceValue(GetLinkReference(node).SizeAsSource);

        /// <summary>
        /// <para>
        /// Sets the balance using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
```

```
        /// </param>
        /// <param name="value">
        /// <para>The value.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetBalance(TLinkAddress node, sbyte value) => SetBalanceValue(ref GetLinkReference(node).SizeA
sSource, value);

        /// <summary>
        /// <para>
        /// Gets the tree root.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetTreeRoot() => GetHeaderReference().RootAsSource;

        /// <summary>
        /// <para>
        /// Gets the base part value using the specified link.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetBasePartValue(TLinkAddress link) => GetLinkReference(link).Source;

        /// <summary>
        /// <para>
        /// Determines whether this instance first is to the left of second.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="firstSource">
        /// <para>The first source.</para>
        /// <para></para>
        /// </param>
        /// <param name="firstTarget">
        /// <para>The first target.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondSource">
        /// <para>The second source.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondTarget">
        /// <para>The second target.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool FirstIsToTheLeftOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress seco
ndSource, TLinkAddress secondTarget) => firstSource < secondSource || (firstSource == secondSource && firstTarget < secondTarg
et);

        /// <summary>
        /// <para>
        /// Determines whether this instance first is to the right of second.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="firstSource">
        /// <para>The first source.</para>
        /// <para></para>
        /// </param>
        /// <param name="firstTarget">
        /// <para>The first target.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondSource">
        /// <para>The second source.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondTarget">
        /// <para>The second target.</para>
        /// <para></para>
        /// </param>
```

```
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool FirstIsToTheRightOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress sec
ondSource, TLinkAddress secondTarget) => firstSource > secondSource || (firstSource == secondSource && firstTarget > secondTar
get);

        /// <summary>
        /// <para>
        /// Clears the node using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void ClearNode(TLinkAddress node)
        {
            ref var link = ref GetLinkReference(node);
            link.LeftAsSource = TLinkAddress.Zero;
            link.RightAsSource = TLinkAddress.Zero;
            link.SizeAsSource = TLinkAddress.Zero;
        }
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/Generic/LinksSourcesRecursionlessSizeBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United.Generic
{
    /// <summary>
    /// <para>
    /// Represents the links sources recursionless size balanced tree methods.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="LinksRecursionlessSizeBalancedTreeMethodsBase{TLinkAddress}"/>
    public unsafe class LinksSourcesRecursionlessSizeBalancedTreeMethods<TLinkAddress> : LinksRecursionlessSizeBalancedTreeMet
hodsBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bo
ol>
    {
        /// <summary>
        /// <para>
        /// Initializes a new <see cref="LinksSourcesRecursionlessSizeBalancedTreeMethods{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="constants">
        /// <para>A constants.</para>
        /// <para></para>
        /// </param>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        /// <param name="header">
        /// <para>A header.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public LinksSourcesRecursionlessSizeBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* links, byte* hea
der) : base(constants, links, header) { }

        /// <summary>
        /// <para>
        /// Gets the left reference using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The ref link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref TLinkAddress GetLeftReference(TLinkAddress node) => ref GetLinkReference(node).LeftAsSource;

        /// <summary>
        /// <para>
        /// Gets the right reference using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The ref link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref TLinkAddress GetRightReference(TLinkAddress node) => ref GetLinkReference(node).RightAsSource;

        /// <summary>
        /// <para>
        /// Gets the left using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetLeft(TLinkAddress node) => GetLinkReference(node).LeftAsSource;
```

```
        /// <summary>
        /// <para>
        /// Gets the right using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetRight(TLinkAddress node) => GetLinkReference(node).RightAsSource;

        /// <summary>
        /// <para>
        /// Sets the left using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="left">
        /// <para>The left.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetLeft(TLinkAddress node, TLinkAddress left) => GetLinkReference(node).LeftAsSource = left;

        /// <summary>
        /// <para>
        /// Sets the right using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="right">
        /// <para>The right.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetRight(TLinkAddress node, TLinkAddress right) => GetLinkReference(node).RightAsSource = righ
t;

        /// <summary>
        /// <para>
        /// Gets the size using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetSize(TLinkAddress node) => GetLinkReference(node).SizeAsSource;

        /// <summary>
        /// <para>
        /// Sets the size using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="size">
        /// <para>The size.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetSize(TLinkAddress node, TLinkAddress size) => GetLinkReference(node).SizeAsSource = size;

        /// <summary>
        /// <para>
        /// Gets the tree root.
        /// </para>
        /// <para></para>
        /// </summary>
```

```
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetTreeRoot() => GetHeaderReference().RootAsSource;

        /// <summary>
        /// <para>
        /// Gets the base part value using the specified link.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetBasePartValue(TLinkAddress link) => GetLinkReference(link).Source;

        /// <summary>
        /// <para>
        /// Determines whether this instance first is to the left of second.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="firstSource">
        /// <para>The first source.</para>
        /// <para></para>
        /// </param>
        /// <param name="firstTarget">
        /// <para>The first target.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondSource">
        /// <para>The second source.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondTarget">
        /// <para>The second target.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool FirstIsToTheLeftOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress seco
ndSource, TLinkAddress secondTarget) => firstSource < secondSource || (firstSource == secondSource && firstTarget < secondTarg
et);

        /// <summary>
        /// <para>
        /// Determines whether this instance first is to the right of second.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="firstSource">
        /// <para>The first source.</para>
        /// <para></para>
        /// </param>
        /// <param name="firstTarget">
        /// <para>The first target.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondSource">
        /// <para>The second source.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondTarget">
        /// <para>The second target.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool FirstIsToTheRightOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress sec
ondSource, TLinkAddress secondTarget) => firstSource > secondSource || (firstSource == secondSource && firstTarget > secondTar
get);

        /// <summary>
        /// <para>
        /// Clears the node using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
```

```
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void ClearNode(TLinkAddress node)
        {
            ref var link = ref GetLinkReference(node);
            link.LeftAsSource = TLinkAddress.Zero;
            link.RightAsSource = TLinkAddress.Zero;
            link.SizeAsSource = TLinkAddress.Zero;
        }
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/Generic/LinksSourcesSizeBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United.Generic
{
    /// <summary>
    /// <para>
    /// Represents the links sources size balanced tree methods.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="LinksSizeBalancedTreeMethodsBase{TLinkAddress}"/>
    public unsafe class LinksSourcesSizeBalancedTreeMethods<TLinkAddress> : LinksSizeBalancedTreeMethodsBase<TLinkAddress> whe
re TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
    {
        /// <summary>
        /// <para>
        /// Initializes a new <see cref="LinksSourcesSizeBalancedTreeMethods{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="constants">
        /// <para>A constants.</para>
        /// <para></para>
        /// </param>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        /// <param name="header">
        /// <para>A header.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public LinksSourcesSizeBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* links, byte* header) : base(c
onstants, links, header) { }

        /// <summary>
        /// <para>
        /// Gets the left reference using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The ref link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref TLinkAddress GetLeftReference(TLinkAddress node) => ref GetLinkReference(node).LeftAsSource;

        /// <summary>
        /// <para>
        /// Gets the right reference using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The ref link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref TLinkAddress GetRightReference(TLinkAddress node) => ref GetLinkReference(node).RightAsSource;

        /// <summary>
        /// <para>
        /// Gets the left using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetLeft(TLinkAddress node) => GetLinkReference(node).LeftAsSource;

        /// <summary>
```

```
        /// <para>
        /// Gets the right using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetRight(TLinkAddress node) => GetLinkReference(node).RightAsSource;

        /// <summary>
        /// <para>
        /// Sets the left using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="left">
        /// <para>The left.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetLeft(TLinkAddress node, TLinkAddress left) => GetLinkReference(node).LeftAsSource = left;

        /// <summary>
        /// <para>
        /// Sets the right using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="right">
        /// <para>The right.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetRight(TLinkAddress node, TLinkAddress right) => GetLinkReference(node).RightAsSource = righ
t;

        /// <summary>
        /// <para>
        /// Gets the size using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetSize(TLinkAddress node) => GetLinkReference(node).SizeAsSource;

        /// <summary>
        /// <para>
        /// Sets the size using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="size">
        /// <para>The size.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetSize(TLinkAddress node, TLinkAddress size) => GetLinkReference(node).SizeAsSource = size;

        /// <summary>
        /// <para>
        /// Gets the tree root.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
```

```
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetTreeRoot() => GetHeaderReference().RootAsSource;

        /// <summary>
        /// <para>
        /// Gets the base part value using the specified link.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetBasePartValue(TLinkAddress link) => GetLinkReference(link).Source;

        /// <summary>
        /// <para>
        /// Determines whether this instance first is to the left of second.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="firstSource">
        /// <para>The first source.</para>
        /// <para></para>
        /// </param>
        /// <param name="firstTarget">
        /// <para>The first target.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondSource">
        /// <para>The second source.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondTarget">
        /// <para>The second target.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool FirstIsToTheLeftOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress seco
ndSource, TLinkAddress secondTarget) => firstSource < secondSource || (firstSource == secondSource && firstTarget < secondTarg
et);

        /// <summary>
        /// <para>
        /// Determines whether this instance first is to the right of second.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="firstSource">
        /// <para>The first source.</para>
        /// <para></para>
        /// </param>
        /// <param name="firstTarget">
        /// <para>The first target.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondSource">
        /// <para>The second source.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondTarget">
        /// <para>The second target.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool FirstIsToTheRightOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress sec
ondSource, TLinkAddress secondTarget) => firstSource > secondSource || (firstSource == secondSource && firstTarget > secondTar
get);

        /// <summary>
        /// <para>
        /// Clears the node using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
```

```
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void ClearNode(TLinkAddress node)
        {
            ref var link = ref GetLinkReference(node);
            link.LeftAsSource = TLinkAddress.Zero;
            link.RightAsSource = TLinkAddress.Zero;
            link.SizeAsSource = TLinkAddress.Zero;
        }
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/Generic/LinksTargetsAvlBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United.Generic
{
    /// <summary>
    /// <para>
    /// Represents the links targets avl balanced tree methods.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="LinksAvlBalancedTreeMethodsBase{TLinkAddress}"/>
    public unsafe class LinksTargetsAvlBalancedTreeMethods<TLinkAddress> : LinksAvlBalancedTreeMethodsBase<TLinkAddress> where
TLinkAddress : IUnsignedNumber<TLinkAddress>, IShiftOperators<TLinkAddress, int, TLinkAddress>, IBitwiseOperators<TLinkAddress
, TLinkAddress, TLinkAddress>, IMinMaxValue<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
    {
        /// <summary>
        /// <para>
        /// Initializes a new <see cref="LinksTargetsAvlBalancedTreeMethods{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="constants">
        /// <para>A constants.</para>
        /// <para></para>
        /// </param>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        /// <param name="header">
        /// <para>A header.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public LinksTargetsAvlBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* links, byte* header) : base(co
nstants, links, header) { }

        /// <summary>
        /// <para>
        /// Gets the left reference using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The ref link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref TLinkAddress GetLeftReference(TLinkAddress node) => ref GetLinkReference(node).LeftAsTarget;

        /// <summary>
        /// <para>
        /// Gets the right reference using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The ref link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref TLinkAddress GetRightReference(TLinkAddress node) => ref GetLinkReference(node).RightAsTarget;

        /// <summary>
        /// <para>
        /// Gets the left using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetLeft(TLinkAddress node) => GetLinkReference(node).LeftAsTarget;
```

```
        /// <summary>
        /// <para>
        /// Gets the right using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetRight(TLinkAddress node) => GetLinkReference(node).RightAsTarget;

        /// <summary>
        /// <para>
        /// Sets the left using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="left">
        /// <para>The left.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetLeft(TLinkAddress node, TLinkAddress left) => GetLinkReference(node).LeftAsTarget = left;

        /// <summary>
        /// <para>
        /// Sets the right using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="right">
        /// <para>The right.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetRight(TLinkAddress node, TLinkAddress right) => GetLinkReference(node).RightAsTarget = righ
t;

        /// <summary>
        /// <para>
        /// Gets the size using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetSize(TLinkAddress node) => GetSizeValue(GetLinkReference(node).SizeAsTarget);

        /// <summary>
        /// <para>
        /// Sets the size using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="size">
        /// <para>The size.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetSize(TLinkAddress node, TLinkAddress size) => SetSizeValue(ref GetLinkReference(node).SizeA
sTarget, size);

        /// <summary>
        /// <para>
        /// Determines whether this instance get left is child.
        /// </para>
        /// <para></para>
```

```
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool GetLeftIsChild(TLinkAddress node) => GetLeftIsChildValue(GetLinkReference(node).SizeAsTarget);

        /// <summary>
        /// <para>
        /// Sets the left is child using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="value">
        /// <para>The value.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetLeftIsChild(TLinkAddress node, bool value) => SetLeftIsChildValue(ref GetLinkReference(node
).SizeAsTarget, value);

        /// <summary>
        /// <para>
        /// Determines whether this instance get right is child.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool GetRightIsChild(TLinkAddress node) => GetRightIsChildValue(GetLinkReference(node).SizeAsTarget
);

        /// <summary>
        /// <para>
        /// Sets the right is child using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="value">
        /// <para>The value.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetRightIsChild(TLinkAddress node, bool value) => SetRightIsChildValue(ref GetLinkReference(no
de).SizeAsTarget, value);

        /// <summary>
        /// <para>
        /// Gets the balance using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The sbyte</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override sbyte GetBalance(TLinkAddress node) => GetBalanceValue(GetLinkReference(node).SizeAsTarget);

        /// <summary>
        /// <para>
        /// Sets the balance using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
```

```
        /// </param>
        /// <param name="value">
        /// <para>The value.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetBalance(TLinkAddress node, sbyte value) => SetBalanceValue(ref GetLinkReference(node).SizeA
sTarget, value);

        /// <summary>
        /// <para>
        /// Gets the tree root.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetTreeRoot() => GetHeaderReference().RootAsTarget;

        /// <summary>
        /// <para>
        /// Gets the base part value using the specified link.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetBasePartValue(TLinkAddress link) => GetLinkReference(link).Target;

        /// <summary>
        /// <para>
        /// Determines whether this instance first is to the left of second.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="firstSource">
        /// <para>The first source.</para>
        /// <para></para>
        /// </param>
        /// <param name="firstTarget">
        /// <para>The first target.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondSource">
        /// <para>The second source.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondTarget">
        /// <para>The second target.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool FirstIsToTheLeftOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress seco
ndSource, TLinkAddress secondTarget) => firstTarget < secondTarget || (firstTarget == secondTarget && firstSource < secondSour
ce);

        /// <summary>
        /// <para>
        /// Determines whether this instance first is to the right of second.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="firstSource">
        /// <para>The first source.</para>
        /// <para></para>
        /// </param>
        /// <param name="firstTarget">
        /// <para>The first target.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondSource">
        /// <para>The second source.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondTarget">
        /// <para>The second target.</para>
        /// <para></para>
        /// </param>
```

```
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool FirstIsToTheRightOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress sec
ondSource, TLinkAddress secondTarget) => firstTarget > secondTarget || (firstTarget == secondTarget && firstSource > secondSou
rce);

        /// <summary>
        /// <para>
        /// Clears the node using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void ClearNode(TLinkAddress node)
        {
            ref var link = ref GetLinkReference(node);
            link.LeftAsTarget = TLinkAddress.Zero;
            link.RightAsTarget = TLinkAddress.Zero;
            link.SizeAsTarget = TLinkAddress.Zero;
        }
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/Generic/LinksTargetsRecursionlessSizeBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United.Generic
{
    /// <summary>
    /// <para>
    /// Represents the links targets recursionless size balanced tree methods.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="LinksRecursionlessSizeBalancedTreeMethodsBase{TLinkAddress}"/>
    public unsafe class LinksTargetsRecursionlessSizeBalancedTreeMethods<TLinkAddress> : LinksRecursionlessSizeBalancedTreeMet
hodsBase<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bo
ol>
    {
        /// <summary>
        /// <para>
        /// Initializes a new <see cref="LinksTargetsRecursionlessSizeBalancedTreeMethods{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="constants">
        /// <para>A constants.</para>
        /// <para></para>
        /// </param>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        /// <param name="header">
        /// <para>A header.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public LinksTargetsRecursionlessSizeBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* links, byte* hea
der) : base(constants, links, header) { }

        /// <summary>
        /// <para>
        /// Gets the left reference using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The ref link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref TLinkAddress GetLeftReference(TLinkAddress node) => ref GetLinkReference(node).LeftAsTarget;

        /// <summary>
        /// <para>
        /// Gets the right reference using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The ref link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref TLinkAddress GetRightReference(TLinkAddress node) => ref GetLinkReference(node).RightAsTarget;

        /// <summary>
        /// <para>
        /// Gets the left using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetLeft(TLinkAddress node) => GetLinkReference(node).LeftAsTarget;
```

```
        /// <summary>
        /// <para>
        /// Gets the right using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetRight(TLinkAddress node) => GetLinkReference(node).RightAsTarget;

        /// <summary>
        /// <para>
        /// Sets the left using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="left">
        /// <para>The left.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetLeft(TLinkAddress node, TLinkAddress left) => GetLinkReference(node).LeftAsTarget = left;

        /// <summary>
        /// <para>
        /// Sets the right using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="right">
        /// <para>The right.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetRight(TLinkAddress node, TLinkAddress right) => GetLinkReference(node).RightAsTarget = righ
t;

        /// <summary>
        /// <para>
        /// Gets the size using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetSize(TLinkAddress node) => GetLinkReference(node).SizeAsTarget;

        /// <summary>
        /// <para>
        /// Sets the size using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="size">
        /// <para>The size.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetSize(TLinkAddress node, TLinkAddress size) => GetLinkReference(node).SizeAsTarget = size;

        /// <summary>
        /// <para>
        /// Gets the tree root.
        /// </para>
        /// <para></para>
        /// </summary>
```

```
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetTreeRoot() => GetHeaderReference().RootAsTarget;

        /// <summary>
        /// <para>
        /// Gets the base part value using the specified link.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetBasePartValue(TLinkAddress link) => GetLinkReference(link).Target;

        /// <summary>
        /// <para>
        /// Determines whether this instance first is to the left of second.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="firstSource">
        /// <para>The first source.</para>
        /// <para></para>
        /// </param>
        /// <param name="firstTarget">
        /// <para>The first target.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondSource">
        /// <para>The second source.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondTarget">
        /// <para>The second target.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool FirstIsToTheLeftOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress seco
ndSource, TLinkAddress secondTarget) => firstTarget < secondTarget || (firstTarget == secondTarget && firstSource < secondSour
ce);

        /// <summary>
        /// <para>
        /// Determines whether this instance first is to the right of second.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="firstSource">
        /// <para>The first source.</para>
        /// <para></para>
        /// </param>
        /// <param name="firstTarget">
        /// <para>The first target.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondSource">
        /// <para>The second source.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondTarget">
        /// <para>The second target.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool FirstIsToTheRightOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress sec
ondSource, TLinkAddress secondTarget) => firstTarget > secondTarget || (firstTarget == secondTarget && firstSource > secondSou
rce);

        /// <summary>
        /// <para>
        /// Clears the node using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
```

```
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void ClearNode(TLinkAddress node)
        {
            ref var link = ref GetLinkReference(node);
            link.LeftAsTarget = TLinkAddress.Zero;
            link.RightAsTarget = TLinkAddress.Zero;
            link.SizeAsTarget = TLinkAddress.Zero;
        }
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/Generic/LinksTargetsSizeBalancedTreeMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United.Generic
{
    /// <summary>
    /// <para>
    /// Represents the links targets size balanced tree methods.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="LinksSizeBalancedTreeMethodsBase{TLinkAddress}"/>
    public unsafe class LinksTargetsSizeBalancedTreeMethods<TLinkAddress> : LinksSizeBalancedTreeMethodsBase<TLinkAddress> whe
re TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
    {
        /// <summary>
        /// <para>
        /// Initializes a new <see cref="LinksTargetsSizeBalancedTreeMethods{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="constants">
        /// <para>A constants.</para>
        /// <para></para>
        /// </param>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        /// <param name="header">
        /// <para>A header.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public LinksTargetsSizeBalancedTreeMethods(LinksConstants<TLinkAddress> constants, byte* links, byte* header) : base(c
onstants, links, header) { }

        /// <summary>
        /// <para>
        /// Gets the left reference using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The ref link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref TLinkAddress GetLeftReference(TLinkAddress node) => ref GetLinkReference(node).LeftAsTarget;

        /// <summary>
        /// <para>
        /// Gets the right reference using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The ref link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref TLinkAddress GetRightReference(TLinkAddress node) => ref GetLinkReference(node).RightAsTarget;

        /// <summary>
        /// <para>
        /// Gets the left using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetLeft(TLinkAddress node) => GetLinkReference(node).LeftAsTarget;

        /// <summary>
```

```
        /// <para>
        /// Gets the right using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetRight(TLinkAddress node) => GetLinkReference(node).RightAsTarget;

        /// <summary>
        /// <para>
        /// Sets the left using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="left">
        /// <para>The left.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetLeft(TLinkAddress node, TLinkAddress left) => GetLinkReference(node).LeftAsTarget = left;

        /// <summary>
        /// <para>
        /// Sets the right using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="right">
        /// <para>The right.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetRight(TLinkAddress node, TLinkAddress right) => GetLinkReference(node).RightAsTarget = righ
t;

        /// <summary>
        /// <para>
        /// Gets the size using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetSize(TLinkAddress node) => GetLinkReference(node).SizeAsTarget;

        /// <summary>
        /// <para>
        /// Sets the size using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        /// <param name="size">
        /// <para>The size.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetSize(TLinkAddress node, TLinkAddress size) => GetLinkReference(node).SizeAsTarget = size;

        /// <summary>
        /// <para>
        /// Gets the tree root.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
```

```
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetTreeRoot() => GetHeaderReference().RootAsTarget;

        /// <summary>
        /// <para>
        /// Gets the base part value using the specified link.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetBasePartValue(TLinkAddress link) => GetLinkReference(link).Target;

        /// <summary>
        /// <para>
        /// Determines whether this instance first is to the left of second.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="firstSource">
        /// <para>The first source.</para>
        /// <para></para>
        /// </param>
        /// <param name="firstTarget">
        /// <para>The first target.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondSource">
        /// <para>The second source.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondTarget">
        /// <para>The second target.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool FirstIsToTheLeftOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress seco
ndSource, TLinkAddress secondTarget) => firstTarget < secondTarget || (firstTarget == secondTarget && firstSource < secondSour
ce);

        /// <summary>
        /// <para>
        /// Determines whether this instance first is to the right of second.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="firstSource">
        /// <para>The first source.</para>
        /// <para></para>
        /// </param>
        /// <param name="firstTarget">
        /// <para>The first target.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondSource">
        /// <para>The second source.</para>
        /// <para></para>
        /// </param>
        /// <param name="secondTarget">
        /// <para>The second target.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override bool FirstIsToTheRightOfSecond(TLinkAddress firstSource, TLinkAddress firstTarget, TLinkAddress sec
ondSource, TLinkAddress secondTarget) => firstTarget > secondTarget || (firstTarget == secondTarget && firstSource > secondSou
rce);

        /// <summary>
        /// <para>
        /// Clears the node using the specified node.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="node">
```

```
        /// <para>The node.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void ClearNode(TLinkAddress node)
        {
            ref var link = ref GetLinkReference(node);
            link.LeftAsTarget = TLinkAddress.Zero;
            link.RightAsTarget = TLinkAddress.Zero;
            link.SizeAsTarget = TLinkAddress.Zero;
        }
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/Generic/UnitedMemoryLinks.cs

```
using System;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Singletons;
using Platform.Memory;
using static System.Runtime.CompilerServices.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United.Generic
{
    /// <summary>
    /// <para>
    /// Represents the united memory links.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="UnitedMemoryLinksBase{TLinkAddress}"/>
    public unsafe class UnitedMemoryLinks<TLinkAddress> : UnitedMemoryLinksBase<TLinkAddress> where TLinkAddress : IUnsignedNu
mber<TLinkAddress>, IShiftOperators<TLinkAddress, int, TLinkAddress>, IBitwiseOperators<TLinkAddress, TLinkAddress, TLinkAddre
ss>, IMinMaxValue<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
    {
        private readonly Func<ILinksTreeMethods<TLinkAddress>> _createSourceTreeMethods;
        private readonly Func<ILinksTreeMethods<TLinkAddress>> _createTargetTreeMethods;
        private byte* _header;
        private byte* _links;

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="UnitedMemoryLinks{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="address">
        /// <para>A address.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public UnitedMemoryLinks(string address) : this(address, DefaultLinksSizeStep) { }

        /// <summary>
        /// Создаёт экземпляр базы данных Links в файле по указанному адресу, с указанным минимальным шагом расширения базы да
нных.
        /// </summary>
        /// <param name="address">Полный пусть к файлу базы данных.</param>
        /// <param name="memoryReservationStep">Минимальный шаг расширения базы данных в байтах.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public UnitedMemoryLinks(string address, long memoryReservationStep) : this(new FileMappedResizableDirectMemory(addres
s, memoryReservationStep), memoryReservationStep) { }

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="UnitedMemoryLinks{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="memory">
        /// <para>A memory.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public UnitedMemoryLinks(IResizableDirectMemory memory) : this(memory, DefaultLinksSizeStep) { }

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="UnitedMemoryLinks{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="memory">
        /// <para>A memory.</para>
        /// <para></para>
        /// </param>
        /// <param name="memoryReservationStep">
        /// <para>A memory reservation step.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public UnitedMemoryLinks(IResizableDirectMemory memory, long memoryReservationStep) : this(memory, memoryReservationSt
ep, Default<LinksConstants<TLinkAddress>>.Instance, IndexTreeType.Default) { }

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="UnitedMemoryLinks{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="memory">
        /// <para>A memory.</para>
        /// <para></para>
        /// </param>
```

```
        /// <param name="memoryReservationStep">
        /// <para>A memory reservation step.</para>
        /// <para></para>
        /// </param>
        /// <param name="constants">
        /// <para>A constants.</para>
        /// <para></para>
        /// </param>
        /// <param name="indexTreeType">
        /// <para>A index tree type.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public UnitedMemoryLinks(IResizableDirectMemory memory, long memoryReservationStep, LinksConstants<TLinkAddress> const
ants, IndexTreeType indexTreeType) : base(memory, memoryReservationStep, constants)
        {
            if (indexTreeType == IndexTreeType.SizedAndThreadedAVLBalancedTree)
            {
                _createSourceTreeMethods = () => new LinksSourcesAvlBalancedTreeMethods<TLinkAddress>(_constants, _links, _hea
der);
                _createTargetTreeMethods = () => new LinksTargetsAvlBalancedTreeMethods<TLinkAddress>(_constants, _links, _hea
der);
            }
            else
            {
                _createSourceTreeMethods = () => new LinksSourcesSizeBalancedTreeMethods<TLinkAddress>(_constants, _links, _he
ader);
                _createTargetTreeMethods = () => new LinksTargetsSizeBalancedTreeMethods<TLinkAddress>(_constants, _links, _he
ader);
            }
            InitializeMemory(memory, memoryReservationStep, () => SetPointersCore(memory));
        }

        /// <summary>
        /// <para>
        /// Sets the pointers using the specified memory.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="memory">
        /// <para>The memory.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetPointers(IResizableDirectMemory memory) => SetPointersCore(memory);

        private void SetPointersCore(IResizableDirectMemory memory)
        {
            _links = (byte*)memory.Pointer;
            _header = _links;
            SourcesTreeMethods = _createSourceTreeMethods();
            TargetsTreeMethods = _createTargetTreeMethods();
            UnusedLinksListMethods = new UnusedLinksListMethods<TLinkAddress>(_links, _header);
        }

        /// <summary>
        /// <para>
        /// Resets the pointers.
        /// </para>
        /// <para></para>
        /// </summary>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void ResetPointers()
        {
            base.ResetPointers();
            _links = null;
            _header = null;
        }

        /// <summary>
        /// <para>
        /// Gets the header reference.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
        /// <para>A ref links header of t link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref LinksHeader<TLinkAddress> GetHeaderReference() => ref AsRef<LinksHeader<TLinkAddress>>(_header)
;

        /// <summary>
        /// <para>
        /// Gets the link reference using the specified link index.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="linkIndex">
        /// <para>The link index.</para>
        /// <para></para>
```

```
        /// </param>
        /// <returns>
        /// <para>A ref raw link of t link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override ref RawLink<TLinkAddress> GetLinkReference(TLinkAddress linkIndex) => ref AsRef<RawLink<TLinkAddres
s>>(_links + (LinkSizeInBytes * long.CreateTruncating(linkIndex)));
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/Generic/UnitedMemoryLinksBase.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Disposables;
using Platform.Singletons;
using Platform.Converters;
using Platform.Numbers;
using Platform.Memory;
using Platform.Data.Exceptions;
using Platform.Delegates;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United.Generic
{
    /// <summary>
    /// <para>
    /// Represents the united memory links base.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="DisposableBase"/>
    /// <seealso cref="ILinks{TLinkAddress}"/>
    public abstract class UnitedMemoryLinksBase<TLinkAddress> : DisposableBase, ILinks<TLinkAddress> where TLinkAddress : IUns
ignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
    {
        private static readonly Comparer<TLinkAddress> _comparer = Comparer<TLinkAddress>.Default;
        private static readonly TLinkAddress _zero = TLinkAddress.Zero;
        private static readonly TLinkAddress _one = TLinkAddress.One;

        /// <summary>Возвращает размер одной связи в байтах.</summary>
        /// <remarks>
        /// Используется только во вне класса, не рекомедуется использовать внутри.
        /// Так как во вне не обязательно будет доступен unsafe C#.
        /// </remarks>
        public static readonly long LinkSizeInBytes = RawLink<TLinkAddress>.SizeInBytes;

        /// <summary>
        /// <para>
        /// The size in bytes.
        /// </para>
        /// <para></para>
        /// </summary>
        public static readonly long LinkHeaderSizeInBytes = LinksHeader<TLinkAddress>.SizeInBytes;

        /// <summary>
        /// <para>
        /// The link size in bytes.
        /// </para>
        /// <para></para>
        /// </summary>
        public static readonly long DefaultLinksSizeStep = LinkSizeInBytes * 1024 * 1024;

        /// <summary>
        /// <para>
        /// The memory.
        /// </para>
        /// <para></para>
        /// </summary>
        protected readonly IResizableDirectMemory _memory;
        /// <summary>
        /// <para>
        /// The memory reservation step.
        /// </para>
        /// <para></para>
        /// </summary>
        protected readonly long _memoryReservationStep;

        /// <summary>
        /// <para>
        /// The targets tree methods.
        /// </para>
        /// <para></para>
        /// </summary>
        protected ILinksTreeMethods<TLinkAddress>? TargetsTreeMethods;
        private ILinksTreeMethods<TLinkAddress> RequiredTargetsTreeMethods => TargetsTreeMethods ?? throw new ObjectDisposedEx
ception(GetType().Name);
        /// <summary>
        /// <para>
        /// The sources tree methods.
        /// </para>
        /// <para></para>
        /// </summary>
        protected ILinksTreeMethods<TLinkAddress>? SourcesTreeMethods;
        private ILinksTreeMethods<TLinkAddress> RequiredSourcesTreeMethods => SourcesTreeMethods ?? throw new ObjectDisposedEx
ception(GetType().Name);
        // TODO: Возможно чтобы гарантированно проверять на то, является ли связь удалённой, нужно использовать не список а де
рево, так как так можно быстрее проверить на наличие связи внутри
        /// <summary>
        /// <para>
```

```
        /// The unused links list methods.
        /// </para>
        /// <para></para>
        /// </summary>
        protected ILinksListMethods<TLinkAddress>? UnusedLinksListMethods;
        private ILinksListMethods<TLinkAddress> RequiredUnusedLinksListMethods => UnusedLinksListMethods ?? throw new ObjectDi
sposedException(GetType().Name);

        /// <summary>
        /// Возвращает общее число связей находящихся в хранилище.
        /// </summary>
        protected virtual TLinkAddress Total
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get
            {
                ref var header = ref GetHeaderReference();
                return Subtract(header.AllocatedLinks, header.FreeLinks);
            }
        }

        /// <summary>
        /// <para>
        /// Gets the constants value.
        /// </para>
        /// <para></para>
        /// </summary>
        protected readonly LinksConstants<TLinkAddress> _constants;

        /// <summary>Gets the configured constants for the store.</summary>
        public virtual LinksConstants<TLinkAddress> Constants => _constants;

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="UnitedMemoryLinksBase{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="memory">
        /// <para>A memory.</para>
        /// <para></para>
        /// </param>
        /// <param name="memoryReservationStep">
        /// <para>A memory reservation step.</para>
        /// <para></para>
        /// </param>
        /// <param name="constants">
        /// <para>A constants.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected UnitedMemoryLinksBase(IResizableDirectMemory memory, long memoryReservationStep, LinksConstants<TLinkAddress
> constants)
        {
            _memory = memory;
            _memoryReservationStep = memoryReservationStep;
            _constants = constants;
        }

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="UnitedMemoryLinksBase{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="memory">
        /// <para>A memory.</para>
        /// <para></para>
        /// </param>
        /// <param name="memoryReservationStep">
        /// <para>A memory reservation step.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected UnitedMemoryLinksBase(IResizableDirectMemory memory, long memoryReservationStep) : this(memory, memoryReserv
ationStep, Default<LinksConstants<TLinkAddress>>.Instance) { }

        /// <summary>
        /// <para>
        /// Inits the memory.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="memory">
        /// <para>The memory.</para>
        /// <para></para>
        /// </param>
        /// <param name="memoryReservationStep">
        /// <para>The memory reservation step.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
```

```
        protected virtual void Init(IResizableDirectMemory memory, long memoryReservationStep)
            => InitializeMemory(memory, memoryReservationStep, () => SetPointers(memory));

        /// <summary>Initializes memory using a concrete pointer initializer during construction.</summary>
        protected unsafe void InitializeMemory(IResizableDirectMemory memory, long memoryReservationStep, Action setPointers)
        {
            if (memory.ReservedCapacity < memoryReservationStep)
            {
                memory.ReservedCapacity = memoryReservationStep;
            }
            setPointers();
            ref var header = ref System.Runtime.CompilerServices.Unsafe.AsRef<LinksHeader<TLinkAddress>>((void*)memory.Pointer
);
            // Гарантия корректности _memory.UsedCapacity относительно _header->AllocatedLinks
            memory.UsedCapacity = (long.CreateTruncating(header.AllocatedLinks) * LinkSizeInBytes) + LinkHeaderSizeInBytes;
            // Гарантия корректности _header->ReservedLinks относительно _memory.ReservedCapacity
            header.ReservedLinks = TLinkAddress.CreateTruncating((memory.ReservedCapacity - LinkHeaderSizeInBytes) / LinkSizeI
nBytes);
        }

        /// <summary>
        /// <para>
        /// Counts the substitution.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="restriction">
        /// <para>The query restriction. Null or an empty list means all links.</para>
        /// <para></para>
        /// </param>
        /// <exception cref="NotSupportedException">
        /// <para>Other restriction sizes and methods are not supported.</para>
        /// <para></para>
        /// </exception>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public virtual TLinkAddress Count(IList<TLinkAddress>? restriction)
        {
            restriction ??= System.Array.Empty<TLinkAddress>();
            // Если нет ограничений, тогда возвращаем общее число связей находящихся в хранилище.
            if (restriction is null || restriction.Count == 0)
            {
                return Total;
            }
            var constants = Constants;
            var any = constants.Any;
            var index = this.GetIndex(restriction);
            if (restriction.Count == 1)
            {
                if (index == any)
                {
                    return Total;
                }
                return Exists(index) ? GetOne() : GetZero();
            }
            if (restriction.Count == 2)
            {
                var value = restriction[1];
                if (index == any)
                {
                    if (value == any)
                    {
                        return Total; // Any - как отсутствие ограничения
                    }
                    var count = TLinkAddress.Zero;
                    if (value == constants.Null)
                    {
                        // Null endpoints are absent from the usage indexes.
                        Each(restriction, _ =>
                        {
                            count += TLinkAddress.One;
                            return constants.Continue;
                        });
                        return count;
                    }
                    count = RequiredSourcesTreeMethods.CountUsages(value);
                    // Count the union, including every distinct raw record with equal endpoints.
                    RequiredTargetsTreeMethods.EachUsage(value, link =>
                    {
                        if (this.GetSource(link) != value)
                        {
                            count += TLinkAddress.One;
                        }
                        return constants.Continue;
                    });
                    return count;
                }
                else
                {
```

```
            if (!Exists(index))
            {
                return GetZero();
            }
            if (value == any)
            {
                return GetOne();
            }
            ref var storedLinkValue = ref GetLinkReference(index);
            if (AreEqual(storedLinkValue.Source, value) || AreEqual(storedLinkValue.Target, value))
            {
                return GetOne();
            }
            return GetZero();
        }
    }
    if (restriction.Count == 3)
    {
        var source = this.GetSource(restriction);
        var target = this.GetTarget(restriction);
        if (index == any)
        {
            if (source == any && target == any)
            {
                return Total;
            }
            else if (source == any)
            {
                return RequiredTargetsTreeMethods.CountUsages(target);
            }
            else if (target == any)
            {
                return RequiredSourcesTreeMethods.CountUsages(source);
            }
            else //if(source != Any && target != Any)
            {
                // Эквивалент Exists(source, target) => Count(Any, source, target) > 0
                var link = RequiredSourcesTreeMethods.Search(source, target);
                return AreEqual(link, constants.Null) ? GetZero() : GetOne();
            }
        }
        else
        {
            if (!Exists(index))
            {
                return GetZero();
            }
            if (source == any && target == any)
            {
                return GetOne();
            }
            ref var storedLinkValue = ref GetLinkReference(index);
            // Three-part restrictions match each endpoint in its own position.
            if ((source == any || storedLinkValue.Source == source) &&
                (target == any || storedLinkValue.Target == target))
            {
                return GetOne();
            }
            return GetZero();
        }
    }
    throw new NotSupportedException("Other restriction sizes and methods are not supported.");
}

/// <summary>
/// <para>
/// Eaches the handler.
/// </para>
/// <para></para>
/// </summary>
/// <param name="handler">
/// <para>The handler.</para>
/// <para></para>
/// </param>
/// <param name="restriction">
/// <para>The query restriction. Null or an empty list means all links.</para>
/// <para></para>
/// </param>
/// <exception cref="NotSupportedException">
/// <para>Other restriction sizes and methods are not supported.</para>
/// <para></para>
/// </exception>
/// <returns>
/// <para>The link</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public virtual TLinkAddress Each(IList<TLinkAddress>? restriction, ReadHandler<TLinkAddress>? handler)
{
    restriction ??= System.Array.Empty<TLinkAddress>();
    handler ??= _ => Constants.Continue;
    var constants = Constants;
```

```
            var @break = constants.Break;
            if (restriction is null || restriction.Count == 0)
            {
                for (var link = GetOne(); LessOrEqualThan(link, GetHeaderReference().AllocatedLinks); link = link + TLinkAddre
ss.One)
                {
                    if (Exists(link) && AreEqual(handler(GetLinkStruct(link)), @break))
                    {
                        return @break;
                    }
                }
                return constants.Continue;
            }
            var @continue = constants.Continue;
            var any = constants.Any;
            var index = this.GetIndex(restriction);
            if (restriction.Count == 1)
            {
                if (index == any)
                {
                    return Each(Array.Empty<TLinkAddress>(), handler);
                }
                if (!Exists(index))
                {
                    return @continue;
                }
                return handler(GetLinkStruct(index));
            }
            if (restriction.Count == 2)
            {
                var value = restriction[1];
                if (index == any)
                {
                    if (value == any)
                    {
                        return Each(Array.Empty<TLinkAddress>(), handler);
                    }
                    if (value == constants.Null)
                    {
                        return Each(Array.Empty<TLinkAddress>(), link =>
                            this.GetSource(link) == value || this.GetTarget(link) == value ? handler(link) : @continue);
                    }
                    if (AreEqual(Each(new Link<TLinkAddress>(index, value, any), handler), @break))
                    {
                        return @break;
                    }
                    return Each(new Link<TLinkAddress>(index, any, value), link =>
                        this.GetSource(link) == value ? @continue : handler(link));
                }
                else
                {
                    if (!Exists(index))
                    {
                        return @continue;
                    }
                    if (value == any)
                    {
                        return handler(GetLinkStruct(index));
                    }
                    ref var storedLinkValue = ref GetLinkReference(index);
                    if (AreEqual(storedLinkValue.Source, value) ||
                        AreEqual(storedLinkValue.Target, value))
                    {
                        return handler(GetLinkStruct(index));
                    }
                    return @continue;
                }
            }
            if (restriction.Count == 3)
            {
                var source = this.GetSource(restriction);
                var target = this.GetTarget(restriction);
                if (index == any)
                {
                    if (source == any && target == any)
                    {
                        return Each(Array.Empty<TLinkAddress>(), handler);
                    }
                    else if (source == any)
                    {
                        return RequiredTargetsTreeMethods.EachUsage(target, handler);
                    }
                    else if (target == any)
                    {
                        return RequiredSourcesTreeMethods.EachUsage(source, handler);
                    }
                    else //if(source != Any && target != Any)
                    {
                        var link = RequiredSourcesTreeMethods.Search(source, target);
                        return AreEqual(link, constants.Null) ? @continue : handler(GetLinkStruct(link));
                    }
                }
```

```
                else
                {
                    if (!Exists(index))
                    {
                        return @continue;
                    }
                    if (source == any && target == any)
                    {
                        return handler(GetLinkStruct(index));
                    }
                    ref var storedLinkValue = ref GetLinkReference(index);
                    // Three-part restrictions match each endpoint in its own position.
                    if ((source == any || storedLinkValue.Source == source) &&
                        (target == any || storedLinkValue.Target == target))
                    {
                        return handler(GetLinkStruct(index));
                    }
                    return @continue;
                }
            }
            throw new NotSupportedException("Other restriction sizes and methods are not supported.");
        }

        /// <summary>Updates the link that matches the restriction.</summary>
        /// <remarks>
        /// The caller must supply a live internal address and valid endpoint/index state.
        /// This raw operation does not validate existence. Select an outer
        /// LinksInnerReferenceExistenceValidator or a per-call EnsureLinkExists check
        /// before resolver work when missing internal addresses must be rejected.
        /// TODO: Возможно можно перемещать значения, если указан индекс, но значение существует в другом месте (но не в менед
жере памяти, а в логике Links)
        /// </remarks>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public virtual TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<T
LinkAddress>? handler)
        {
            var constants = Constants;
            var @null = constants.Null;
            var linkIndex = this.GetIndex(restriction);
            var before = GetLinkStruct(linkIndex);
            ref var link = ref GetLinkReference(linkIndex);
            ref var header = ref GetHeaderReference();
            ref var firstAsSource = ref header.RootAsSource;
            ref var firstAsTarget = ref header.RootAsTarget;
            // Будет корректно работать только в том случае, если пространство выделенной связи предварительно заполнено нулям
и
            if (!AreEqual(link.Source, @null))
            {
                RequiredSourcesTreeMethods.Detach(ref firstAsSource, linkIndex);
            }
            if (!AreEqual(link.Target, @null))
            {
                RequiredTargetsTreeMethods.Detach(ref firstAsTarget, linkIndex);
            }
            link.Source = this.GetSource(substitution);
            link.Target = this.GetTarget(substitution);
            if (!AreEqual(link.Source, @null))
            {
                RequiredSourcesTreeMethods.Attach(ref firstAsSource, linkIndex);
            }
            if (!AreEqual(link.Target, @null))
            {
                RequiredTargetsTreeMethods.Attach(ref firstAsTarget, linkIndex);
            }
            return handler != null ? handler(before, GetLinkStruct(linkIndex)) : Constants.Continue;
        }

        /// <summary>Creates a link and invokes the optional callback.</summary>
        /// <remarks>
        /// TODO: Возможно нужно будет заполнение нулями, если внешнее API ими не заполняет пространство
        /// </remarks>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public virtual TLinkAddress Create(IList<TLinkAddress>? substitution, WriteHandler<TLinkAddress>? handler)
        {
            ref var header = ref GetHeaderReference();
            var freeLink = header.FirstFreeLink;
            if (!AreEqual(freeLink, Constants.Null))
            {
                RequiredUnusedLinksListMethods.Detach(freeLink);
            }
            else
            {
                var maximumPossibleInnerReference = Constants.InternalReferencesRange.Maximum;
                if (GreaterOrEqualThan(header.AllocatedLinks, maximumPossibleInnerReference))
                {
                    throw new LinksLimitReachedException<TLinkAddress>(maximumPossibleInnerReference);
                }
                if (GreaterOrEqualThan(header.AllocatedLinks, header.ReservedLinks - TLinkAddress.One))
                {
                    _memory.ReservedCapacity += _memoryReservationStep;
                    SetPointers(_memory);
                    header = ref GetHeaderReference();
```

```
                    header.ReservedLinks = TLinkAddress.CreateTruncating(_memory.ReservedCapacity / LinkSizeInBytes);
                }
                freeLink = header.AllocatedLinks = header.AllocatedLinks + TLinkAddress.One;
                _memory.UsedCapacity += LinkSizeInBytes;
            }
            return handler != null ? handler(null, new Link<TLinkAddress>(freeLink, Constants.Null, Constants.Null)) : Constan
ts.Continue;
        }

        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Deletes a live link whose endpoints have already been reset. The caller owns these preconditions.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="restriction">
        /// <para>The restriction containing the index of the link to delete.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public virtual TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
        {
            System.ArgumentNullException.ThrowIfNull(restriction);
            ref var header = ref GetHeaderReference();
            var link = restriction[Constants.IndexPart];
            var before = GetLinkStruct(link);
            if (LessThan(link, header.AllocatedLinks))
            {
                RequiredUnusedLinksListMethods.AttachAsFirst(link);
                // return handler?.Invoke(before, null);
                return handler != null ? handler(before, null) : Constants.Continue;
            }
            else if (AreEqual(link, header.AllocatedLinks))
            {
                header.AllocatedLinks = header.AllocatedLinks - TLinkAddress.One;
                _memory.UsedCapacity -= LinkSizeInBytes;
                // Убираем все связи, находящиеся в списке свободных в конце файла, до тех пор, пока не дойдём до первой сущес
твующей связи
                // Позволяет оптимизировать количество выделенных связей (AllocatedLinks)
                while (GreaterThan(header.AllocatedLinks, GetZero()) && IsUnusedLink(header.AllocatedLinks))
                {
                    RequiredUnusedLinksListMethods.Detach(header.AllocatedLinks);
                    header.AllocatedLinks = header.AllocatedLinks - TLinkAddress.One;
                    _memory.UsedCapacity -= LinkSizeInBytes;
                }
                return handler != null ? handler(before, null) : Constants.Continue;
            }
            return Constants.Continue;
        }

        /// <summary>
        /// <para>
        /// Gets the link struct using the specified link index.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="linkIndex">
        /// <para>The link index.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>A list of t link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public IList<TLinkAddress> GetLinkStruct(TLinkAddress linkIndex)
        {
            ref var link = ref GetLinkReference(linkIndex);
            return new Link<TLinkAddress>(linkIndex, link.Source, link.Target);
        }

        /// <remarks>
        /// TODO: Возможно это должно быть событием, вызываемым из IMemory, в том случае, если адрес реально поменялся
        ///
        /// Указатель this.links может быть в том же месте,
        /// так как 0-я связь не используется и имеет такой же размер как Header,
        /// поэтому header размещается в том же месте, что и 0-я связь
        /// </remarks>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected abstract void SetPointers(IResizableDirectMemory memory);

        /// <summary>
        /// <para>
        /// Resets the pointers.
        /// </para>
        /// <para></para>
        /// </summary>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected virtual void ResetPointers()
        {
            SourcesTreeMethods = null;
```

```
    TargetsTreeMethods = null;
    UnusedLinksListMethods = null;
}

/// <summary>
/// <para>
/// Gets the header reference.
/// </para>
/// <para></para>
/// </summary>
/// <returns>
/// <para>A ref links header of t link</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
protected abstract ref LinksHeader<TLinkAddress> GetHeaderReference();

/// <summary>
/// <para>
/// Gets the link reference using the specified link index.
/// </para>
/// <para></para>
/// </summary>
/// <param name="linkIndex">
/// <para>The link index.</para>
/// <para></para>
/// </param>
/// <returns>
/// <para>A ref raw link of t link</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
protected abstract ref RawLink<TLinkAddress> GetLinkReference(TLinkAddress linkIndex);

/// <summary>
/// <para>
/// Determines whether this instance exists.
/// </para>
/// <para></para>
/// </summary>
/// <param name="link">
/// <para>The link.</para>
/// <para></para>
/// </param>
/// <returns>
/// <para>The bool</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
protected virtual bool Exists(TLinkAddress link)
    => GreaterOrEqualThan(link, Constants.InternalReferencesRange.Minimum)
    && LessOrEqualThan(link, GetHeaderReference().AllocatedLinks)
    && !IsUnusedLink(link);

/// <summary>
/// <para>
/// Determines whether this instance is unused link.
/// </para>
/// <para></para>
/// </summary>
/// <param name="linkIndex">
/// <para>The link index.</para>
/// <para></para>
/// </param>
/// <returns>
/// <para>The bool</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
protected virtual bool IsUnusedLink(TLinkAddress linkIndex)
{
    if (!AreEqual(GetHeaderReference().FirstFreeLink, linkIndex)) // May be this check is not needed
    {
        ref var link = ref GetLinkReference(linkIndex);
        return AreEqual(link.SizeAsSource, TLinkAddress.Zero) && !AreEqual(link.Source, TLinkAddress.Zero);
    }
    else
    {
        return true;
    }
}

/// <summary>
/// <para>
/// Gets the one.
/// </para>
/// <para></para>
/// </summary>
/// <returns>
/// <para>The link</para>
/// <para></para>
/// </returns>
```

```
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected virtual TLinkAddress GetOne() => _one;

        /// <summary>
        /// <para>
        /// Gets the zero.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected virtual TLinkAddress GetZero() => TLinkAddress.Zero;

        /// <summary>
        /// <para>
        /// Determines whether this instance are equal.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="first">
        /// <para>The first.</para>
        /// <para></para>
        /// </param>
        /// <param name="second">
        /// <para>The second.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected virtual bool AreEqual(TLinkAddress first, TLinkAddress second) => first == second;

        /// <summary>
        /// <para>
        /// Determines whether this instance less than.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="first">
        /// <para>The first.</para>
        /// <para></para>
        /// </param>
        /// <param name="second">
        /// <para>The second.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected virtual bool LessThan(TLinkAddress first, TLinkAddress second) => _comparer.Compare(first, second) < 0;

        /// <summary>
        /// <para>
        /// Determines whether this instance less or equal than.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="first">
        /// <para>The first.</para>
        /// <para></para>
        /// </param>
        /// <param name="second">
        /// <para>The second.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected virtual bool LessOrEqualThan(TLinkAddress first, TLinkAddress second) => _comparer.Compare(first, second) <=
0;

        /// <summary>
        /// <para>
        /// Determines whether this instance greater than.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="first">
        /// <para>The first.</para>
        /// <para></para>
        /// </param>
        /// <param name="second">
        /// <para>The second.</para>
```

```
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected virtual bool GreaterThan(TLinkAddress first, TLinkAddress second) => _comparer.Compare(first, second) > 0;

        /// <summary>
        /// <para>
        /// Determines whether this instance greater or equal than.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="first">
        /// <para>The first.</para>
        /// <para></para>
        /// </param>
        /// <param name="second">
        /// <para>The second.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected virtual bool GreaterOrEqualThan(TLinkAddress first, TLinkAddress second) => _comparer.Compare(first, second)
>= 0;

        /// <summary>
        /// <para>
        /// Adds the first.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="first">
        /// <para>The first.</para>
        /// <para></para>
        /// </param>
        /// <param name="second">
        /// <para>The second.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected virtual TLinkAddress Add(TLinkAddress first, TLinkAddress second) => first + second;

        /// <summary>
        /// <para>
        /// Subtracts the first.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="first">
        /// <para>The first.</para>
        /// <para></para>
        /// </param>
        /// <param name="second">
        /// <para>The second.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected virtual TLinkAddress Subtract(TLinkAddress first, TLinkAddress second) => first - second;

        #region Disposable

        /// <summary>
        /// <para>
        /// Gets the allow multiple dispose calls value.
        /// </para>
        /// <para></para>
        /// </summary>
        protected override bool AllowMultipleDisposeCalls
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get => true;
        }

        /// <summary>
        /// <para>
        /// Disposes the manual.
        /// </para>
        /// <para></para>
```

```
        /// </summary>
        /// <param name="manual">
        /// <para>The manual.</para>
        /// <para></para>
        /// </param>
        /// <param name="wasDisposed">
        /// <para>The was disposed.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void Dispose(bool manual, bool wasDisposed)
        {
            if (!wasDisposed)
            {
                ResetPointers();
                _memory.DisposeIfPossible();
            }
        }

        #endregion
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/Generic/UnusedLinksListMethods.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Collections.Methods.Lists;
using Platform.Converters;
using static System.Runtime.CompilerServices.Unsafe;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United.Generic
{
    /// <summary>
    /// <para>
    /// Represents the unused links list methods.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="AbsoluteCircularDoublyLinkedListMethods{TLinkAddress}"/>
    /// <seealso cref="ILinksListMethods{TLinkAddress}"/>
    public unsafe class UnusedLinksListMethods<TLinkAddress> : AbsoluteCircularDoublyLinkedListMethods<TLinkAddress>, ILinksLi
stMethods<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, b
ool>
    {
        private readonly byte* _links;
        private readonly byte* _header;

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="UnusedLinksListMethods{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        /// <param name="header">
        /// <para>A header.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public UnusedLinksListMethods(byte* links, byte* header)
        {
            _links = links;
            _header = header;
        }

        /// <summary>
        /// <para>
        /// Gets the header reference.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
        /// <para>A ref links header of t link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected virtual ref LinksHeader<TLinkAddress> GetHeaderReference() => ref AsRef<LinksHeader<TLinkAddress>>(_header);

        /// <summary>
        /// <para>
        /// Gets the link reference using the specified link.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>A ref raw link of t link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected virtual ref RawLink<TLinkAddress> GetLinkReference(TLinkAddress link) => ref AsRef<RawLink<TLinkAddress>>(_l
inks + (RawLink<TLinkAddress>.SizeInBytes * long.CreateTruncating(link)));

        /// <summary>
        /// <para>
        /// Gets the first.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override TLinkAddress GetFirst() => GetHeaderReference().FirstFreeLink;

        /// <summary>
```

```
/// <para>
/// Gets the last.
/// </para>
/// <para></para>
/// </summary>
/// <returns>
/// <para>The link</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetLast() => GetHeaderReference().LastFreeLink;

/// <summary>
/// <para>
/// Gets the previous using the specified element.
/// </para>
/// <para></para>
/// </summary>
/// <param name="element">
/// <para>The element.</para>
/// <para></para>
/// </param>
/// <returns>
/// <para>The link</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetPrevious(TLinkAddress element) => GetLinkReference(element).Source;

/// <summary>
/// <para>
/// Gets the next using the specified element.
/// </para>
/// <para></para>
/// </summary>
/// <param name="element">
/// <para>The element.</para>
/// <para></para>
/// </param>
/// <returns>
/// <para>The link</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetNext(TLinkAddress element) => GetLinkReference(element).Target;

/// <summary>
/// <para>
/// Gets the size.
/// </para>
/// <para></para>
/// </summary>
/// <returns>
/// <para>The link</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
protected override TLinkAddress GetSize() => GetHeaderReference().FreeLinks;

/// <summary>
/// <para>
/// Sets the first using the specified element.
/// </para>
/// <para></para>
/// </summary>
/// <param name="element">
/// <para>The element.</para>
/// <para></para>
/// </param>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
protected override void SetFirst(TLinkAddress element) => GetHeaderReference().FirstFreeLink = element;

/// <summary>
/// <para>
/// Sets the last using the specified element.
/// </para>
/// <para></para>
/// </summary>
/// <param name="element">
/// <para>The element.</para>
/// <para></para>
/// </param>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
protected override void SetLast(TLinkAddress element) => GetHeaderReference().LastFreeLink = element;

/// <summary>
/// <para>
/// Sets the previous using the specified element.
/// </para>
/// <para></para>
/// </summary>
/// <param name="element">
```

```
        /// <para>The element.</para>
        /// <para></para>
        /// </param>
        /// <param name="previous">
        /// <para>The previous.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetPrevious(TLinkAddress element, TLinkAddress previous) => GetLinkReference(element).Source =
previous;

        /// <summary>
        /// <para>
        /// Sets the next using the specified element.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="element">
        /// <para>The element.</para>
        /// <para></para>
        /// </param>
        /// <param name="next">
        /// <para>The next.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetNext(TLinkAddress element, TLinkAddress next) => GetLinkReference(element).Target = next;

        /// <summary>
        /// <para>
        /// Sets the size using the specified size.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="size">
        /// <para>The size.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void SetSize(TLinkAddress size) => GetHeaderReference().FreeLinks = size;
    }
}
```

csharp/Platform.Data.Doublets/Memory/United/RawLink.cs

```
using Platform.Unsafe;
using System;
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Memory.United
{
    /// <summary>
    /// <para>
    /// The raw link.
    /// </para>
    /// <para></para>
    /// </summary>
    public struct RawLink<TLinkAddress> : IEquatable<RawLink<TLinkAddress>> where TLinkAddress : IUnsignedNumber<TLinkAddress>
    {

        /// <summary>
        /// <para>
        /// The size.
        /// </para>
        /// <para></para>
        /// </summary>
        public static readonly long SizeInBytes = Structure<RawLink<TLinkAddress>>.Size;

        /// <summary>
        /// <para>
        /// The source.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress Source;
        /// <summary>
        /// <para>
        /// The target.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress Target;
        /// <summary>
        /// <para>
        /// The left as source.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress LeftAsSource;
        /// <summary>
        /// <para>
        /// The right as source.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress RightAsSource;
        /// <summary>
        /// <para>
        /// The size as source.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress SizeAsSource;
        /// <summary>
        /// <para>
        /// The left as target.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress LeftAsTarget;
        /// <summary>
        /// <para>
        /// The right as target.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress RightAsTarget;
        /// <summary>
        /// <para>
        /// The size as target.
        /// </para>
        /// <para></para>
        /// </summary>
        public TLinkAddress SizeAsTarget;

        /// <summary>
        /// <para>
        /// Determines whether this instance equals.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="obj">
```

```
        /// <para>The obj.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override bool Equals(object? obj) { return obj is RawLink<TLinkAddress> link && Equals(link); }

        /// <summary>
        /// <para>
        /// Determines whether this instance equals.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="other">
        /// <para>The other.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The bool</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public bool Equals(RawLink<TLinkAddress> other)
            => Source == other.Source
            && Target == other.Target
            && LeftAsSource == other.LeftAsSource
            && RightAsSource == other.RightAsSource
            && SizeAsSource == other.SizeAsSource
            && LeftAsTarget == other.LeftAsTarget
            && RightAsTarget == other.RightAsTarget
            && SizeAsTarget == other.SizeAsTarget;

        /// <summary>
        /// <para>
        /// Gets the hash code.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
        /// <para>The int</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override int GetHashCode() { return (Source, Target, LeftAsSource, RightAsSource, SizeAsSource, LeftAsTarget, R
ightAsTarget, SizeAsTarget).GetHashCode(); }

        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator ==(RawLink<TLinkAddress> left, RawLink<TLinkAddress> right) { return left.Equals(right); }

        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator !=(RawLink<TLinkAddress> left, RawLink<TLinkAddress> right) { return !(left == right); }
    }
}
```

csharp/Platform.Data.Doublets/NullableLinkHelpers.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;

namespace Platform.Data.Doublets;

// Platform.Data 0.16.1 annotates these callback snapshots as non-null even
// though creation/deletion legitimately supply null. Keep their semantics
// until the upstream annotations are corrected.
internal static class NullableLinkHelpers
{
    internal static T HandleNullable<T>(this ref WriteHandlerState<T> state, IList<T>? before, IList<T>? after) where T : IUns
ignedNumber<T>
    {
        if (state.Handler is { } handler)
        {
            state.Apply(handler(before, after));
        }
        return state.Result;
    }

    internal static IList<T> GetRequiredLink<T>(this ILinks<T> links, T link) where T : IUnsignedNumber<T>
        => links.GetLink(link) ?? throw new KeyNotFoundException($"Link {link} does not exist.");
}
```

csharp/Platform.Data.Doublets/Properties/AssemblyInfo.cs

```
using System.Runtime.CompilerServices;

[assembly: InternalsVisibleTo("Platform.Data.Doublets.Tests")]
```

csharp/Platform.Data.Doublets/PropertyOperators/PropertiesOperator.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Interfaces;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.PropertyOperators
{
    /// <summary>
    /// <para>
    /// Represents the properties operator.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="LinksOperatorBase{TLinkAddress}"/>
    /// <seealso cref="IProperties{TLinkAddress, TLinkAddress, TLinkAddress}"/>
    public class PropertiesOperator<TLinkAddress> : LinksOperatorBase<TLinkAddress>, IProperties<TLinkAddress, TLinkAddress, T
LinkAddress> where TLinkAddress : IUnsignedNumber<TLinkAddress>
    {

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="PropertiesOperator{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public PropertiesOperator(ILinks<TLinkAddress> links) : base(links) { }
        /// <summary>
        /// <para>
        /// Gets the value using the specified object.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="object">
        /// <para>The object.</para>
        /// <para></para>
        /// </param>
        /// <param name="property">
        /// <para>The property.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public TLinkAddress GetValue(TLinkAddress @object, TLinkAddress property)
        {
            var links = _links;
            var objectProperty = links.SearchOrDefault(@object, property);
            if (objectProperty == TLinkAddress.Zero)
            {
                return TLinkAddress.Zero;
            }
            var constants = links.Constants;
            var any = constants.Any;
            var query = new Link<TLinkAddress>(any, objectProperty, any);
            var valueLink = links.SingleOrDefault(query);
            if (valueLink == null)
            {
                return TLinkAddress.Zero;
            }
            return links.GetTarget(links.GetIndex(valueLink));
        }
        /// <summary>
        /// <para>
        /// Sets the value using the specified object.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="object">
        /// <para>The object.</para>
        /// <para></para>
        /// </param>
        /// <param name="property">
        /// <para>The property.</para>
        /// <para></para>
        /// </param>
        /// <param name="value">
        /// <para>The value.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public void SetValue(TLinkAddress @object, TLinkAddress property, TLinkAddress value)
        {
```

```
            var links = _links;
            var objectProperty = links.GetOrCreate(@object, property);
            links.DeleteMany(links.AllIndices(links.Constants.Any, objectProperty));
            links.GetOrCreate(objectProperty, value);
        }
    }
}
```

csharp/Platform.Data.Doublets/PropertyOperators/PropertyOperator.cs

```
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Interfaces;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.PropertyOperators
{
    /// <summary>
    /// <para>
    /// Represents the property operator.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="LinksOperatorBase{TLinkAddress}"/>
    /// <seealso cref="IProperty{TLinkAddress, TLinkAddress}"/>
    public class PropertyOperator<TLinkAddress> : LinksOperatorBase<TLinkAddress>, IProperty<TLinkAddress, TLinkAddress> where
TLinkAddress : IUnsignedNumber<TLinkAddress>
    {
        private readonly TLinkAddress _propertyMarker;
        private readonly TLinkAddress _propertyValueMarker;

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="PropertyOperator{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        /// <param name="propertyMarker">
        /// <para>A property marker.</para>
        /// <para></para>
        /// </param>
        /// <param name="propertyValueMarker">
        /// <para>A property value marker.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public PropertyOperator(ILinks<TLinkAddress> links, TLinkAddress propertyMarker, TLinkAddress propertyValueMarker) : b
ase(links)
        {
            _propertyMarker = propertyMarker;
            _propertyValueMarker = propertyValueMarker;
        }

        /// <summary>
        /// <para>
        /// Gets the link.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public TLinkAddress Get(TLinkAddress link)
        {
            var property = _links.SearchOrDefault(link, _propertyMarker);
            return GetValue(GetContainer(property));
        }
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        private TLinkAddress GetContainer(TLinkAddress property)
        {
            var valueContainer = TLinkAddress.Zero;
            if (property == TLinkAddress.Zero)
            {
                return valueContainer;
            }
            var links = _links;
            var constants = links.Constants;
            var countinueConstant = constants.Continue;
            var breakConstant = constants.Break;
            var anyConstant = constants.Any;
            var query = new Link<TLinkAddress>(anyConstant, property, anyConstant);
            links.Each(candidate =>
            {
                var candidateTarget = links.GetTarget(candidate);
                var valueTarget = links.GetTarget(candidateTarget);
                if (valueTarget == _propertyValueMarker)
                {
                    valueContainer = links.GetIndex(candidate);
                    return breakConstant;
```

```
                }
                return countinueConstant;
            }, query);
            return valueContainer;
        }
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        private TLinkAddress GetValue(TLinkAddress container) { return container == TLinkAddress.Zero ? TLinkAddress.Zero : _l
inks.GetTarget(container); }

        /// <summary>
        /// <para>
        /// Sets the link.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="link">
        /// <para>The link.</para>
        /// <para></para>
        /// </param>
        /// <param name="value">
        /// <para>The value.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public void Set(TLinkAddress link, TLinkAddress value)
        {
            var links = _links;
            var property = links.GetOrCreate(link, _propertyMarker);
            var container = GetContainer(property);
            if (container == TLinkAddress.Zero)
            {
                links.GetOrCreate(property, value);
            }
            else
            {
                links.Update(container, property, value);
            }
        }
    }
}
```

csharp/Platform.Data.Doublets/Stacks/Stack.cs

```
using System.Collections;
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Collections.Stacks;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Stacks
{
    /// <summary>
    /// <para>
    /// Represents the stack.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="LinksOperatorBase{TLinkAddress}"/>
    /// <seealso cref="IStack{TLinkAddress}"/>
    public class Stack<TLinkAddress> : LinksOperatorBase<TLinkAddress>, IStack<TLinkAddress> where TLinkAddress : IUnsignedNum
ber<TLinkAddress>
    {
        private readonly TLinkAddress _stack;

        /// <summary>
        /// <para>
        /// Gets the is empty value.
        /// </para>
        /// <para></para>
        /// </summary>
        public bool IsEmpty
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get => Peek() == _stack;
        }

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="Stack"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        /// <param name="stack">
        /// <para>A stack.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Stack(ILinks<TLinkAddress> links, TLinkAddress stack) : base(links) { _stack = stack; }
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        private TLinkAddress GetStackMarker() { return _links.GetSource(_stack); }
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        private TLinkAddress GetTop() { return _links.GetTarget(_stack); }

        /// <summary>
        /// <para>
        /// Peeks this instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
        /// <para>The link</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public TLinkAddress Peek() { return _links.GetTarget(GetTop()); }

        /// <summary>
        /// <para>
        /// Pops this instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <returns>
        /// <para>The element.</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public TLinkAddress Pop()
        {
            var element = Peek();
            if (element != _stack)
            {
                var top = GetTop();
                var previousTop = _links.GetSource(top);
                _links.Update(_stack, GetStackMarker(), previousTop);
                _links.Delete(top);
            }
            return element;
```

```
        }

        /// <summary>
        /// <para>
        /// Pushes the element.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="element">
        /// <para>The element.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public void Push(TLinkAddress element) { _links.Update(_stack, GetStackMarker(), _links.GetOrCreate(GetTop(), element)
); }
    }
}
```

csharp/Platform.Data.Doublets/Stacks/StackExtensions.cs

```
using System.Numerics;
using System.Runtime.CompilerServices;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets.Stacks
{
    /// <summary>
    /// <para>
    /// Represents the stack extensions.
    /// </para>
    /// <para></para>
    /// </summary>
    public static class StackExtensions
    {
        /// <summary>
        /// <para>
        /// Creates the stack using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLinkAddress">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="stackMarker">
        /// <para>The stack marker.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The stack.</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
         public static TLinkAddress CreateStack<TLinkAddress>(this ILinks<TLinkAddress> links, TLinkAddress stackMarker) where
TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            var stackPoint = links.CreatePoint();
            var stack = links.Update(stackPoint, stackMarker, stackPoint);
            return stack;
        }
    }
}
```

csharp/Platform.Data.Doublets/SynchronizedLinks.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;
using System.Runtime.CompilerServices;
using Platform.Data.Doublets;
using Platform.Delegates;
using Platform.Threading.Synchronization;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets
{
    /// <summary>Synchronizes access to an underlying link store.</summary>
    /// <remarks>
    /// TODO: Autogeneration of synchronized wrapper (decorator).
    /// TODO: Try to unfold code of each method using IL generation for performance improvements.
    /// TODO: Or even to unfold multiple layers of implementations.
    /// </remarks>
    public class SynchronizedLinks<TLinkAddress> : ISynchronizedLinks<TLinkAddress> where TLinkAddress : IUnsignedNumber<TLink
Address>
    {
        /// <summary>
        /// <para>
        /// Gets the constants value.
        /// </para>
        /// <para></para>
        /// </summary>
        public LinksConstants<TLinkAddress> Constants
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get;
        }

        /// <summary>
        /// <para>
        /// Gets the sync root value.
        /// </para>
        /// <para></para>
        /// </summary>
        public ISynchronization SyncRoot
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get;
        }

        /// <summary>
        /// <para>
        /// Gets the sync value.
        /// </para>
        /// <para></para>
        /// </summary>
        public ILinks<TLinkAddress> Sync
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get;
        }

        /// <summary>
        /// <para>
        /// Gets the unsync value.
        /// </para>
        /// <para></para>
        /// </summary>
        public ILinks<TLinkAddress> Unsync
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get;
        }

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="SynchronizedLinks{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public SynchronizedLinks(ILinks<TLinkAddress> links) : this(new ReaderWriterLockSynchronization(), links) { }

        /// <summary>
        /// <para>
        /// Initializes a new <see cref="SynchronizedLinks{TLinkAddress}"/> instance.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="synchronization">
        /// <para>A synchronization.</para>
        /// <para></para>
```

```
        /// </param>
        /// <param name="links">
        /// <para>A links.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public SynchronizedLinks(ISynchronization synchronization, ILinks<TLinkAddress> links)
        {
            SyncRoot = synchronization;
            Sync = this;
            Unsync = links;
            Constants = links.Constants;
        }

        /// <summary>
        /// <para>
        /// Counts the restriction.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="restriction">
        /// <para>The restriction.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link address</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public TLinkAddress Count(IList<TLinkAddress>? restriction) { return SyncRoot.DoRead(restriction, Unsync.Count); }

        /// <summary>
        /// <para>
        /// Eaches the handler.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="handler">
        /// <para>The handler.</para>
        /// <para></para>
        /// </param>
        /// <param name="restriction">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link address</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public TLinkAddress Each(IList<TLinkAddress>? restriction, ReadHandler<TLinkAddress>? handler) { return SyncRoot.DoRea
d(restriction, handler, Unsync.Each); }

        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Creates the substitution.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="substitution">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link address</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public TLinkAddress Create(IList<TLinkAddress>? substitution, WriteHandler<TLinkAddress>? handler) { return SyncRoot.D
oWrite(substitution, handler, Unsync.Create); }

        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Updates the substitution.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="restriction">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        /// <param name="substitution">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link address</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
```

```
        public TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<TLinkAddr
ess>? handler) { return SyncRoot.DoWrite(restriction, substitution, handler, Unsync.Update); }

        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Deletes the substitution.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="restriction">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler) { return SyncRoot.Do
Write(restriction, handler, Unsync.Delete); }

        //public T Trigger(IList<T> restriction, Func<IList<T>, IList<T>, T> matchedHandler, IList<T> substitution, Func<IList
<T>, IList<T>, T> substitutedHandler)
        //{
        //    if (restriction != null && substitution != null && !substitution.EqualTo(restriction))
        //        return SyncRoot.ExecuteWriteOperation(restriction, matchedHandler, substitution, substitutedHandler, Unsync.
Trigger);

        //    return SyncRoot.ExecuteReadOperation(restriction, matchedHandler, substitution, substitutedHandler, Unsync.Trigg
er);
        //}
    }
}
```

csharp/Platform.Data.Doublets/Tests/ILinksExtensions.cs

```
using System;
using System.Runtime.CompilerServices;
using Platform.Ranges;
using Platform.Random;
using Platform.Converters;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using Platform.Numbers;
using Platform.Setters;
using Range = System.Range;

namespace Platform.Data.Doublets.Tests
{
    /// <summary>
    /// <para>
    /// Represents the links extensions.
    /// </para>
    /// <para></para>
    /// </summary>
    public static class ILinksExtensions
    {
        private static void EnsureTrue(bool boolean)
        {
            EnsureTrue(boolean, default);
        }

        private static void EnsureTrue(bool boolean, string? message)
        {
            if (boolean)
            {
                return;
            }
            string? messageBuilder() => message;
            throw new ArgumentException("EnsureTrue Failed. The value is not a true. " + messageBuilder());
        }

        private static void EnsureEqual<T>(T expected, T actual) where T : IUnsignedNumber<T>
        {
            EnsureEqual(expected, actual, default);
        }

        private static void EnsureEqual<T>(T expected, T actual, string? message) where T : IUnsignedNumber<T>
        {
            EnsureEqual(expected, actual, message, EqualityComparer<T>.Default);
        }

        private static void EnsureEqual<T>(T expected, T actual, string? message, IEqualityComparer<T> equalityComparer) where
T : IUnsignedNumber<T>
        {
            if (expected == actual)
            {
                return;
            }
            string? messageBuilder() => message;
            throw new ArgumentException("EnsureEqual Failed. The values are not equal. " + messageBuilder());
        }

        /// <summary>
        /// <para>
        /// Tests the crud operations using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="T">
        /// <para>The .</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        public static void TestCRUDOperations<T>(this ILinks<T> links) where T : IUnsignedNumber<T>
        {
            var constants = links.Constants;
            var zero = T.Zero;
            var one = T.One;

            // Create Link
            EnsureTrue(links.Count() == zero);
            var setter = new Setter<T, T>(constants.Continue, constants.Break, constants.Null);
            links.Each(setter.SetFirstFromNonNullListAndReturnTrue, constants.Any, constants.Any, constants.Any);
            EnsureTrue(setter.Result == constants.Null);
            var linkAddress = links.Create();
            var link = new Link<T>(links.GetRequiredLink(linkAddress));
            EnsureTrue(link.Count == 3);
            EnsureTrue(link.Index == linkAddress);
            EnsureTrue(link.Source == constants.Null);
            EnsureTrue(link.Target == constants.Null);
            EnsureTrue(links.Count() == one);
```

```
    // Get first link
    setter = new Setter<T, T>(constants.Continue, constants.Break, constants.Null);
    links.Each(setter.SetFirstFromNonNullListAndReturnTrue, constants.Any, constants.Any, constants.Any);
    EnsureTrue(setter.Result == linkAddress);

    // Update link to reference itself
    links.Update(linkAddress, linkAddress, linkAddress);
    link = new Link<T>(links.GetRequiredLink(linkAddress));
    EnsureTrue(link.Source == linkAddress);
    EnsureTrue(link.Target == linkAddress);

    // Update link to reference null (prepare for delete)
    var updated = links.Update(linkAddress, constants.Null, constants.Null);
    EnsureTrue(updated == linkAddress);
    link = new Link<T>(links.GetRequiredLink(linkAddress));
    EnsureTrue(link.Source == constants.Null);
    EnsureTrue(link.Target == constants.Null);

    // Delete link
    links.Delete(linkAddress);
    EnsureTrue(links.Count() == zero);
    setter = new Setter<T, T>(constants.Continue, constants.Break, constants.Null);
    links.Each(setter.SetFirstFromNonNullListAndReturnTrue, constants.Any, constants.Any, constants.Any);
    EnsureTrue(setter.Result == constants.Null);
}

/// <summary>
/// <para>
/// Tests the raw numbers crud operations using the specified links.
/// </para>
/// <para></para>
/// </summary>
/// <typeparam name="T">
/// <para>The .</para>
/// <para></para>
/// </typeparam>
/// <param name="links">
/// <para>The links.</para>
/// <para></para>
/// </param>
public static void TestRawNumbersCRUDOperations<T>(this ILinks<T> links) where T : IUnsignedNumber<T>
{
    // Constants
    var constants = links.Constants;
    var one = T.One;
    var two = ++one;
    var h106E = new Hybrid<T>(106L, isExternal: true);
    var h107E = new Hybrid<T>(-char.ConvertFromUtf32(107)[0]);
    var h108E = new Hybrid<T>(-108L);
    EnsureEqual(T.CreateTruncating(106L), T.CreateTruncating(h106E.AbsoluteValue));
    EnsureEqual(T.CreateTruncating(107L), T.CreateTruncating(h107E.AbsoluteValue));
    EnsureEqual(T.CreateTruncating(108L), T.CreateTruncating(h108E.AbsoluteValue));

    // Create Link (External -> External)
    var linkAddress1 = links.Create();
    links.Update(linkAddress1, h106E, h108E);
    var link1 = new Link<T>(links.GetRequiredLink(linkAddress1));
    EnsureTrue(link1.Source == h106E.Value);
    EnsureTrue(link1.Target == h108E.Value);

    // Create Link (Internal -> External)
    var linkAddress2 = links.Create();
    links.Update(linkAddress2, linkAddress1, h108E);
    var link2 = new Link<T>(links.GetRequiredLink(linkAddress2));
    EnsureTrue(link2.Source == linkAddress1);
    EnsureTrue(link2.Target == h108E.Value);

    // Create Link (Internal -> Internal)
    var linkAddress3 = links.Create();
    links.Update(linkAddress3, linkAddress1, linkAddress2);
    var link3 = new Link<T>(links.GetRequiredLink(linkAddress3));
    EnsureTrue(link3.Source == linkAddress1);
    EnsureTrue(link3.Target == linkAddress2);

    // Search for created link
    var setter1 = new Setter<T, T>(constants.Continue, constants.Break, constants.Null);
    links.Each(setter1.SetFirstAndReturnFalse, constants.Any, h106E, h108E);
    EnsureTrue(setter1.Result == linkAddress1);

    // Search for nonexistent link
    var setter2 = new Setter<T, T>(constants.Continue, constants.Break, constants.Null);
    links.Each(setter2.SetFirstAndReturnFalse, constants.Any, h106E, h107E);
    EnsureTrue(setter2.Result == constants.Null);

    // Update link to reference null (prepare for delete)
    var updated = links.Update(linkAddress3, constants.Null, constants.Null);
    EnsureTrue(updated == linkAddress3);
    link3 = new Link<T>(links.GetRequiredLink(linkAddress3));
    EnsureTrue(link3.Source == constants.Null);
    EnsureTrue(link3.Target == constants.Null);

    // Delete link
```

```
            links.Delete(linkAddress3);
            EnsureTrue(links.Count() == two);
            var isLinkAddress2Found = links.All().Any(link => link![constants.IndexPart] == linkAddress2);
            EnsureTrue(isLinkAddress2Found);
        }

        /// <summary>
        /// <para>
        /// Tests the multiple random creations and deletions using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLink">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="maximumOperationsPerCycle">
        /// <para>The maximum operations per cycle.</para>
        /// <para></para>
        /// </param>
        public static void TestMultipleRandomCreationsAndDeletions<TLink>(this ILinks<TLink> links, int maximumOperationsPerCy
cle) where TLink : IUnsignedNumber<TLink>
        {
            var comparer = Comparer<TLink>.Default;
            var addressToUInt64Converter = CheckedConverter<TLink, ulong>.Default;
            for (var N = 1; N < maximumOperationsPerCycle; N++)
            {
                var random = new System.Random(N);
                var createdAddresses = new List<TLink>();
                var created = 0;
                for (var i = 0; i < N; i++)
                {
                    var createPoint = random.NextBoolean();
                    if (created >= 2 && createPoint)
                    {
                        var linksAddressRange = new Range<int>(0, created);
                        var source = createdAddresses[random.Next(linksAddressRange.Minimum, linksAddressRange.Maximum)];
                         var target = createdAddresses[random.Next(linksAddressRange.Minimum, linksAddressRange.Maximum)]; //-V
3086
                        var resultLink = links.SearchOrDefault(source, target);
                        if (resultLink != default)
                        {
                            continue;
                        }
                        resultLink = links.CreateAndUpdate(source, target);
                        if (comparer.Compare(resultLink, default) > 0)
                        {
                            createdAddresses.Add(resultLink);
                            created++;
                        }
                    }
                    else
                    {
                        createdAddresses.Add(links.Create());
                        created++;
                    }
                }
                EnsureTrue((ulong)created == addressToUInt64Converter.Convert(links.Count()));
                var deletedLinksAddressRange = new Range<int>(0, created);
                // Random deletions
                for (var i = 0; i < N; i++)
                {
                     var id = createdAddresses[random.Next(deletedLinksAddressRange.Minimum, deletedLinksAddressRange.Maximum)]
;
                    if (links.Exists(id))
                    {
                        links.Delete(id);
                    }
                }
                // Delete all remaining links
                for (var i = 0; i < createdAddresses.Count; i++)
                {
                    if (links.Exists(createdAddresses[i]))
                    {
                        links.Delete(createdAddresses[i]);
                    }
                }
                EnsureTrue(addressToUInt64Converter.Convert(links.Count()) == 0L);
            }
        }

        /// <summary>
        ///
        /// </summary>
        /// <param name="links"></param>
        /// <param name="numberOfOperations"></param>
        /// <typeparam name="TLinkAddress"></typeparam>
        public static void TestMultipleCreationsAndDeletions<TLinkAddress>(this ILinks<TLinkAddress> links, int numberOfOperat
```

```
ions) where TLinkAddress : IUnsignedNumber<TLinkAddress>
        {
            for (int i = 0; i < numberOfOperations; i++)
            {
                links.Create();
            }
            for (int i = 0; i < numberOfOperations; i++)
            {
                links.Delete(links.Count());
            }
        }

        /// <summary>
        /// <para>
        /// Runs the random creations using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLink">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="amountOfCreations">
        /// <para>The amount of creations.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void RunRandomCreations<TLink>(this ILinks<TLink> links, ulong amountOfCreations) where TLink : IUnsigne
dNumber<TLink>
        {
            var random = RandomHelpers.Default;
            for (var i = 0UL; i < amountOfCreations; i++)
            {
                var linksAddressRange = new Range<ulong>(ulong.CreateTruncating(TLink.Zero), ulong.CreateTruncating(links.Coun
t()));
                var source = TLink.CreateTruncating(random.NextUInt64(linksAddressRange));
                var target = TLink.CreateTruncating(random.NextUInt64(linksAddressRange));
                links.GetOrCreate(source, target);
            }
        }

        /// <summary>
        /// <para>
        /// Runs the random searches using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLink">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
        /// <param name="amountOfSearches">
        /// <para>The amount of searches.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void RunRandomSearches<TLink>(this ILinks<TLink> links, ulong amountOfSearches) where TLink : IUnsignedN
umber<TLink>
        {
            var random = RandomHelpers.Default;
            for (var i = 0UL; i < amountOfSearches; i++)
            {
                var linksAddressRange = new Range<ulong>(ulong.CreateTruncating(TLink.Zero), ulong.CreateTruncating(links.Coun
t()));
                var source = TLink.CreateTruncating(random.NextUInt64(linksAddressRange));
                var target = TLink.CreateTruncating(random.NextUInt64(linksAddressRange));
                links.SearchOrDefault(source, target);
            }
        }

        /// <summary>
        /// <para>
        /// Runs the random deletions using the specified links.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <typeparam name="TLink">
        /// <para>The link.</para>
        /// <para></para>
        /// </typeparam>
        /// <param name="links">
        /// <para>The links.</para>
        /// <para></para>
        /// </param>
```

```
        /// <param name="amountOfDeletions">
        /// <para>The amount of deletions.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static void RunRandomDeletions<TLink>(this ILinks<TLink> links, TLink amountOfDeletions) where TLink : IUnsigne
dNumber<TLink>, IComparisonOperators<TLink, TLink, bool>
        {
            var random = RandomHelpers.Default;
            var linksCount = links.Count();
            var min = amountOfDeletions > linksCount ? TLink.Zero : linksCount - amountOfDeletions;
            for (var i = TLink.Zero; i < amountOfDeletions; i++)
            {
                linksCount = links.Count();
                if (linksCount <= min)
                {
                    break;
                }
                var linksAddressRange = new Range<ulong>(ulong.CreateTruncating(min), ulong.CreateTruncating(linksCount));
                var link = TLink.CreateTruncating(random.NextUInt64(linksAddressRange));
                links.Delete(link);
            }
        }
    }
}
```

csharp/Platform.Data.Doublets/UInt64LinksTransactionsLayer.cs

```
using System;
using System.Linq;
using System.Collections.Generic;
using System.IO;
using System.Runtime.CompilerServices;
using System.Threading;
using System.Threading.Tasks;
using Platform.Disposables;
using Platform.Timestamps;
using Platform.Unsafe;
using Platform.IO;
using Platform.Data.Doublets.Decorators;
using Platform.Delegates;
using Platform.Exceptions;
using TLinkAddress = System.UInt64;

#pragma warning disable CS1591 // Missing XML comment for publicly visible type or member

namespace Platform.Data.Doublets
{
    /// <summary>
    /// <para>
    /// Represents the int 64 links transactions layer.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <seealso cref="LinksDisposableDecoratorBase{TLinkAddress}"/>
    public class UInt64LinksTransactionsLayer : LinksDisposableDecoratorBase<TLinkAddress> //-V3073
    {
        /// <summary>Records a transaction transition between two link states.</summary>
        /// <remarks>
        /// Альтернативные варианты хранения трансформации (элемента транзакции):
        ///
        /// private enum TransitionType
        /// {
        ///     Creation,
        ///     UpdateOf,
        ///     UpdateTo,
        ///     Deletion
        /// }
        ///
        /// private struct Transition
        /// {
        ///     public TLinkAddress TransactionId;
        ///     public UniqueTimestamp Timestamp;
        ///     public TransactionItemType Type;
        ///     public Link Source;
        ///     public Link Linker;
        ///     public Link Target;
        /// }
        ///
        /// Или
        ///
        /// public struct TransitionHeader
        /// {
        ///     public TLinkAddress TransactionIdCombined;
        ///     public TLinkAddress TimestampCombined;
        ///
        ///     public TLinkAddress TransactionId
        ///     {
        ///         get
        ///         {
        ///             return (TLinkAddress) mask &amp;amp; TransactionIdCombined;
        ///         }
        ///     }
        ///
        ///     public UniqueTimestamp Timestamp
        ///     {
        ///         get
        ///         {
        ///             return (UniqueTimestamp)mask &amp;amp; TransactionIdCombined;
        ///         }
        ///     }
        ///
        ///     public TransactionItemType Type
        ///     {
        ///         get
        ///         {
        ///             // Использовать по одному биту из TransactionId и Timestamp,
        ///             // для значения в 2 бита, которое представляет тип операции
        ///             throw new NotImplementedException();
        ///         }
        ///     }
        /// }
        ///
        /// private struct Transition
        /// {
        ///     public TransitionHeader Header;
        ///     public Link Source;
        ///     public Link Linker;
        ///     public Link Target;
```

```
        /// }
        ///
        /// </remarks>
        public struct Transition : IEquatable<Transition>
        {
            /// <summary>
            /// <para>
            /// The size.
            /// </para>
            /// <para></para>
            /// </summary>
            public static readonly long Size = Structure<Transition>.Size;

            /// <summary>
            /// <para>
            /// The transaction id.
            /// </para>
            /// <para></para>
            /// </summary>
            public readonly TLinkAddress TransactionId;
            /// <summary>
            /// <para>
            /// The before.
            /// </para>
            /// <para></para>
            /// </summary>
            public readonly Link<TLinkAddress> Before;
            /// <summary>
            /// <para>
            /// The after.
            /// </para>
            /// <para></para>
            /// </summary>
            public readonly Link<TLinkAddress> After;
            /// <summary>
            /// <para>
            /// The timestamp.
            /// </para>
            /// <para></para>
            /// </summary>
            public readonly Timestamp Timestamp;

            /// <summary>
            /// <para>
            /// Initializes a new <see cref="Transition"/> instance.
            /// </para>
            /// <para></para>
            /// </summary>
            /// <param name="uniqueTimestampFactory">
            /// <para>A unique timestamp factory.</para>
            /// <para></para>
            /// </param>
            /// <param name="transactionId">
            /// <para>A transaction id.</para>
            /// <para></para>
            /// </param>
            /// <param name="before">
            /// <para>A before.</para>
            /// <para></para>
            /// </param>
            /// <param name="after">
            /// <para>A after.</para>
            /// <para></para>
            /// </param>
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            public Transition(UniqueTimestampFactory uniqueTimestampFactory, TLinkAddress transactionId, Link<TLinkAddress> be
fore, Link<TLinkAddress> after)
            {
                TransactionId = transactionId;
                Before = before;
                After = after;
                Timestamp = uniqueTimestampFactory.Create();
            }

             public Transition(UniqueTimestampFactory uniqueTimestampFactory, TLinkAddress transactionId, IList<TLinkAddress>?
before, IList<TLinkAddress>? after) : this(uniqueTimestampFactory, transactionId, new Link<TLinkAddress>(before), new Link<TLi
nkAddress>(after)) { }

            /// <summary>
            /// <para>
            /// Initializes a new <see cref="Transition"/> instance.
            /// </para>
            /// <para></para>
            /// </summary>
            /// <param name="uniqueTimestampFactory">
            /// <para>A unique timestamp factory.</para>
            /// <para></para>
            /// </param>
            /// <param name="transactionId">
            /// <para>A transaction id.</para>
            /// <para></para>
            /// </param>
```

```
            /// <param name="before">
            /// <para>A before.</para>
            /// <para></para>
            /// </param>
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            public Transition(UniqueTimestampFactory uniqueTimestampFactory, TLinkAddress transactionId, Link<TLinkAddress> be
fore) : this(uniqueTimestampFactory, transactionId, before, default) { }

            /// <summary>
            /// <para>
            /// Initializes a new <see cref="Transition"/> instance.
            /// </para>
            /// <para></para>
            /// </summary>
            /// <param name="uniqueTimestampFactory">
            /// <para>A unique timestamp factory.</para>
            /// <para></para>
            /// </param>
            /// <param name="transactionId">
            /// <para>A transaction id.</para>
            /// <para></para>
            /// </param>
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            public Transition(UniqueTimestampFactory uniqueTimestampFactory, TLinkAddress transactionId) : this(uniqueTimestam
pFactory, transactionId, default, default) { }

            /// <summary>
            /// <para>
            /// Returns the string.
            /// </para>
            /// <para></para>
            /// </summary>
            /// <returns>
            /// <para>The string</para>
            /// <para></para>
            /// </returns>
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            public override string ToString() => $"{Timestamp} {TransactionId}: {Before} => {After}";

            /// <summary>
            /// <para>
            /// Determines whether this instance equals.
            /// </para>
            /// <para></para>
            /// </summary>
            /// <param name="obj">
            /// <para>The obj.</para>
            /// <para></para>
            /// </param>
            /// <returns>
            /// <para>The bool</para>
            /// <para></para>
            /// </returns>
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            public override bool Equals(object? obj) => obj is Transition transition && Equals(transition);

            /// <summary>
            /// <para>
            /// Gets the hash code.
            /// </para>
            /// <para></para>
            /// </summary>
            /// <returns>
            /// <para>The int</para>
            /// <para></para>
            /// </returns>
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            public override int GetHashCode() => (TransactionId, Before, After, Timestamp).GetHashCode();

            /// <summary>
            /// <para>
            /// Determines whether this instance equals.
            /// </para>
            /// <para></para>
            /// </summary>
            /// <param name="other">
            /// <para>The other.</para>
            /// <para></para>
            /// </param>
            /// <returns>
            /// <para>The bool</para>
            /// <para></para>
            /// </returns>
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            public bool Equals(Transition other) => TransactionId == other.TransactionId && Before == other.Before && After ==
other.After && Timestamp == other.Timestamp;

            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            public static bool operator ==(Transition left, Transition right) => left.Equals(right);

            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            public static bool operator !=(Transition left, Transition right) => !(left == right);
```

```
}

/// <summary>Groups link changes for commit or rollback.</summary>
/// <remarks>
/// Другие варианты реализации транзакций (атомарности):
///     1. Разделение хранения значения связи ((Source Target) или (Source Linker Target)) и индексов.
///     2. Хранение трансформаций/операций в отдельном хранилище Links, но дополнительно потребуется решить вопрос
///        со ссылками на внешние идентификаторы, или как-то иначе решить вопрос с пересечениями идентификаторов.
///
/// Где хранить промежуточный список транзакций?
///
/// В оперативной памяти:
///  Минусы:
///     1. Может усложнить систему, если она будет функционировать самостоятельно,
///     так как нужно отдельно выделять память под список трансформаций.
///     2. Выделенной оперативной памяти может не хватить, в том случае,
///     если транзакция использует слишком много трансформаций.
///         -> Можно использовать жёсткий диск для слишком длинных транзакций.
///         -> Максимальный размер списка трансформаций можно ограничить / задать константой.
///     3. При подтверждении транзакции (Commit) все трансформации записываются разом создавая задержку.
///
/// На жёстком диске:
///  Минусы:
///     1. Длительный отклик, на запись каждой трансформации.
///     2. Лог транзакций дополнительно наполняется отменёнными транзакциями.
///         -> Это может решаться упаковкой/исключением дублирующих операций.
///         -> Также это может решаться тем, что короткие транзакции вообще
///            не будут записываться в случае отката.
///     3. Перед тем как выполнять отмену операций транзакции нужно дождаться пока все операции (трансформации)
///        будут записаны в лог.
///
/// </remarks>
public class Transaction : DisposableBase
{
    private readonly Queue<Transition> _transitions;
    private readonly UInt64LinksTransactionsLayer _layer;
    /// <summary>
    /// <para>
    /// Gets or sets the is committed value.
    /// </para>
    /// <para></para>
    /// </summary>
    public bool IsCommitted { get; private set; }
    /// <summary>
    /// <para>
    /// Gets or sets the is reverted value.
    /// </para>
    /// <para></para>
    /// </summary>
    public bool IsReverted { get; private set; }

    /// <summary>
    /// <para>
    /// Initializes a new <see cref="Transaction"/> instance.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <param name="layer">
    /// <para>A layer.</para>
    /// <para></para>
    /// </param>
    /// <exception cref="NotSupportedException">
    /// <para>Nested transactions not supported.</para>
    /// <para></para>
    /// </exception>
    [MethodImpl(MethodImplOptions.AggressiveInlining)]
    public Transaction(UInt64LinksTransactionsLayer layer)
    {
        _layer = layer;
        if (_layer._currentTransactionId != 0)
        {
            throw new NotSupportedException("Nested transactions not supported.");
        }
        IsCommitted = false;
        IsReverted = false;
        _transitions = new Queue<Transition>();
        SetCurrentTransaction(layer, this);
    }

    /// <summary>
    /// <para>
    /// Commits this instance.
    /// </para>
    /// <para></para>
    /// </summary>
    [MethodImpl(MethodImplOptions.AggressiveInlining)]
    public void Commit()
    {
        EnsureTransactionAllowsWriteOperations(this);
        while (_transitions.Count > 0)
        {
            var transition = _transitions.Dequeue();
```

```
            _layer._transitions.Enqueue(transition);
        }
        _layer._lastCommitedTransactionId = _layer._currentTransactionId;
        IsCommitted = true;
    }
    [MethodImpl(MethodImplOptions.AggressiveInlining)]
    private void Revert()
    {
        EnsureTransactionAllowsWriteOperations(this);
        var transitionsToRevert = new Transition[_transitions.Count];
        _transitions.CopyTo(transitionsToRevert, 0);
        for (var i = transitionsToRevert.Length - 1; i >= 0; i--)
        {
            _layer.RevertTransition(transitionsToRevert[i]);
        }
        IsReverted = true;
    }

    /// <summary>
    /// <para>
    /// Sets the current transaction using the specified layer.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <param name="layer">
    /// <para>The layer.</para>
    /// <para></para>
    /// </param>
    /// <param name="transaction">
    /// <para>The transaction.</para>
    /// <para></para>
    /// </param>
    [MethodImpl(MethodImplOptions.AggressiveInlining)]
    public static void SetCurrentTransaction(UInt64LinksTransactionsLayer layer, Transaction transaction)
    {
        layer._currentTransactionId = layer._lastCommitedTransactionId + 1;
        layer._currentTransactionTransitions = transaction._transitions;
        layer._currentTransaction = transaction;
    }

    /// <summary>
    /// <para>
    /// Ensures the transaction allows write operations using the specified transaction.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <param name="transaction">
    /// <para>The transaction.</para>
    /// <para></para>
    /// </param>
    /// <exception cref="InvalidOperationException">
    /// <para>Transation is commited.</para>
    /// <para></para>
    /// </exception>
    /// <exception cref="InvalidOperationException">
    /// <para>Transation is reverted.</para>
    /// <para></para>
    /// </exception>
    [MethodImpl(MethodImplOptions.AggressiveInlining)]
    public static void EnsureTransactionAllowsWriteOperations(Transaction transaction)
    {
        if (transaction.IsReverted)
        {
            throw new InvalidOperationException("Transation is reverted.");
        }
        if (transaction.IsCommitted)
        {
            throw new InvalidOperationException("Transation is commited.");
        }
    }

    /// <summary>
    /// <para>
    /// Disposes the manual.
    /// </para>
    /// <para></para>
    /// </summary>
    /// <param name="manual">
    /// <para>The manual.</para>
    /// <para></para>
    /// </param>
    /// <param name="wasDisposed">
    /// <para>The was disposed.</para>
    /// <para></para>
    /// </param>
    [MethodImpl(MethodImplOptions.AggressiveInlining)]
    protected override void Dispose(bool manual, bool wasDisposed)
    {
        if (!wasDisposed && _layer != null && !_layer.Disposable.IsDisposed)
        {
            if (!IsCommitted && !IsReverted)
            {
```

```
                Revert();
            }
            _layer.ResetCurrentTransation();
        }
    }
}

/// <summary>
/// <para>
/// The from seconds.
/// </para>
/// <para></para>
/// </summary>
public static readonly TimeSpan DefaultPushDelay = TimeSpan.FromSeconds(0.1);
private readonly string _logAddress;
private readonly FileStream _log;
private readonly Queue<Transition> _transitions;
private readonly UniqueTimestampFactory _uniqueTimestampFactory;
private Task? _transitionsPusher;
private Transition _lastCommitedTransition;
private TLinkAddress _currentTransactionId;
private Queue<Transition>? _currentTransactionTransitions;
private Transaction? _currentTransaction;
private TLinkAddress _lastCommitedTransactionId;

/// <summary>
/// <para>
/// Initializes a new <see cref="UInt64LinksTransactionsLayer"/> instance.
/// </para>
/// <para></para>
/// </summary>
/// <param name="links">
/// <para>A links.</para>
/// <para></para>
/// </param>
/// <param name="logAddress">
/// <para>A log address.</para>
/// <para></para>
/// </param>
/// <exception cref="ArgumentNullException">
/// <para></para>
/// <para></para>
/// </exception>
/// <exception cref="NotSupportedException">
/// <para>Database is damaged, autorecovery is not supported yet.</para>
/// <para></para>
/// </exception>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public UInt64LinksTransactionsLayer(ILinks<TLinkAddress> links, string logAddress)
    : base(links)
{
    if (string.IsNullOrWhiteSpace(logAddress))
    {
        throw new ArgumentNullException(nameof(logAddress));
    }
    // В первой строке файла хранится последняя закоммиченную транзакцию.
    // При запуске это используется для проверки удачного закрытия файла лога.
    // In the first line of the file the last committed transaction is stored.
    // On startup, this is used to check that the log file is successfully closed.
    var lastCommitedTransition = FileHelpers.ReadFirstOrDefault<Transition>(logAddress);
    var lastWrittenTransition = FileHelpers.ReadLastOrDefault<Transition>(logAddress);
    if (!lastCommitedTransition.Equals(lastWrittenTransition))
    {
        Dispose();
        throw new NotSupportedException("Database is damaged, autorecovery is not supported yet.");
    }
    if (lastCommitedTransition == default)
    {
        FileHelpers.WriteFirst(logAddress, lastCommitedTransition);
    }
    _lastCommitedTransition = lastCommitedTransition;
    // TODO: Think about a better way to calculate or store this value
    var allTransitions = FileHelpers.ReadAll<Transition>(logAddress);
    _lastCommitedTransactionId = allTransitions.Length > 0 ? allTransitions.Max(x => x.TransactionId) : 0;
    _uniqueTimestampFactory = new UniqueTimestampFactory();
    _logAddress = logAddress;
    _log = FileHelpers.Append(logAddress);
    _transitions = new Queue<Transition>();
    _transitionsPusher = new Task(TransitionsPusher);
    _transitionsPusher.Start();
}

/// <summary>
/// <para>
/// Gets the link value using the specified link.
/// </para>
/// <para></para>
/// </summary>
/// <param name="link">
/// <para>The link.</para>
/// <para></para>
/// </param>
```

```
        /// <returns>
        /// <para>A list of TLinkAddress</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public IList<TLinkAddress> GetLinkValue(TLinkAddress link) => _links.GetRequiredLink(link);

        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Creates the substitution.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="substitution">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The created link index.</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override TLinkAddress Create(IList<TLinkAddress>? substitution, WriteHandler<TLinkAddress>? handler)
        {
            return _links.Create(new Link<TLinkAddress>(), (before, after) =>
            {
                CommitTransition(new Transition(_uniqueTimestampFactory, _currentTransactionId, new Link<TLinkAddress>(before)
, new Link<TLinkAddress>(after)));
                return handler?.Invoke(before, after) ?? Links.Constants.Continue;
            });
        }

        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Updates the substitution.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="restriction">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        /// <param name="substitution">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        /// <returns>
        /// <para>The link index.</para>
        /// <para></para>
        /// </returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override TLinkAddress Update(IList<TLinkAddress>? restriction, IList<TLinkAddress>? substitution, WriteHandler<
TLinkAddress>? handler)
        {
            return _links.Update(restriction, substitution, (before, after) =>
                {
                    CommitTransition(new Transition(_uniqueTimestampFactory, _currentTransactionId, new Link<TLinkAddress>(bef
ore), new Link<TLinkAddress>(after)));
                    return handler != null ? handler(before, after) : Constants.Continue;
                }
            );
        }

        /// <param name="handler">Optional callback; null continues without invoking a callback.</param>
        /// <summary>
        /// <para>
        /// Deletes the substitution.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="restriction">
        /// <para>The substitution.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override TLinkAddress Delete(IList<TLinkAddress>? restriction, WriteHandler<TLinkAddress>? handler)
        {
            return _links.Delete(restriction, (before, after) =>
            {
                CommitTransition(new Transition(_uniqueTimestampFactory, _currentTransactionId, before, after));
                return handler != null ? handler(before, after) : Constants.Continue;
            });
        }
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        private Queue<Transition> GetCurrentTransitions() => _currentTransactionTransitions ?? _transitions;
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        private void CommitTransition(Transition transition)
        {
            if (_currentTransaction != null)
            {
```

```
        Transaction.EnsureTransactionAllowsWriteOperations(_currentTransaction);
    }
    var transitions = GetCurrentTransitions();
    transitions.Enqueue(transition);
}
[MethodImpl(MethodImplOptions.AggressiveInlining)]
private void RevertTransition(Transition transition)
{
    if (transition.After.IsNull()) // Revert Deletion with Creation
    {
        _links.Create();
    }
    else if (transition.Before.IsNull()) // Revert Creation with Deletion
    {
        _links.Delete(transition.After.Index);
    }
    else // Revert Update
    {
        _links.Update(new[] { transition.After.Index, transition.Before.Source, transition.Before.Target });
    }
}
[MethodImpl(MethodImplOptions.AggressiveInlining)]
private void ResetCurrentTransation()
{
    _currentTransactionId = 0;
    _currentTransactionTransitions = null;
    _currentTransaction = null;
}
[MethodImpl(MethodImplOptions.AggressiveInlining)]
private void PushTransitions()
{
    if (_log == null || _transitions == null)
    {
        return;
    }
    for (var i = 0; i < _transitions.Count; i++)
    {
        var transition = _transitions.Dequeue();

        _log.Write(transition);
        _lastCommitedTransition = transition;
    }
}
[MethodImpl(MethodImplOptions.AggressiveInlining)]
private void TransitionsPusher()
{
    while (!Disposable.IsDisposed && _transitionsPusher != null)
    {
        Thread.Sleep(DefaultPushDelay);
        PushTransitions();
    }
}

/// <summary>
/// <para>
/// Begins the transaction.
/// </para>
/// <para></para>
/// </summary>
/// <returns>
/// <para>The transaction</para>
/// <para></para>
/// </returns>
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public Transaction BeginTransaction() => new Transaction(this);
[MethodImpl(MethodImplOptions.AggressiveInlining)]
private void DisposeTransitions(bool manual)
{
    // A constructor may reject a damaged log before opening the stream.
    if (_log == null) return;
    try
    {
        try
        {
            var pusher = _transitionsPusher;
            if (pusher != null)
            {
                _transitionsPusher = null;
                pusher.Wait();
            }
            if (_transitions != null)
            {
                PushTransitions();
            }
        }
        finally
        {
            _log.DisposeIfPossible();
        }
        FileHelpers.WriteFirst(_logAddress, _lastCommitedTransition);
    }
    catch (Exception ex) when (!manual)
```

```
            {
                ex.Ignore();
            }
        }

        #region DisposalBase

        /// <summary>
        /// <para>
        /// Disposes the manual.
        /// </para>
        /// <para></para>
        /// </summary>
        /// <param name="manual">
        /// <para>The manual.</para>
        /// <para></para>
        /// </param>
        /// <param name="wasDisposed">
        /// <para>The was disposed.</para>
        /// <para></para>
        /// </param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        protected override void Dispose(bool manual, bool wasDisposed)
        {
            try
            {
                if (!wasDisposed)
                {
                    DisposeTransitions(manual);
                }
            }
            finally
            {
                base.Dispose(manual, wasDisposed);
            }
        }

        #endregion
    }
}
```

csharp/Platform.Data.Doublets.Benchmarks/CountBenchmarks.cs

```
using System.Collections.Generic;
using BenchmarkDotNet.Attributes;
using Platform.Collections.Arrays;
using Platform.Collections.Lists;
using Platform.Converters;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Memory;
using TLinkAddress = System.UInt64;

#pragma warning disable CA1822 // Mark members as static

namespace Platform.Data.Doublets.Benchmarks
{
    [SimpleJob]
    [MemoryDiagnoser]
    public class CountBenchmarks
    {
        private static ILinks<TLinkAddress>? _storage;
        private static ILinks<TLinkAddress> _links => _storage ?? throw new System.InvalidOperationException("Run GlobalSetup
before a benchmark.");
        private static TLinkAddress _any;
        private static readonly CheckedConverter<TLinkAddress, long> _addressToInt64Converter = CheckedConverter<TLinkAddress,
long>.Default;
        private static HeapResizableDirectMemory? _dataMemory;

        [Params(10, 100, 1000, 10000, 100000)]
        public static ulong N;

        [GlobalSetup]
        public static void Setup()
        {
            _dataMemory = new HeapResizableDirectMemory();
            _storage = new UnitedMemoryLinks<TLinkAddress>(_dataMemory).DecorateWithAutomaticUniquenessAndUsagesResolution();
            _any = _links.Constants.Any;
            var firstLink = _links.CreatePoint();
            for (ulong i = 0; i < N; i++)
            {
                var link = _links.Create();
                _links.Update(link, firstLink, link);
            }
            for (ulong i = 0; i < N; i++)
            {
                var link = _links.Create();
                _links.Update(link, link, firstLink);
            }
        }

        [GlobalCleanup]
        public static void Cleanup()
        {
            (_storage as System.IDisposable)?.Dispose();
            _storage = null;
            _dataMemory?.Dispose();
            _dataMemory = null;
        }

        [Benchmark]
        public IList<IList<TLinkAddress>?> Array()
        {
            var addressToInt64Converter = CheckedConverter<TLinkAddress, long>.Default;
            var usagesAsSourceQuery = new Link<TLinkAddress>(_any, 1UL, _any);
            var usagesAsSourceCount = addressToInt64Converter.Convert(_links.Count(usagesAsSourceQuery));
            var usagesAsTargetQuery = new Link<TLinkAddress>(_any, _any, 1UL);
            var usagesAsTargetCount = addressToInt64Converter.Convert(_links.Count(usagesAsTargetQuery));
            var totalUsages = usagesAsSourceCount + usagesAsTargetCount;
            var usages = new IList<TLinkAddress>?[totalUsages];
            var usagesFiller = new ArrayFiller<IList<TLinkAddress>?, TLinkAddress>(usages, _links.Constants.Continue);
            _links.Each(usagesFiller.AddAndReturnConstant, usagesAsSourceQuery);
            _links.Each(usagesFiller.AddAndReturnConstant, usagesAsTargetQuery);
            return usages;
        }

        [Benchmark]
        public IList<IList<TLinkAddress>?> List()
        {
            var usagesAsSourceQuery = new Link<TLinkAddress>(_any, 1UL, _any);
            var usagesAsTargetQuery = new Link<TLinkAddress>(_any, _any, 1UL);
            var usages = new List<IList<TLinkAddress>?>();
            var usagesFiller = new ListFiller<IList<TLinkAddress>?, TLinkAddress>(usages, _links.Constants.Continue);
            _links.Each(usagesFiller.AddAndReturnConstant, usagesAsSourceQuery);
            _links.Each(usagesFiller.AddAndReturnConstant, usagesAsTargetQuery);
            return usages;
        }

        [Benchmark]
        public IList<IList<TLinkAddress>?> ListWithCapacity()
        {
            var addressToInt64Converter = CheckedConverter<TLinkAddress, long>.Default;
            var usagesAsSourceQuery = new Link<TLinkAddress>(_any, 1UL, _any);
            var usagesAsSourceCount = addressToInt64Converter.Convert(_links.Count(usagesAsSourceQuery));
            var usagesAsTargetQuery = new Link<TLinkAddress>(_any, _any, 1UL);
```

```
            var usagesAsTargetCount = addressToInt64Converter.Convert(_links.Count(usagesAsTargetQuery));
            var totalUsages = usagesAsSourceCount + usagesAsTargetCount;
            var usages = new List<IList<TLinkAddress>?>((int)totalUsages);
            var usagesFiller = new ListFiller<IList<TLinkAddress>?, TLinkAddress>(usages, _links.Constants.Continue);
            _links.Each(usagesFiller.AddAndReturnConstant, usagesAsSourceQuery);
            _links.Each(usagesFiller.AddAndReturnConstant, usagesAsTargetQuery);
            return usages;
        }
    }
}
```

csharp/Platform.Data.Doublets.Benchmarks/LinkStructBenchmarks.cs

```
using System.Collections.Generic;
using BenchmarkDotNet.Attributes;
using TLinkAddress = System.UInt64;

#pragma warning disable CA1822 // Mark members as static

namespace Platform.Data.Doublets.Benchmarks
{
    [SimpleJob]
    [MemoryDiagnoser]
    public class LinkStructBenchmarks
    {
        [GlobalSetup]
        public static void Setup()
        {
        }

        [GlobalCleanup]
        public static void Cleanup() { }

        [Benchmark]
        public IList<TLinkAddress>? Struct()
        {
            return new Link<TLinkAddress>(1UL, 1UL, 1UL);
        }

        [Benchmark]
        public IList<TLinkAddress>? Array()
        {
            return new TLinkAddress[] { 1UL, 1UL, 1UL };
        }

        [Benchmark]
        public IList<TLinkAddress>? List()
        {
            return new List<TLinkAddress> { 1UL, 1UL, 1UL };
        }
    }
}
```

csharp/Platform.Data.Doublets.Benchmarks/MemoryBenchmarks.cs

```
/*
using System.IO;
using BenchmarkDotNet.Attributes;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Data.Doublets.Tests;
using Platform.Memory;

#pragma warning disable CA1822 // Mark members as static

namespace Platform.Data.Doublets.Benchmarks
{
    [SimpleJob]
    [MemoryDiagnoser]
    public class MemoryBenchmarks
    {
        private const int N = 1000;
        private static SplitMemoryLinks<uint> _splitMemory;
        private static ILinks<uint> _splitMemoryLinks;
        private static UnitedMemoryLinks<uint> _unitedMemory;
        private static ILinks<uint> _unitedMemoryLinks;
        private static Ffi.Links<uint> _ffi;
        private static ILinks<uint> _ffiUnitedMemoryLinks;
        private static Ffi.UInt32Links _ffiUInt32;
        private static ILinks<uint> _ffiUInt32UnitedMemoryLinks;

        [GlobalSetup]
        public static void Setup()
        {
            var dataMemory = new HeapResizableDirectMemory();
            var indexMemory = new HeapResizableDirectMemory();
            _splitMemory = new SplitMemoryLinks<uint>(dataMemory, indexMemory);
            _splitMemoryLinks = _splitMemory.DecorateWithAutomaticUniquenessAndUsagesResolution();

            var memory = new HeapResizableDirectMemory();
            _unitedMemory = new UnitedMemoryLinks<uint>(memory);
            _unitedMemoryLinks = _unitedMemory.DecorateWithAutomaticUniquenessAndUsagesResolution();

            File.Delete("db.links");
            _ffi = new Ffi.Links<uint>("db.links");
            _ffiUnitedMemoryLinks = _ffi;

            File.Delete("db.links1");
            _ffiUInt32 = new Ffi.UInt32Links("db.links1");
            _ffiUInt32UnitedMemoryLinks = _ffiUInt32.DecorateWithAutomaticUniquenessAndUsagesResolution();
        }

        [GlobalCleanup]
        public static void Cleanup()
        {
            _splitMemory.Dispose();
            _unitedMemory.Dispose();
            _ffi.Dispose();
            File.Delete("db.links");
            _ffiUInt32.Dispose();
            File.Delete("db.links1");
        }

        [Benchmark]
        public void Split()
        {
            _splitMemoryLinks.TestMultipleRandomCreationsAndDeletions(N);
        }

        [Benchmark]
        public void United()
        {
            _unitedMemoryLinks.TestMultipleRandomCreationsAndDeletions(N);
        }

        [Benchmark]
        public void FfiUnited()
        {
            _ffiUnitedMemoryLinks.TestMultipleRandomCreationsAndDeletions(N);
        }

        [Benchmark]
        public void FfiUInt32United()
        {
            _ffiUInt32UnitedMemoryLinks.TestMultipleRandomCreationsAndDeletions(N);
        }
    }
}
*/
```

csharp/Platform.Data.Doublets.Benchmarks/Program.cs

```
using BenchmarkDotNet.Running;

namespace Platform.Data.Doublets.Benchmarks
{
    class Program
    {
        static void Main()
        {
            BenchmarkRunner.Run<CountBenchmarks>();
            BenchmarkRunner.Run<LinkStructBenchmarks>();
            // BenchmarkRunner.Run<MemoryBenchmarks>();
        }
    }
}
```

csharp/Platform.Data.Doublets.Benchmarks/SerialCreationsAndDeletionsBenchmark.cs

```
/*
using System.IO;
using BenchmarkDotNet.Attributes;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Data.Doublets.Tests;
using Platform.Memory;

#pragma warning disable CA1822 // Mark members as static

namespace Platform.Data.Doublets.Benchmarks
{
    [SimpleJob]
    [MemoryDiagnoser]
    public class SerialCreationsAndDeletionsBenchmark
    {
        private static SplitMemoryLinks<uint> _splitMemory;
        private static UnitedMemoryLinks<uint> _unitedMemory;
        private static Ffi.Links<uint> _ffi;
        private static Ffi.UInt32Links _uint32Ffi;

        [GlobalSetup]
        public static void Setup()
        {
            var dataMemory = new HeapResizableDirectMemory();
            var indexMemory = new HeapResizableDirectMemory();
            _splitMemory = new SplitMemoryLinks<uint>(dataMemory, indexMemory);

            var memory = new HeapResizableDirectMemory();
            _unitedMemory = new UnitedMemoryLinks<uint>(memory);

            File.Delete("db.links");
            _ffi = new Ffi.Links<uint>("db.links");

            File.Delete("db.links");
            _uint32Ffi = new Ffi.UInt32Links("db.links");
        }

        [GlobalCleanup]
        public static void Cleanup()
        {
            _splitMemory.Dispose();
            _unitedMemory.Dispose();
             _ffi.Dispose();
             _uint32Ffi.Dispose();
            File.Delete("db.links");
        }

        [Benchmark]
        public void Split()
        {
            _splitMemory.TestMultipleCreationsAndDeletions(10000);
        }

        [Benchmark]
        public void United()
        {
            _unitedMemory.TestMultipleCreationsAndDeletions(10000);
        }

        [Benchmark]
        public void FfiUnited()
        {
            _ffi.TestMultipleCreationsAndDeletions(10000);
        }

        [Benchmark]
        public void UInt32FfiUnited()
        {
            _uint32Ffi.TestMultipleCreationsAndDeletions(10000);
        }
    }
}
*/
```

csharp/Platform.Data.Doublets.Tests/AddressLimitTests.cs

```
using System;
using Platform.Data.Doublets.Memory;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Data.Exceptions;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests;

public class AddressLimitTests
{
    [Theory]
    [InlineData(false)]
    [InlineData(true)]
    public void ExhaustedAddressesAreRejectedButFreedLinksCanBeReused(bool split)
    {
        using var data = new HeapResizableDirectMemory();
        using var index = new HeapResizableDirectMemory();
        var constants = new LinksConstants<ulong>((1, 9), (0, 0));
        using var disposable = split
            ? (IDisposable)new SplitMemoryLinks<ulong>(data, index, 4096, constants)
            : new UnitedMemoryLinks<ulong>(data, 4096, constants, IndexTreeType.Default);
        var links = (ILinks<ulong>)disposable;
        var maximum = constants.InternalReferencesRange.Maximum;
        Assert.InRange(maximum, 1UL, 9UL);
        for (ulong address = 1; address <= maximum; address++)
            Assert.Equal(address, links.Create());
        Assert.Throws<LinksLimitReachedException<ulong>>(() => links.Create());
        Assert.Equal(maximum, links.Count());
        links.Delete(1UL);
        Assert.Equal(1UL, links.Create());
        Assert.Throws<LinksLimitReachedException<ulong>>(() => links.Create());
    }
}
```

csharp/Platform.Data.Doublets.Tests/ConstructorSafetyTests.cs

```
using System;
using Platform.Memory;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Xunit;

namespace Platform.Data.Doublets.Tests;

public class ConstructorSafetyTests
{
    [Fact]
    public void UnitedConstructionDoesNotCallOverridesBeforeDerivedFieldsAreReady()
    {
        using var memory = new HeapResizableDirectMemory();
        using var links = new UnitedProbe(memory);
        Assert.Equal(1UL, links.Create());
    }

    [Fact]
    public void SplitConstructionDoesNotCallOverridesBeforeDerivedFieldsAreReady()
    {
        using var data = new HeapResizableDirectMemory();
        using var index = new HeapResizableDirectMemory();
        using var links = new SplitProbe(data, index);
        Assert.Equal(1UL, links.Create());
    }

    private class UnitedProbe : UnitedMemoryLinks<ulong>
    {
        private readonly bool _ready;
        public UnitedProbe(IResizableDirectMemory memory) : base(memory, 4096) => _ready = true;
        protected override void Init(IResizableDirectMemory memory, long memoryReservationStep)
        {
            if (!_ready) throw new InvalidOperationException("Derived fields are not initialized");
            base.Init(memory, memoryReservationStep);
        }
        protected override void SetPointers(IResizableDirectMemory memory)
        {
            if (!_ready) throw new InvalidOperationException("Derived fields are not initialized");
            base.SetPointers(memory);
        }
    }

    private class SplitProbe : SplitMemoryLinks<ulong>
    {
        private readonly bool _ready;
        public SplitProbe(IResizableDirectMemory data, IResizableDirectMemory index) : base(data, index, 64) => _ready = true;
        protected override void Init(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory)
        {
            if (!_ready) throw new InvalidOperationException("Derived fields are not initialized");
            base.Init(dataMemory, indexMemory);
        }
        protected override void SetPointers(IResizableDirectMemory dataMemory, IResizableDirectMemory indexMemory)
        {
            if (!_ready) throw new InvalidOperationException("Derived fields are not initialized");
            base.SetPointers(dataMemory, indexMemory);
        }
    }
}
```

csharp/Platform.Data.Doublets.Tests/DeleteAllTests.cs

```
using System;
using System.Collections.Generic;
using System.Numerics;
using Platform.Data.Doublets.Decorators;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests;

// Issue #354: DeleteAll must delete every link and leave a store that works like a new one.
public class DeleteAllTests
{
    public static readonly TheoryData<bool> Stores = new() { false, true };

    [Theory]
    [MemberData(nameof(Stores))]
    public void DeletesContiguousPoints(bool split)
    {
        // The original loop deleted 3, skipped 2 and deleted 1.
        Check<byte>(split, links => CreatePoints(links, 3));
        Check<ushort>(split, links => CreatePoints(links, 3));
        Check<uint>(split, links => CreatePoints(links, 3));
        Check<ulong>(split, links => CreatePoints(links, 3));
    }

    [Theory]
    [MemberData(nameof(Stores))]
    public void DeletesLinksBetweenPoints(bool split)
    {
        // Deleting without resetting endpoints first leaves stale entries in the endpoint indexes.
        Check<uint>(split, links => links.CreateAndUpdate(links.CreatePoint(), links.CreatePoint()));
        Check<uint>(split, links =>
        {
            var (first, second) = (links.CreatePoint(), links.CreatePoint());
            links.CreateAndUpdate(first, second);
            links.CreateAndUpdate(second, first);
        });
    }

    [Theory]
    [MemberData(nameof(Stores))]
    public void DeletesLinksAroundFreedAddresses(bool split)
    {
        // Count() is 3 while the highest address is 5: links 4 and 5 are above the count.
        Check<uint>(split, links =>
        {
            var points = CreatePoints(links, 5);
            links.Delete(points[1], null);
            links.Delete(points[2], null);
            links.Update(points[4], points[0], points[3]);
        });
    }

    [Theory]
    [MemberData(nameof(Stores))]
    public void DeletesLinksThatUseHigherAddresses(bool split)
    {
        Check<uint>(split, links =>
        {
            var points = CreatePoints(links, 3);
            links.Update(points[0], points[1], points[2]);
        });
    }

    [Theory]
    [MemberData(nameof(Stores))]
    public void EmptyStoreStaysEmpty(bool split) => Check<uint>(split, _ => { });

    [Theory]
    [MemberData(nameof(Stores))]
    public void WorksThroughCascadeDecorators(bool split)
    {
        Check<uint>(split, links =>
        {
            var points = CreatePoints(links, 4);
            links.Delete(points[1], null);
            links.GetOrCreate(points[0], points[3]);
            links.GetOrCreate(points[3], points[2]);
        }, links => links.DecorateWithAutomaticUniquenessAndUsagesResolution());
    }

    [Theory]
    [MemberData(nameof(Stores))]
    public void DeletesUsagesBeforeTheLinksTheyUse(bool split)
    {
        // The validator rejects deleting a link that is still used by another link.
        Check<uint>(split, links =>
        {
            var (first, second) = (links.CreatePoint(), links.CreatePoint());
```

```
            links.CreateAndUpdate(first, second);
        }, links => new LinksUsagesValidator<uint>(links));
    }

    private static void Check<TLinkAddress>(bool split, Action<ILinks<TLinkAddress>> fill, Func<ILinks<TLinkAddress>, ILinks<T
LinkAddress>>? decorate = null)
        where TLinkAddress : IUnsignedNumber<TLinkAddress>, IShiftOperators<TLinkAddress, int, TLinkAddress>,
            IBitwiseOperators<TLinkAddress, TLinkAddress, TLinkAddress>, IMinMaxValue<TLinkAddress>,
            IComparisonOperators<TLinkAddress, TLinkAddress, bool>
    {
        using var sources = new HeapResizableDirectMemory();
        using var targets = new HeapResizableDirectMemory();
        if (split)
        {
            using var store = new SplitMemoryLinks<TLinkAddress>(sources, targets, 4096);
            DeleteAllAndReuse(store, fill, decorate);
        }
        else
        {
            using var store = new UnitedMemoryLinks<TLinkAddress>(sources, 4096);
            DeleteAllAndReuse(store, fill, decorate);
        }
    }

    private static void DeleteAllAndReuse<TLinkAddress>(ILinks<TLinkAddress> raw, Action<ILinks<TLinkAddress>> fill, Func<ILin
ks<TLinkAddress>, ILinks<TLinkAddress>>? decorate)
        where TLinkAddress : IUnsignedNumber<TLinkAddress>
    {
        fill(raw);
        var links = decorate?.Invoke(raw) ?? raw;
        links.DeleteAll();
        Assert.Equal(TLinkAddress.Zero, links.Count());
        Assert.Empty(links.All());
        // Addresses start from 1 again, and the endpoint indexes contain only new links.
        var first = links.CreatePoint();
        var second = links.CreatePoint();
        // GetOrCreate searches the index first, so a stale entry would be returned instead of a new link.
        var link = links.GetOrCreate(first, second);
        Assert.Equal(TLinkAddress.One, first);
        Assert.Equal(TLinkAddress.CreateTruncating(3), link);
        Assert.Equal(TLinkAddress.CreateTruncating(3), links.Count());
        Assert.Equal(link, links.SearchOrDefault(first, second));
        var any = links.Constants.Any;
        Assert.Equal(TLinkAddress.CreateTruncating(2), links.Count(any, first, any));
        Assert.Equal(TLinkAddress.CreateTruncating(2), links.Count(any, any, second));
    }

    private static List<TLinkAddress> CreatePoints<TLinkAddress>(ILinks<TLinkAddress> links, int count)
        where TLinkAddress : IUnsignedNumber<TLinkAddress>
    {
        var points = new List<TLinkAddress>();
        for (var i = 0; i < count; i++)
        {
            points.Add(links.CreatePoint());
        }
        return points;
    }
}
```

csharp/Platform.Data.Doublets.Tests/DeletionContractTests.cs

```
using System;
using System.Collections.Generic;
using Platform.Data.Doublets.Decorators;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Delegates;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests
{
    public static class DeletionContractTests
    {
        [Fact]
        public static void UnitedCoreDeletionDoesNotValidateCallerContract()
        {
            using var links = new UnitedWithoutExistenceChecks();
            var index = links.CreatePoint();
            links.Update(index, 0u, 0u);
            links.RejectExistenceChecks = true;
            links.Delete(new[] { index }, null);
            links.RejectExistenceChecks = false;
            Assert.Equal(0u, links.Count());
            Assert.Equal(index, links.CreatePoint());
        }

        [Fact]
        public static void SplitCoreDeletionDoesNotValidateCallerContract()
        {
            using var links = new SplitWithoutExistenceChecks();
            var index = links.CreatePoint();
            links.Update(index, 0u, 0u);
            links.RejectExistenceChecks = true;
            links.Delete(new[] { index }, null);
            links.RejectExistenceChecks = false;
            Assert.Equal(0u, links.Count());
            Assert.Equal(index, links.CreatePoint());
        }

        [Fact]
        public static void ResetResolverDoesNotAlsoValidateExistence()
        {
            using var raw = new UnitedMemoryLinks<uint>(new HeapResizableDirectMemory(), 4096);
            var inner = new ResetLinkStub(raw);
            var links = new NonNullContentsLinkDeletionResolver<uint>(inner);
            Assert.Equal(links.Constants.Continue, links.Delete(new[] { 1u }, null));
            Assert.True(inner.Deleted);
        }

        private sealed class UnitedWithoutExistenceChecks : UnitedMemoryLinks<uint>
        {
            public bool RejectExistenceChecks { get; set; }
            public UnitedWithoutExistenceChecks() : base(new HeapResizableDirectMemory(), 4096) { }
            protected override bool Exists(uint link) => RejectExistenceChecks
                ? throw new InvalidOperationException("Core deletion must trust its caller.")
                : base.Exists(link);
        }

        private sealed class SplitWithoutExistenceChecks : SplitMemoryLinks<uint>
        {
            public bool RejectExistenceChecks { get; set; }
            public SplitWithoutExistenceChecks()
                : base(new HeapResizableDirectMemory(), new HeapResizableDirectMemory(), 4096) { }
            protected override bool Exists(uint link) => RejectExistenceChecks
                ? throw new InvalidOperationException("Core deletion must trust its caller.")
                : base.Exists(link);
        }

        private sealed class ResetLinkStub : LinksDecoratorBase<uint>
        {
            public bool Deleted { get; private set; }
            public ResetLinkStub(ILinks<uint> links) : base(links) { }
            public override uint Count(IList<uint>? restriction)
                => throw new InvalidOperationException($"Unexpected validation with {restriction?.Count} restriction parts.");
            public override uint Each(IList<uint>? restriction, ReadHandler<uint>? handler)
            {
                Assert.Equal(1u, Assert.Single(restriction!));
                return handler!(new[] { 1u, 0u, 0u });
            }
            public override uint Delete(IList<uint>? restriction, WriteHandler<uint>? handler)
            {
                Assert.Equal(1u, Assert.Single(restriction!));
                Deleted = true;
                return handler?.Invoke(new[] { 1u, 0u, 0u }, null) ?? Constants.Continue;
            }
        }
    }
}
```

csharp/Platform.Data.Doublets.Tests/EitherEndpointQueryTests.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using Platform.Data.Doublets.Memory;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests;

public class EitherEndpointQueryTests
{
    public static IEnumerable<object[]> Stores()
    {
        foreach (var decorated in new[] { false, true })
        {
            foreach (var tree in new[] { IndexTreeType.Default, IndexTreeType.SizedAndThreadedAVLBalancedTree })
            {
                yield return new object[] { false, tree, true, decorated };
            }
            foreach (var tree in new[] { IndexTreeType.Default, IndexTreeType.SizeBalancedTree, IndexTreeType.RecursionlessSiz
eBalancedTree })
            {
                foreach (var linkedList in new[] { false, true })
                {
                    yield return new object[] { true, tree, linkedList, decorated };
                }
            }
        }
    }

    [Theory]
    [MemberData(nameof(Stores))]
    public void CountMatchesOriginalReproduction(bool split, IndexTreeType tree, bool linkedList, bool decorated)
        => CheckOriginal(split, tree, linkedList, decorated, count: true);

    [Theory]
    [MemberData(nameof(Stores))]
    public void EachMatchesOriginalReproduction(bool split, IndexTreeType tree, bool linkedList, bool decorated)
        => CheckOriginal(split, tree, linkedList, decorated, count: false);

    private static void CheckOriginal(bool split, IndexTreeType tree, bool linkedList, bool decorated, bool count)
    {
        Using<uint>(split, tree, linkedList, decorated, links =>
        {
            links.CreatePoint();
            links.CreatePoint();
            links.Update(2u, 1u, 2u);
            var query = new[] { links.Constants.Any, 1u };
            if (count)
            {
                Assert.Equal(2u, links.Count(query));
            }
            else
            {
                Assert.Equal(new[] { 1u, 2u }, Find(links, query));
            }
        });
    }

    [Theory]
    [MemberData(nameof(Stores))]
    public void MatchingSnapshotCanBeDeletedOnce(bool split, IndexTreeType tree, bool linkedList, bool decorated)
    {
        Using<uint>(split, tree, linkedList, decorated, links =>
        {
            var a = links.CreatePoint();
            var b = links.CreatePoint();
            links.Update(b, a, b);
            var matches = Find(links, new[] { links.Constants.Any, a });
            foreach (var index in Enumerable.Reverse(matches))
            {
                Assert.True(links.Exists(index));
                links.Update(index, 0u, 0u);
                links.Delete(index);
            }
            Assert.Equal(0u, links.Count());
        });
    }

    [Theory]
    [MemberData(nameof(Stores))]
    public void UnionMatchesScanAcrossAddressTypes(bool split, IndexTreeType tree, bool linkedList, bool decorated)
    {
        CheckUnion<byte>(split, tree, linkedList, decorated);
        CheckUnion<ushort>(split, tree, linkedList, decorated);
        CheckUnion<uint>(split, tree, linkedList, decorated);
        CheckUnion<ulong>(split, tree, linkedList, decorated);
    }
```

```
private static void CheckUnion<T>(bool split, IndexTreeType tree, bool linkedList, bool decorated)
    where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
        IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
{
    Using<T>(split, tree, linkedList, decorated, links =>
    {
        var any = links.Constants.Any;
        AssertUnion(links, T.One);
        var a = links.CreatePoint();
        var b = links.CreatePoint();
        var sourceOnly = links.CreateAndUpdate(a, b);
        var targetOnly = links.CreateAndUpdate(b, a);
        var overlap = decorated ? links.CreateAndUpdate(a, a) : a;
        var values = new List<T> { a, b, T.Zero, T.CreateChecked(50) };
        var external = T.Zero;
        if (split)
        {
            external = new Hybrid<T>(T.CreateChecked(20), isExternal: true).Value;
            links.CreateAndUpdate(external, external);
            links.CreateAndUpdate(external, a);
            links.CreateAndUpdate(b, external);
            values.Add(external);
        }
        foreach (var value in values)
        {
            AssertUnion(links, value);
        }
        var empty = links.Create();
        AssertUnion(links, T.Zero);
        links.Delete(empty);
        Assert.Equal(Find(links, Array.Empty<T>()).OrderBy(x => x), Find(links, new[] { any, any }).OrderBy(x => x));
        Assert.Equal(links.Count(), links.Count(new[] { any, any }));
        Assert.Equal(new[] { a }, Find(links, new[] { a, a }));
        Assert.Equal(T.One, links.Count(new[] { a, a }));

        // Exercise index changes, removal and free-slot reuse.
        links.Update(sourceOnly, b, sourceOnly);
        links.Update(targetOnly, T.Zero, T.Zero);
        links.Delete(targetOnly);
        links.CreateAndUpdate(b, a);
        foreach (var value in values)
        {
            AssertUnion(links, value);
        }
        Assert.True(links.Exists(overlap));

        // AVL indexes reject duplicate keys even on raw stores.
        if (!decorated && tree != IndexTreeType.SizedAndThreadedAVLBalancedTree)
        {
            links.CreateAndUpdate(a, a);
            links.CreateAndUpdate(a, a);
            if (split)
            {
                links.CreateAndUpdate(external, external);
            }
            foreach (var value in values)
            {
                AssertUnion(links, value);
            }
        }
    });
}

[Theory]
[MemberData(nameof(Stores))]
public void BreakStopsInEitherPass(bool split, IndexTreeType tree, bool linkedList, bool decorated)
{
    Using<uint>(split, tree, linkedList, decorated, links =>
    {
        var a = links.CreatePoint();
        var b = links.CreatePoint();
        links.CreateAndUpdate(a, b);
        links.CreateAndUpdate(b, a);
        var query = new[] { links.Constants.Any, a };
        var all = Find(links, query);
        for (var stopAt = 1; stopAt <= all.Length; stopAt++)
        {
            var visited = new List<uint>();
            var result = links.Each(query, link =>
            {
                visited.Add(link![0]);
                return visited.Count == stopAt ? links.Constants.Break : links.Constants.Continue;
            });
            Assert.Equal(links.Constants.Break, result);
            Assert.Equal(all.Take(stopAt), visited);
            Assert.Equal(visited.Count, visited.Distinct().Count());
        }
        Assert.Equal(links.Constants.Continue, links.Each(query, null));
    });
}
```

```
    private static void AssertUnion<T>(ILinks<T> links, T value) where T : IUnsignedNumber<T>
    {
        var expected = new List<T>();
        links.Each(Array.Empty<T>(), link =>
        {
            if (link![1] == value || link[2] == value)
            {
                expected.Add(link[0]);
            }
            return links.Constants.Continue;
        });
        var query = new[] { links.Constants.Any, value };
        Assert.Equal(T.CreateChecked(expected.Count), links.Count(query));
        Assert.Equal(expected.OrderBy(x => x), Find(links, query).OrderBy(x => x));
    }

    private static T[] Find<T>(ILinks<T> links, T[] query) where T : IUnsignedNumber<T>
    {
        var found = new List<T>();
        Assert.Equal(links.Constants.Continue, links.Each(query, link =>
        {
            found.Add(link![0]);
            return links.Constants.Continue;
        }));
        return found.ToArray();
    }

    private static void Using<T>(bool split, IndexTreeType tree, bool linkedList, bool decorated, Action<ILinks<T>> action)
        where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
            IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
    {
        var constants = new LinksConstants<T>(enableExternalReferencesSupport: true);
        using var united = split ? null : new UnitedMemoryLinks<T>(new HeapResizableDirectMemory(), 4096, constants, tree);
        using var separated = split ? new SplitMemoryLinks<T>(new HeapResizableDirectMemory(), new HeapResizableDirectMemory()
,
            4096, constants, tree, linkedList) : null;
        ILinks<T> raw = split ? separated! : united!;
        action(decorated ? raw.DecorateWithAutomaticUniquenessAndUsagesResolution() : raw);
    }
}
```

csharp/Platform.Data.Doublets.Tests/ExceptionDiagnosticsTests.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Reflection;
using Platform.Data.Doublets.Decorators;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests;

public class ExceptionDiagnosticsTests
{
    [Fact]
    public void ErrorConversionPreservesDefaultBehaviorAndSupportsOptInDiagnosticsAndFiltering()
    {
        using var memory = new HeapResizableDirectMemory();
        using var store = new UnitedMemoryLinks<ulong>(memory);
        var decorator = new NoExceptionsDecorator<ulong>(new ThrowingLinks(store));
        Assert.Null(decorator.ExceptionHandler);
        Assert.Equal(store.Constants.Error, decorator.Count(null));
        var errors = new List<Exception>();
        decorator.ExceptionHandler = errors.Add;
        Assert.Equal(store.Constants.Error, decorator.Count(null));
        Assert.IsType<InvalidOperationException>(Assert.Single(errors));
        decorator.ExceptionFilter = _ => false;
        Assert.Throws<InvalidOperationException>(() => decorator.Count(null));
        Assert.Single(errors);
    }

    [Fact]
    public void ExplicitTransactionDisposalReportsFailureToPersistCommitState()
    {
        var directory = Path.Join(Path.GetTempPath(), Guid.NewGuid().ToString("N"));
        Directory.CreateDirectory(directory);
        try
        {
            var logPath = Path.Join(directory, "transactions.log");
            using var memory = new HeapResizableDirectMemory();
            using var store = new UnitedMemoryLinks<ulong>(memory);
            var owner = new DisposalProbe(store);
            var layer = new UInt64LinksTransactionsLayer(owner, logPath);
            // Inject a real I/O failure after construction without changing filesystem
            // permissions (which behave differently for root and on Windows).
            var field = typeof(UInt64LinksTransactionsLayer).GetField("_log", BindingFlags.Instance | BindingFlags.NonPublic);
            Assert.NotNull(field);
            Assert.IsType<FileStream>(field.GetValue(layer)).Dispose();
            using var exclusive = new FileStream(logPath, FileMode.Open, FileAccess.ReadWrite, FileShare.None);
            Assert.ThrowsAny<IOException>(() => layer.Dispose());
            Assert.True(owner.IsDisposed);
            GC.SuppressFinalize(layer);
        }
        finally
        {
            Directory.Delete(directory, true);
        }
    }

    [Fact]
    public void DamagedTransactionLogKeepsItsOriginalDiagnosticDuringConstructorCleanup()
    {
        var path = Path.GetTempFileName();
        try
        {
            var bytes = new byte[checked((int)UInt64LinksTransactionsLayer.Transition.Size * 2)];
            bytes[UInt64LinksTransactionsLayer.Transition.Size] = 1;
            File.WriteAllBytes(path, bytes);
            using var memory = new HeapResizableDirectMemory();
            using var store = new UnitedMemoryLinks<ulong>(memory);
            Assert.Throws<NotSupportedException>(() => new UInt64LinksTransactionsLayer(store, path));
        }
        finally
        {
            File.Delete(path);
        }
    }

    private class DisposalProbe : LinksDecoratorBase<ulong>, IDisposable
    {
        public bool IsDisposed { get; private set; }
        public DisposalProbe(ILinks<ulong> storage) : base(storage) { }
        public void Dispose()
        {
            IsDisposed = true;
            ((IDisposable)_links).Dispose();
        }
    }

    private class ThrowingLinks : LinksDecoratorBase<ulong>
    {
        public ThrowingLinks(ILinks<ulong> storage) : base(storage) { }
```

```
        public override ulong Count(IList<ulong>? restriction) => throw new InvalidOperationException("reproduction");
    }
}
```

csharp/Platform.Data.Doublets.Tests/GarbageCollectionTests.cs

```
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Memory;
using Xunit;
using TLinkAddress = System.UInt64;

namespace Platform.Data.Doublets.Tests;

public class GarbageCollectionTests
{
    public IResizableDirectMemory LinksMemory;
    public ILinks<TLinkAddress> Links;


    public GarbageCollectionTests()
    {
        LinksMemory = new HeapResizableDirectMemory();
        Links = new UnitedMemoryLinks<TLinkAddress>(LinksMemory);
    }

    [Fact]
    public void ClearGarbagePartialPointDependency()
    {
        TLinkAddress link = Links.CreatePoint();
        Links.ClearGarbage(link);
        Assert.True(Links.Exists(link));
    }

    [Fact]
    public void ClearGarbageFullPointDependency()
    {
        TLinkAddress link = Links.CreatePoint();
        Links.ClearGarbage(link);
        Assert.True(Links.Exists(link));
    }

    [Fact]
    public void ClearGarbageWithInDependency()
    {
        TLinkAddress link = Links.CreatePoint();
        TLinkAddress dependant = Links.GetOrCreate(TLinkAddress.CreateTruncating(10), link);
        Links.ClearGarbage(link);
        Assert.True(Links.Exists(link));
        Assert.True(Links.Exists(dependant));
    }

    [Fact]
    public void ClearGarbageWithOutDependency()
    {
        TLinkAddress link = Links.CreatePoint();
        TLinkAddress dependant = Links.GetOrCreate(link, TLinkAddress.CreateTruncating(10));
        Links.ClearGarbage(link);
        Assert.True(Links.Exists(link));
        Assert.True(Links.Exists(dependant));
    }

    [Fact]
    public void ClearGarbageWithoutDependency()
    {
        TLinkAddress link1 = Links.CreatePoint();
        TLinkAddress link2 = Links.CreatePoint();
        TLinkAddress link3 = Links.GetOrCreate(link1, link2);
        Links.ClearGarbage(link3);
        Assert.False(Links.Exists(link3));
        Assert.True(Links.Exists(link2));
        Assert.True(Links.Exists(link1));
    }

    [Fact]
    public void ComplexgarbageCollectionTest()
    {
        /*
           (1: 1 1)
           (2: 2 2)
           (3: 1 2)
           (4: 3 2)
         */
        TLinkAddress link1 = Links.CreatePoint();
        TLinkAddress link2 = Links.CreatePoint();
        TLinkAddress link3 = Links.GetOrCreate(link1, link2);
        TLinkAddress link4 = Links.GetOrCreate(link3, link2);
        Links.ClearGarbage(link4);
        Assert.False(Links.Exists(link4));
        Assert.False(Links.Exists(link3));
        Assert.True(Links.Exists(link2));
        Assert.True(Links.Exists(link1));

    }
}
```

csharp/Platform.Data.Doublets.Tests/GenericLinksTests.cs

```
using System;
using System.IO;
using System.Numerics;
using Platform.Data.Doublets.Decorators;
using Xunit;

using Platform.Memory;

using Platform.Data.Doublets.Memory.United.Generic;

namespace Platform.Data.Doublets.Tests
{
    public static class GenericLinksTests
    {
        [Fact]
        public static void CRUDTest()
        {
            Using<byte>(links => links.TestCRUDOperations());
            Using<ushort>(links => links.TestCRUDOperations());
            Using<uint>(links => links.TestCRUDOperations());
            Using<ulong>(links => links.TestCRUDOperations());
        }

        [Fact]
        public static void RawNumbersCRUDTest()
        {
            Using<byte>(links => links.TestRawNumbersCRUDOperations());
            Using<ushort>(links => links.TestRawNumbersCRUDOperations());
            Using<uint>(links => links.TestRawNumbersCRUDOperations());
            Using<ulong>(links => links.TestRawNumbersCRUDOperations());
        }

        [Fact]
        public static void MultipleRandomCreationsAndDeletionsTest()
        {
            Using<byte>(links => links.DecorateWithAutomaticUniquenessAndUsagesResolution().TestMultipleRandomCreationsAndDele
tions(16)); // Cannot use more because current implementation of tree cuts out 5 bits from the address space.
            Using<ushort>(links => links.DecorateWithAutomaticUniquenessAndUsagesResolution().TestMultipleRandomCreationsAndDe
letions(100));
            Using<uint>(links => links.DecorateWithAutomaticUniquenessAndUsagesResolution().TestMultipleRandomCreationsAndDele
tions(100));
            Using<ulong>(links => links.DecorateWithAutomaticUniquenessAndUsagesResolution().TestMultipleRandomCreationsAndDel
etions(100));
        }
        private static void Using<TLinkAddress>(Action<ILinks<TLinkAddress>> action) where TLinkAddress : IUnsignedNumber<TLin
kAddress>, IShiftOperators<TLinkAddress, int, TLinkAddress>, IBitwiseOperators<TLinkAddress, TLinkAddress, TLinkAddress>, IMin
MaxValue<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
        {
            var unitedMemoryLinks = new UnitedMemoryLinks<TLinkAddress>(new HeapResizableDirectMemory());
            using (var logFile = File.Open("linksLogger.txt", FileMode.Create, FileAccess.Write))
            {
                LoggingDecorator<TLinkAddress> decoratedStorage = new(unitedMemoryLinks, logFile);
                action(decoratedStorage);
            }

            /*
            File.Delete("db.links");
            using var ffiLinks = new Ffi.Links<TLinkAddress>("db.links");
            action(ffiLinks);
        */
        }
    }
}
```

csharp/Platform.Data.Doublets.Tests/ILinksBasicTests.cs

```
using System.IO;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Memory;
using Xunit;


namespace Platform.Data.Doublets.Tests
{
    public static class ILinksBasicTests
    {
        [Fact]
        public static void DeleteAllUsages()
        {
            var mem = new HeapResizableDirectMemory();
            var links = new UnitedMemoryLinks<uint>(mem);

            var root = links.CreatePoint();

            var a = links.CreatePoint();
            var b = links.CreatePoint();

            links.CreateAndUpdate(a, root);
            links.CreateAndUpdate(b, root);

            Assert.Equal(5U, links.Count());

            links.DeleteAllUsages(root);

            Assert.Equal(3U, links.Count());
        }

        /*
        [Fact]
        public static void FfiDeleteAllUsages()
        {
            File.Delete("db.links");
            var links = new Ffi.Links<uint>("db.links");

            var root = links.CreatePoint();

            var a = links.CreatePoint();
            var b = links.CreatePoint();

            links.CreateAndUpdate(a, root);
            links.CreateAndUpdate(b, root);

            Assert.Equal(5U, links.Count());

            links.DeleteAllUsages(root);

            Assert.Equal(3U, links.Count());
        }
    */
    }
}
```

csharp/Platform.Data.Doublets.Tests/LinksConstantsTests.cs

```
using Xunit;

namespace Platform.Data.Doublets.Tests
{
    public static class LinksConstantsTests
    {
        [Fact]
        public static void ExternalReferencesTest()
        {
            LinksConstants<ulong> constants = new LinksConstants<ulong>((1, long.MaxValue), (long.MaxValue + 1UL, ulong.MaxVal
ue));

            //var minimum = new Hybrid<ulong>(0, isExternal: true);
            var minimum = new Hybrid<ulong>(1, isExternal: true);
            var maximum = new Hybrid<ulong>(long.MaxValue, isExternal: true);

            Assert.True(constants.IsExternalReference(minimum));
            Assert.True(constants.IsExternalReference(maximum));
        }
    }
}
```

csharp/Platform.Data.Doublets.Tests/MergeUsagesTests.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests;

// Issue #515: re-pointing a usage must replace only the endpoint naming the merged link.
public class MergeUsagesTests
{
    [Theory]
    [InlineData(false)]
    [InlineData(true)]
    public void MergeUsagesRewritesOnlyTheMergedEndpoint(bool split)
    {
        Check<byte>(split, Merge.Usages);
        Check<ushort>(split, Merge.Usages);
        Check<uint>(split, Merge.Usages);
        Check<ulong>(split, Merge.Usages);
    }

    [Theory]
    [InlineData(false)]
    [InlineData(true)]
    public void MergeUsagesWithHandlerRewritesOnlyTheMergedEndpoint(bool split)
    {
        Check<byte>(split, Merge.UsagesWithHandler);
        Check<ulong>(split, Merge.UsagesWithHandler);
    }

    [Theory]
    [InlineData(false)]
    [InlineData(true)]
    public void MergeAndDeleteRewritesOnlyTheMergedEndpoint(bool split)
    {
        Check<uint>(split, Merge.AndDelete);
        Check<uint>(split, Merge.AndDeleteWithHandler);
    }

    [Fact]
    public void MergeUsagesReportsEveryRewrittenLink()
    {
        using var links = new UnitedMemoryLinks<uint>(new HeapResizableDirectMemory(), 4096);
        var (one, two, three, asSource, asTarget, asBoth) = CreateIssueLinks(links);
        var changes = new List<string>();
        var result = links.MergeUsages(one, two, (before, after) =>
        {
            changes.Add($"({string.Join(" ", before!)}) -> ({string.Join(" ", after!)})");
            return links.Constants.Continue;
        });
        Assert.Equal(links.Constants.Continue, result);
        // A link using the merged link on both ends is rewritten once per endpoint.
        Assert.Equal(new[]
        {
            $"({asSource} {one} {three}) -> ({asSource} {two} {three})",
            $"({asBoth} {one} {one}) -> ({asBoth} {two} {one})",
            $"({asTarget} {three} {one}) -> ({asTarget} {three} {two})",
            $"({asBoth} {two} {one}) -> ({asBoth} {two} {two})",
        }.Order(), changes.Order());
    }

    private enum Merge { Usages, UsagesWithHandler, AndDelete, AndDeleteWithHandler }

    private static void Check<T>(bool split, Merge merge)
        where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
            IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
    {
        using var united = split ? null : new UnitedMemoryLinks<T>(new HeapResizableDirectMemory(), 4096);
        using var separated = split ? new SplitMemoryLinks<T>(new HeapResizableDirectMemory(), new HeapResizableDirectMemory()
, 4096) : null;
        ILinks<T> links = split ? separated! : united!;
        var (one, two, three, asSource, asTarget, asBoth) = CreateIssueLinks(links);
        var continueResult = links.Constants.Continue;
        switch (merge)
        {
            case Merge.Usages:
                links.MergeUsages(one, two);
                break;
            case Merge.UsagesWithHandler:
                Assert.Equal(continueResult, links.MergeUsages(one, two, (_, _) => continueResult));
                break;
            case Merge.AndDelete:
                Assert.Equal(two, links.MergeAndDelete(one, two));
                break;
            case Merge.AndDeleteWithHandler:
                Assert.Equal(continueResult, links.MergeAndDelete(one, two, (_, _) => continueResult));
                break;
```

```
            default:
                throw new ArgumentOutOfRangeException(nameof(merge), merge, null);
        }
        var deleted = merge is Merge.AndDelete or Merge.AndDeleteWithHandler;
        Assert.Equal(!deleted, links.Exists(one));
        if (!deleted)
        {
            Assert.Equal(new[] { one, T.Zero, T.Zero }, links.GetLink(one));
        }
        Assert.Equal(new[] { two, T.Zero, T.Zero }, links.GetLink(two));
        Assert.Equal(new[] { three, T.Zero, T.Zero }, links.GetLink(three));
        // The issue expects (4: 1 3) -> (4: 2 3) and (5: 3 1) -> (5: 3 2), not (4: 3 0) and (5: 2 0).
        Assert.Equal(new[] { asSource, two, three }, links.GetLink(asSource));
        Assert.Equal(new[] { asTarget, three, two }, links.GetLink(asTarget));
        Assert.Equal(new[] { asBoth, two, two }, links.GetLink(asBoth));
        Assert.Equal(T.CreateTruncating(deleted ? 5 : 6), links.Count());
        Assert.Equal(T.Zero, links.Count(new Link<T>(links.Constants.Any, one, links.Constants.Any)));
        Assert.Equal(T.Zero, links.Count(new Link<T>(links.Constants.Any, links.Constants.Any, one)));
    }

    private static (T One, T Two, T Three, T AsSource, T AsTarget, T AsBoth) CreateIssueLinks<T>(ILinks<T> links)
        where T : IUnsignedNumber<T>
    {
        var nothing = links.Constants.Null;
        var one = links.CreateAndUpdate(nothing, nothing);
        var two = links.CreateAndUpdate(nothing, nothing);
        var three = links.CreateAndUpdate(nothing, nothing);
        var asSource = links.CreateAndUpdate(one, three);
        var asTarget = links.CreateAndUpdate(three, one);
        var asBoth = links.CreateAndUpdate(one, one);
        Assert.Equal(new[] { asSource, one, three }, links.GetLink(asSource));
        Assert.Equal(new[] { asTarget, three, one }, links.GetLink(asTarget));
        return (one, two, three, asSource, asTarget, asBoth);
    }
}
```

csharp/Platform.Data.Doublets.Tests/MissingLinkDeletionTests.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using Platform.Data.Doublets.Decorators;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Data.Exceptions;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests
{
    public static class MissingLinkDeletionTests
    {
        public static IEnumerable<object[]> MissingDeleteCases()
        {
            foreach (var split in new[] { false, true })
            {
                foreach (var decorated in new[] { false, true })
                {
                    foreach (var direct in new[] { false, true })
                    {
                        foreach (var scenario in new[] { "never-created", "deleted-middle", "deleted-last", "empty", "null-add
ress", "out-of-range" })
                        {
                            foreach (var ignoreMissing in new[] { false, true })
                            {
                                yield return new object[] { split, decorated, direct, scenario, ignoreMissing };
                            }
                        }
                    }
                }
            }
        }

        [Theory]
        [MemberData(nameof(MissingDeleteCases))]
        public static void MissingDeleteUsesSelectedPolicyWithoutChangingStore(bool split, bool decorated, bool direct, string
scenario, bool ignoreMissing)
        {
            CheckMissingDelete<byte>(split, decorated, direct, scenario, ignoreMissing);
            CheckMissingDelete<ushort>(split, decorated, direct, scenario, ignoreMissing);
            CheckMissingDelete<uint>(split, decorated, direct, scenario, ignoreMissing);
            CheckMissingDelete<ulong>(split, decorated, direct, scenario, ignoreMissing);
        }

        private static void CheckMissingDelete<T>(bool split, bool decorated, bool direct, string scenario, bool ignoreMissing
)
            where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
                IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
        {
            Using<T>(split, decorated, inner =>
            {
                ILinks<T> links = ignoreMissing
                    ? new LinksNonExistentDeletionResolver<T>(inner)
                    : new LinksInnerReferenceExistenceValidator<T>(inner);
                var first = links.CreatePoint();
                var middle = links.CreatePoint();
                var last = links.CreatePoint();
                var missing = T.CreateTruncating(10);
                var next = last + T.One;
                switch (scenario)
                {
                    case "never-created":
                        break;
                    case "deleted-middle":
                        ResetAndDelete(links, middle);
                        missing = next = middle;
                        break;
                    case "deleted-last":
                        ResetAndDelete(links, last);
                        missing = next = last;
                        break;
                    case "empty":
                        ResetAndDelete(links, last);
                        ResetAndDelete(links, middle);
                        ResetAndDelete(links, first);
                        missing = next = first;
                        break;
                    case "null-address":
                        missing = links.Constants.Null;
                        break;
                    case "out-of-range":
                        missing = T.MaxValue;
                        break;
                    default:
                        throw new ArgumentOutOfRangeException(nameof(scenario), scenario, "Unknown deletion scenario.");
                }

                Assert.False(links.Exists(missing));
```

```
        var before = Snapshot(links);
        var count = links.Count();
        var calls = 0;
        // Check both handler paths, repeating the invalid deletion each time.
        for (var attempt = 0; attempt < 2; attempt++)
        {
            void DeleteMissing()
            {
                T result;
                if (direct)
                {
                    result = links.Delete(new[] { missing }, attempt == 0 ? null : (_, _) =>
                    {
                        calls++;
                        return links.Constants.Continue;
                    });
                }
                else if (attempt == 0)
                {
                    result = links.Delete(missing, null);
                }
                else
                {
                    result = links.Delete(missing, (_, _) =>
                    {
                        calls++;
                        return links.Constants.Continue;
                    });
                }
                Assert.Equal(links.Constants.Continue, result);
            }
            if (ignoreMissing)
            {
                DeleteMissing();
            }
            else
            {
                Assert.Throws<ArgumentLinkDoesNotExistsException<T>>(DeleteMissing);
            }
            Assert.Equal(0, calls);
            Assert.Equal(count, links.Count());
            AssertSnapshot(links, before);
            Assert.False(links.Exists(missing));
        }

        // The no-handler adapter returns an address, rather than a flow constant.
        if (ignoreMissing)
        {
            Assert.Equal(links.Constants.Null, links.Delete(missing));
        }
        else
        {
            Assert.Throws<ArgumentLinkDoesNotExistsException<T>>(() => links.Delete(missing));
        }
        Assert.Equal(count, links.Count());
        AssertSnapshot(links, before);

        // A double free corrupts allocation even when enumeration still looks right.
        Assert.Equal(next, links.CreatePoint());
        var following = scenario == "deleted-middle" ? last + T.One : next + T.One;
        Assert.Equal(following, links.CreatePoint());
        Assert.Equal(count + T.CreateTruncating(2), links.Count());
        Assert.Equal(int.CreateTruncating(links.Count()), Snapshot(links).Length);
        foreach (var row in before)
        {
            Assert.Equal(row, links.GetLink(row[0]));
            Assert.Equal(row[0], links.SearchOrDefault(row[1], row[2]));
        }
        Assert.Equal(new[] { next, next, next }, links.GetLink(next));
        Assert.Equal(new[] { following, following, following }, links.GetLink(following));
    });
}

[Theory]
[InlineData(false, false, 0)]
[InlineData(false, false, 1)]
[InlineData(false, false, 2)]
[InlineData(false, true, 0)]
[InlineData(false, true, 1)]
[InlineData(false, true, 2)]
[InlineData(true, false, 0)]
[InlineData(true, false, 1)]
[InlineData(true, false, 2)]
[InlineData(true, true, 0)]
[InlineData(true, true, 1)]
[InlineData(true, true, 2)]
public static void ExistingResetLinkCanBeDeletedWithBreakHandler(bool split, bool decorated, int policy)
{
    CheckExistingDelete<byte>(split, decorated, policy);
    CheckExistingDelete<ushort>(split, decorated, policy);
    CheckExistingDelete<uint>(split, decorated, policy);
    CheckExistingDelete<ulong>(split, decorated, policy);
```

```
        }

        private static void CheckExistingDelete<T>(bool split, bool decorated, int policy)
            where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
                IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
        {
            Using<T>(split, decorated, inner =>
            {
                ILinks<T> links = policy switch
                {
                    1 => new LinksInnerReferenceExistenceValidator<T>(inner),
                    2 => new LinksNonExistentDeletionResolver<T>(inner),
                    _ => inner
                };
                var index = links.CreatePoint();
                links.Update(index, T.Zero, T.Zero);
                var calls = 0;
                var result = links.Delete(new[] { index }, (before, after) =>
                {
                    calls++;
                    Assert.Equal(new[] { index, T.Zero, T.Zero }, before);
                    Assert.Null(after);
                    return links.Constants.Break;
                });
                Assert.Equal(links.Constants.Break, result);
                Assert.Equal(1, calls);
                Assert.False(links.Exists(index));
                Assert.Equal(T.Zero, links.Count());
                Assert.Equal(index, links.CreatePoint());
            });
        }

        private static void Using<T>(bool split, bool decorated, Action<ILinks<T>> action)
            where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
                IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
        {
            using var united = split ? null : new UnitedMemoryLinks<T>(new HeapResizableDirectMemory(), 4096);
            using var splitLinks = split
                ? new SplitMemoryLinks<T>(new HeapResizableDirectMemory(), new HeapResizableDirectMemory(), 4096)
                : null;
            ILinks<T> links = split ? splitLinks! : united!;
            action(decorated ? links.DecorateWithAutomaticUniquenessAndUsagesResolution() : links);
        }

        private static T[][] Snapshot<T>(ILinks<T> links) where T : IUnsignedNumber<T>
        {
            var rows = new List<T[]>();
            links.Each(Array.Empty<T>(), link =>
            {
                rows.Add(link!.ToArray());
                return links.Constants.Continue;
            });
            return rows.ToArray();
        }

        private static void ResetAndDelete<T>(ILinks<T> links, T index) where T : IUnsignedNumber<T>
        {
            // Plain stores require endpoints to be reset before a valid deletion.
            links.Update(index, T.Zero, T.Zero);
            links.Delete(index);
        }

        private static void AssertSnapshot<T>(ILinks<T> links, T[][] expected) where T : IUnsignedNumber<T>
        {
            var actual = Snapshot(links);
            Assert.Equal(expected.Length, actual.Length);
            for (var i = 0; i < expected.Length; i++)
            {
                Assert.Equal(expected[i], actual[i]);
            }
        }
    }
}
```

csharp/Platform.Data.Doublets.Tests/NullableContractTests.cs

```
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Memory;
using Platform.Data.Doublets.Decorators;
using Xunit;

namespace Platform.Data.Doublets.Tests;

public class NullableContractTests
{
    [Theory]
    [InlineData(false)]
    [InlineData(true)]
    public void OptionalRestrictionsAndReadHandlersAreSupported(bool split)
    {
        using var first = new HeapResizableDirectMemory();
        using var second = new HeapResizableDirectMemory();
        using var storage = split
            ? (System.IDisposable)new SplitMemoryLinks<ulong>(first, second)
            : new UnitedMemoryLinks<ulong>(first);
        var links = (ILinks<ulong>)storage;
        links.Create();
        Assert.Equal(1UL, links.Count(null));
        Assert.Equal(links.Constants.Continue, links.Each(null, null));
        var visited = 0;
        links.Each(null, _ => { visited++; return links.Constants.Continue; });
        Assert.Equal(1, visited);
        links.Create();
        visited = 0;
        Assert.Equal(links.Constants.Break, links.Each(null, _ => { visited++; return links.Constants.Break; }));
        Assert.Equal(1, visited);
    }

    [Theory]
    [InlineData(false)]
    [InlineData(true)]
    public void UniversalCreationAndDeletionAllowIndependentOptionalHandlers(bool split)
    {
        using var first = new HeapResizableDirectMemory();
        using var second = new HeapResizableDirectMemory();
        using var storage = split
            ? (System.IDisposable)new SplitMemoryLinks<ulong>(first, second)
            : new UnitedMemoryLinks<ulong>(first);
        var links = (ILinks<ulong>)storage;
        var universal = new UniLinks<ulong>(links);
        var calls = 0;
        var created = 0UL;
        universal.Trigger(null, (Platform.Delegates.ReadHandler<ulong>?)null, new ulong[] { 0, 0, 0 }, (before, after) =>
        {
            System.ArgumentNullException.ThrowIfNull(after);
            created = after[0];
            calls++;
            return links.Constants.Continue;
        });
        Assert.Equal(1, calls);
        Assert.NotEqual(0UL, created);
        // A match handler must not cause invocation of an absent write handler.
        universal.Trigger(new[] { created }, _ => links.Constants.Continue, null, null);
        Assert.Equal(0UL, links.Count());
    }

    [Theory]
    [InlineData(false)]
    [InlineData(true)]
    public void EnsureCreatedFillsAllRequestedFreeAddresses(bool split)
    {
        using var first = new HeapResizableDirectMemory();
        using var second = new HeapResizableDirectMemory();
        using var storage = split
            ? (System.IDisposable)new SplitMemoryLinks<ulong>(first, second)
            : new UnitedMemoryLinks<ulong>(first);
        var links = (ILinks<ulong>)storage;
        for (var i = 0; i < 4; i++) links.Create();
        links.Delete(1UL);
        links.Delete(3UL);
        // The free list returns 3 before 1. Stopping at the maximum misses 1.
        links.EnsureCreated(1UL, 3UL);
        Assert.True(links.Exists(1UL));
        Assert.True(links.Exists(3UL));
        Assert.Equal(4UL, links.Count());
    }

    [Theory]
    [InlineData(false)]
    [InlineData(true)]
    public void EnsurePointsCreatedRetainsExistingLinksAndOnlyRequestedNewPoints(bool split)
    {
        using var first = new HeapResizableDirectMemory();
        using var second = new HeapResizableDirectMemory();
        using var storage = split
            ? (System.IDisposable)new SplitMemoryLinks<ulong>(first, second)
```

```
            : new UnitedMemoryLinks<ulong>(first);
        var links = (ILinks<ulong>)storage;
        var existing = links.Create();
        links.EnsurePointsCreated(existing, 0UL, 3UL, 5UL, 3UL, links.Constants.Any);
        Assert.True(links.Exists(existing));
        Assert.False(links.Exists(2UL));
        Assert.False(links.Exists(4UL));
        Assert.Equal(3UL, links.GetSource(3UL));
        Assert.Equal(3UL, links.GetTarget(3UL));
        Assert.Equal(5UL, links.GetSource(5UL));
        Assert.Equal(5UL, links.GetTarget(5UL));
        Assert.Equal(3UL, links.Count());
    }

    [Theory]
    [InlineData(false)]
    [InlineData(true)]
    public void EnsureCreatedRejectsARepeatedCreatorResult(bool split)
    {
        using var first = new HeapResizableDirectMemory();
        using var second = new HeapResizableDirectMemory();
        using var storage = split
            ? (System.IDisposable)new SplitMemoryLinks<ulong>(first, second)
            : new UnitedMemoryLinks<ulong>(first);
        var links = (ILinks<ulong>)storage;
        Assert.Throws<System.InvalidOperationException>(() => links.EnsureCreated(() => 1UL, 3UL));
    }

    [Fact]
    public void UniversalUpdateUsesTheExistingLinkInsteadOfCreatingAnother()
    {
        using var memory = new HeapResizableDirectMemory();
        using var links = new UnitedMemoryLinks<ulong>(memory);
        var original = links.Create();
        var universal = new UniLinks<ulong>(links);
        universal.Trigger(new[] { original }, (Platform.Delegates.ReadHandler<ulong>?)null,
            new ulong[] { 0, original, original }, null);
        Assert.Equal(1UL, links.Count(null));
        Assert.Equal(original, links.GetSource(original));
    }
}
```

csharp/Platform.Data.Doublets.Tests/PositionalQueryTests.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using Platform.Data.Doublets.Memory;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests;

public class PositionalQueryTests
{
    [Theory]
    [MemberData(nameof(EitherEndpointQueryTests.Stores), MemberType = typeof(EitherEndpointQueryTests))]
    public void CountMatchesEndpointPositions(bool split, IndexTreeType tree, bool linkedList, bool decorated)
        => CheckAllWidths(split, tree, linkedList, decorated, count: true);

    [Theory]
    [MemberData(nameof(EitherEndpointQueryTests.Stores), MemberType = typeof(EitherEndpointQueryTests))]
    public void EachMatchesEndpointPositions(bool split, IndexTreeType tree, bool linkedList, bool decorated)
        => CheckAllWidths(split, tree, linkedList, decorated, count: false);

    private static void CheckAllWidths(bool split, IndexTreeType tree, bool linkedList, bool decorated, bool count)
    {
        Check<byte>(split, tree, linkedList, decorated, count);
        Check<ushort>(split, tree, linkedList, decorated, count);
        Check<uint>(split, tree, linkedList, decorated, count);
        Check<ulong>(split, tree, linkedList, decorated, count);
    }

    private static void Check<T>(bool split, IndexTreeType tree, bool linkedList, bool decorated, bool count)
        where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
            IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
    {
        var constants = new LinksConstants<T>(enableExternalReferencesSupport: true);
        using var united = split ? null : new UnitedMemoryLinks<T>(new HeapResizableDirectMemory(), 4096, constants, tree);
        using var separated = split ? new SplitMemoryLinks<T>(new HeapResizableDirectMemory(), new HeapResizableDirectMemory()
,
            4096, constants, tree, linkedList) : null;
        ILinks<T> raw = split ? separated! : united!;
        var links = decorated ? raw.DecorateWithAutomaticUniquenessAndUsagesResolution() : raw;
        var a = links.CreatePoint();
        var b = links.CreatePoint();
        var asymmetric = links.CreateAndUpdate(a, b);
        links.CreateAndUpdate(b, a);
        var external = new Hybrid<T>(T.CreateChecked(20), isExternal: true).Value;
        links.CreateAndUpdate(external, a);
        links.CreateAndUpdate(b, external);
        links.CreateAndUpdate(external, external);
        links.CreateAndUpdate(T.Zero, a);
        links.CreateAndUpdate(b, T.Zero);
        links.Create(); // A live record with two null endpoints.
        var deleted = links.CreatePoint();
        var last = links.CreatePoint(); // Keep the deleted address inside the allocated range.
        links.Update(deleted, T.Zero, T.Zero);
        links.Delete(deleted);
        var values = new[] { constants.Any, T.Zero, a, b, external, T.CreateChecked(50) };
        AssertQueries(links, values, deleted, count);

        // Recheck after changing endpoint indexes and recycling a deleted slot.
        links.Update(asymmetric, b, asymmetric);
        Assert.Equal(deleted, links.CreateAndUpdate(last, a));
        AssertQueries(links, values, T.CreateChecked(50), count);
    }

    private static void AssertQueries<T>(ILinks<T> links, T[] values, T deleted, bool count)
        where T : IUnsignedNumber<T>
    {
        var rows = new List<T[]>();
        links.Each(Array.Empty<T>(), link =>
        {
            rows.Add(link!.ToArray());
            return links.Constants.Continue;
        });
        foreach (var index in rows.Select(row => row[0]).Concat(new[] { T.Zero, deleted, T.CreateChecked(50), values[4] }))
        {
            var row = rows.SingleOrDefault(row => row[0] == index);
            foreach (var source in values)
            {
                foreach (var target in values)
                {
                    var matches = row != null && (source == links.Constants.Any || row[1] == source)
                        && (target == links.Constants.Any || row[2] == target);
                    var query = new[] { index, source, target };
                    if (count)
                    {
                        Assert.True(links.Count(query) == (matches ? T.One : T.Zero),
                            $"Count [{index}, {source}, {target}] should match {matches}");
                    }
```

```
                    else
                    {
                        AssertEach(links, query, row, matches);
                    }
                }
                // Two-part queries intentionally match either endpoint.
                var either = row != null && (source == links.Constants.Any || row[1] == source || row[2] == source);
                var shortQuery = new[] { index, source };
                if (count)
                {
                    Assert.Equal(either ? T.One : T.Zero, links.Count(shortQuery));
                }
                else
                {
                    AssertEach(links, shortQuery, row, either);
                }
            }
        }
    }

    private static void AssertEach<T>(ILinks<T> links, T[] query, T[]? expected, bool matches)
        where T : IUnsignedNumber<T>
    {
        foreach (var flow in new[] { links.Constants.Continue, links.Constants.Break })
        {
            var calls = 0;
            var result = links.Each(query, link =>
            {
                calls++;
                Assert.Equal(expected, link!.ToArray());
                return flow;
            });
            Assert.True(calls == (matches ? 1 : 0), $"Each [{string.Join(", ", query)}] should match {matches}");
            Assert.Equal(matches ? flow : links.Constants.Continue, result);
        }
        Assert.Equal(links.Constants.Continue, links.Each(query, null));
    }
}
```

csharp/Platform.Data.Doublets.Tests/QueryRestrictionTests.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests
{
    public static class QueryRestrictionTests
    {
        [Theory]
        [InlineData(false, false)]
        [InlineData(false, true)]
        [InlineData(true, false)]
        [InlineData(true, true)]
        public static void NullCountRestrictionCountsAllLiveLinks(bool split, bool decorated)
        {
            CheckNullCountRestrictionCountsAllLiveLinks<byte>(split, decorated);
            CheckNullCountRestrictionCountsAllLiveLinks<ushort>(split, decorated);
            CheckNullCountRestrictionCountsAllLiveLinks<uint>(split, decorated);
            CheckNullCountRestrictionCountsAllLiveLinks<ulong>(split, decorated);
        }

        private static void CheckNullCountRestrictionCountsAllLiveLinks<T>(bool split, bool decorated)
            where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
                IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
        {
            Using<T>(split, decorated, links =>
            {
                Assert.Equal(T.Zero, links.Count(restriction: null));
                Assert.Equal(links.Count(Array.Empty<T>()), links.Count(restriction: null));
                var first = links.CreatePoint();
                var deleted = links.CreatePoint();
                var last = links.CreatePoint();
                AssertCount(links, 3);

                ResetAndDelete(links, deleted);
                AssertCount(links, 2);
                Assert.Equal(deleted, links.CreatePoint());
                AssertCount(links, 3);

                ResetAndDelete(links, last);
                ResetAndDelete(links, deleted);
                ResetAndDelete(links, first);
                AssertCount(links, 0);
            });
        }

        [Theory]
        [InlineData(false, false)]
        [InlineData(false, true)]
        [InlineData(true, false)]
        [InlineData(true, true)]
        public static void NullEachRestrictionEnumeratesAllLiveLinks(bool split, bool decorated)
        {
            CheckNullEachRestrictionEnumeratesAllLiveLinks<byte>(split, decorated);
            CheckNullEachRestrictionEnumeratesAllLiveLinks<ushort>(split, decorated);
            CheckNullEachRestrictionEnumeratesAllLiveLinks<uint>(split, decorated);
            CheckNullEachRestrictionEnumeratesAllLiveLinks<ulong>(split, decorated);
        }

        private static void CheckNullEachRestrictionEnumeratesAllLiveLinks<T>(bool split, bool decorated)
            where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
                IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
        {
            Using<T>(split, decorated, links =>
            {
                AssertScan(links);
                var first = links.CreatePoint();
                var deleted = links.CreatePoint();
                var last = links.CreatePoint();
                AssertScan(links, first, deleted, last);

                ResetAndDelete(links, deleted);
                AssertScan(links, first, last);
                Assert.Equal(deleted, links.CreatePoint());
                AssertScan(links, first, deleted, last);

                ResetAndDelete(links, last);
                ResetAndDelete(links, deleted);
                ResetAndDelete(links, first);
                AssertScan(links);
            });
        }

        [Theory]
        [InlineData(false, false)]
        [InlineData(false, true)]
```

```
[InlineData(true, false)]
[InlineData(true, true)]
public static void NullEachRestrictionHonorsBreak(bool split, bool decorated)
{
    CheckNullEachRestrictionHonorsBreak<byte>(split, decorated);
    CheckNullEachRestrictionHonorsBreak<ushort>(split, decorated);
    CheckNullEachRestrictionHonorsBreak<uint>(split, decorated);
    CheckNullEachRestrictionHonorsBreak<ulong>(split, decorated);
}

private static void CheckNullEachRestrictionHonorsBreak<T>(bool split, bool decorated)
    where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
        IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
{
    Using<T>(split, decorated, links =>
    {
        var first = links.CreatePoint();
        links.CreatePoint();
        foreach (var restriction in new T[]?[] { null, Array.Empty<T>() })
        {
            var calls = 0;
            var result = links.Each(restriction, link =>
            {
                calls++;
                Assert.Equal(first, link![0]);
                return links.Constants.Break;
            });
            Assert.Equal(1, calls);
            Assert.Equal(links.Constants.Break, result);
        }
    });
}

[Theory]
[InlineData(false, false)]
[InlineData(false, true)]
[InlineData(true, false)]
[InlineData(true, true)]
public static void UnsupportedCountRestrictionHasEnglishMessage(bool split, bool decorated)
{
    CheckUnsupportedCountRestrictionHasEnglishMessage<byte>(split, decorated);
    CheckUnsupportedCountRestrictionHasEnglishMessage<ushort>(split, decorated);
    CheckUnsupportedCountRestrictionHasEnglishMessage<uint>(split, decorated);
    CheckUnsupportedCountRestrictionHasEnglishMessage<ulong>(split, decorated);
}

private static void CheckUnsupportedCountRestrictionHasEnglishMessage<T>(bool split, bool decorated)
    where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
        IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
{
    Using<T>(split, decorated, links =>
    {
        var any = links.Constants.Any;
        var exception = Assert.Throws<NotSupportedException>(() => links.Count(new[] { any, any, any, any }));
        Assert.Equal("Other restriction sizes and methods are not supported.", exception.Message);
    });
}

[Theory]
[InlineData(false, false)]
[InlineData(false, true)]
[InlineData(true, false)]
[InlineData(true, true)]
public static void UnsupportedEachRestrictionHasEnglishMessage(bool split, bool decorated)
{
    CheckUnsupportedEachRestrictionHasEnglishMessage<byte>(split, decorated);
    CheckUnsupportedEachRestrictionHasEnglishMessage<ushort>(split, decorated);
    CheckUnsupportedEachRestrictionHasEnglishMessage<uint>(split, decorated);
    CheckUnsupportedEachRestrictionHasEnglishMessage<ulong>(split, decorated);
}

private static void CheckUnsupportedEachRestrictionHasEnglishMessage<T>(bool split, bool decorated)
    where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
        IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
{
    Using<T>(split, decorated, links =>
    {
        var any = links.Constants.Any;
        var calls = 0;
        var exception = Assert.Throws<NotSupportedException>(() => links.Each(new[] { any, any, any, any }, _ =>
        {
            calls++;
            return links.Constants.Continue;
        }));
        Assert.Equal("Other restriction sizes and methods are not supported.", exception.Message);
        Assert.Equal(0, calls);
    });
}

private static void Using<T>(bool split, bool decorated, Action<ILinks<T>> action)
    where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
        IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
```

```
        {
            // Keep these small regression stores within a single allocator page.
            using var united = split ? null : new UnitedMemoryLinks<T>(new HeapResizableDirectMemory(), 4096);
            using var splitLinks = split
                ? new SplitMemoryLinks<T>(new HeapResizableDirectMemory(), new HeapResizableDirectMemory(), 4096)
                : null;
            ILinks<T> links = split ? splitLinks! : united!;
            action(decorated ? links.DecorateWithAutomaticUniquenessAndUsagesResolution() : links);
        }

        private static void AssertCount<T>(ILinks<T> links, int expected) where T : IUnsignedNumber<T>
        {
            Assert.Equal(T.CreateTruncating(expected), links.Count(restriction: null));
            Assert.Equal(links.Count(Array.Empty<T>()), links.Count(restriction: null));
        }

        private static void AssertScan<T>(ILinks<T> links, params T[] expected) where T : IUnsignedNumber<T>
        {
            var emptyRows = new List<T[]>();
            var emptyResult = links.Each(Array.Empty<T>(), link =>
            {
                emptyRows.Add(link!.ToArray());
                return links.Constants.Continue;
            });
            var nullRows = new List<T[]>();
            var nullResult = links.Each(restriction: null, handler: link =>
            {
                nullRows.Add(link!.ToArray());
                return links.Constants.Continue;
            });
            Assert.Equal(expected, nullRows.Select(link => link[0]));
            Assert.Equal(emptyResult, nullResult);
            Assert.Equal(emptyRows.Count, nullRows.Count);
            for (var i = 0; i < nullRows.Count; i++)
            {
                Assert.Equal(new[] { expected[i], expected[i], expected[i] }, nullRows[i]);
                Assert.Equal(emptyRows[i], nullRows[i]);
            }
        }

        private static void ResetAndDelete<T>(ILinks<T> links, T index) where T : IUnsignedNumber<T>
        {
            // Raw stores require endpoint values to be reset before deletion.
            links.Update(index, links.Constants.Null, links.Constants.Null);
            links.Delete(index);
        }
    }
}
```

csharp/Platform.Data.Doublets.Tests/ResizableDirectMemoryLinksTests.cs

```
using System.IO;
using Xunit;
using Platform.Singletons;
using Platform.Memory;
using Platform.Data.Doublets.Memory.United.Generic;

namespace Platform.Data.Doublets.Tests
{
    public static class ResizableDirectMemoryLinksTests
    {
        private static readonly LinksConstants<ulong> _constants = Default<LinksConstants<ulong>>.Instance;

        [Fact]
        public static void BasicFileMappedMemoryTest()
        {
            var tempFilename = Path.GetTempFileName();
            using (var memoryAdapter = new TemporaryFileMappedResizableDirectMemory())
            {
                using (var unitedMemoryLinksStorage = new UnitedMemoryLinks<ulong>(memoryAdapter))
                {
                    unitedMemoryLinksStorage.TestBasicMemoryOperations();
                }
            }
            File.Delete(tempFilename);
        }

        [Fact]
        public static void BasicHeapMemoryTest()
        {
            using (var memory = new HeapResizableDirectMemory(UnitedMemoryLinks<ulong>.DefaultLinksSizeStep))
            using (var memoryAdapter = new UnitedMemoryLinks<ulong>(memory, UnitedMemoryLinks<ulong>.DefaultLinksSizeStep))
            {
                memoryAdapter.TestBasicMemoryOperations();
            }
        }
        private static void TestBasicMemoryOperations(this ILinks<ulong> memoryAdapter)
        {
            var link = memoryAdapter.Create();
            memoryAdapter.Delete(link);
        }

        [Fact]
        public static void NonexistentReferencesHeapMemoryTest()
        {
            using (var memory = new HeapResizableDirectMemory(UnitedMemoryLinks<ulong>.DefaultLinksSizeStep))
            using (var memoryAdapter = new UnitedMemoryLinks<ulong>(memory, UnitedMemoryLinks<ulong>.DefaultLinksSizeStep))
            {
                memoryAdapter.TestNonexistentReferences();
            }
        }
        private static void TestNonexistentReferences(this ILinks<ulong> memoryAdapter)
        {
            var link = memoryAdapter.Create();
            memoryAdapter.Update(link, ulong.MaxValue, ulong.MaxValue);
            var resultLink = _constants.Null;
            memoryAdapter.Each(foundLink =>
            {
                resultLink = memoryAdapter.GetIndex(foundLink);
                return _constants.Break;
            }, _constants.Any, ulong.MaxValue, ulong.MaxValue);
            Assert.True(resultLink == link);
            Assert.True(memoryAdapter.Count(ulong.MaxValue) == 0);
            memoryAdapter.Delete(link);
        }
    }
}
```

csharp/Platform.Data.Doublets.Tests/ScopeTests.cs

```
using Xunit;
using Platform.Scopes;
using Platform.Memory;
using Platform.Data.Doublets.Decorators;
using Platform.Reflection;
using Platform.Data.Doublets.Memory.United.Generic;
using TLinkAddress = System.UInt64;

namespace Platform.Data.Doublets.Tests
{
    public static class ScopeTests
    {
        [Fact]
        public static void SingleDependencyTest()
        {
            using (var scope = new Scope())
            {
                scope.IncludeAssemblyOf<IMemory>();
                var instance = scope.Use<IDirectMemory>();
                Assert.IsType<HeapResizableDirectMemory>(instance);
            }
        }

        [Fact]
        public static void CascadeDependencyTest()
        {
            using (var scope = new Scope())
            {
                scope.Include<TemporaryFileMappedResizableDirectMemory>();
                scope.Include<UnitedMemoryLinks<TLinkAddress>>();
                var instance = scope.Use<ILinks<TLinkAddress>>();
                Assert.IsType<UnitedMemoryLinks<TLinkAddress>>(instance);
            }
        }

        [Fact]
        public static void FullAutoResolutionTest()
        {
            using (var scope = new Scope(autoInclude: true, autoExplore: true))
            {
                var instance = scope.Use<CombinedDecorator<TLinkAddress>>();
                Assert.IsType<CombinedDecorator<TLinkAddress>>(instance);
            }
        }

        [Fact]
        public static void TypeParametersTest()
        {
            using (var scope = new Scope<Types<HeapResizableDirectMemory, UnitedMemoryLinks<TLinkAddress>>>())
            {
                var links = scope.Use<ILinks<TLinkAddress>>();
                Assert.IsType<UnitedMemoryLinks<TLinkAddress>>(links);
            }
        }
    }
}
```

csharp/Platform.Data.Doublets.Tests/SplitGrowthTests.cs

```
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests;

public class SplitGrowthTests
{
    [Theory]
    [InlineData(4096, 4096)]
    [InlineData(8192, 4096)]
    [InlineData(4096, 8192)]
    public void GrowthRespectsTheCapacityOfBothBuffers(long dataCapacity, long indexCapacity)
    {
        using var data = new HeapResizableDirectMemory(dataCapacity);
        using var index = new HeapResizableDirectMemory(indexCapacity);
        using var links = new SplitMemoryLinks<ulong>(data, index, 4);
        for (ulong address = 1; address <= 128; address++)
        {
            Assert.Equal(address, links.Create());
            Assert.InRange(data.UsedCapacity, 0, data.ReservedCapacity);
            Assert.InRange(index.UsedCapacity, 0, index.ReservedCapacity);
        }
        Assert.Equal(128UL, links.Count());
    }
}
```

csharp/Platform.Data.Doublets.Tests/SplitMemoryExternalReferencesTests.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using Platform.Data.Doublets.Memory;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests
{
    public static class SplitMemoryExternalReferencesTests
    {
        public static IEnumerable<object[]> Configurations()
        {
            yield return new object[] { IndexTreeType.Default, true };
            yield return new object[] { IndexTreeType.SizeBalancedTree, true };
            yield return new object[] { IndexTreeType.SizeBalancedTree, false };
            yield return new object[] { IndexTreeType.RecursionlessSizeBalancedTree, true };
            yield return new object[] { IndexTreeType.RecursionlessSizeBalancedTree, false };
        }

        public static IEnumerable<object[]> ExactQueries()
        {
            foreach (var configuration in Configurations())
            {
                foreach (var operation in new[] { "Each", "Count", "SearchOrDefault" })
                {
                    yield return new[] { configuration[0], configuration[1], operation };
                }
            }
        }

        [Theory]
        [MemberData(nameof(ExactQueries))]
        public static void InternalSourceAndExternalTargetAreFound(IndexTreeType tree, bool useLinkedList, string operation)
        {
            CheckExactQuery<byte>(tree, useLinkedList, operation);
            CheckExactQuery<ushort>(tree, useLinkedList, operation);
            CheckExactQuery<uint>(tree, useLinkedList, operation);
            CheckExactQuery<ulong>(tree, useLinkedList, operation);
        }

        private static void CheckExactQuery<T>(IndexTreeType tree, bool useLinkedList, string operation)
            where T : IUnsignedNumber<T>, IComparisonOperators<T, T, bool>
        {
            using var links = Create<T>(tree, useLinkedList);
            var source = links.CreatePoint();
            var target = new Hybrid<T>(T.CreateTruncating(20), isExternal: true).Value;
            var link = links.CreateAndUpdate(source, target);
            Assert.Equal(source, links.GetSource(link));
            Assert.Equal(target, links.GetTarget(link));
            switch (operation)
            {
                case "Each":
                    Assert.Equal(new[] { link }, Find(links, source, target));
                    break;
                case "Count":
                    Assert.Equal(T.One, links.Count(links.Constants.Any, source, target));
                    break;
                case "SearchOrDefault":
                    Assert.Equal(link, links.SearchOrDefault(source, target));
                    break;
                default:
                    throw new ArgumentOutOfRangeException(nameof(operation), operation, "Unknown query operation.");
            }
        }

        [Theory]
        [MemberData(nameof(Configurations))]
        public static void QueriesMatchScanAfterEndpointChanges(IndexTreeType tree, bool useLinkedList)
        {
            using var links = Create<ulong>(tree, useLinkedList);
            var a = links.CreatePoint();
            var b = links.CreatePoint();
            var x = new Hybrid<ulong>(20, isExternal: true).Value;
            var y = new Hybrid<ulong>(21, isExternal: true).Value;
            var values = new[] { a, b, x, y };
            var mixed = links.CreateAndUpdate(a, x);
            links.CreateAndUpdate(a, y); // Same source, different external target.
            links.CreateAndUpdate(b, x); // Same external target, different source.
            links.CreateAndUpdate(x, a);
            links.CreateAndUpdate(x, y);
            AssertQueriesMatchScan(links, values);

            // Exercise detachment and attachment across all endpoint categories.
            foreach (var pair in new[] { (b, y), (y, b), (y, x), (a, b), (a, x) })
            {
                links.Update(mixed, pair.Item1, pair.Item2);
                AssertQueriesMatchScan(links, values);
            }
```

```
            Assert.Equal(mixed, links.GetOrCreate(a, x));
            AssertQueriesMatchScan(links, values);

            // Raw stores require endpoints to be reset before deletion.
            links.Update(mixed, links.Constants.Null, links.Constants.Null);
            links.Delete(mixed);
            AssertQueriesMatchScan(links, values);
            Assert.Equal(mixed, links.CreateAndUpdate(a, x)); // Reuse the free slot.
            AssertQueriesMatchScan(links, values);
        }

        private static SplitMemoryLinks<T> Create<T>(IndexTreeType tree, bool useLinkedList)
            where T : IUnsignedNumber<T>, IComparisonOperators<T, T, bool>
        {
            // Reserve at least one page for the memory allocator, without the default million-link allocation.
            return new SplitMemoryLinks<T>(new HeapResizableDirectMemory(), new HeapResizableDirectMemory(),
                4096, new LinksConstants<T>(enableExternalReferencesSupport: true), tree, useLinkedList);
        }

        private static T[] Find<T>(ILinks<T> links, T source, T target) where T : IUnsignedNumber<T>
        {
            var found = new List<T>();
            links.Each(link =>
            {
                found.Add(link![0]);
                return links.Constants.Continue;
            }, links.Constants.Any, source, target);
            return found.OrderBy(index => index).ToArray();
        }

        private static void AssertQueriesMatchScan(ILinks<ulong> links, ulong[] values)
        {
            var all = new List<ulong[]>();
            links.Each(link =>
            {
                all.Add(link!.ToArray());
                return links.Constants.Continue;
            });
            var any = links.Constants.Any;
            foreach (var source in values.Append(any))
            {
                foreach (var target in values.Append(any))
                {
                    var expected = all.Where(link => (source == any || link[1] == source)
                        && (target == any || link[2] == target)).Select(link => link[0]).OrderBy(index => index).ToArray();
                    Assert.Equal(expected, Find(links, source, target));
                    Assert.Equal((ulong)expected.Length, links.Count(any, source, target));
                    if (source != any && target != any)
                    {
                        Assert.Equal(expected.SingleOrDefault(), links.SearchOrDefault(source, target));
                        var calls = 0;
                        var result = links.Each(link =>
                        {
                            calls++;
                            Assert.Equal(expected.Single(), link![0]);
                            return links.Constants.Break;
                        }, any, source, target);
                        Assert.Equal(expected.Length, calls);
                        Assert.Equal(expected.Length == 0 ? links.Constants.Continue : links.Constants.Break, result);
                    }
                }
            }
        }
    }
}
```

csharp/Platform.Data.Doublets.Tests/SplitMemoryGenericLinksTests.cs

```
using System;
using System.Numerics;
using Xunit;
using Platform.Memory;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory;

namespace Platform.Data.Doublets.Tests
{
    public unsafe static class SplitMemoryGenericLinksTests
    {
        [Fact]
        public static void CRUDTest()
        {
            Using<byte>(links => links.TestCRUDOperations());
            Using<ushort>(links => links.TestCRUDOperations());
            Using<uint>(links => links.TestCRUDOperations());
            Using<ulong>(links => links.TestCRUDOperations());
        }

        [Fact]
        public static void RawNumbersCRUDTest()
        {
            UsingWithExternalReferences<byte>(links => links.TestRawNumbersCRUDOperations());
            UsingWithExternalReferences<ushort>(links => links.TestRawNumbersCRUDOperations());
            UsingWithExternalReferences<uint>(links => links.TestRawNumbersCRUDOperations());
            UsingWithExternalReferences<ulong>(links => links.TestRawNumbersCRUDOperations());
        }

        [Fact]
        public static void MultipleRandomCreationsAndDeletionsTest()
        {
            Using<byte>(links => links.DecorateWithAutomaticUniquenessAndUsagesResolution().TestMultipleRandomCreationsAndDele
tions(16)); // Cannot use more because current implementation of tree cuts out 5 bits from the address space.
            Using<ushort>(links => links.DecorateWithAutomaticUniquenessAndUsagesResolution().TestMultipleRandomCreationsAndDe
letions(100));
            Using<uint>(links => links.DecorateWithAutomaticUniquenessAndUsagesResolution().TestMultipleRandomCreationsAndDele
tions(100));
            Using<ulong>(links => links.DecorateWithAutomaticUniquenessAndUsagesResolution().TestMultipleRandomCreationsAndDel
etions(100));
        }
        private static void Using<TLinkAddress>(Action<ILinks<TLinkAddress>> action) where TLinkAddress : IUnsignedNumber<TLin
kAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
        {
            using (var dataMemory = new HeapResizableDirectMemory())
            using (var indexMemory = new HeapResizableDirectMemory())
            using (var memory = new SplitMemoryLinks<TLinkAddress>(dataMemory, indexMemory))
            {
                action(memory);
            }
        }
        private static void UsingWithExternalReferences<TLinkAddress>(Action<ILinks<TLinkAddress>> action) where TLinkAddress
: IUnsignedNumber<TLinkAddress>, IComparisonOperators<TLinkAddress, TLinkAddress, bool>
        {
            var contants = new LinksConstants<TLinkAddress>(enableExternalReferencesSupport: true);
            using (var dataMemory = new HeapResizableDirectMemory())
            using (var indexMemory = new HeapResizableDirectMemory())
            using (var memory = new SplitMemoryLinks<TLinkAddress>(dataMemory, indexMemory, SplitMemoryLinks<TLinkAddress>.Def
aultLinksSizeStep, contants))
            {
                action(memory);
            }
        }
    }
}
```

csharp/Platform.Data.Doublets.Tests/UniquenessResultTests.cs

```
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using Platform.Data.Doublets.Decorators;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Delegates;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests;

public class UniquenessResultTests
{
    public static IEnumerable<object[]> Stores()
    {
        foreach (var split in new[] { false, true })
        {
            foreach (var policy in new[] { "raw", "unique", "automatic" })
            {
                yield return new object[] { split, policy };
            }
        }
    }

    [Theory]
    [MemberData(nameof(Stores))]
    public void CreationReturnsLiveAddresses(bool split, string policy)
    {
        CheckCreation<byte>(split, policy);
        CheckCreation<ushort>(split, policy);
        CheckCreation<uint>(split, policy);
        CheckCreation<ulong>(split, policy);
    }

    private static void CheckCreation<T>(bool split, string policy)
        where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
            IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
    {
        using var united = split ? null : new UnitedMemoryLinks<T>(new HeapResizableDirectMemory(), 4096);
        using var separated = split ? new SplitMemoryLinks<T>(new HeapResizableDirectMemory(), new HeapResizableDirectMemory()
, 4096) : null;
        ILinks<T> raw = split ? separated! : united!;
        var links = Decorate(raw, policy);
        var first = links.CreatePoint();
        var second = links.CreatePoint();
        var created = links.CreateAndUpdate(first, first);
        Assert.Equal(policy == "raw" ? T.CreateTruncating(3) : first, created);
        Assert.True(links.Exists(created));
        Assert.Equal(new[] { created, first, first }, links.GetLink(created));

        // No (first, second) exists: this is the CreateAndUpdate fallback in #519.
        var updated = links.UpdateOrCreateOrGet(first, second, second, second);
        Assert.Equal(policy == "raw" ? T.CreateTruncating(4) : second, updated);
        Assert.True(links.Exists(updated));
        Assert.Equal(new[] { updated, second, second }, links.GetLink(updated));
        Assert.Equal(T.CreateTruncating(policy == "raw" ? 4 : 2), links.Count());

        // An absent target pair still creates a new link through every stack.
        var unique = links.CreateAndUpdate(first, second);
        Assert.True(links.Exists(unique));
        Assert.Equal(unique, links.SearchOrDefault(first, second));
        Assert.Equal(new[] { unique, first, second }, links.GetLink(unique));
    }

    [Theory]
    [MemberData(nameof(Stores))]
    public void UpdateOrCreateOrGetReturnsAddressForEveryBranch(bool split, string policy)
    {
        using var united = split ? null : new UnitedMemoryLinks<uint>(new HeapResizableDirectMemory(), 4096);
        using var separated = split ? new SplitMemoryLinks<uint>(new HeapResizableDirectMemory(), new HeapResizableDirectMemor
y(), 4096) : null;
        var links = Decorate<uint>(split ? separated! : united!, policy);
        var first = links.CreatePoint();
        var second = links.CreatePoint();
        Assert.Equal(first, links.UpdateOrCreateOrGet(first, first, first, first));
        Assert.Equal(first, links.UpdateOrCreateOrGet(first, first, first, second));
        var merged = links.UpdateOrCreateOrGet(first, second, second, second);
        Assert.Equal(policy == "raw" ? first : second, merged);
        Assert.True(links.Exists(merged));
        Assert.Equal(new[] { merged, second, second }, links.GetLink(merged));
    }

    [Theory]
    [InlineData(false, "unique")]
    [InlineData(false, "automatic")]
    [InlineData(true, "unique")]
    [InlineData(true, "automatic")]
    public void MergeReportsSurvivorAfterDeletionCallbacks(bool split, string policy)
    {
        using var united = split ? null : new UnitedMemoryLinks<uint>(new HeapResizableDirectMemory(), 4096);
```

```
        using var separated = split ? new SplitMemoryLinks<uint>(new HeapResizableDirectMemory(), new HeapResizableDirectMemor
y(), 4096) : null;
        var links = Decorate<uint>(split ? separated! : united!, policy);
        var survivor = links.CreatePoint();
        var redundant = links.CreatePoint();
        var anchor = links.CreatePoint();
        var usage = policy == "automatic" ? links.CreateAndUpdate(redundant, anchor) : 0;
        var events = new List<(uint[]? Before, uint[]? After)>();
        Assert.Equal(links.Constants.Continue, links.Update(redundant, survivor, survivor, (before, after) =>
        {
            events.Add((before?.ToArray(), after?.ToArray()));
            return links.Constants.Continue;
        }));
        Assert.Contains(events, change => change.Before?[0] == redundant && change.After == null);
        Assert.Equal(new[] { survivor, survivor, survivor }, events[^1].After);
        Assert.False(links.Exists(redundant));
        Assert.True(links.Exists(survivor));
        if (policy == "automatic")
        {
            Assert.Equal(new[] { usage, survivor, anchor }, links.GetLink(usage));
        }
        // Updating the survivor to its current pair must also supply the result.
        Assert.Equal(survivor, links.Update(survivor, survivor, survivor));
        Assert.Equal(links.Constants.Continue, links.Update(survivor, survivor, survivor, null));
    }

    [Theory]
    [MemberData(nameof(Stores))]
    public void BreakSuppressesLaterCallbacksButCompletesCreation(bool split, string policy)
    {
        using var united = split ? null : new UnitedMemoryLinks<uint>(new HeapResizableDirectMemory(), 4096);
        using var separated = split ? new SplitMemoryLinks<uint>(new HeapResizableDirectMemory(), new HeapResizableDirectMemor
y(), 4096) : null;
        var links = Decorate<uint>(split ? separated! : united!, policy);
        var first = links.CreatePoint();
        var second = links.CreatePoint();
        foreach (var collide in new[] { false, true })
        {
            foreach (var stopAt in new[] { 1, 2 })
            {
                var calls = 0;
                Assert.Equal(links.Constants.Break, links.CreateAndUpdate(first, collide ? first : second, (_, _) =>
                    ++calls == stopAt ? links.Constants.Break : links.Constants.Continue));
                Assert.Equal(stopAt, calls);
                Assert.NotEqual(0u, links.SearchOrDefault(first, collide ? first : second));
            }
        }
        Assert.Equal(links.Constants.Continue, links.CreateAndUpdate(second, first, null));
        Assert.NotEqual(0u, links.SearchOrDefault(second, first));
    }

    [Theory]
    [InlineData(false)]
    [InlineData(true)]
    public void MergePreservesBothEndpointsOfEveryUsage(bool split)
    {
        using var united = split ? null : new UnitedMemoryLinks<uint>(new HeapResizableDirectMemory(), 4096);
        using var separated = split ? new SplitMemoryLinks<uint>(new HeapResizableDirectMemory(), new HeapResizableDirectMemor
y(), 4096) : null;
        ILinks<uint> raw = split ? separated! : united!;
        var links = raw.DecorateWithAutomaticUniquenessAndUsagesResolution();
        var first = links.CreatePoint();
        var second = links.CreatePoint();
        var survivor = links.CreateAndUpdate(first, second);
        var redundant = links.CreateAndUpdate(second, first);
        var sourceUsage = links.CreateAndUpdate(redundant, second);
        var targetUsage = links.CreateAndUpdate(first, redundant);
        var bothUsage = links.CreateAndUpdate(redundant, redundant);
        Assert.Equal(survivor, links.Update(redundant, first, second));
        Assert.False(links.Exists(redundant));
        Assert.Equal(new[] { sourceUsage, survivor, second }, links.GetLink(sourceUsage));
        Assert.Equal(new[] { targetUsage, first, survivor }, links.GetLink(targetUsage));
        Assert.Equal(new[] { bothUsage, survivor, survivor }, links.GetLink(bothUsage));
        Assert.Equal(6u, links.Count());
    }

    [Fact]
    public void CreateAndUpdateRetainsBreakEvenWhenInnerUpdateReturnsContinue()
    {
        using var raw = new UnitedMemoryLinks<uint>(new HeapResizableDirectMemory(), 4096);
        var links = new ContinueAfterUpdate(raw);
        var calls = 0;
        Assert.Equal(links.Constants.Break, links.CreateAndUpdate(0u, 0u, (_, _) =>
            ++calls == 2 ? links.Constants.Break : links.Constants.Continue));
        Assert.Equal(2, calls);
        Assert.Equal(1u, raw.Count());
    }

    private static ILinks<T> Decorate<T>(ILinks<T> raw, string policy) where T : IUnsignedNumber<T>
        => policy switch
        {
            "unique" => new LinksUniquenessResolver<T>(new NonNullContentsLinkDeletionResolver<T>(raw)),
```

```
            "automatic" => raw.DecorateWithAutomaticUniquenessAndUsagesResolution(),
            _ => raw
        };

    private sealed class ContinueAfterUpdate(ILinks<uint> links) : LinksDecoratorBase<uint>(links)
    {
        public override uint Update(IList<uint>? restriction, IList<uint>? substitution, WriteHandler<uint>? handler)
        {
            base.Update(restriction, substitution, handler);
            return Constants.Continue;
        }
    }
}
```

csharp/Platform.Data.Doublets.Tests/UpdateContractTests.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using Platform.Data.Doublets.Decorators;
using Platform.Data.Doublets.Memory.Split.Generic;
using Platform.Data.Doublets.Memory.United.Generic;
using Platform.Data.Exceptions;
using Platform.Delegates;
using Platform.Memory;
using Xunit;

namespace Platform.Data.Doublets.Tests
{
    public static class UpdateContractTests
    {
        public static IEnumerable<object[]> MissingUpdateCases()
        {
            foreach (var split in new[] { false, true })
            {
                foreach (var automatic in new[] { false, true })
                {
                    foreach (var policy in new[] { "validator", "per-call", "ignore" })
                    {
                        foreach (var scenario in new[] { "never-created", "deleted-middle", "deleted-last", "empty" })
                        {
                            foreach (var collision in new[] { false, true })
                            {
                                yield return new object[] { split, automatic, policy, scenario, collision };
                            }
                        }
                    }
                }
            }
        }

        [Theory]
        [MemberData(nameof(MissingUpdateCases))]
        public static void SelectedMissingUpdatePolicyPreservesStore(bool split, bool automatic, string policy, string scenari
o, bool collision)
        {
            CheckMissingUpdate<byte>(split, automatic, policy, scenario, collision);
            CheckMissingUpdate<ushort>(split, automatic, policy, scenario, collision);
            CheckMissingUpdate<uint>(split, automatic, policy, scenario, collision);
            CheckMissingUpdate<ulong>(split, automatic, policy, scenario, collision);
        }

        private static void CheckMissingUpdate<T>(bool split, bool automatic, string policy, string scenario, bool collision)
            where T : IUnsignedNumber<T>, IShiftOperators<T, int, T>,
                IBitwiseOperators<T, T, T>, IMinMaxValue<T>, IComparisonOperators<T, T, bool>
        {
            using var united = split ? null : new UnitedMemoryLinks<T>(new HeapResizableDirectMemory(), 4096);
            using var separated = split ? new SplitMemoryLinks<T>(new HeapResizableDirectMemory(), new HeapResizableDirectMemo
ry(), 4096) : null;
            ILinks<T> raw = split ? separated! : united!;
            ILinks<T> inner = automatic ? raw.DecorateWithAutomaticUniquenessAndUsagesResolution() : raw;
            ILinks<T> links = policy == "validator" ? new LinksInnerReferenceExistenceValidator<T>(inner) : inner;
            var first = links.CreatePoint();
            var middle = links.CreatePoint();
            var last = links.CreatePoint();
            var missing = T.CreateTruncating(10);
            var next = last + T.One;
            if (scenario == "deleted-middle")
            {
                links.Delete(middle, null);
                missing = next = middle;
            }
            else if (scenario == "deleted-last")
            {
                links.Delete(last, null);
                missing = next = last;
            }
            else if (scenario == "empty")
            {
                links.Delete(last, null);
                links.Delete(middle, null);
                links.Delete(first, null);
                missing = next = first;
            }

            var source = scenario == "empty" ? T.Zero : first;
            var target = scenario == "empty" ? T.Zero : collision ? first : last;
            var before = Snapshot(raw);
            var count = raw.Count();
            var calls = 0;
            Assert.False(raw.Exists(missing));
            // All invalid addresses stop at a selected policy, never at raw memory.
            for (var shape = 0; shape < 3; shape++)
            {
                var api = shape;
                void UpdateMissing()
```

```
                {
                    if (policy == "per-call")
                    {
                        links.EnsureLinkExists(missing, nameof(missing));
                    }
                    if (policy == "ignore" && !links.Exists(missing))
                    {
                        return;
                    }
                    WriteHandler<T> handler = (_, _) => { calls++; return links.Constants.Break; };
                    if (api == 0)
                    {
                        links.Update(missing, source, target);
                    }
                    else if (api == 1)
                    {
                        links.Update(missing, source, target, handler);
                    }
                    else
                    {
                        links.Update(new[] { missing }, new[] { missing, source, target }, null);
                    }
                }
                if (policy == "ignore")
                {
                    UpdateMissing();
                }
                else
                {
                    Assert.Throws<ArgumentLinkDoesNotExistsException<T>>(UpdateMissing);
                }
                Assert.Equal(0, calls);
                Assert.Equal(count, raw.Count());
                AssertRows(before, Snapshot(raw));
                Assert.False(raw.Exists(missing));
                Assert.Equal(collision && scenario != "empty" ? first : T.Zero, raw.SearchOrDefault(source, target));
                foreach (var row in before)
                {
                    Assert.Equal(row[0], raw.SearchOrDefault(row[1], row[2]));
                    AssertRows(before.Where(x => x[1] == row[1]).ToArray(), Snapshot(raw, new[] { raw.Constants.Any, row[1], r
aw.Constants.Any }));
                    AssertRows(before.Where(x => x[2] == row[2]).ToArray(), Snapshot(raw, new[] { raw.Constants.Any, raw.Const
ants.Any, row[2] }));
                }
            }
            // Both the phantom-pair and free-list failure from #517 stay observable.
            if (scenario != "empty" && !collision)
            {
                Assert.Equal(next, links.CreateAndUpdate(source, target));
                Assert.Equal(next, raw.SearchOrDefault(source, target));
            }
            else
            {
                Assert.Equal(next, links.CreatePoint());
            }
            var following = scenario == "deleted-middle" ? last + T.One : next + T.One;
            Assert.Equal(following, links.CreatePoint());
            Assert.Equal(count + T.CreateTruncating(2), raw.Count());
            Assert.Equal(int.CreateTruncating(raw.Count()), Snapshot(raw).Length);
            foreach (var row in before)
            {
                Assert.Equal(row, raw.GetLink(row[0]));
            }
        }

        [Theory]
        [InlineData(false)]
        [InlineData(true)]
        public static void ValidRawUpdateOmitsExistenceChecksAndPreservesCallbackFlow(bool split)
        {
            using var united = split ? null : new UnitedWithoutChecks();
            using var separated = split ? new SplitWithoutChecks() : null;
            ILinks<uint> links = split ? separated! : united!;
            var first = links.CreatePoint();
            var second = links.CreatePoint();
            if (split)
            {
                separated!.RejectChecks = true;
            }
            else
            {
                united!.RejectChecks = true;
            }
            var calls = 0;
            var result = links.Update(new[] { second }, new[] { second, first, second }, (before, after) =>
            {
                calls++;
                Assert.Equal(new[] { second, second, second }, before);
                Assert.Equal(new[] { second, first, second }, after);
                return links.Constants.Break;
            });
            if (split)
```

```
            {
                separated!.RejectChecks = false;
            }
            else
            {
                united!.RejectChecks = false;
            }
            Assert.Equal(links.Constants.Break, result);
            Assert.Equal(1, calls);
            Assert.Equal(2u, links.Count());
            Assert.Equal(second, links.SearchOrDefault(first, second));
            Assert.Equal(second, links.Update(second, second, first));
            Assert.Equal(links.Constants.Continue, links.Update(new[] { second }, new[] { second, first, second }, null));
        }

        [Fact]
        public static void OuterValidatorRejectsBeforeAnyResolverOperation()
        {
            using var raw = new UnitedMemoryLinks<uint>(new HeapResizableDirectMemory(), 4096);
            var first = raw.CreatePoint();
            var spy = new TerminalSpy(raw);
            var links = new LinksInnerReferenceExistenceValidator<uint>(spy.DecorateWithAutomaticUniquenessAndUsagesResolution
());
            Assert.Throws<ArgumentLinkDoesNotExistsException<uint>>(() => links.Update(10u, first, first));
            Assert.Equal(0, spy.Operations);
            // Valid id but missing endpoint is also rejected by this broader policy.
            Assert.Throws<ArgumentLinkDoesNotExistsException<uint>>(() => links.Update(first, first, 10u));
            Assert.Equal(0, spy.Operations);
        }

        [Fact]
        public static void AutomaticCompositionDoesNotSelectExistenceValidation()
        {
            using var raw = new UnitedMemoryLinks<uint>(new HeapResizableDirectMemory(), 4096);
            var first = raw.CreatePoint();
            var spy = new TerminalSpy(raw);
            var links = spy.DecorateWithAutomaticUniquenessAndUsagesResolution();
            // The terminal spy absorbs the request: no invalid raw access is performed.
            Assert.Equal(links.Constants.Continue, links.Update(new[] { 10u }, new[] { 10u, first, first }, null));
            Assert.Equal(2, spy.Operations); // Search, then update; no implicit validator.
            Assert.Equal(new[] { nameof(ILinks<uint>.Each), nameof(ILinks<uint>.Update) }, spy.Calls.Select(call => call.Opera
tion));
            Assert.Equal(new[] { 10u }, spy.Calls[1].Restriction);
            Assert.Equal(new[] { 10u, first, first }, spy.Calls[1].Substitution);
            Assert.Null(spy.Calls[1].Handler);
            Assert.Equal(1u, raw.Count());
            Assert.Equal(new[] { first, first, first }, raw.GetLink(first));
        }

        [Theory]
        [InlineData(false, false)]
        [InlineData(false, true)]
        [InlineData(true, false)]
        [InlineData(true, true)]
        public static void CheckedValidUpdatePreservesResultAndCallback(bool split, bool automatic)
        {
            using var united = split ? null : new UnitedMemoryLinks<uint>(new HeapResizableDirectMemory(), 4096);
            using var separated = split ? new SplitMemoryLinks<uint>(new HeapResizableDirectMemory(), new HeapResizableDirectM
emory(), 4096) : null;
            ILinks<uint> raw = split ? separated! : united!;
            var links = new LinksInnerReferenceExistenceValidator<uint>(automatic ? raw.DecorateWithAutomaticUniquenessAndUsag
esResolution() : raw);
            var first = links.CreatePoint();
            var second = links.CreatePoint();
            var calls = 0;
            Assert.Equal(links.Constants.Break, links.Update(second, first, second, (before, after) =>
            {
                calls++;
                Assert.Equal(new[] { second, second, second }, before);
                Assert.Equal(new[] { second, first, second }, after);
                return links.Constants.Break;
            }));
            Assert.Equal(1, calls);
            Assert.Equal(second, links.Update(second, second, first));
            Assert.Equal(second, raw.SearchOrDefault(second, first));
            Assert.Equal(2u, raw.Count());
        }

        [Fact]
        public static void InnerReferenceValidatorDoesNotValidateServiceConstantsAsAddresses()
        {
            using var raw = new UnitedMemoryLinks<uint>(new HeapResizableDirectMemory(), 4096);
            var first = raw.CreatePoint();
            var spy = new TerminalSpy(raw);
            var links = new LinksInnerReferenceExistenceValidator<uint>(spy);
            // These are not live link addresses. The spy prevents raw memory access.
            foreach (var address in new[] { links.Constants.Null, links.Constants.Any })
            {
                Assert.False(links.Constants.IsInternalReference(address));
                Assert.Equal(links.Constants.Continue, links.Update(new[] { address }, new[] { address, first, first }, null))
;
            }
```

```
            Assert.Equal(2, spy.Operations);
        }

        private static T[][] Snapshot<T>(ILinks<T> links, IList<T>? restriction = null) where T : IUnsignedNumber<T>
        {
            var rows = new List<T[]>();
            links.Each(restriction, row => { rows.Add(row!.ToArray()); return links.Constants.Continue; });
            return rows.OrderBy(row => row[0]).ToArray();
        }

        private static void AssertRows<T>(T[][] expected, T[][] actual)
        {
            Assert.Equal(expected.Length, actual.Length);
            for (var i = 0; i < expected.Length; i++)
            {
                Assert.Equal(expected[i], actual[i]);
            }
        }

        private sealed class UnitedWithoutChecks : UnitedMemoryLinks<uint>
        {
            public bool RejectChecks { get; set; }
            public UnitedWithoutChecks() : base(new HeapResizableDirectMemory(), 4096) { }
            protected override bool Exists(uint link) => RejectChecks ? throw new InvalidOperationException("Unexpected core v
alidation") : base.Exists(link);
        }

        private sealed class SplitWithoutChecks : SplitMemoryLinks<uint>
        {
            public bool RejectChecks { get; set; }
            public SplitWithoutChecks() : base(new HeapResizableDirectMemory(), new HeapResizableDirectMemory(), 4096) { }
            protected override bool Exists(uint link) => RejectChecks ? throw new InvalidOperationException("Unexpected core v
alidation") : base.Exists(link);
        }

        private sealed class TerminalSpy : LinksDecoratorBase<uint>
        {
            public List<(string Operation, IList<uint>? Restriction, IList<uint>? Substitution, Delegate? Handler)> Calls { ge
t; } = new();
            public int Operations => Calls.Count;
            public TerminalSpy(ILinks<uint> links) : base(links) { }
            public override uint Each(IList<uint>? restriction, ReadHandler<uint>? handler)
            {
                Calls.Add((nameof(Each), restriction, null, handler));
                return Constants.Continue;
            }
            public override uint Update(IList<uint>? restriction, IList<uint>? substitution, WriteHandler<uint>? handler)
            {
                Calls.Add((nameof(Update), restriction, substitution, handler));
                return Constants.Continue;
            }
            public override uint Delete(IList<uint>? restriction, WriteHandler<uint>? handler)
            {
                Calls.Add((nameof(Delete), restriction, null, handler));
                return Constants.Continue;
            }
        }
    }
}
```